Министерство транспорта Российской Федерации

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«РОССИЙСКИЙ УНИВЕРСИТЕТ ТРАНСПОРТА»**

Кафедра «Вычислительные системы, сети и информационная безопасность»

М.А. Давыдовский, М.Н. Никольская

ПРОЕКТИРОВАНИЕ ПРОГРАММНОЙ СИСТЕМЫ В UML DESIGNER

Учебное пособие

Москва – 2019

Министерство транспорта Российской Федерации

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «РОССИЙСКИЙ УНИВЕРСИТЕТ ТРАНСПОРТА»

Кафедра «Вычислительные системы, сети и информационная безопасность»

М.А. Давыдовский, М.Н. Никольская

ПРОЕКТИРОВАНИЕ ПРОГРАММНОЙ СИСТЕМЫ В UML DESIGNER

Учебное пособие для бакалавров, обучающихся по направлениям: «Информатика и вычислительная техника» и «Информационная безопасность»

Москва – 2019

УДК 004.4
Д13

Давыдовский М.А., Никольская М.Н. Проектирование программной системы в UML Designer: Учебное пособие. – М.: РУТ (МИИТ), 2019. – 129 с.

Рецензируемое учебное пособие описывает построение моделей программных систем в среде UML Designer. Разработка UML-моделей позволяет подробно и всесторонне описать проектируемое программное обеспечение (ПО), что положительно влияет на качество создаваемого ПО. Поэтому очень важно ознакомить студентов профилей обучения, связанных с созданием программных систем, с особенностями построения UML-диаграмм.

В учебном пособии рассмотрены диаграммы языка UML и особенности их использования при разработке проекта программной системы.

Также описана работа в среде UML Designer и инструменты построения диаграмм языка UML. Рассмотрены, как графические элементы рисования диаграмм, так и описание этих элементов на языке UML. Показана связь различных диаграмм за счет использования одних и тех же элементов на разных диаграммах и применения одних диаграмм для более подробного описания элементов, показанных на других диаграммах. Приведен пример модели программной системы.

В целом учебное пособие написано хорошим профессиональным языком, построено методически грамотно, содержит всю информацию, необходимую для работы в рассматриваемой среде. Считаю, что пособие может быть рекомендовано для бакалавров направления 09.03.01 «Информатика и вычислительная техника» и направления 10.03.01 «Информационная безопасность»

Рецензенты:

– д.т.н, профессор, профессор кафедры «Компьютерные системы и сети» МГТУ им. Н.Э. Баумана Г.С. Иванова

– старший преподаватель кафедры «Управление транспортным бизнесом и интеллектуальные системы» РУТ (МИИТ) А.И. Новиков

© РУТ(МИИТ), 2019

ОГЛАВЛЕНИЕ

Введение 5
1. Настройка UML Designer 6
1.1. Начало работы 6
1.2. Перспективы 7
1.3. Переключение рабочих областей 11
1.4. Пример проекта TravelAgency 12
2. Виды UML Designer 13
2.1. Вид Model Explorer 13
2.2. Вид Outline 15
2.3. Вид Properties 16
3. Редактор диаграмм 19
3.1. Палитра инструментов 19
3.2. Панель вкладок 21
3.3. Область редактирования 24
4. Работа с проектом 27
4.1. Создание нового проекта 27
4.2. Создание новой модели 28
4.3. Панель инструментов 29
4.4. Создание диаграммы 31
4.5. Экспорт и импорт проекта 33
5. Диаграмма вариантов использования (Use Case Diagram) 34
5.1. Элементы диаграммы вариантов использования 34
5.2. Отношения между элементами диаграммы вариантов использования 37
5.3. Пример диаграммы вариантов использования 39
6. Диаграмма пакетов (Package Hierarchy Diagram) 40
6.1. Создание диаграммы пакетов 40
6.2. Элементы диаграммы пакетов 41
6.3. Отношения между элементами диаграммы пакетов 43
7. Диаграмма классов (Class Diagram) 45
7.1. Создание диаграммы классов 46
7.2. Элементы диаграммы классов 46
7.3. Описание структуры элементов 49
7.3.1. Свойства элемента 50
7.3.2. Операции элемента 54
7.4. Отношения между элементами диаграммы классов 56
7.4.1. Отношения ассоциации 57
7.4.2. Отношение обобщения 66
7.4.3. Отношение реализации интерфейса 66
7.4.4. Отношения зависимости 67
7.4.5. Отношение вложенности 68
7.4.6. Отношение связывания 69
8. Диаграмма деятельности (Activity Diagram) 72
8.1. Создание диаграммы деятельности 73
8.2. Элементы диаграммы деятельности 75
8.3. Исполняемые узлы 77
8.4. Объектные узлы 80
8.5. Управляющие узлы 85
8.6. Отношения между элементами диаграммы деятельности 87
8.7. Примеры диаграмм деятельности 89

9. Диаграмма состояний (State Machine Diagram) 92
9.1. Создание диаграммы состояний 92
9.2. Элементы диаграммы состояний 94
9.3. Переход между состояниями 98
9.4. Действия в состоянии 105
9.5. Пример диаграммы состояний 106
10. Диаграмма последовательности (Sequence Diagram) 107
10.1. Создание диаграммы последовательности 108
10.2. Элементы диаграммы последовательности 108
10.2.1. Актеры и объекты 108
10.2.2. Сообщения 112
10.3. Пример диаграммы последовательности 116
11. Диаграмма компонентов (Component Diagram) 117
11.1. Создание диаграммы компонентов 117
11.2. Элементы диаграммы компонентов 117
11.3. Отношения между элементами диаграммы компонентов 121
12. Диаграмма развертывания (Deployment Diagram) 123
12.1. Создание диаграммы развертывания 123
12.2. Элементы диаграммы развертывания 124
12.3. Отношения между элементами диаграммы развертывания 126
12.4. Пример диаграммы развертывания 128
Литература 129

Введение

UML Designer является свободно распространяемым программным продуктом с открытым исходным кодом для построения графических моделей программных систем и баз данных на языке унифицированного моделирования (Unified Modeling Language - UML).

Язык UML применяется для разработки и наглядного представления модели программной системы. Модель может быть использована как на этапе выработки требований к системе, этапе анализа, так и на этапе разработки программной системы. Она позволяет описать функциональные возможности системы для различных пользователей, ее структуру, информационное содержимое, алгоритмы функционирования системы. Модель помогает заказчику и разработчику программного продукта выработать единое мнение о требованиях, предъявляемых к системе. На основе модели разрабатывается программный код системы. Модель является хорошим средством для документации возможностей и алгоритмов реализации системы.

Для разработки программных продуктов используются системы автоматизации проектирования программного обеспечения. Их называют CASE-системами (Computer Aided Software Engineering systems). UML Designer – одна из таких систем.

В учебном пособии рассмотрены следующие диаграммы языка UML стандарта 2.4, которые можно разработать в UML Designer:

- диаграмму вариантов использования (Use Case Diagram);
- диаграмму пакетов (Package Diagram);
- диаграмму классов (Class Diagram);
- диаграмму деятельности (Activity Diagram);
- диаграмму состояний (State Machine Diagram);
- диаграмму последовательности (Sequence Diagram);
- диаграмму компонентов (Component Diagram);
- диаграмма развертывания (размещения) (Deployment Diagram).

Стандарт UML[10] не определяет порядок разработки диаграмм при проектировании программного обеспечения. Мы будем рассматривать данные диаграммы в перечисленной последовательности как один из возможных способов разработки модели программной системы.

Диаграммы можно разбить на два класса:

- структурные диаграммы описывают статические элементы системы;
- диаграммы поведения описывают работу системы во времени.

Из перечисленных диаграмм к структурным диаграммам относятся:

- диаграмма пакетов;
- диаграмма классов;
- диаграмма компонентов;
- диаграмма развертывания.

Из перечисленных диаграмм к диаграммам поведения относятся:

- диаграмма вариантов использования;
- диаграмма деятельности;
- диаграмма состояний;
- диаграмма последовательности.

В данном учебном пособии рассматриваются вопросы проектирования программных систем на языке UML, диаграммы языка UML, используемые для наглядного представления проекта системы. Описана работа в среде UML Designer по построению модели программной системы. Приведены примеры построения диаграмм на языке UML.

Для более углубленного изучения языка UML и среды UML Designer рекомендуется литература [1-10].

1. Настройка UML Designer

1.1. Начало работы

Для установки UML Designer зайдите на сайт http://www.umldesigner.org/ и скачайте последнюю версию продукта в соответствии с вашей операционной системой.

Запустите UML Designer, выбрав файл UMLDesigner.exe.

UML Designer хранит все свои файлы в рабочей области (workspace). Это некоторая папка, в которой хранятся все файлы проектов. Укажите ее в поле Workspace (рис. 1.1).

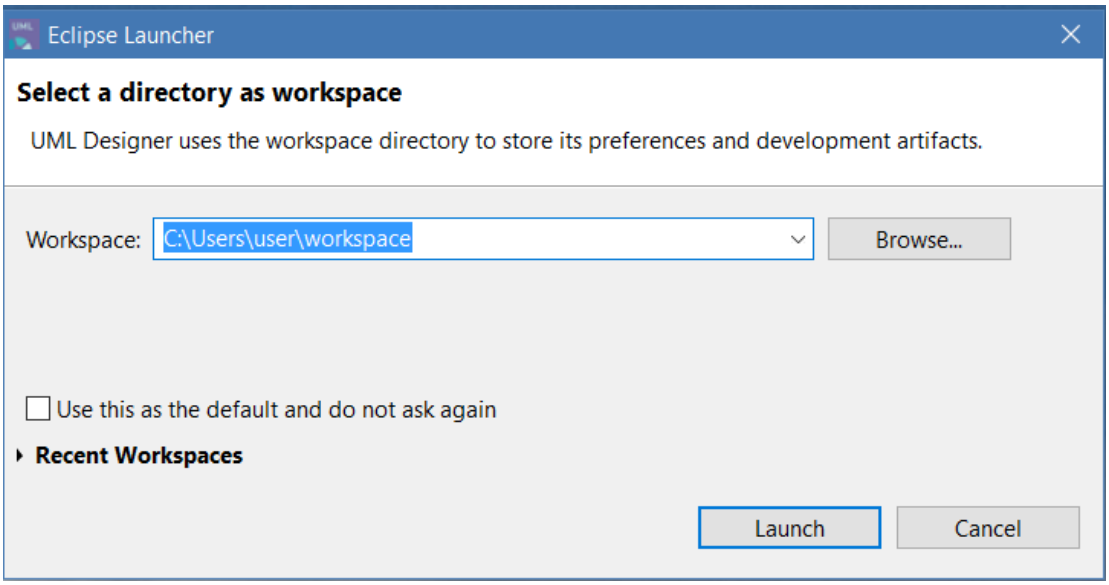


Рис. 1.1. Определение рабочей области

Если вы будете работать с одной и той же рабочей областью, то поставьте галочку в строке «Use this as the default and do not ask again».

Нажмите кнопку **Launch** для начала работы.

При первоначальном запуске UML Designer появляется страница Welcome (рис. 1.2).

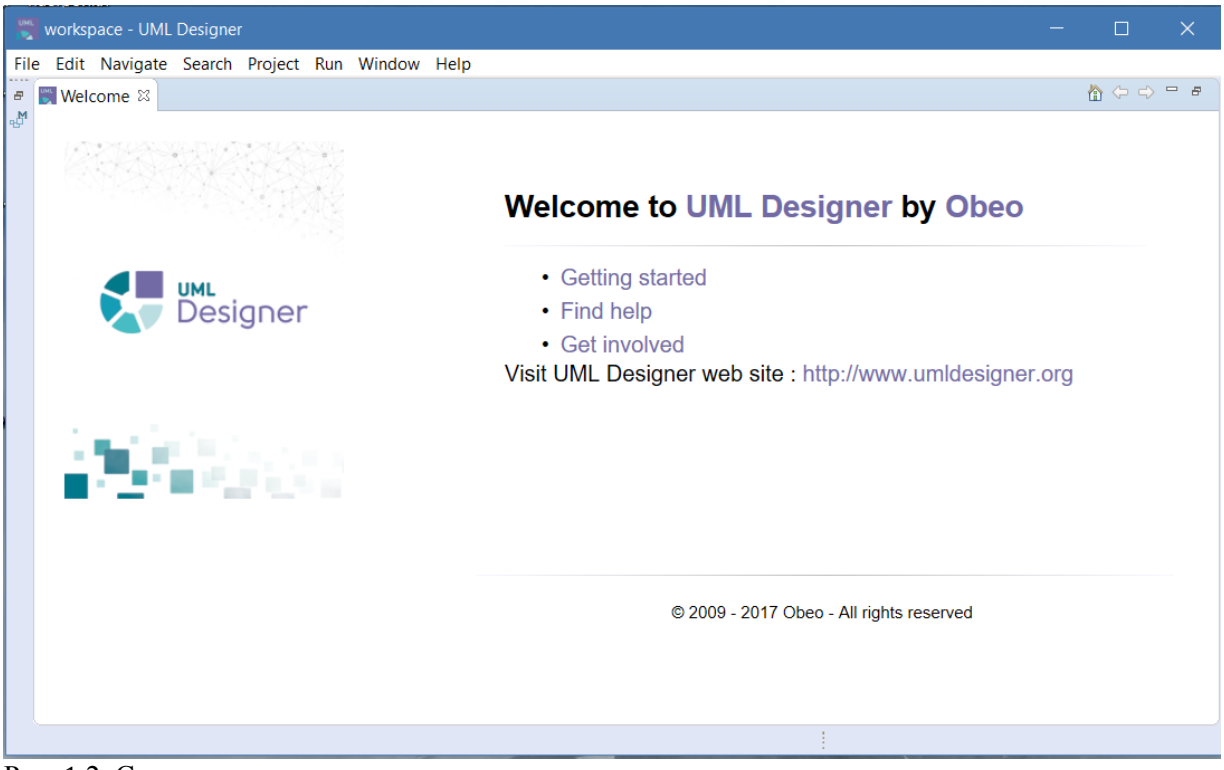


Рис. 1.2. Страница приветствия

Вы можете перейти к изучению начала работы с UML Designer по ссылке Getting started, или приступить к работе, закрыв эту страницу. Данную страницу всегда можно открыть из меню Help/Welcome (рис. 1.3).



Рис. 1.3. Меню и панель инструментов системы

1.2. Перспективы

Первоначальный вид окна UML Designer показан на рис. 1.4.

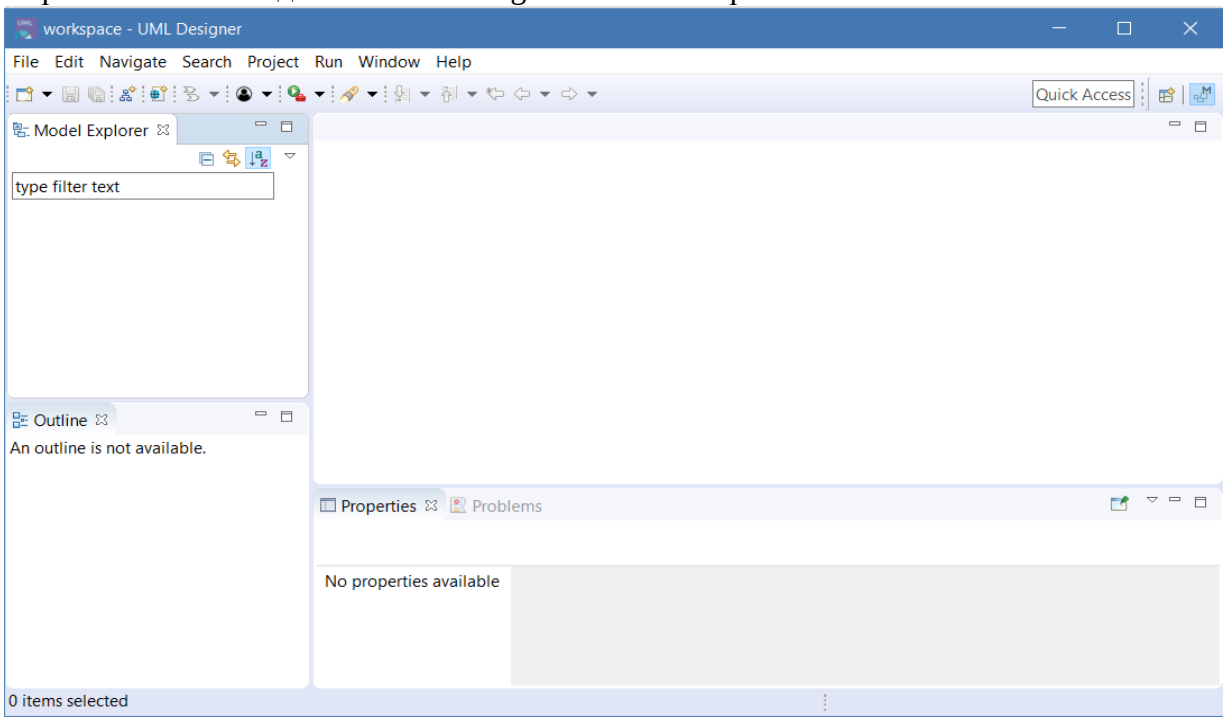


Рис. 1.4. Первоначальное окно UML Designer

В первой строке показана текущая рабочая область, во второй строке - меню UML Designer, в третьей строке - панель инструментов. Окно имеет область редактирования и нескольких видов (View): Model Explorer, Otline и др. Область редактирования (справа от Model Explorer) предназначена для рисования диаграмм. Виды используются для выбора диаграмм и описания элементов диаграмм. Определенная группа видов, которая показывается при работе в UML Designer, называется перспективой (Perspective). Данная перспектива используется по умолчанию и имеет имя Modeling.

Открыть другую перспективу можно с помощью меню Window/Perspective/Open Perspective/Other (рис. 1.5).

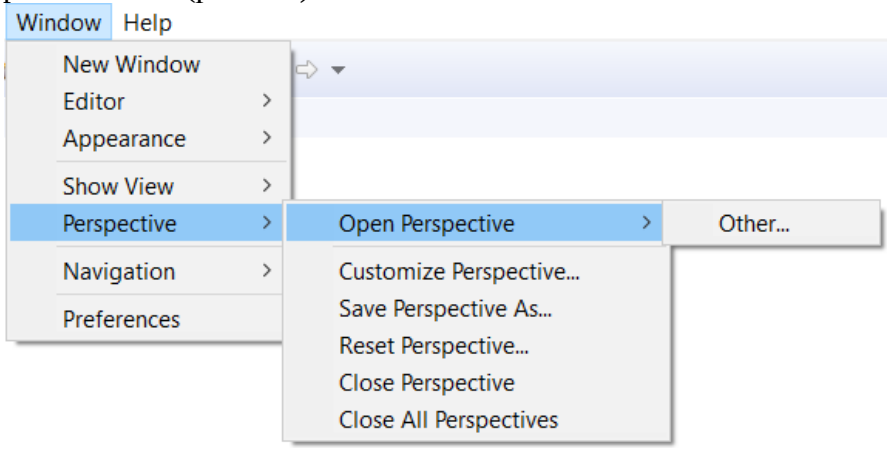


Рис. 1.5. Открытие перспективы

Перспектива выбирается в появившемся окне Open Perspective (рис. 1.6).

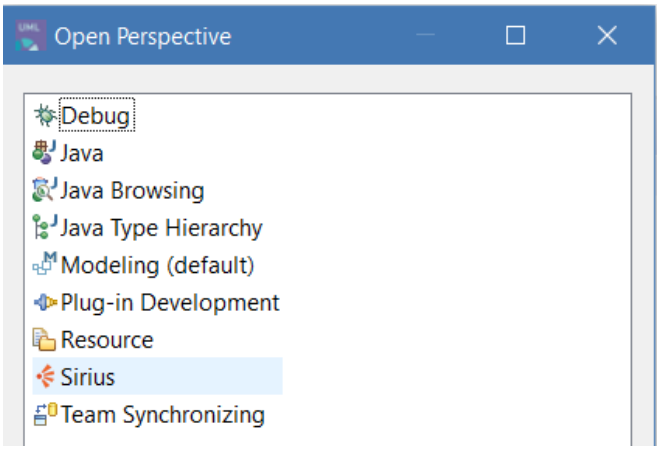


Рис. 1.6. Выбор перспективы

Одновременно может быть открыто несколько перспектив, например, Modeling и Sirius. При этом в правом верхнем углу окна показываются иконки открытых перспектив  . Нажимая на них мышкой можно менять перспективы. При нажатии на иконку правой кнопкой мышки появляется меню (рис. 1.7).

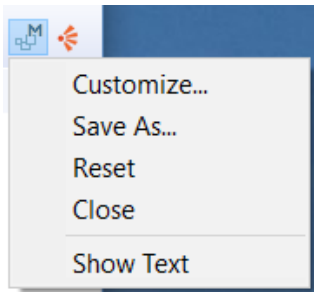


Рис. 1.7. Меню перспективы

С помощью этого меню можно настроить перспективу (Customize), создать новую перспективу (Save As), вернуть перспективу к исходному виду (Reset), закрыть перспективу (Close) и показать вместе со значками перспектив их названия (рис. 1.8) или вернуться обратно к показу иконок без названия, если названия уже показаны (Show Text).

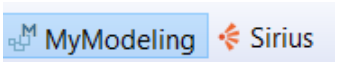


Рис. 1.8. Показ названия перспектив

Для быстрого открытия перспективы можно воспользоваться иконкой, расположенной слева от иконок открытых перспектив:  .

При работе с некоторой перспективой можно закрывать или минимизировать (сворачивать) отдельные виды. Закрыть вид можно, кликнув по иконке  рядом с именем вида (рис. 1.9).

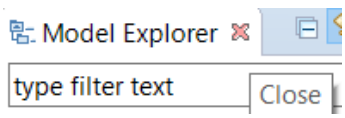


Рис. 1.9. Закрытие вида

Минимизировать вид можно, кликнув по символу в строке с именем вида (рис. 1.10).

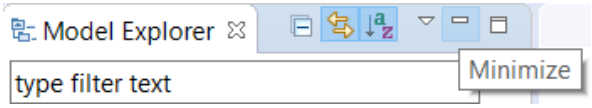


Рис. 1.10. Минимизация вида

При минимизации вида или области редактирования его иконка появляется слева или справа в окне UML Designer (в зависимости от расположения самого вида в окне). Над иконкой вида располагается иконка Restore

Показ иконок свернутого вида Model Explorer (слева) и области редактирования (справа) (рис. 1.11).

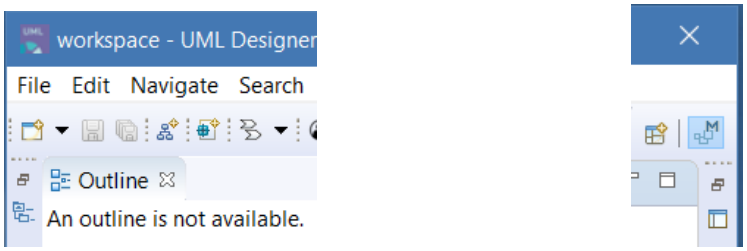


Рис. 1.11. Показ иконок свернутого вида Model Explorer и области редактирования

Нажатие на иконку вида приводит к показу вида в уменьшенном размере, а нажатие на иконку Restore – к показу вида в первоначальном размере.

При показе вида в уменьшенном размере иконка вида и иконка Restore не исчезают. Нажатие на иконку вида приводит к сворачиванию вида. Для показа вида в прежнем размере можно использовать иконку Restore, расположенную над иконкой вида или в правом углу самого вида (рис.1.12).



Рис. 1.12. Расположение иконки Restore

Открыть закрытый или свернутый вид можно в меню Window/Show View (рис. 1.13).

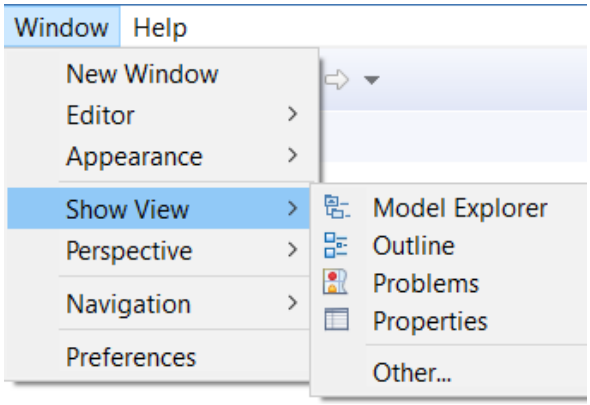


Рис. 1.13. Меню Show View

Вернуть показ перспективы в исходное состояние можно через меню Window/Perspective/Reset Perspective.

Перспективу можно настроить с помощью меню Window/Perspective/Customize Perspective (рис. 1.14).

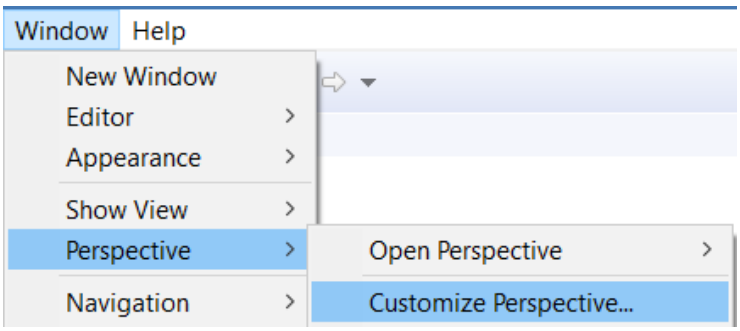


Рис. 1.14. Меню Perspective

Окно Customize Perspective позволяет выбрать элементы, которые показываются на панели инструментов (закладка Tool Bar Visibility), элементы меню (закладка Menu Visibility), наборы элементов меню (закладка Action Set Availability), иконки и элементы (Shortcuts), показываемые при нажатии левой кнопки панели инструментов  (рис. 1.15).

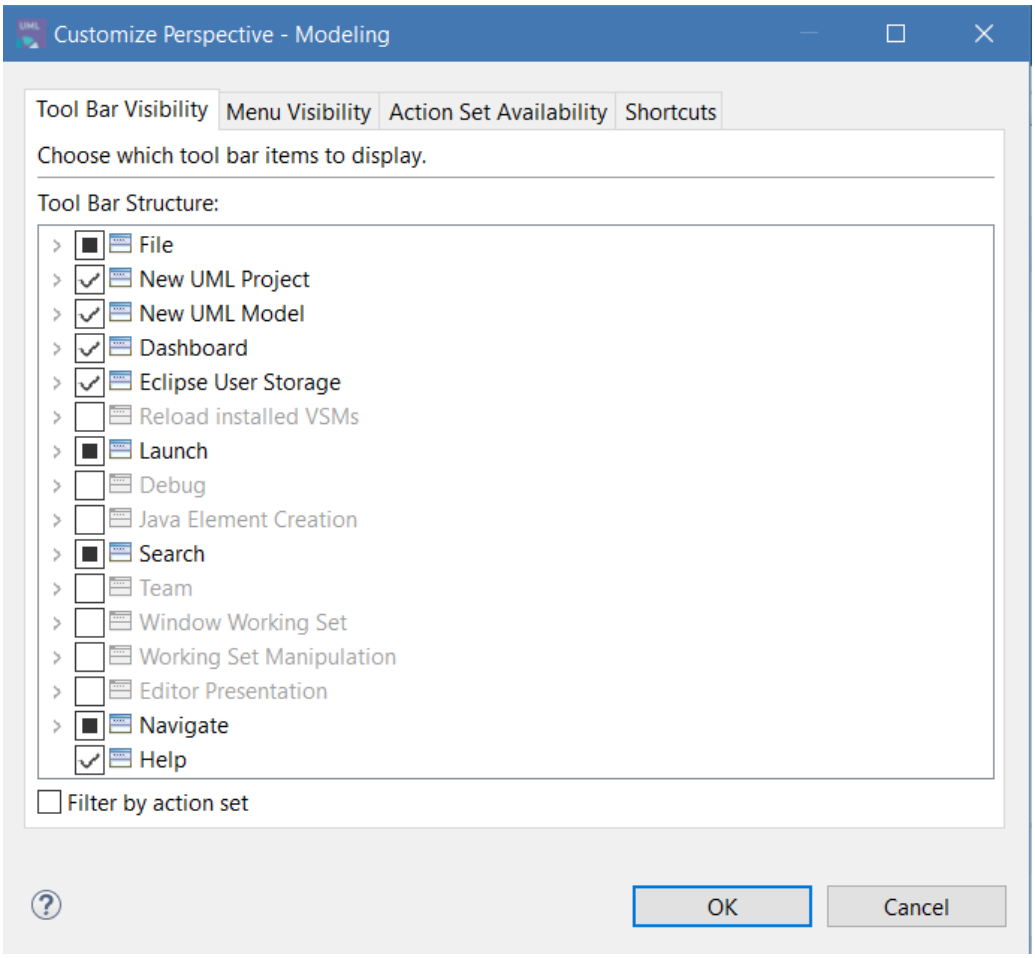


Рис. 1.15. Окно Customize Perspective

Настроенную перспективу можно сохранить под новым именем для дальнейшего использования с помощью Window/Perspective/Save Perspective As и указания ее имени в окне Save Perspective As (рис. 1.16).

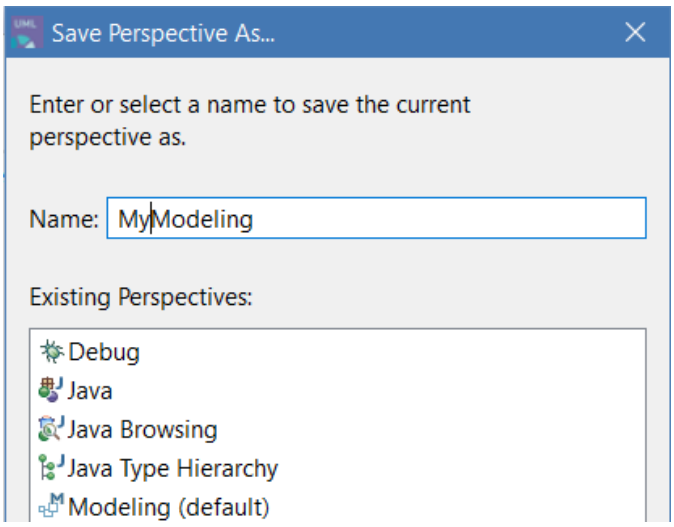


Рис. 1.16. Окно Save Perspective As

1.3. Переключение рабочих областей

Если вы используете несколько рабочих областей, то в любой момент можно сменить рабочую область через меню File/Switch Workspace (рис. 1.17).

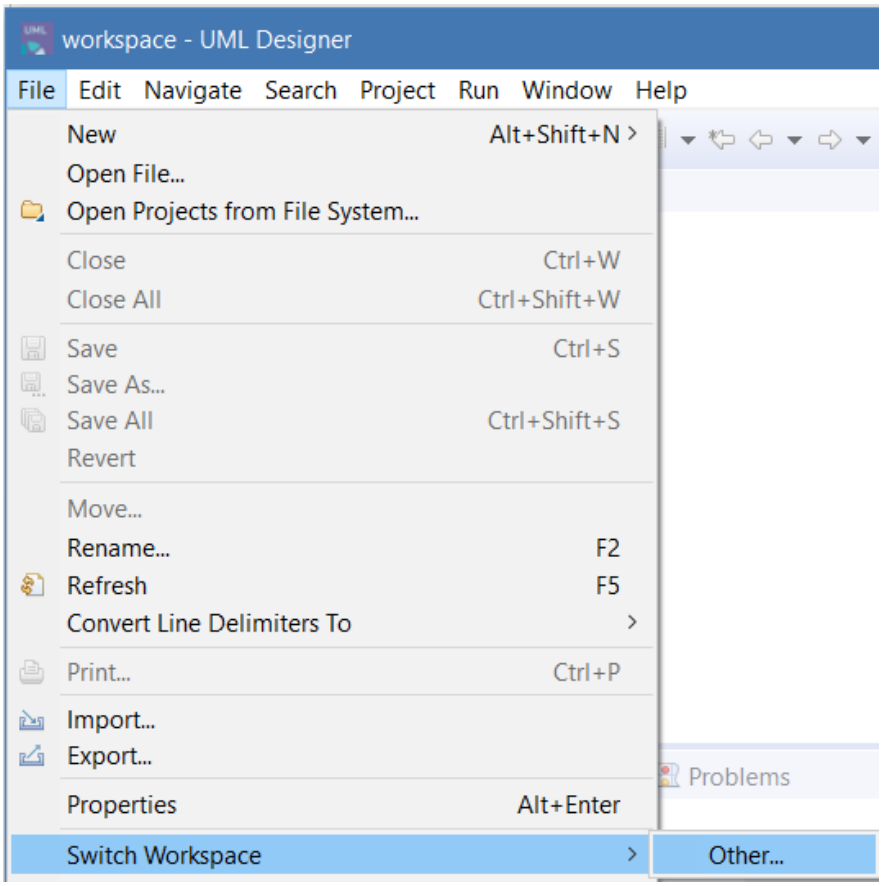


Рис. 1.17. Меню File/ Switch Workspace

Выберите одну из областей или пункт Other. При выборе Other открывается окно Eclipse Launcher (рис. 1.18).

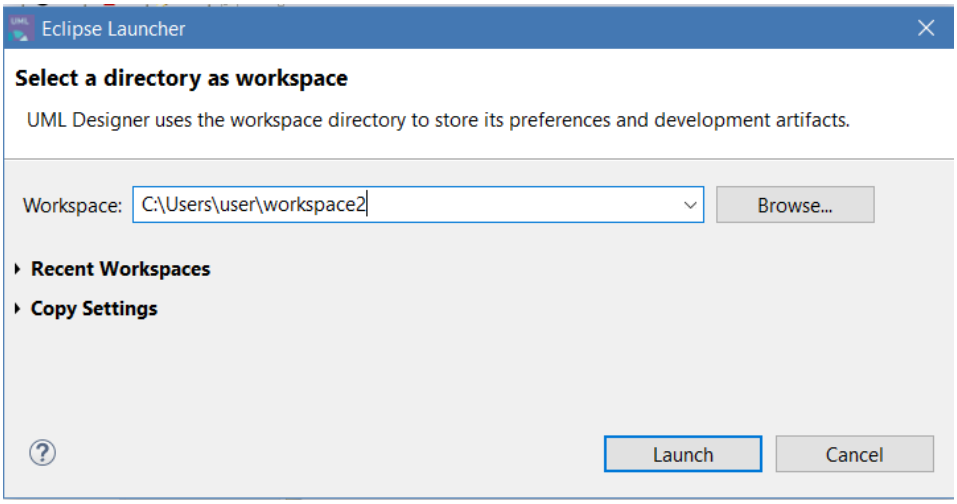


Рис. 1.18. Окно Eclipse Launcher

В этом окне вы можете выбрать одну из существующих областей (Resent Workspaces), или ввести название новой рабочей области в поле Workspace (будет создана новая директория), или выбрать некоторую существующую директорию (она должна быть пустой), нажав кнопку Browse, в качестве новой рабочей области.

1.4. Пример проекта TravelAgency

В UML Designer есть пример проекта TravelAgency.

Для открытия примера выберите меню File/New/Example (рис. 1.19) и пример проекта Travel Agency UML в окне New Example (рис. 1.20).

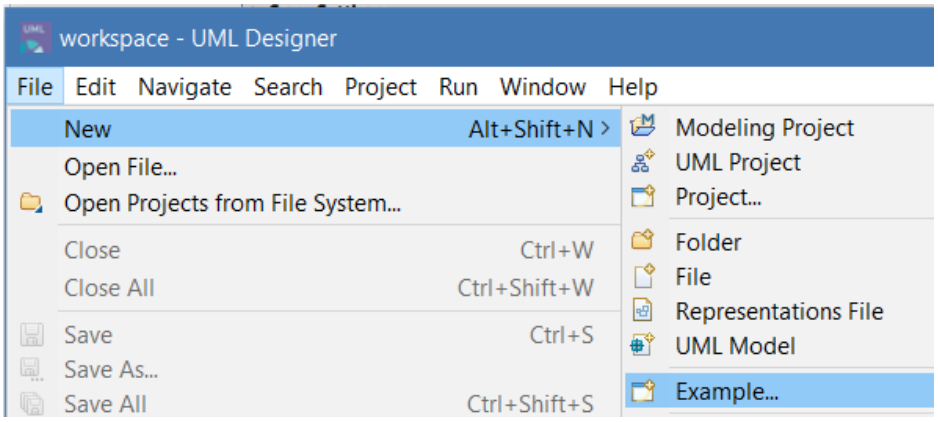


Рис. 1.19. Меню File/New/Example

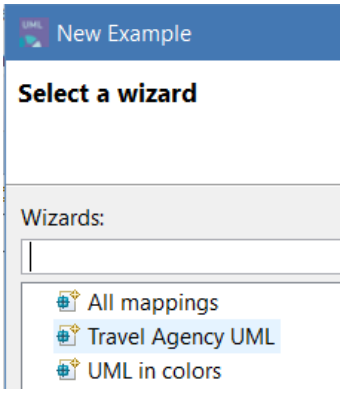


Рис. 1.20. Окно New Example

После выбора примера проекта его имя появится в виде Model Explorer (рис. 1.21).

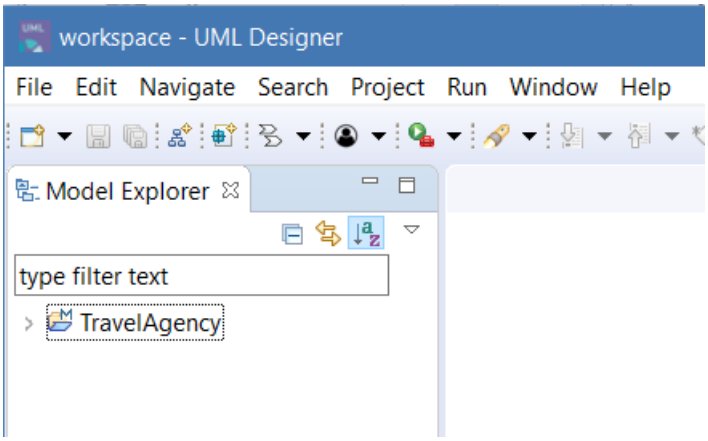


Рис. 1.21. Пример проекта в Model Explorer

2. Виды UML Designer

2.1. Вид Model Explorer

Вид Model Explorer используется для просмотра существующих и создания новых проектов и их диаграмм.

Элементы, показанные в данном виде, могут состоять из других элементов. В этом случае слева от имени элемента показывается значок «>». Кликнув по нему мышкой вы откроете список вложенных элементов. Значок «>» поменяется при этом на значок «∨». Клик мышкой по значку «∨» закрывает список вложенных элементов.

Можно показывать не все элементы с помощью ввода текста в строке type filter text. В этом случае будут показываться только элементы, в имени которых содержится набранный текст, и элементы их содержащие. Если в тексте встречается символ *, то на этом месте может быть любая последовательность символов, а если символ ?, то – любой символ. Например, набрав текст «*class*», легко найти все классы (рис. 2.1).

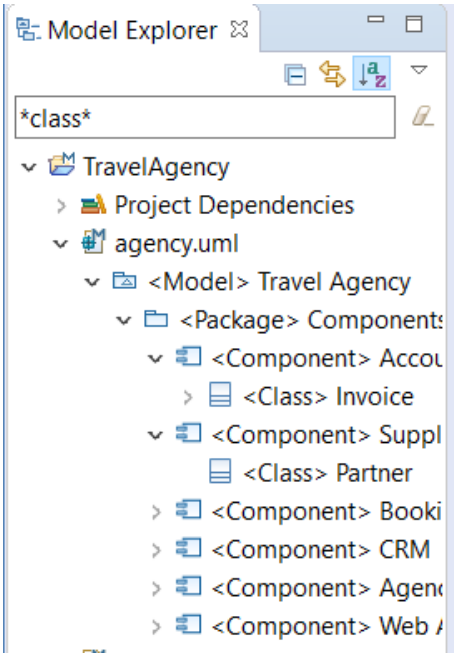


Рис. 2.1. Использование фильтра в Model Explorer

Элемент agency.uml содержит элемент **<Model> Travel Agency**, который содержит диаграммы и все графические элементы диаграмм, представленных в UML модели проекта. С его помощью можно также открыть любую диаграмму. Например, для открытия диаграммы вариантов использования откройте последовательно элементы **Travel Agency**,

agency.uml, **<Model> Travel Agency**. Далее двойным кликом по имени диаграммы **Use Cases** открывается сама диаграмма в области редактирования (рис. 2.2).

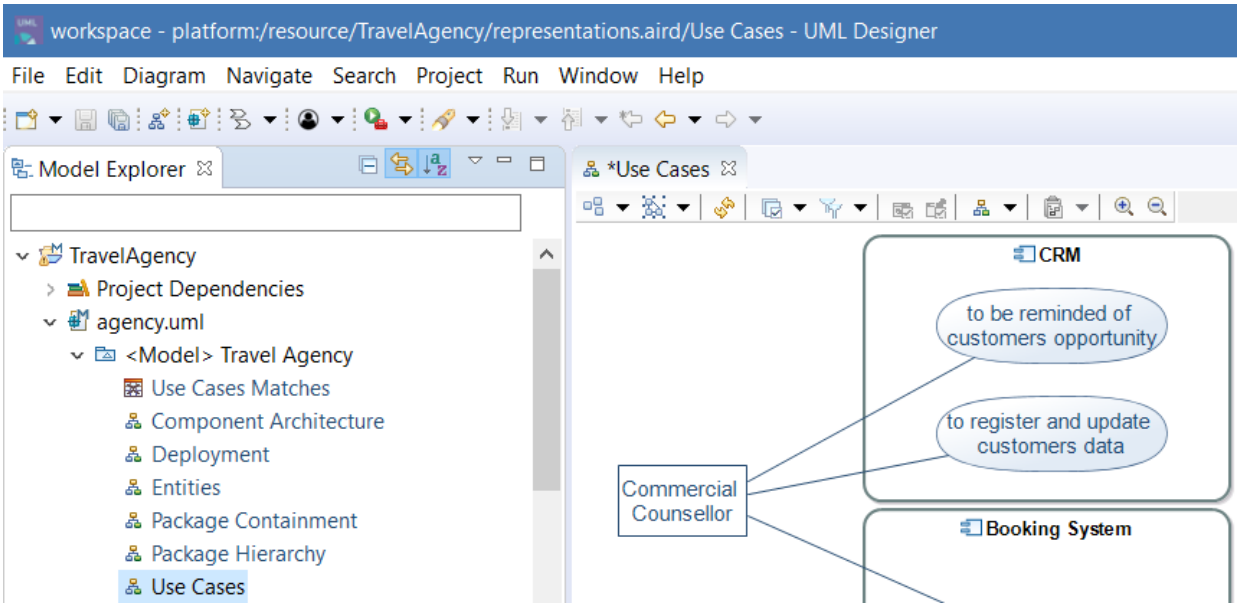


Рис. 2.2. Открытие диаграммы с использованием Model Explorer

В элементе representation.aid диаграммы сгруппированы в элементы Capture, Design, Extend, Reused и Review (рис. 2.2).

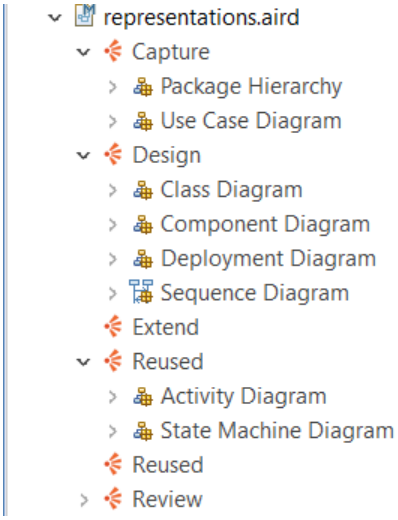


Рис. 2.2. Группировка диаграмм

Если в области редактирования открыта некоторая диаграмма, то кликнув на любой графический элемент, мы увидим его в элементе **<Model> Travel Agency**. Можно поступить и обратно: выбрать название графического элемента (кликнув на него или используя стрелки для перемещения по элементам) в элементе **<Model> Travel Agency** и он выделится на соответствующей диаграмме. Эта возможность называется Link with Editor. Ее можно отключить или включить, кликнув на значок  в первой строке вида Model Explorer.

Например, если кликнуть на элемент **<Actor>Comercial Counsellor**, то он будет выделен на диаграмме вариантов использования **Use Cases** (рис. 2.4).

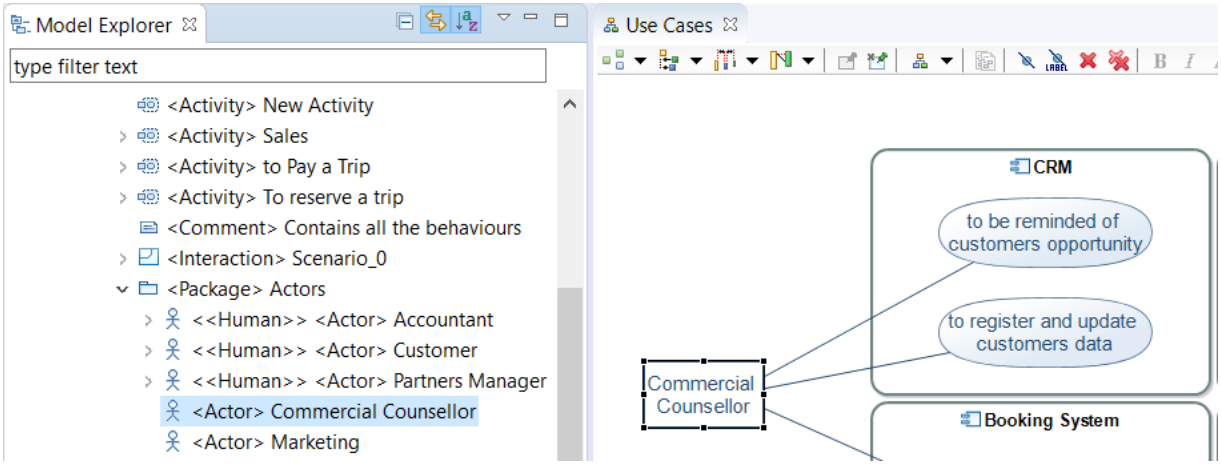


Рис. 2.4. Перемещение по элементам модели

Описание элемента **<Actor>Comercial Counsellor** также появится в Properties (рис. 2.5).

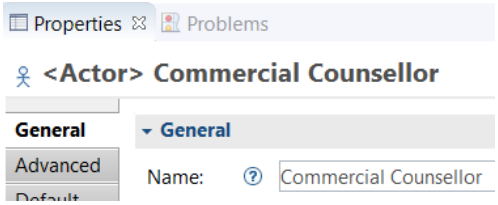


Рис. 2.5. Показ элемента модели в виде Properties

2.2. Вид Outline

В общем случае в окне редактирования видна только часть диаграммы. Вид Outline используется для выбора той части диаграммы, которая показывается в окне редактирования, а также для поиска нужного графического элемента на диаграмме.

Вид имеет два представления: Outline и Overwiew. Выбор одного из них осуществляется с помощью клика на соответствующую иконку в верхней строке вида (рис. 2.6).



Рис. 2.6. Представления Outline и Overwiew вида Outline

В представлении **Outline** показывается список всех графических элементов диаграммы, открытой в области редактирования. После выбора элемента с помощью клика мышкой или стрелок вверх и вниз он выделяется на диаграмме и показывается в окне редактирования (рис. 2.7).

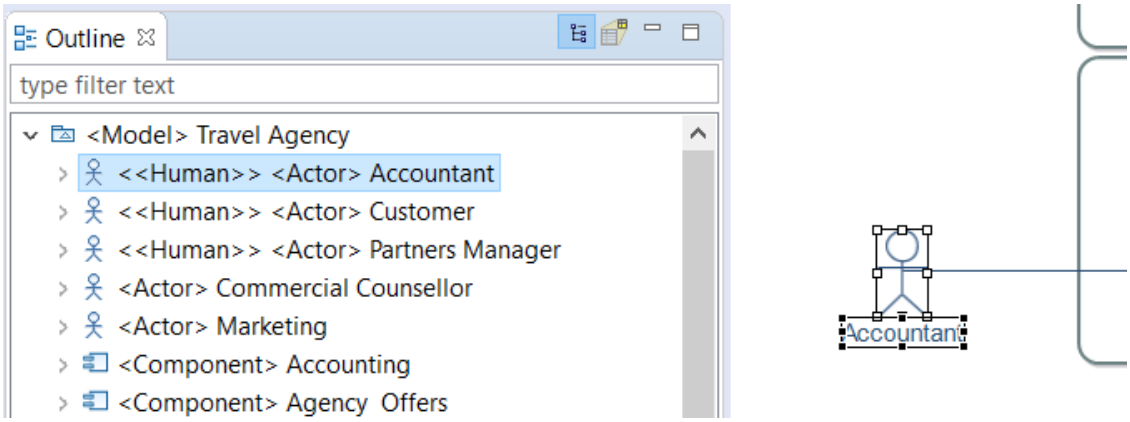


Рис. 2.7. Использование представления Outline

В представлении **Overwiew** показывается вся диаграмма в уменьшенном виде. Та часть диаграммы, которая видна в области редактирования, выделена прямоугольником синего цвета. Перемещая этот прямоугольник с помощью стрелок или мышки (нажав на левую кнопку мышки) можно увидеть в области редактирования нужную часть диаграммы (рис. 2.8).

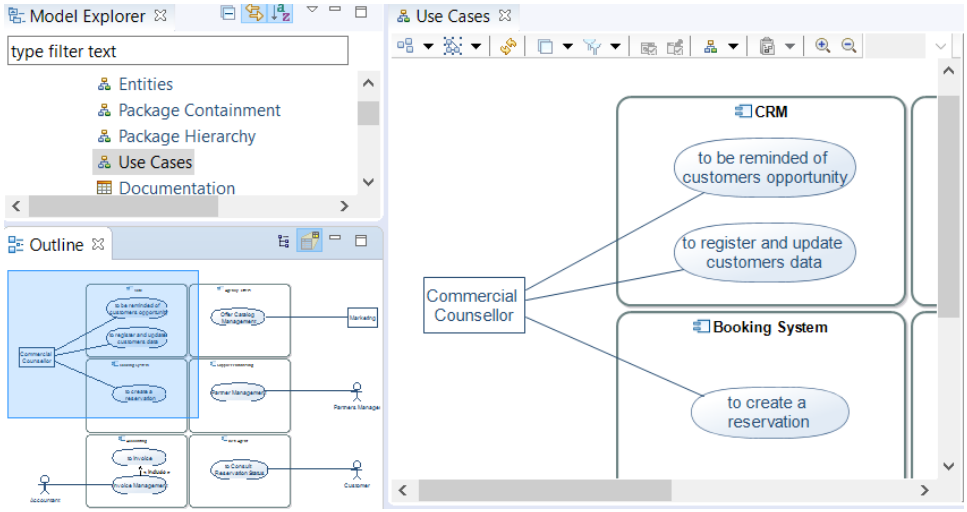


Рис. 2.8. Использование представления Overwiew

2.3. Вид Properties

Диаграмма UML содержит графические элементы. Для обозначения элемента используется некоторый графический символ. Обычно элемент имеет только имя, слева от которого может изображаться значок (пиктограмма) элемента. Например, актер на диаграмме вариантов использования изображается с помощью графического элемента и имени, а класс с помощью графического элемента, значка и имени (рис. 2.9).

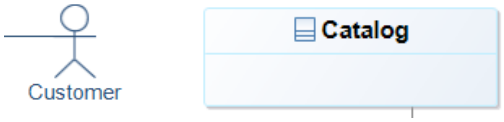


Рис. 2.9. Изображения графических элементов на диаграмме

Также элемент может иметь на диаграмме дополнительную информацию. Например, для класса показываются атрибуты класса. Каждый атрибут обозначается значком, именем, имеет название типа атрибута и множественность, т.е. количество значений (рис. 2.10).

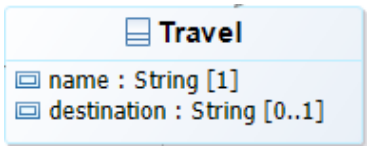


Рис. 2.10. Дополнительная информация графического элемента

Все графические элементы диаграммы, а также элементы, имеющие значки (например, атрибут класса), могут иметь дополнительный набор свойств, которые описываются в виде Properties. Описание элемента в виде Properties открывается при клике на элемент в области редактирования, или выборе элемента в представлении Outline вида Outline, или виде Model Explorer (рис. 2.11).

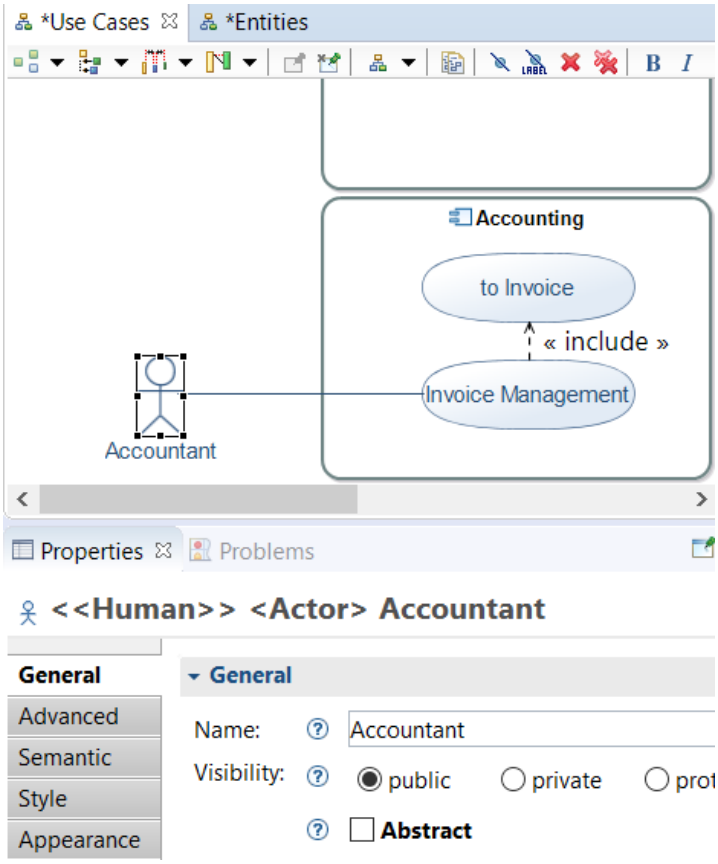


Рис. 2.11. Показ элемента на диаграмме и его описание в виде Properties

Можно закрепить вид Properties за конкретным элементом диаграммы, кликнув на значок Pin  (рис. 2.12).

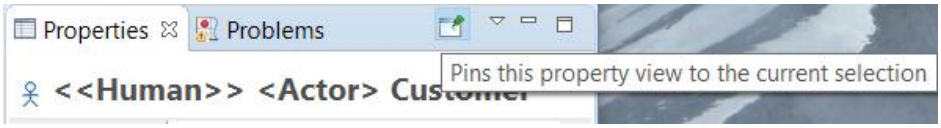


Рис. 2.12. Закрепление вида Properties за конкретным элементом диаграммы

В виде Properties появляется сообщение о закреплении данного элемента для диаграммы (Selection from Use Cases) (рис. 2.13).

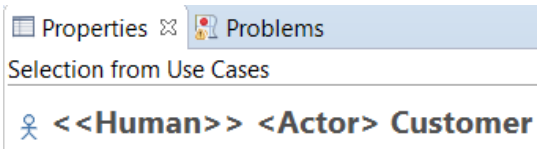


Рис. 2.13. Сообщение о закреплении элемента

Открепление вида Properties от элемента диаграммы осуществляется также с помощью клика на этот значок Pin.

Если в виде Properties закреплен некоторый элемент, то он не меняется при выборе других элементов диаграммы.

Можно создать несколько видов Properties с помощью клика на стрелку рядом со значком Pin и выбора New Properties View (рис. 2.14).

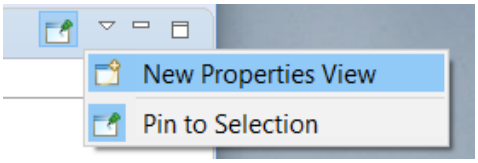


Рис. 2.14. Создание нескольких видов Properties

Обычно один из видов Properties не закрепляется за элементом и используется для показа свойств текущего выбранного элемента, а другие виды закрепляются за некоторыми элементами (в общем случае из разных диаграмм). Это дает возможность легко преключаться между свойствами различных элементов.

Свойства элемента разбиты на группы General, Advanced, Semantic, Style и Appearance. Каждая группа свойств представлена на своей закладке.

Свойства описывают, как характеристики элемента с точки зрения описания модели (свойства на закладках General, Advanced, Semantic), так и визуальное представление элемента на диаграмме (свойства на закладках Style и Appearance).

Например, на закладке General можно увидеть имя (Name), видимость (Visibility), тип (Type) и другие свойства атрибута destination класса Travel (рис. 2.15).

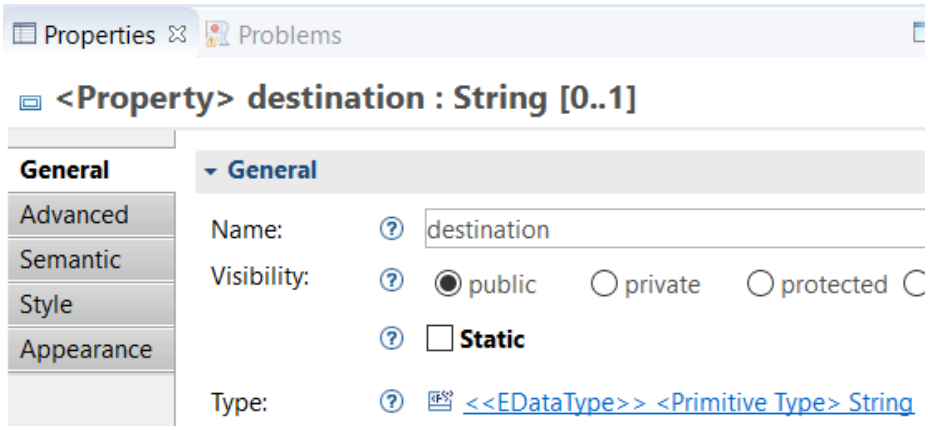


Рис. 2.15. Описание элемента на закладке General

На закладке Advanced, например для операции класса можно увидеть свойство параллельности выполнения операции (Concurrency) (рис. 2.16).

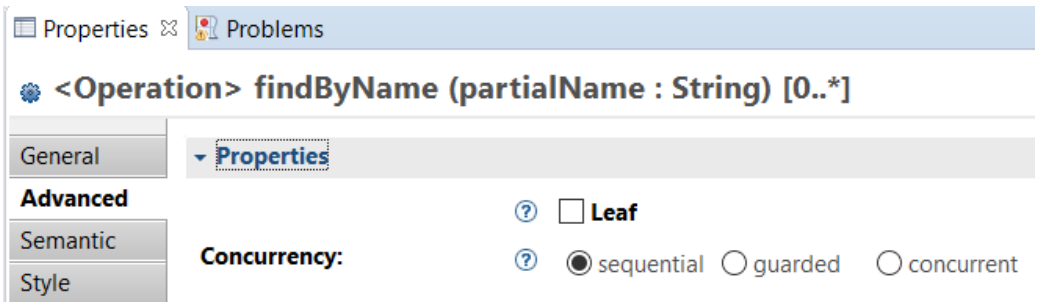


Рис. 2.16. Описание элемента на закладке Advanced

На закладке Appearance можно поменять шрифт и цвет имен элементов на диаграмме (рис. 2.17).

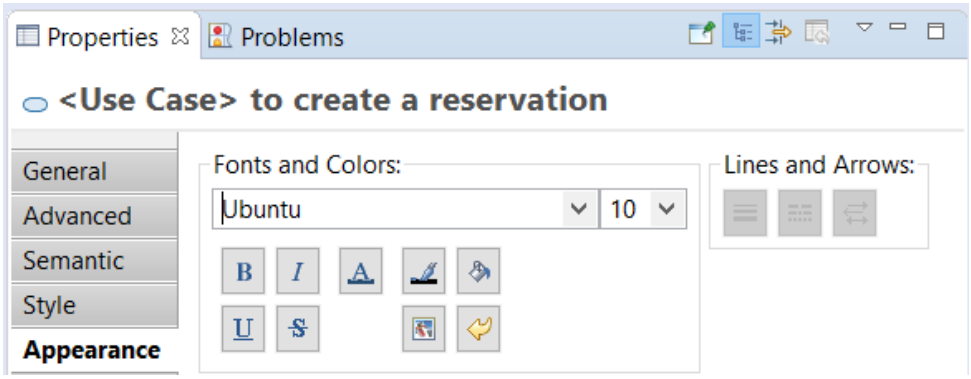


Рис. 2.17. Описание элемента на закладке Appearance

Свойства, представленные на закладке General, можно посмотреть и отредактировать также в окне Edit, которое появляется при двойном клике на элементе диаграммы. Например, так выглядит окно редактирования для атрибута name класса Travel (рис. 2.18).

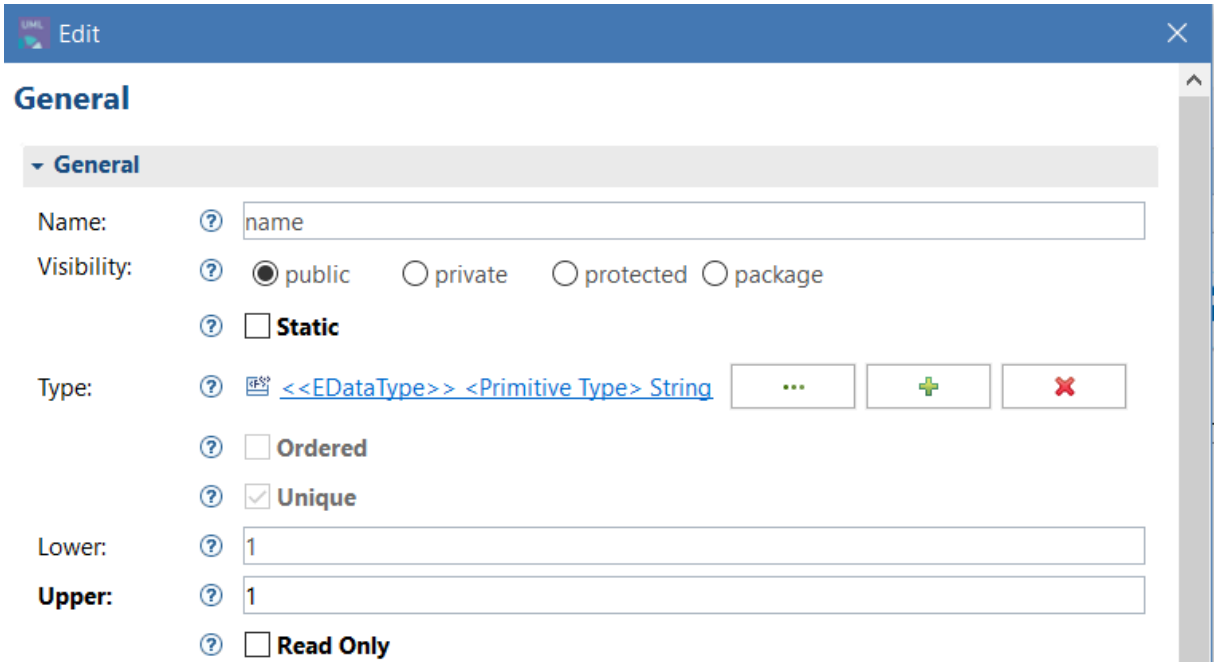


Рис. 2.18. Окно редактирования описания элемента

Использование окна Edit дает возможность просмотра и редактирования основных свойств элемента при закрытом виде Properties. Это удобно, т.к. при закрытом виде Properies и Problems область редактирования занимает больше места на экране и можно увидеть всю диаграмму целиком или, по крайней мере, большую ее часть.

3. Редактор диаграмм

Для рисования диаграмм используется редактор диаграмм. Он состоит из трех частей:

- палитра инструментов (palette) позволяет рисовать диаграмму;
- панель вкладок (tab bar) используется для выполнения ряда операций по работе с диаграммой (например, масштабирования изображения);
- область редактирования (main graphical area), на которой рисуется диаграмма.

3.1. Палитра инструментов

Каждая диаграмма в UML Designer имеет палитру инструментов (Palette), которая располагается справа от области редактирования. В зависимости от диаграммы на палитре имеется разный набор инструментов. Общие для всех диаграмм инструменты расположены в верхней ее части. Пример палитры инструментов показан на рис. 3.1.

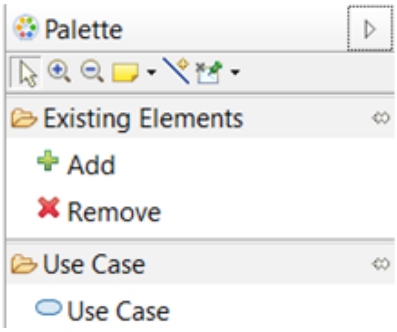


Рис. 3.1. Пример палитры инструментов

Размер палитры инструментов может быть изменен по ширине с помощью нажатия и перемещения мышки на вертикальной линии, отделяющей область редактирования и палитру инструментов. Можно также нажать правую кнопку мышки, наведя ее на строку Palette, и в появившемся меню выбрать Resize (рис. 3.2).

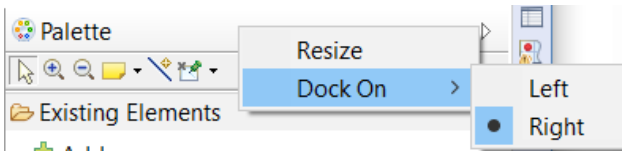


Рис. 3.2. Изменение размера и расположения палитры инструментов

Меню на рис. 3.2 также позволяет расположить палитру инструментов слева от области редактирования, выбрав Doc On -> Left, или справа, выбрав Doc On -> Right.

Можно временно скрыть палитру инструментов, кликнув на значок  в ее правом верхнем углу. На месте этого значка появляется значок  , который используется для восстановления показа палитры.

Основной набор инструментов  расположен в верхней строке палитры инструментов.

Инструмент **Select**  выбирается по умолчанию при открытии диаграммы. Он позволяет выбрать любой элемент на диаграмме с помощью клика или несколько элементов, если нажать клавишу Ctrl и кликать на элементы. Можно также выбрать несколько элементов, наведя мышку на пустое место на диаграмме, нажав левую кнопку мышки и двигая мышку. При движении мышки рисуется прямоугольник и выделяются все элементы, попавшие в него.

Инструмент **Zoom** имеет две кнопки  и позволяет увеличить или уменьшить диаграмму, кликая в произвольном месте диаграммы. Можно также при нажатой клавише Ctrl нажимать + или – или крутить колесико мышки.

Инструмент **Note**  позволяет создавать заметки на диаграмме, содержащие поясняющий диаграмму текст. Заметку можно выбрать, нажав на стрелку рядом с символом заметки (рис. 3.3).

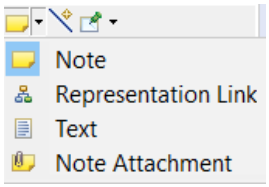


Рис. 3.3. Меню инструмента Note

Есть несколько видов заметок:

- **Note** позволяет написать текст внутри желтого прямоугольника;
- **Text** позволяет написать текст в произвольном месте диаграммы;
- **Representation Link** позволяет создать ссылку на любую другую диаграмму;
- **Note Attachment** аналогична Note, но после клика на этот элемент нужно кликнуть на некоторый элемент диаграммы, а потом на свободное место диаграммы. В результате, заметка будет соединена линией с элементом диаграммы.

Любая заметка, кроме Note Attachment, создается с помощью клика на значок заметки, а затем клика на пустом месте диаграммы.

Инструмент **Generic Connection**  позволяет соединить два элемента на диаграмме. Кликните на эту иконку, а затем последовательно на два соединяемых элемента. Если между

соединяемыми элементами возможны разные отношения, то будет показано меню выбора. Например, при соединении двух вариантов использования показывается меню (рис. 3.4).

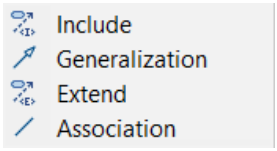


Рис. 3.4. Меню инструмента General Connection

Инструмент **Pin/Unpin** позволяет закрепить или открепить любой элемент диаграммы, кликнув на него. Смена Pin на Unpin осуществляется с помощью клика на стрелку и выбора (рис. 3.5).

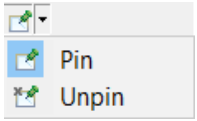


Рис. 3.5. Меню инструмента Pin

Закрепленные элементы не меняют размера и своего положения на диаграмме при выполнении автоматического изменения вида диаграммы, но их размер можно изменить и их можно перемещать вручную.

3.2. Панель вкладок

Панель вкладок UML Designer, показанная на рис. 3.6, имеет инструменты, которые выбираются по клику на иконку инструмента или на стрелку рядом с иконкой, а затем на имя инструмента в открывшемся меню. Данная панель показывается, если на диаграмме не выбран ни один элемент.



Рис. 3.6. Панель вкладок

Инструменты **Arrange All, Arrange Linked Border Nodes** позволяют автоматически изменить положение и размер не закрепленных элементов на диаграмме.

Результатом применения **Arrange All** является диаграмма, в которой элементы не наложены друг на друга, линии по возможности не пересекаются и элементы, содержащие внутри себя другие элементы, имеют достаточный размер для показа внутренних элементов.

Если на диаграмме выбран один или несколько элементов, то доступен только инструмент **Arrange Selection**, который выполняет указанные действия только с выбранными элементам.

Данные инструменты доступны также с помощью пункта Layout из меню, открывающегося по нажатию правой кнопки мышки на элементе диаграммы или пустом месте диаграммы (рис. 3.7).

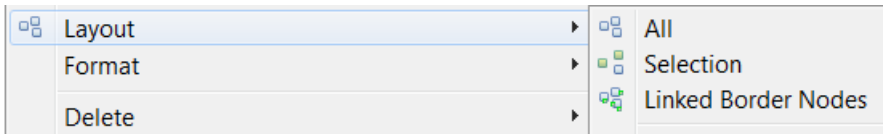


Рис. 3.7. Меню Layout

Инструменты **Select All, Select All Connectors, Select All Shapes** позволяют, соответственно, выбрать все графические элементы на диаграмме; все соединительные элементы; все графические элементы, кроме соединительных. Для отмены операции достаточно кликнуть мышкой на пустое место диаграммы или нажать клавишу Esc.

Инструмент **Show/Hide** позволяет скрыть или показать любой из элементов диаграммы или сразу все элементы. После клика на иконку открывается окно со списком элементов, в котором показываемые элементы отмечены галочкой. Если элемент имеет вложенные элементы, то их список можно увидеть кликнув по значку рядом с элементом, например, как показано на рис. 3.8.

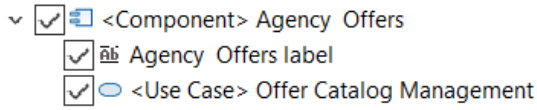


Рис. 3.8. Просмотр вложенных элементов

Среди списка вложенных элементов можно увидеть метку элемента – это имя элемента и значок (пиктограмма) элемента, если он есть. Можно скрыть метку элемента, а не сам элемент. Можно также скрыть элемент и/или его метку с помощью нажатия правой кнопки мышки на элементе диаграммы и выбора соответствующего пункта меню (рис. 3.9).

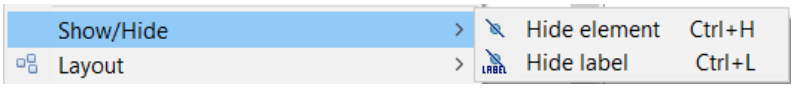


Рис. 3.9. Меню скрытия элемента и метки

Используя правую кнопку мышки, можно аналогично восстановить скрытую метку (рис. 3.10).



Рис. 3.10. Восстановление скрытой метки

Инструмент **Edition mode** позволяет задать один из режимов редактирования диаграммы (рис. 3.11).

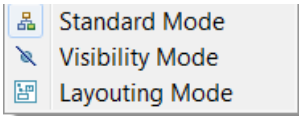


Рис. 3.11. Меню инструмента Edition mode

В стандартном режиме (**Standart Mode)** можно редактировать все элементы диаграммы.

В режиме видимости (**Visibility Mode)** можно увидеть все элементы диаграммы, включая и скрытые. Редактировать элементы нельзя. Скрытые элементы показываются бледным цветом. Двойным кликом мышки по скрытому элементу можно перевести его в режим показа. Если элемент не скрыт, то его можно скрыть двойным кликом мышки.

В режим размещения (**Layouting Mode)** предназначен для изменения расположения элементов диаграммы, их размеров, цвета, шрифтов текста. В этом режиме нельзя изменить семантику элемента, т.е. поменять имя элемента или его свойства. Например, нельзя открыть окно со свойствами элемента с помощью двойного клика на элементе. К сожалению, если открыт вид Properties, то в нем можно менять свойства элемента даже в этом режиме.

Инструмент **Pin/Unpin** позволяет закрепить или открепить любой из элементов диаграммы или сразу все элементы. После клика на иконку открывается окно со списком элементов, в котором закрепленные элементы отмечены галочкой. Если элемент имеет вложенные элементы, то их список можно увидеть кликнув по значку рядом с элементом. Закрепленные элементы не перемещаются при выполнении действий по автоматическому расположению элементов (**Arrange All**).

Инструменты **Zoom In/Zoom Out** и соседний с ними инструмент 100% используются для изменения масштаба изображения диаграммы. Увеличение или

уменьшение масштаба можно выполнять также с помощью клавиш Ctrl+ и Ctrl- или вращая колесико мышки при нажатой клавише Ctrl.

Инструмент позволяет сохранить изображение диаграммы в графическом файле.

Если на диаграмме выбран хотя бы один элемент, то панель вкладок имеет вид, показанный на рис. 3.12.



Рис. 3.12. Вид панели вкладок при выбранном элементе

Инструмент **Align** позволяет выровнять два или более выбранных элементов по одной из границ, выбрав соответствующий пункт меню, показанный на рис. 3.13.

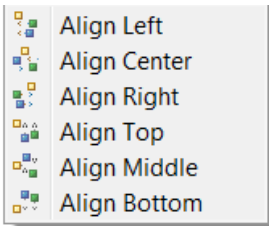


Рис. 3.13. Меню инструмента Align

Инструмент **Gaps** позволяет при выборе более двух элементов выровнять расстояние между элементами по горизонтали или вертикали, выбрав соответствующий пункт меню, показаннный на рис. 3.14.

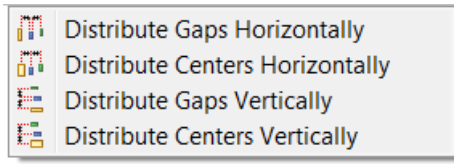


Рис. 3.14. Меню инструмента Gaps

Инструмент **straighten** используется для выравнивания выделенной линии связи по горизонтали (по верхнему, нижнему; левому или правому краю линии) или вертикали (по левому, правому; верхнему или нижнему краю линии).

Если линия соединяет два горизонтально расположенных элемента, то для ее выравнивания используется верхняя половина меню инструмента **straighten** (рис. 3.15).

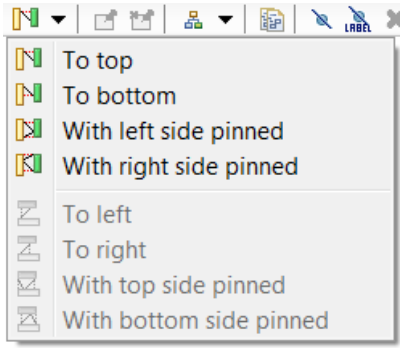


Рис. 3.15. Меню инструмента straighten для горизонтального выравнивания линии

Например, если нужно правый конец наклонной линии, соединяющей класс и компонент на диаграмме компонентов, поднять на уровень левого конца линии, то выберите пункт **With left pinned** или **To top** (рис. 3.16).

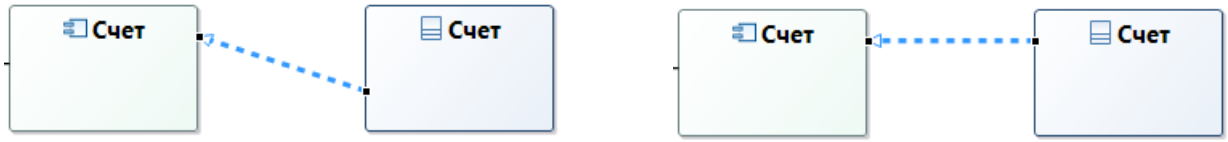


Рис. 3.16. Выравнивание линии по горизонтали

Если линия соединяет два вертикально расположенных элемента, то для ее выравнивания используется нижняя половина меню инструмента **straighten** (рис. 3.17).

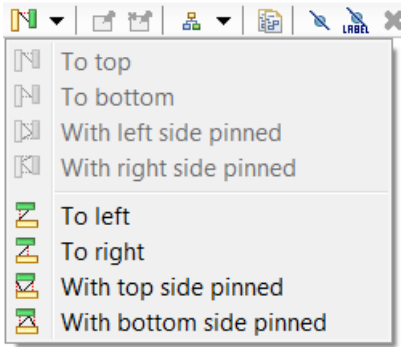


Рис. 3.17. Меню инструмента straighten для вертикального выравнивания линии

Например, если нужно верхний конец наклонной линии, соединяющей класс и компонент на диаграмме компонентов, сдвинуть вправо на уровень нижнего конца линии, то выберите пункт **With bottom side pinned** или **To right** (рис. 3.18).



Рис. 3.18. Выравнивание линии по вертикали

3.3. Область редактирования

Область редактирования предназначена для рисования UML-диаграмм. Графические элементы диаграмм помещаются в область редактирования из палитры инструментов. Перетащите мышкой элемент из палитры инструментов в область редактирования или кликните на значок элемента в палитре инструментов, а затем в области редактирования.

Если задержать указатель мышки в некотором месте области редактирования, то в зависимости от расположения мышки появляется окно с определенным набором инструментов. Например, если на диаграмме вариантов использования остановиться на пустом месте диаграммы, то появится окно с инструментами, показанное на рис. 3.19.



Рис. 3.19. Окно с инструмеетами

Действия инструментов такие же, как если бы вы выбрали соответствующий инструмент в палитре инструментов.

Имя любого графического элемента в области редактирования можно непосредственно изменить следующими способами:

- кликните мышкой на элемент, а затем нажмите F2;
- дважды с небольшой задержкой кликните на элемент;

- кликните на элемент и начните набирать текст.

Для поиска нужного вам элемента в окне редактирования нажмите правую кнопку мышки на пустом месте и выберите пункт Quick search в открывшемся меню (рис. 3.20).

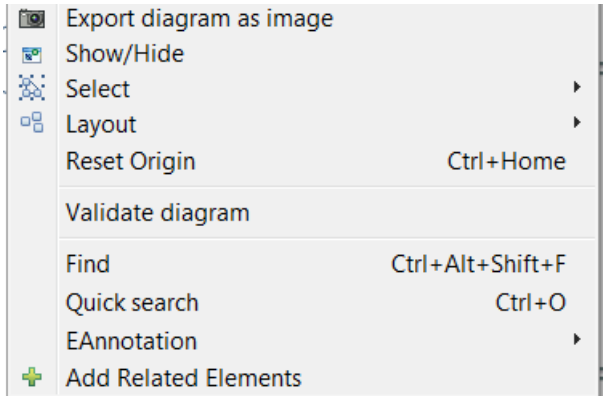


Рис. 3.20. Меню диаграммы

Для быстрого открытия окна поиска элементов можно просто нажать Ctrl+O. В окне поиска показывается весь список элементов или же часть элементов, имена которых начинаются со слова, введенного в строке поиска, расположенной в верхней части окна. Если поисковое слово может встречаться внутри имен элементов, то укажите перед ним символ *. Например, так выглядит окно поиска, когда открыта диаграмма вариантов использования проекта TravelAgency и в строке поиска набрано «par» (рис. 3.21).

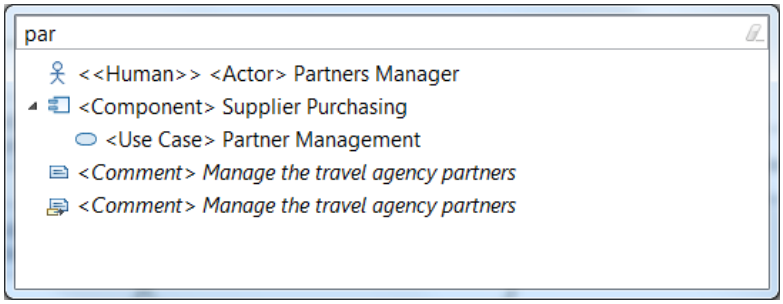


Рис. 3.21. Окно поиска

Переход к элементу из окна поиска осуществляется двойным кликом мышки на строке элемента.

Для редактирования элемента нажмите правую кнопку мышки на элементе и в появившемся меню выберите Edit (рис. 3.22).

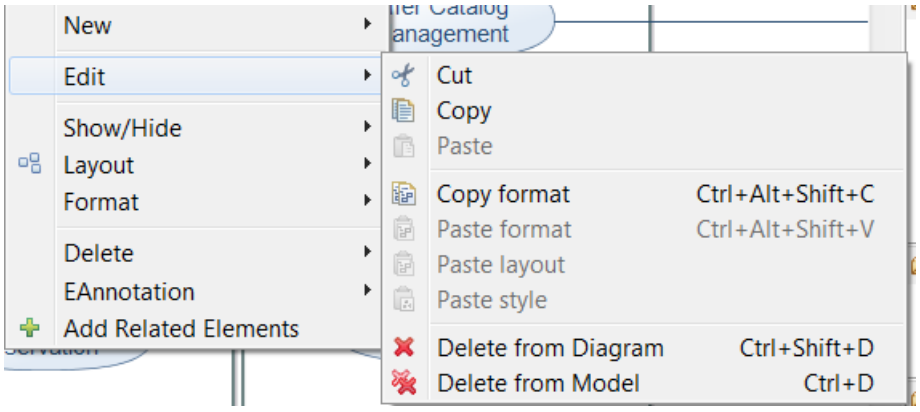


Рис. 3.22. Меню редактирования элемента

Если вы вырезаете элемент (Cut) для последующего его размещения (Paste), то вместе с элементом будут удалены все его связи с другими элементами.

При выполнении команды Paste появится только сам элемент. Поэтому, чтобы не потерять связи, нужно перед выполнением команды Cut выделить сам элемент и все его связи.

Для редактирования элемента можно пользоваться клавишами Ctrl+X (Cut), Ctrl+C (Copy), Ctrl+V (Paste).

Для отмены действий по редактированию нажмите Ctr+Z (Undo).

Обратите внимание на два режима удаления:

- **Delete from Diagram** удаляет элемент из текущей диаграммы, расположенной в окне редактирования. При этом элемент остается в модели и его можно добавлять на диаграммы с мощью элемента Add в палитре инструментов.
- **Delete from Model** удаляет элемент из диаграммы и модели.

Команды редактирования можно выполнять как с одним элементом, так и с несколькими выделенными элементами. Выделение нескольких элементов осуществляется с помощью клика мышкой на эти элементы при нажатой клавише Ctrl. Можно выделить несколько элементов, нажав мышку в некоторой точке области редактирования и перемещая ее нарисовать прямоугольник. Все элементы, попавшие в него, будут выделены. Если необходимо выбрать несколько элементов, расположенных внутри некоторого другого элемента, то прежде чем нарисовать вокруг них прямоугольник с помощью мышки нажмите и удерживайте клавишу Alt.

Для отмены выделения одного или нескольких элементов кликните мышкой на пустом месте или нажмите клавишу Esc. Для отмены любого действия используйте Ctrl+Z.

Меню редактирования элементов можно также вызвать, выбрав пункт Edit в главном меню UML Designer (рис. 3.23).

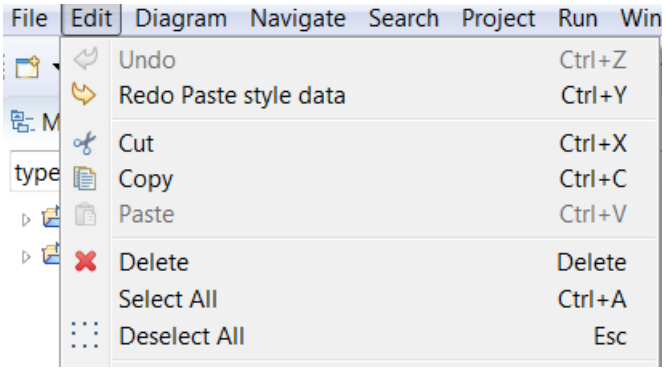


Рис. 3.23. Меню редактирования элементов

В этом меню пункт Delete означает удаление элемента из диаграммы и модели.

Для более точного рисования элементов на диаграмме можно воспользоваться пунктом **Diagram** из главного меню (рис. 3.24).

Данное меню частично дублирует инструменты панели вкладок, но имеет и дополнительные инструменты.

Выбрав в меню пункт **View**, можно далее выбрать пункт **Grid** для показа сетки в области редактирования и пункт **Rules** для показа линеек по горизонтали и вертикали области редактирования.

Пункт **Page Breaks** позволяет увидеть сколько страниц при печати занимает диаграмма.

Выбор **Snap to Grid** означает, что при перетаскивании графического элемента (нажимаем мышку на элементе и не отпуская перемещаем мышку) по области редактирования его границы (прямоугольник, в который вписан элемент) всегда будут расположены на линиях сетки. При отсутствии **Snap to Grid** границы элемента могут располагаться внутри сетки.

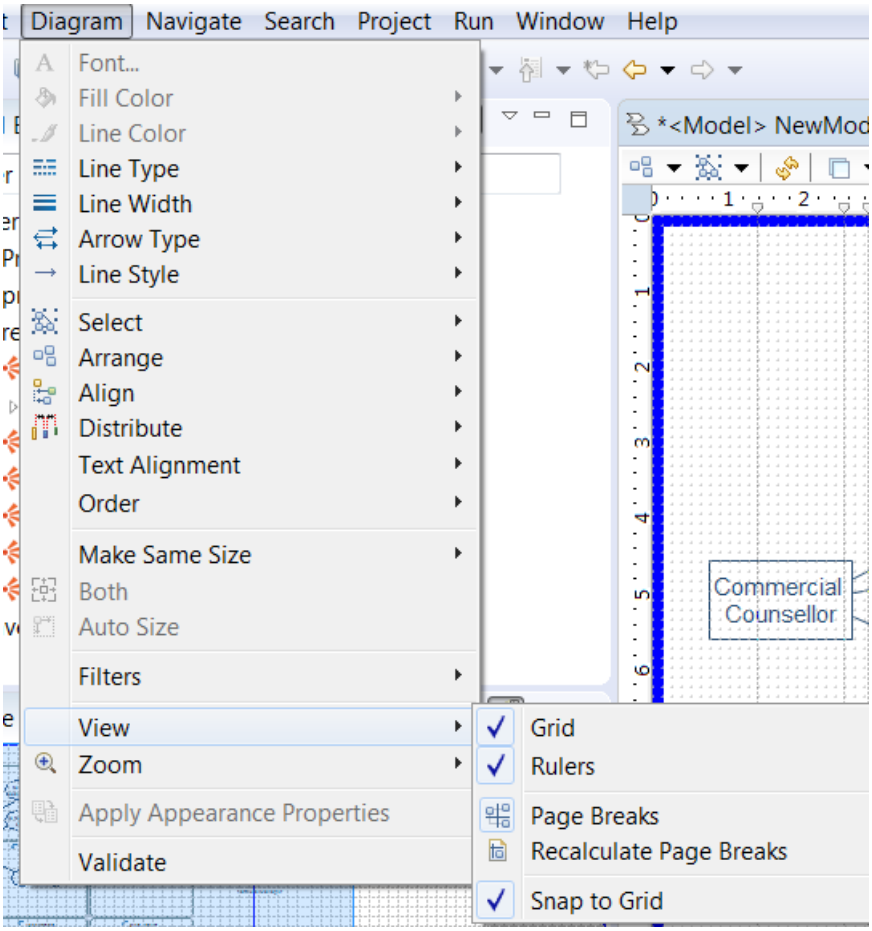


Рис. 3.24. Меню Diagram

Также может быть полезен пункт меню **Make Same Size** (рис. 3.25).

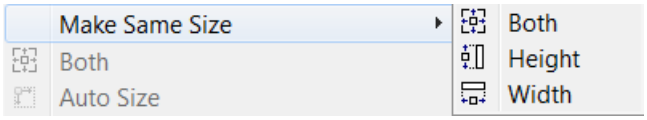


Рис. 3.25. Подменю Make Same Size

Выделите последовательно несколько графических элементов диаграммы. После этого можно установить высоту (**Height**), или ширину (**Width**), или оба размера (**Both**) всех элементов по последнему выбранному элементу.

На панели вкладок также доступен инструмент **Both** . А еще на панели вкладок есть инструмент **Auto Size** , позволяющий автоматически установить размеры всех выбранных элементов.

4. Работа с проектом

4.1. Создание нового проекта

В качестве примера будем рассматривать создание проекта работы компании, являющейся поставщиком интернет-услуг (интернет-провайдер).

Для создания нового проекта выберите в меню File/New/UML Project (рис. 4.1).

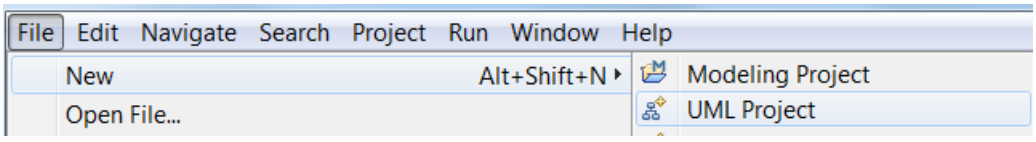


Рис. 4.1. Меню создания проекта

В появившемся окне введите имя проекта InternetProvider. При желании вы можете разместить проект в другой рабочей области, сняв выбор Use defaul location и указав новую рабочую область в поле Location. Далее нажмите кнопку Finish (рис. 4.2).

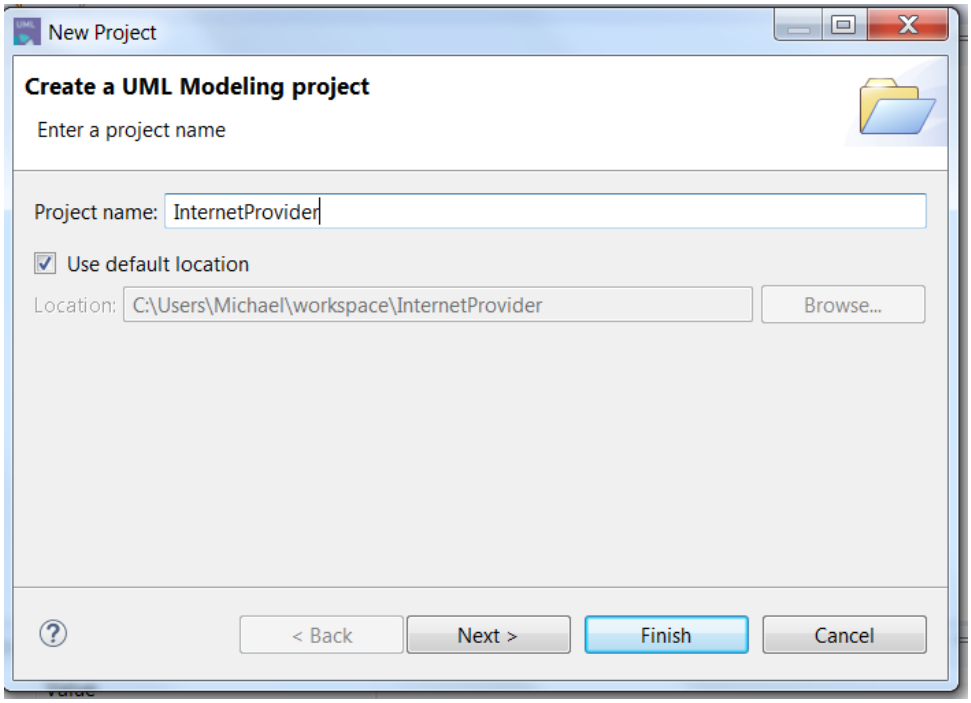


Рис. 4.2. Окно создания нового проекта

В виде Model Explorer появится новый проект. По умолчанию UML-модель этого проекта находится в фале model.uml и имеет имя NewModel Нажав правую кнопку мышки на элементе model.uml и выбрав пункт **Rename**, поменяем его имя на provider.uml. Аналогично поменяем имя модели на Internet Provider (рис. 4.3).

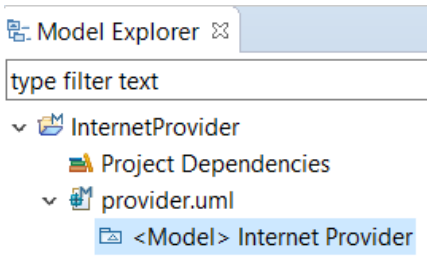


Рис. 4.3. Изменение имени модели в Model Explorer

4.2. Создание новой модели

В общем случае один проект может содержать несколько UML-моделей. Например, вы хотите создать несколько вариантов одного и того же проекта или же вы хотите разделить проект на несколько подсистем и для каждой подсистемы описать свою модель.

Для создания новой UML-модели нажмите правую кнопку мышки на элементе с именем проекта InternetProvider или на одном из подчиненных ему элементов и в появившемся окне выберите New, а затем UML Model (рис. 4.4).

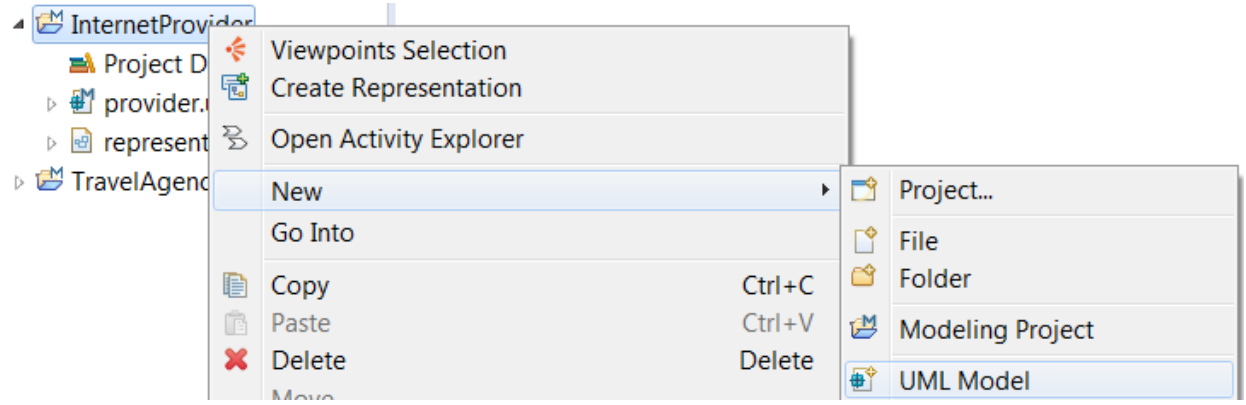


Рис. 4.4. Создание новой модели

В появившемся окне введите имя модели в поле **File name** и нажмите кнопку **Finish**. Если у вас есть несколько проектов, то в этом окне можно выбрать проект, которому будет принадлежать новая модель (рис. 4.5).

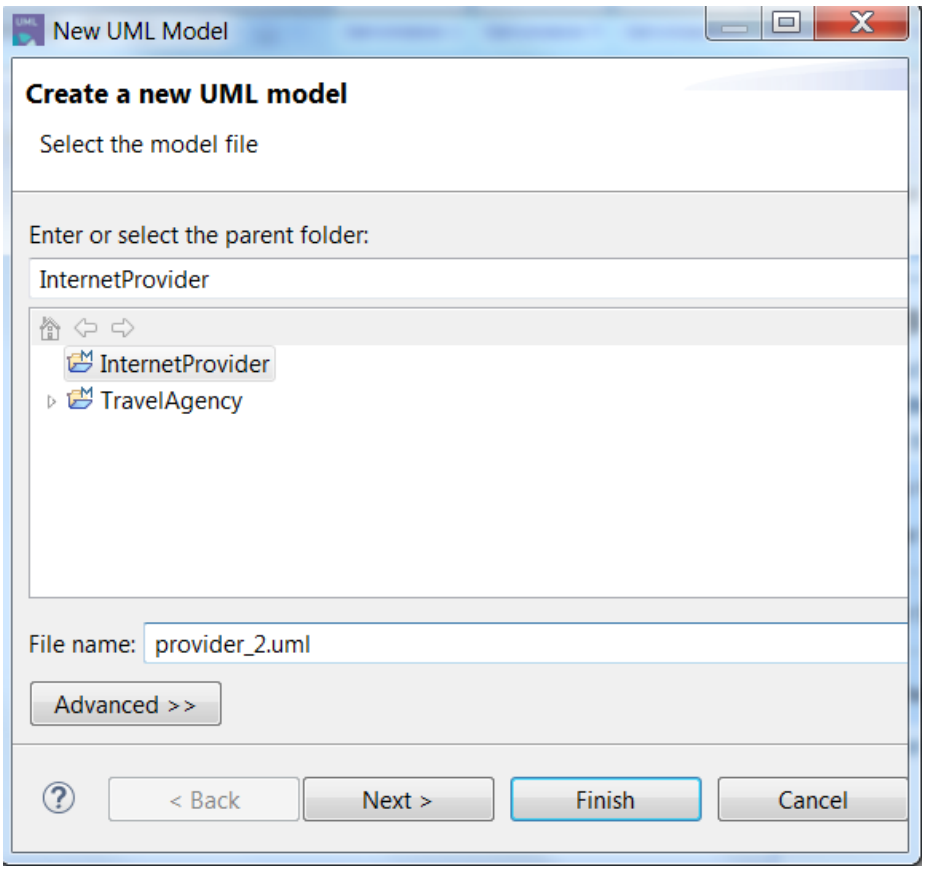


Рис. 4.5. Окно создания новой модели

4.3. Панель инструментов

Для каждого проекта создается панель инструментов (Dashboard). Открыть ее можно с помощью выбора пункта Open Activity Explorer в меню проекта, которое показывается по нажатию правой кнопки мышки на имени проекта в виде Model Explorer (рис. 4.6).

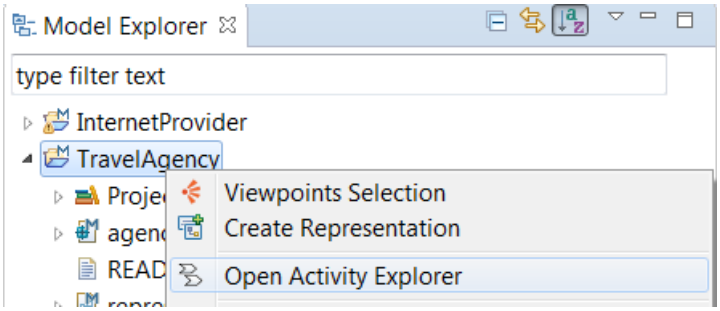


Рис. 4.6. Открытие панели инструментов

Панель имеет закладки Process Map, Overview, Capture, Design, Review Extend и Help. На закладке Process Map перечислены инструменты панели. Выбор инструмента осуществляется кликом мышки на иконку инструмента или же на закладку внизу окна (рис. 4.7).

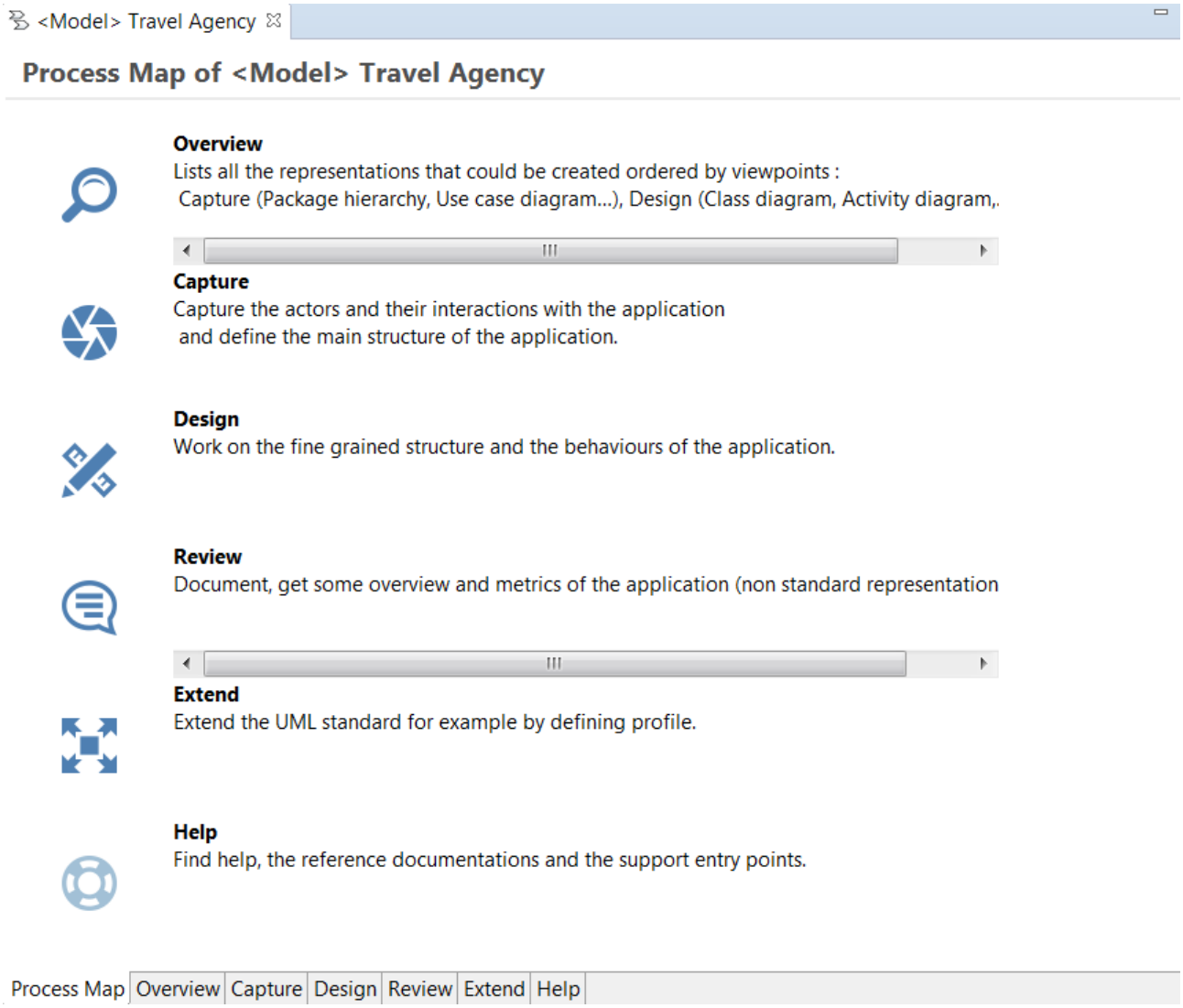


Рис. 4.7. Панель инструментов проекта

Закладка Overview позволяет создать новую диаграмму или посмотреть и выбрать одну из существующих диаграмм с помощью Diagrams Viewer.

Модель можно рассматривать с разных точек зрения (viewpoint). Все диаграммы разбиты по точкам зрения Capture, Design, Reused, Review и Extend. Обратите внимание, что диаграммы точек зрения Design и Reused, показанные в окне Diagrams Viewer, объединены в одну точку зрения Design в левой части панели инструментов (рис. 4.8).

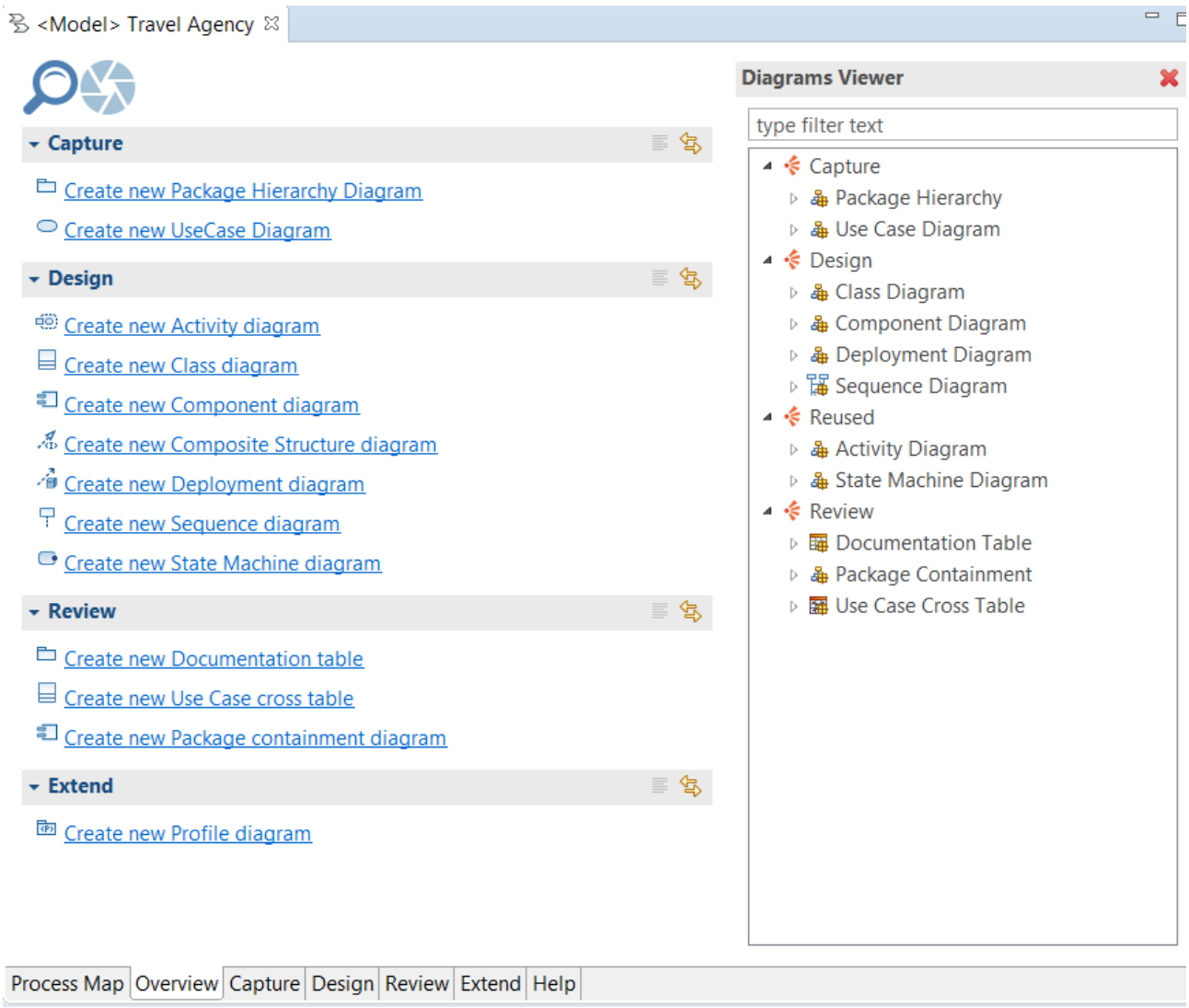


Рис. 4.8. Точки зрения на панели инструментов

Для каждой точки зрения, представленной в левой части панели инструментов, существует своя закладка на панели инструментов. Например, перейдя на закладку Capture, мы увидим в левой части панели список диаграмм, которые можно создать, а в правой части имена типов диаграмм, для которых уже созданы диаграммы (рис. 4.9).

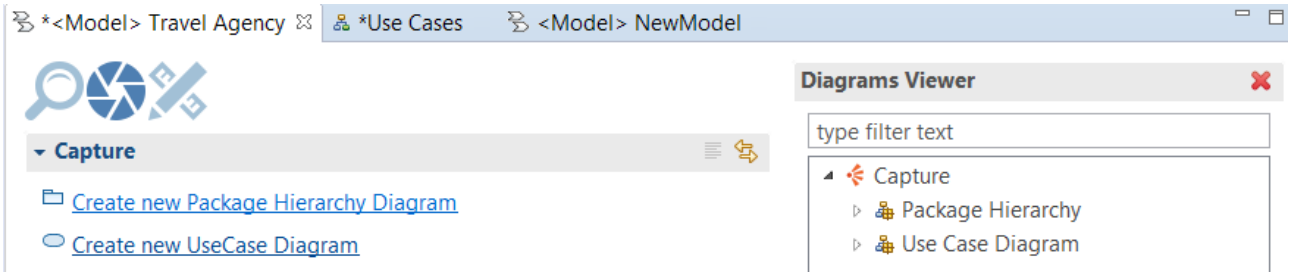


Рис. 4.9. Создание новой и просмотр существующих диаграмм

Чтобы увидеть созданные диаграммы нужно кликнуть на имя типа диаграммы в окне Diagrams Viewer.

4.4. Создание диаграммы

UML Designer предназначен для создания различного рода диаграмм (representations). Откройте панель инструментов, как описано в предыдущем разделе, выберите закладку Overview для создания любой диаграммы или одну из закладок Capture, Design, Review и Extend для создания диаграммы, относящейся к определенной точке зрения. Кликните на

строку создания диаграммы, например, Create new Use Case Diagram. Появится окно, в котором можно ввести имя диаграммы или оставить имя, которое предлагает UML Designer (рис. 4.10).

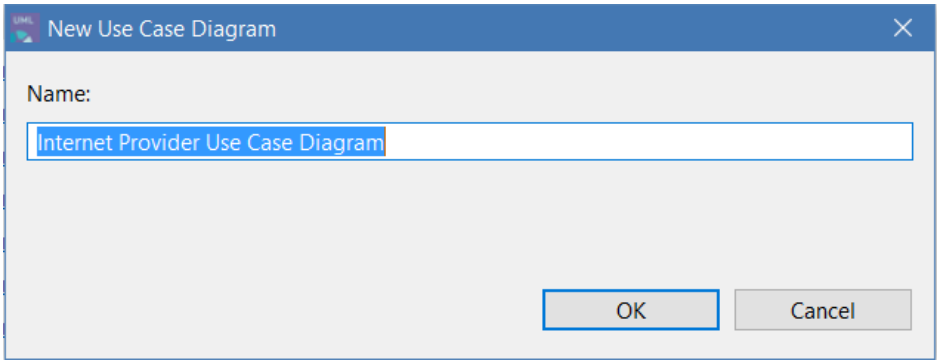


Рис. 4.10. Окно создания новой диаграммы

Заменим имя на **Use Case** и нажмем кнопку **OK**. В виде Model Explorer появится имя созданной диаграммы, а в окне редактирования откроется новая закладка с именем диаграммы (рис. 4.11).

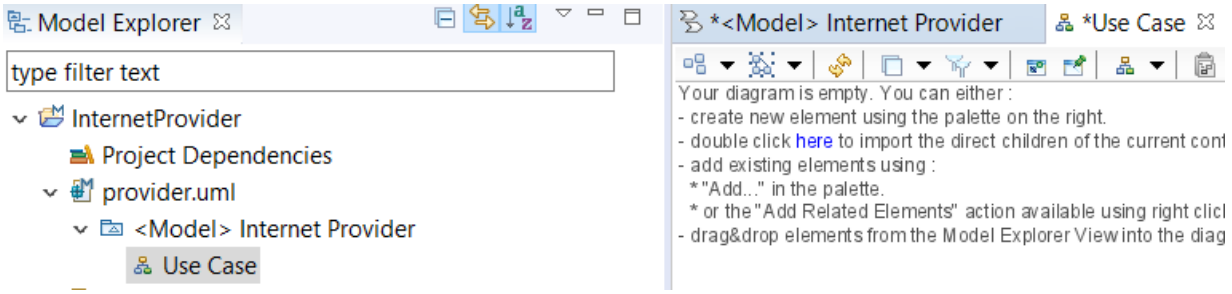


Рис. 4.11. Показ диаграммы в Model Explorer и окне редактирования

Новую диаграмму можно также создать, нажав правую кнопку мышки на строке **<Model> Internet Provider** и выбрав в меню пункт **New Representation**. В появившемся подменю выберите диаграмму, которую вы хотите создать (рис. 4.12).

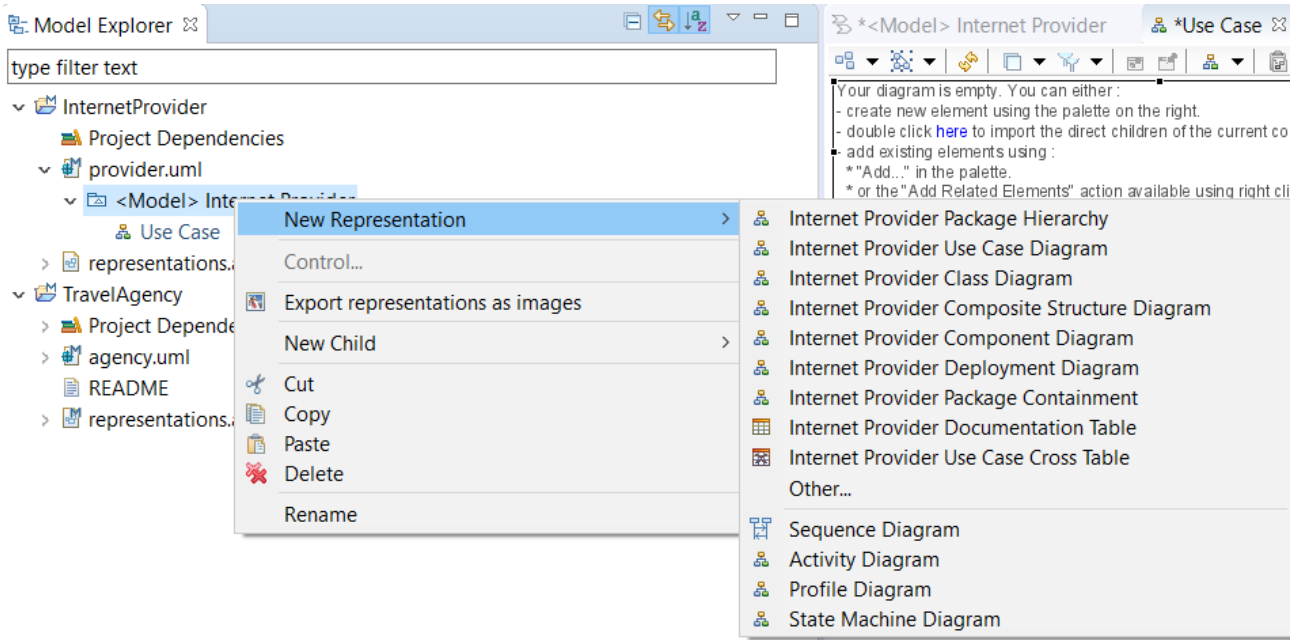


Рис. 4.12. Меню создания новой диаграммы

4.5. Экспорт и импорт проекта

Экспорт проекта осуществляется с помощью меню File/Export. В окне Export выберите **Archive File** для создания архива проекта или **File System** для копирования файлов проекта (рис. 4.13).

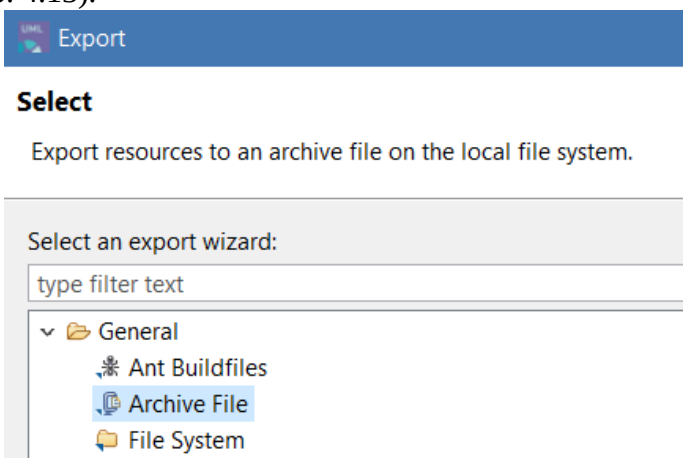


Рис. 4.13. Окно экспорта проекта

После выбора, например, **Archive File** отметьте имя проекта и все или часть файлов проекта, которые нужно сохранить. Можно сохранить сразу несколько проектов в одном архиве. В поле **To archive file** укажите полное имя архивного файла (рис. 4.14).

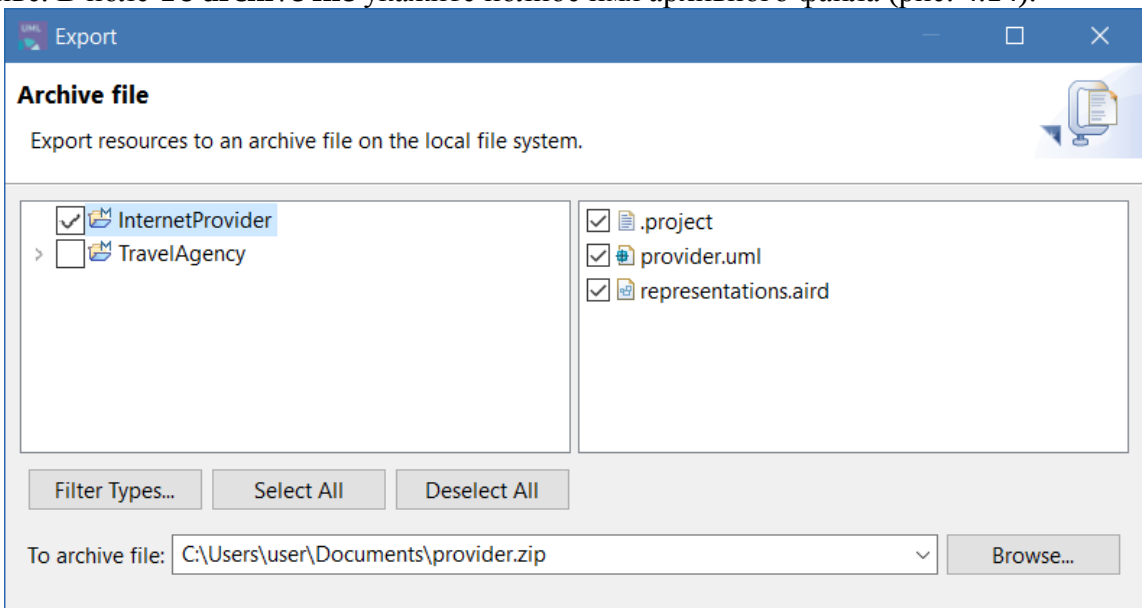


Рис. 4.14. Окно формирования архивного файла

Аналогично можно сохранить один или несколько проектов в не архивированном виде. В этом случае указывается папка, в которую копируются файлы проектов.

Для импорта проекта в рабочую область выберите меню File/Import и в открывшемся окне пункт **Existing Projects to Workspace**. Далее в окне Import укажите имя архива или имя папки с сохраненным проектом, а также выберите проект, который нужно импортировать. В папке рабочей области не должно быть данного проекта (рис. 4.15).

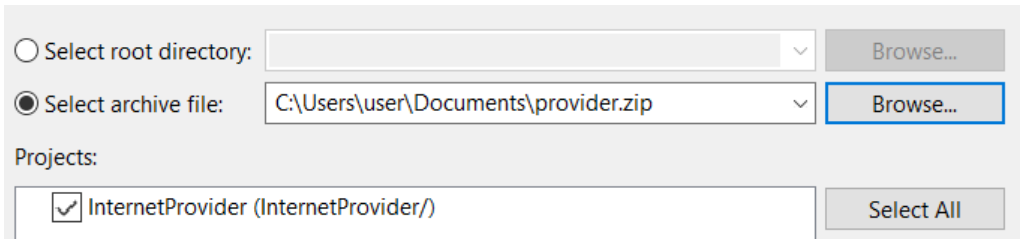


Рис. 4.15. Выбор импортируемого проекта

5. Диаграмма вариантов использования (Use Case Diagram)

Диаграмма вариантов использования предназначена для описания взаимодействия пользователей с системой. На диаграмме показываются не конкретные пользователи, а их роли, которые они выполняют при взаимодействии с системой. С системой могут взаимодействовать как люди, так и некоторые приложения, а также некоторые устройства. Например, в системе «Интернет-провайдер» с системой может взаимодействовать клиент, менеджер, подсистема бухгалтерии, а можно подключить к системе и оборудование (например, маршрутизатор), которое устанавливается у клиента.

Каждый пользователь выполняет определенные функции при взаимодействии с системой. На диаграмме они показываются с помощью вариантов использования (Use case). Их еще называют прецендентами. Варианты использования определяются на основе требований, которые пользователи предъявляют к системе. Они описывают что пользователь может сделать при взаимодействии с системой и определяют некоторый сценарий взаимодействия пользователя с системой.

5.1. Элементы диаграммы вариантов использования

Палитра инструментов диаграммы вариантов использования показана на рис. 5.1.

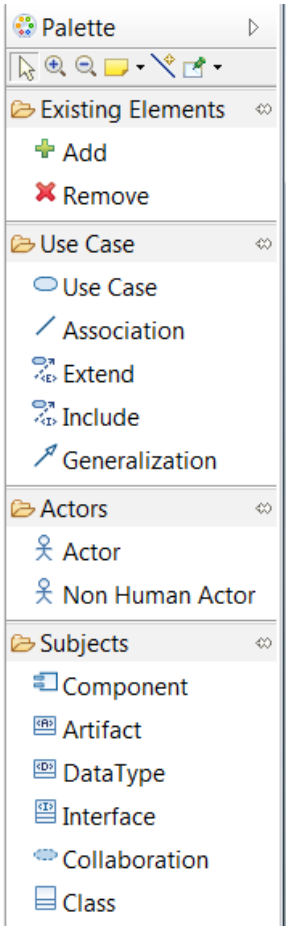


Рис. 5.1. Палитра инструментов диаграммы вариантов использования

Для перемещения элемента из палитры на диаграмму кликните на значок элемента, а затем на пустое место на диаграмме или на элемент внутри диаграммы, если один элемент должен быть размещен внутри другого элемента.

Каждый пользователь называется актером (Actor). Для актера, которым является человек, используется графический элемент 

(рис. 5.2).

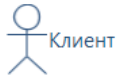


Рис. 5.2. Актер, являющийся человеком

Для обозначения актеров, не являющихся людьми, используется элемент Non Human Actor (рис. 5.3).



Рис. 5.3. Неодушевленный актер

Для обозначения варианта использования применяется элемент Use Case (рис. 5.4).

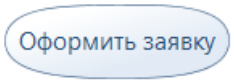


Рис. 5.4. Вариант использования

Варианты использования могут быть реализованы внутри некоторого компонента (подробнее о компонентах написано в разделе 21. Диаграмма компонентов). Распределение вариантов использования по компонентам может осуществляться по разным критериям. Например, можно поместить все варианты использования, с которыми взаимодействует один пользователь, внутрь одного компонента. Другой вариант – распределить варианты использования по подсистемам нашей системы. Например, выделить подсистемы «Взаимодействие с клиентом», «Тарификация разговоров», «Бухгалтерия». Третий вариант – поместить варианты использования, которые будут реализованы с помощью некоторого множества классов, расположенного в одном файле, в один компонент.

Для обозначения компонента применяется элемент Component. Вариант использования, реализуемый некоторым компонентом, размещается на диаграмме внутри этого компонента (рис. 5.5).

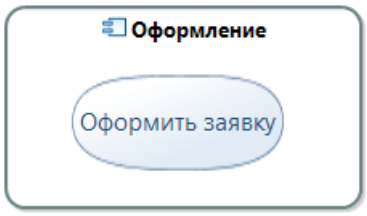


Рис. 5.5. Компонент и реализуемый им вариант использования

Все графические элементы и сами диаграммы располагаются в UML-модели проекта. В данном примере их можно увидеть в элементе **<Model> Internet Provider** вида Model Explorer (рис. 5.6).

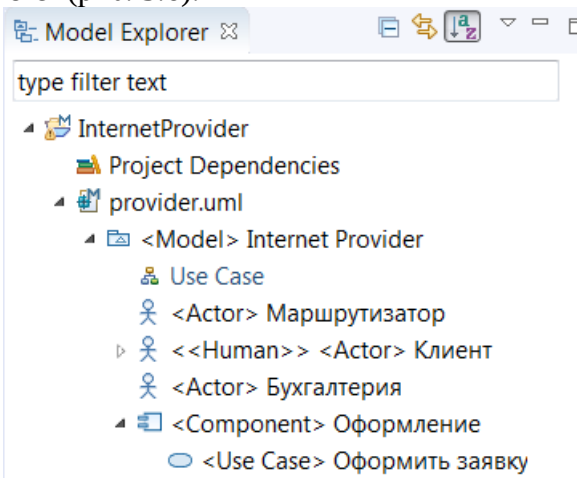


Рис. 5.6. Показ диаграмм и элементов в виде Model Explorer

Каждый элемент имеет пиктограмму элемента, класс элемента, указанный в угловых скобках, и имя элемента. Класс элемента модели определяется стандартом языка UML. Для диаграмм (например, диаграмма с именем Use Case) класс элемента не указывается. Имя элемента также может быть не указано. Например, для соединительных линий.

Если графический элемент располагается внутри другого графического элемента, то его описание смещено относительно описания родительского элемента вправо. А у родительского элемента слева появляется значок треугольника, позволяющий скрыть или открыть описание дочерних элементов. Например, видно, что родительский элемент **<Component> Оформление** имеет дочерний элемент **<Use Case> Оформить заявку**.

Вариант использования может быть сразу размещен внутри компонента или же его можно перетащить мышкой на компонент из другой части диаграммы. Если Вы попытаетесь перетащить мышкой вариант использования из компонента на диаграмму, то он исчезнет с диаграммы, но не из модели. В виде Model Explorer при этом видно, что элемент **<Use Case> Оформить заявку** уже не подчинен элементу **<Component> Оформление** (рис. 5.7).

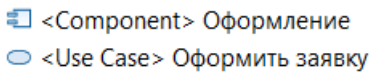


Рис. 5.7. Компонент и отдельный вариант использования

Для того, чтобы после этого вернуть вариант использования на диаграмму кликните элемент Add , затем кликните на пустое место диаграммы и в открывшемся окне выберите нужный элемент с помощью клика на прямоугольник слева от имени элемента (рис. 5.8).

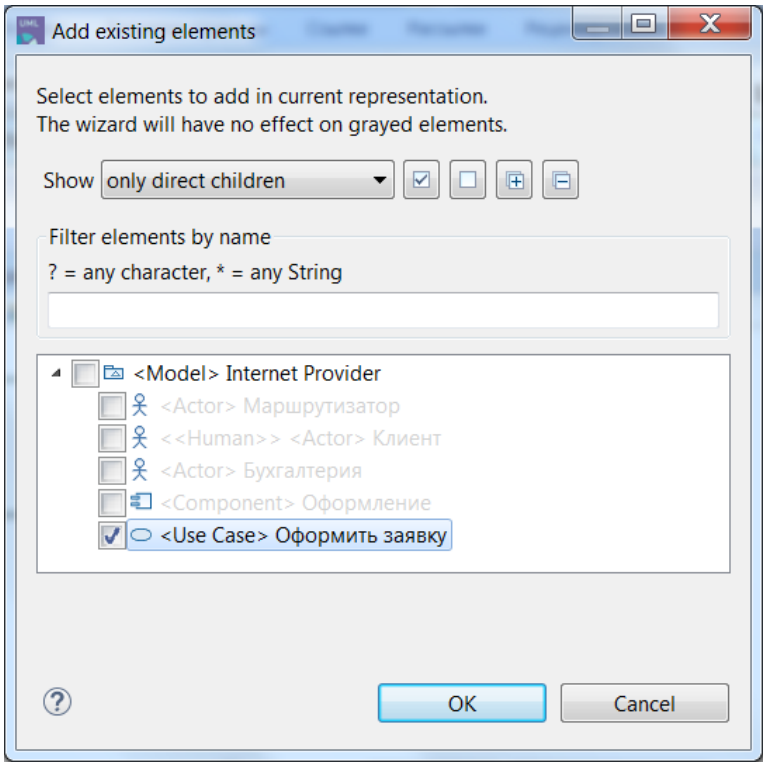


Рис. 5.8. Добавление существующих элементов на диаграмму

В окне **Add existing element** также показывается иерархия элементов.

Элемент **<Use Case> Оформить заявку** можно удалить из диаграммы с помощью меню Edit, вызываемого по правой кнопке мышки (рис. 5.9).

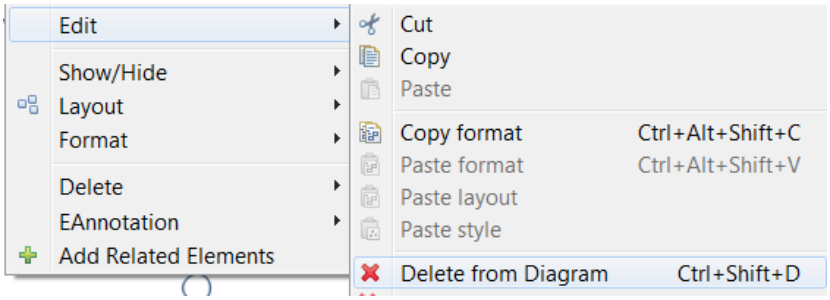


Рис. 5.9. Удаление элемента с диаграммы

В этом случае элемент останется внутри элемента **<Component> Оформление**, что видно в виде Model Explorer (рис. 5.10).

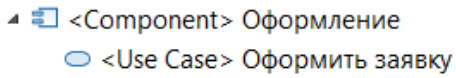


Рис. 5.10. Вариант использования, размещенный внутри компонента

Тогда с помощью 

элемент **<Use Case> Оформить заявку** нужно возвращать, кликая на элемент **<Component> Оформление**. Помещение же элемента на пустое место приводит к ошибке в структуре элементов – в модели он остается внутри элемента **<Component> Оформление** (надеемся, что в следующих версиях UML Designer эта ошибка будет исправлена).

Варианты использования могут быть сгруппированы не только с помощью компонентов, но также с помощью таких объектов как:

- Артифакт (Artifact);
- Тип данных (DataType);
- Интерфейс (Interface);
- Коммуникация (Collaboration);
- Класс (Class).

5.2. Отношения между элементами диаграммы вариантов использования

Между элементами диаграммы вариантов использования можно установить следующие отношения:

- отношение ассоциации (Association);
- отношение включения (Include);
- отношение расширения (Extend);
- отношение (Generalization).

Для установки отношения кликните на него в палитре инструментов, а затем последовательно на соединяемые элементы. Если отношение направленное (со стрелкой), то сначала нужно кликнуть на элемент-источник, а затем на элемент-цель. Стрелка отношения указывает на элемент-цель.

Линию отношения можно изогнуть, кликнув на нее (линия выделяется синим цветом) и переместив мышку. Выделенную ломанную линию можно выпрямить, нажав Ctrl+Shift-.

Отношение ассоциации используется для связи актера и варианта использования. Оно показывает, что актер может выполнить данный варинт использования при взаимодействии с системой. Например, техник подбирает оборудование, которое нужно установить у клиента (рис. 5.11).



Рис. 5.11. Отношение ассоциации

Отношение включения используется для связи двух вариантов использования, чтобы показать, что при работе одного варианта использования выполняется другой вариант использования. Актер напрямую не соединяется со включаемым вариантом использования.

Например, клиент при изменении тарифного плана просматривает список возможных тарифов (рис. 5.12).

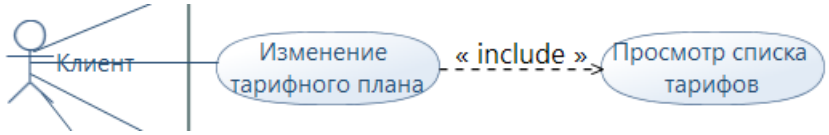


Рис. 5.12. Отношение включения

Распространенной ошибкой является применение отношения включения для показа иерархии пунктов меню, где каждый пункт меню изображается в виде варианта использования. Также ошибкой является использование отношения включения для показа последовательности действий пользователя (для этого применяется диаграмма деятельности). Например, показ того, что пользователь сначала подает заявку на подключение, а затем выбирает тарифный план.

Отношение расширения используется для связи двух вариантов использования, чтобы показать, что некоторый вариант использования расширяет возможности основного варианта использования. Стрелка в этом случае направлена на основной вариант использования. Актер напрямую не соединяется с расширяющим вариантом использования.

Например, менеджер обычно передает заказ в техотдел для подбора оборудования, которое нужно установить у клиента, но может использовать и стандартное оборудование (рис. 5.13).

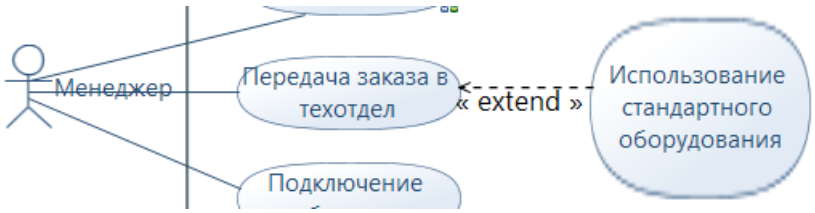


Рис. 5.13. Отношение расширения

Отношение обобщения связывает либо два варианта использования, либо двух актеров. В случае двух вариантов использования один вариант использования (обобщающий) содержит описание основных действий пользователя, а наследуемые от него варианты использования описывают действия пользователя в конкретных ситуациях. В отличие от отношения расширения, где действия варианта использования, расширяющего основной вариант использования, выполняются не всегда, в случае обобщения всегда должен выполнятся один из наследуемых вариантов использования. Актер напрямую не соединяется с наследуемыми вариантами использования.

Например, при оплате клиентом подключения к Интернету, он может это сделать либо в офисе провайдера, либо на сайте провайдера (рис. 5.14).



Рис. 5.14. Отношение обобщения между вариантами использования

Отношение обобщения между двумя актерами служит для того, чтобы варианты использования, которые выполняются разными актерами, не связывать с каждым из этих актеров. Для этого вводится новый актер, который обобщает действия других актеров. Все

варианты использования, которые привязываются к этому актеру, доступны для любого актера, наследуемого от обобщающегося актера. Например, в системе все актеры выполняют регистрацию, поэтому можно ввести обобщающего актера **Пользователь** и связать его с другими актерами отношением обобщения (рис. 5.15).

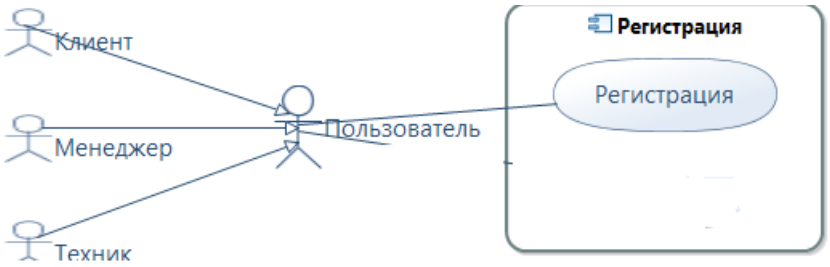


Рис. 5.15. Отношение обобщения между актерами

5.3. Пример диаграммы вариантов использования

Общий вид диаграммы вариантов использования для модели Internet Provider имеет вид, показанный на рис. 5.16.

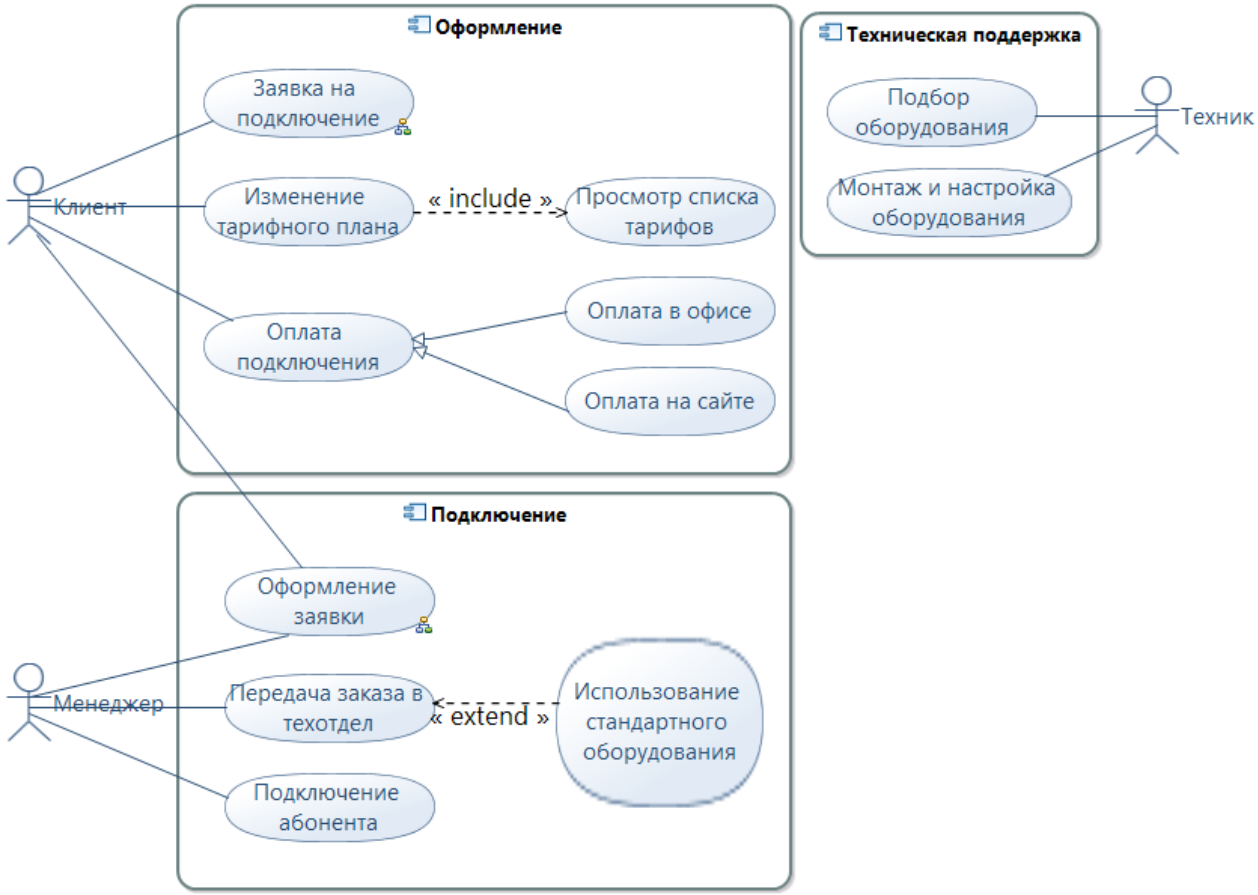


Рис. 5.16. Пример диаграммы вариантов использования

На одной диаграмме может не удасться разместить все варианты использования и всех актеров системы так, чтобы диаграмма уместилась на одной странице при печати или же так, чтобы не пересекались линии связей между элементами . В этом случае создайте несколько диаграмм вариантов использования. Если один и тот же элемент, обычно это актер, используется на разных диаграммах, то разместив элемент на одной диаграмме, на другую диаграмму его нужно поместить, используя инструмент Add .

Отношение обобщения для нескольких актеров может использоваться для показа того, что все эти актеры взаимодействуют с одними и теми же вариантами использования.

Например, все актеры в модели Internet Provider выполняют регистрацию в системе, а для входа в систему должны пройти аутентификацию. Поэтому введем еще одного актера **Пользователь,** который будет являться обобщением актеров **Клиент**, **Менеджер** и **Техник**. Если мы попытаемся нарисовать эти варианты использования на той же диаграмме, то получим пересекающиеся линии отношений, связывающие актеров. Поэтому создадим отдельную диаграмму, на которой покажем нового актера и варианты использования (рис. 5.17).

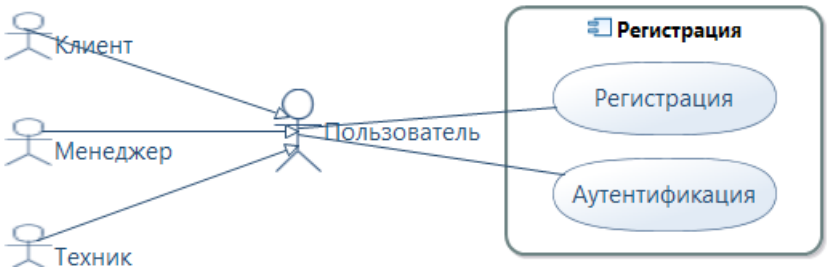


Рис. 5.17. Пример диаграммы вариантов использования с отношениями обобщения

6. Диаграмма пакетов (Package Hierarchy Diagram)

Диаграмма пакетов служит для описания общей структуры UML-модели с помощью объединения элементов модели в пакеты. Пакет может содержать в себе любые элементы модели. Пакет может также содержать в себе другие пакеты. Между пакетами устанавливаются отношения.

6.1. Создание диаграммы пакетов

Диаграмма пакетов содержит все другие элементы модели, поэтому поместим ее непосредственно в элемент **<Model> Internet Provider**. Откроем меню по правой кнопке мышки, встав на строку элемента **<Model> Internet Provider** в виде Model Explorer. Выберем пункт **New Representation** и далее в открывшемся подменю пункт **Internet Provider Package Hierarchy** (рис. 6.1).



Рис. 6.1. Создание диаграммы пакетов

В открывшемся окне **New Package Hierarchy** изменим имя диаграммы на **Package Hierarchy** (рис. 6.2).

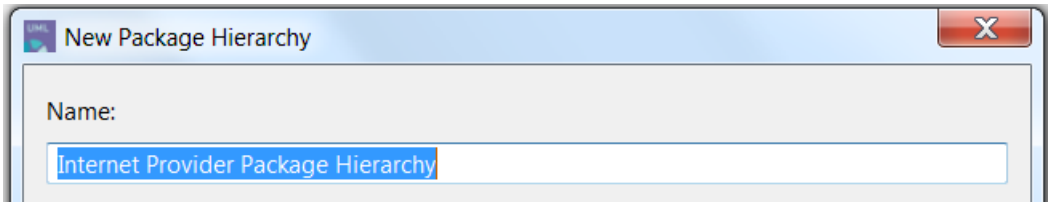


Рис. 6.2. А

В Model Explorer увидим появившуюся диаграмму (рис. 6.3).

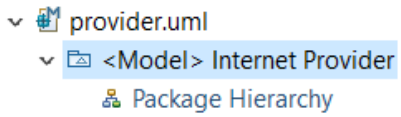


Рис. 6.3. Диаграмма пакетов в Model Explorer

6.2. Элементы диаграммы пакетов

Диаграмма пакетов имеет следующую палитру инструментов, показанную на рис. 6.4.

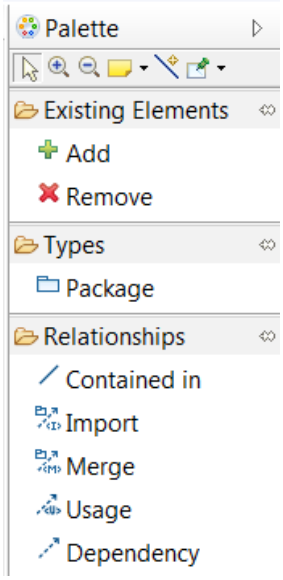


Рис. 6.4. Палитра инструментов диаграммы пакетов

На диаграмме пакетов показываются элементы всего одного типа – это пакеты (**Package**).

Для размещения пакета на диаграмме кликните на элемент **Package** в палитре, а затем на пустое место диаграммы.

Изображение пакета появится на диаграмме а его имя в виде Model Explorer.
Измените имя пакета на диаграмме, как это описано в разделе **Область редактирования.**

Например, так будет выглядеть вид Model Explorer и диаграмма после создания двух пакетов (рис. 6.5).

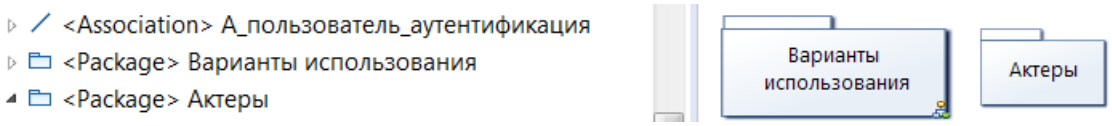


Рис. 6.5. Два пакета на диаграмме

Для выполнения действий с пакетом используйте меню, открывающееся по правой кнопке мышки, когда выбрана строка в виде Model Explorer (рис. 6.6).

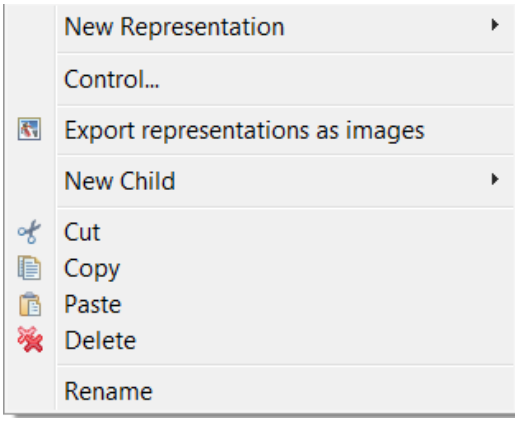


Рис. 6.6. Меню действий с пакетом

Имя пакета можно изменить с помощью пункта **Rename**.

Пункт **New Representation** позволяет создать новую диаграмму внутри пакета (рис. 6.7).

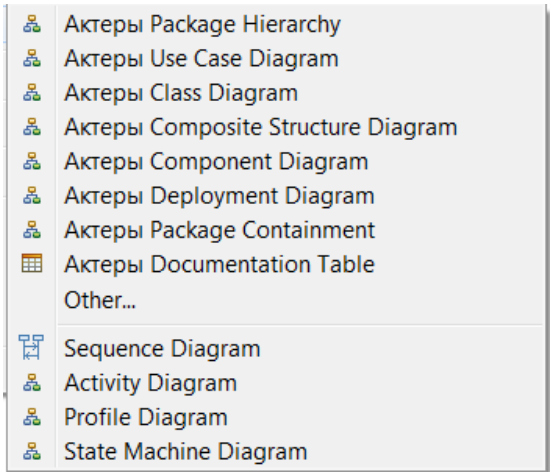


Рис. 6.7. Меню создания новой диаграммы внутри пакета

Если диаграмма создана в пакете, то она и все ее элементы будут содержаться в пакете.

Например, можно создать внутри пакета **Варианты использования** новую диаграмму **Use Case Test**, на которой расположен элемент **UseCase1** (рис. 6.8).

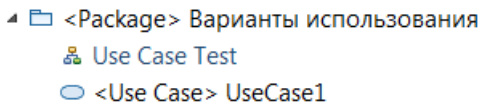


Рис. 6.8. Диаграмма и ее элемент в Model Explorer

Для того, чтобы поместить ранее созданный элемент модели в пакет, например, элемент **<<Human>><Actor > Пользователь**, нажмите на него мышкой в виде Model Explorer и перетащите в папку пакета, например, папку **<Package> Актеры**. Откроется окно Deletion (рис. 6.9).

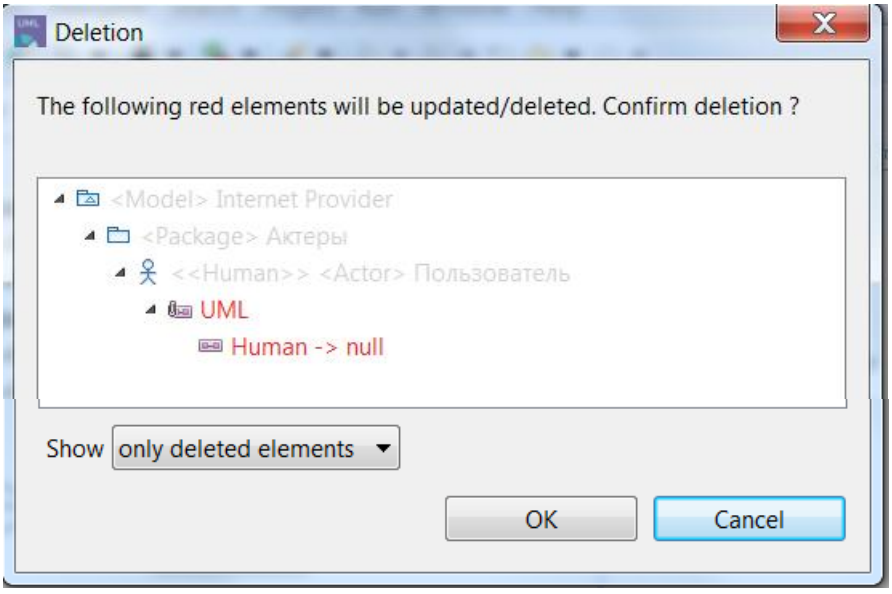


Рис. 6.9. Удаление элементов при перемещении элемента в пакет

После нажатия OK элемент переместится в папку пакета в виде Model Explorer (рис. 6.10).

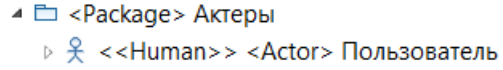


Рис. 6.10. Результат перемещения в Model Explorer

Можно выделить сразу несколько элементов и также перетащить их в пакет (рис. 6.11).

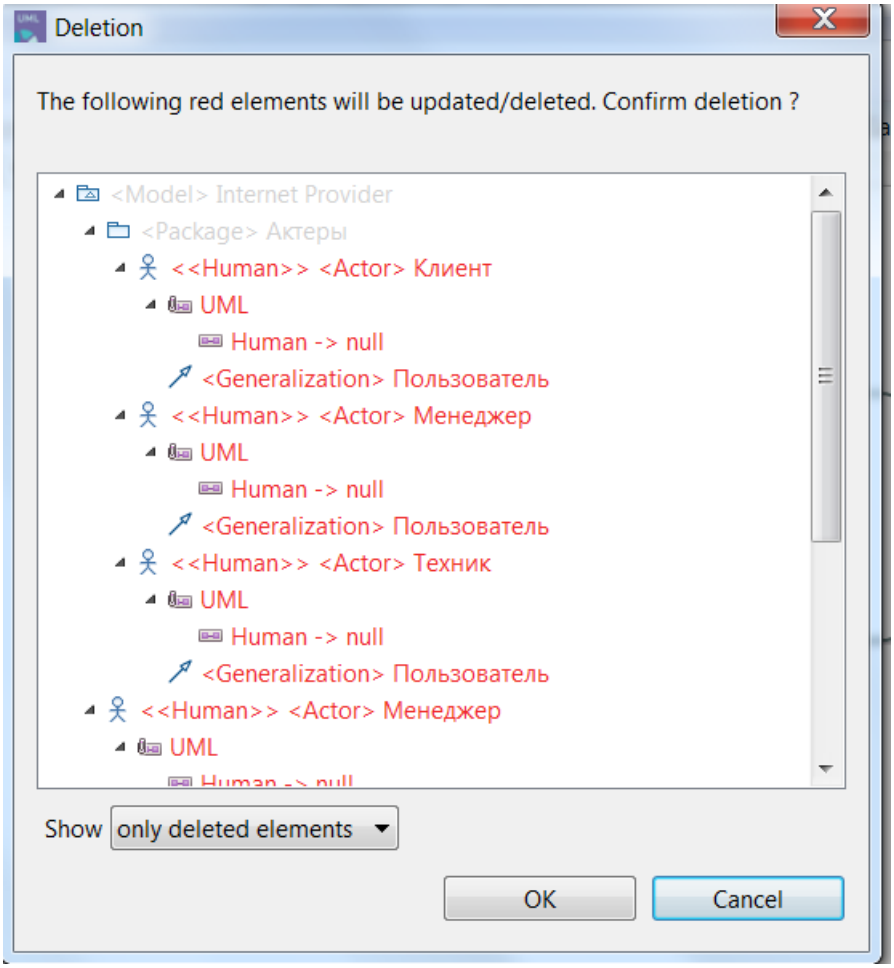


Рис. 6.11. Удаление элементов при их перемещении в пакет

Обратите внимание, что каждая связь типа Generalization находится внутри элемента типа <<Human>><Actor>. Это видно, как в окне Deletion, так и виде Model Explorer (рис. 6.12).

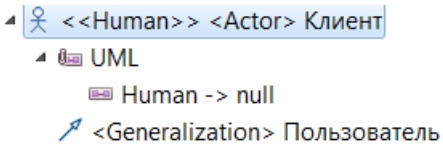


Рис. 6.12. Размещение связи обобщения внутри элемента актера

Поэтому при перемещении этих актеров они будут перемещены в пакет вместе со своими связями типа Generalization.

Ни в коем случае не перемещайте элемент с помощью пунктов Cut и Paste в меню. На первый взгляд, после выполнения этих действий вы увидите, что элемент переместился в папку пакета. Однако, при выполнении Cut сам элемент и все его связи останется только в модели, но будет удален с диаграммы, на которой расположен этот элемент.

6.3. Отношения между элементами диаграммы пакетов

Отношение использования (**Usage)** показывает, что элементы одного пакета требуют наличия элементов другого пакета.

Разместим в пакете **Актеры** всех актеров и все отношения между актерами и между вариантами использования, а в пакете **Варианты использования** - все варианты использования. Так как отношения ассоциации связывают актеров и варианты использования, то для их описания требуется знание о наличии вариантов использования.

Поэтому соединим эти два пакета отношением использования. Стрелка показывает, что пакет **Актеры** зависит от пакета **Варианты использования** (рис. 6.13).

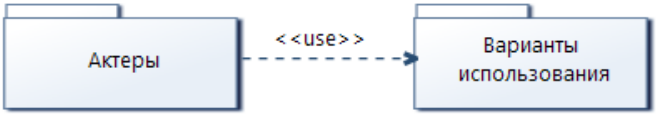


Рис. 6.13. Отношение использования

Отношение принадлежности (**Contained in**) между пакетами показывает, что один пакет вложен в другой пакет.

Создадим пакет **Поведение** и соединим его с пакетами **Актеры** и **Варианты использования** отношениями принадлежности. При соединении двух пакетов отношением принадлежности сначала выберите мышкой дочерний пакет, а потом родительский пакет. В результате появляется окно **Deletion** как в случае перемещения элементов в пакет (рис. 6.14).

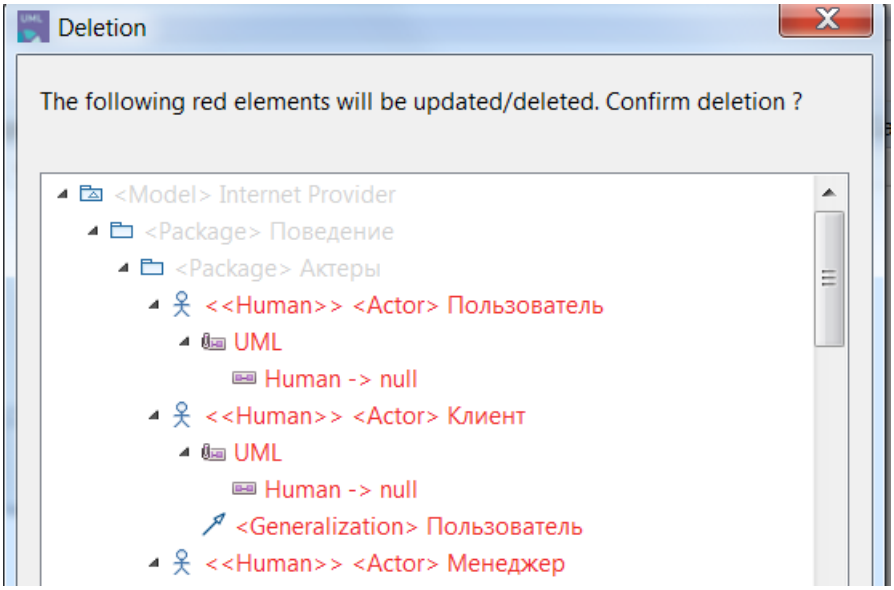


Рис. 6.14. Удаление элементов при использовании отношения принадлежности

Дочерний пакет, т.е. тот, который принадлежит родительскому пакету, перемещается вместе со всеми своими элементами в родительский пакет. Это видно в виде Model Explor er. Стрелка отношения принадлежности направлена в сторону родительского пакета (рис. 6.15).

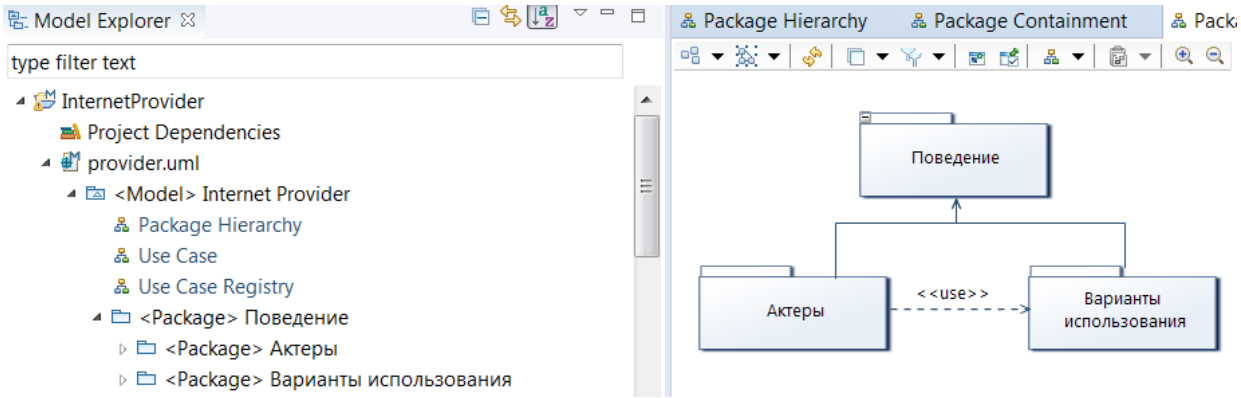


Рис. 6.15. Расположение пакетов в модели при использовании отношения принадлежности

Отношение зависимости (**Dependency**) показывает, что изменения некоторого элемента модели (**поставщика** независимого элемента), принадлежащего одному (независимому) пакету, могут вызвать изменения в другом элементе модели (**клиент** или зависимый

элемент), принадлежащего другому (зависимому) пакету. Стрелка отношения направлена от зависимого пакета к независимому пакету.

Создадим пакет **Компоненты** и пакет **Интерфейсы** по аналогии с тем, как это сделано в примере Travel Abgency. В пакете **Компоненты** содержаться компоненты и классы, относящиеся к этим компонентам. В пакете **Интерфейсы** содержаться интерфейсы (абстрактные классы). Так как классы реализуют методы, указанные в интерфейсах, то они зависят от интерфейсов (рис. 6.16).



Рис. 6.16. Отношение зависимости

Пакеты позволяют не только группировать элементы, но скрывать их. Так, например, открытые классы пакета **A** не видят открытых классов пакета **B** до тех пор пока не будет установлено отношение зависимости пакета **A** от пакета **B**.

Отношение импорта (**Import**) между пакетами **B** и **C**, где **B** – это клиент зависимости, а **C** – поставщик, как бы перемещает все открытые классы пакета **C** в пакет **B** и теперь они становятся видны для пакета **A** (рис. 6.17).

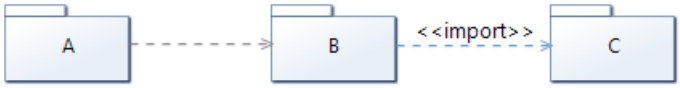


Рис. 6.17. Отношение импорта

При обращении к элементам пакета извне нужно указывать имя пакета и имя элемента. В случае отношения импорта между пакетами из пакета клиета можно обращаться к элементам пакета источника просто по именам.

Отношение слияния (соединение) (**Merge**) означает, что все элементы двух пакетов объединяются. Если в соединяемых пакетах есть элементы одного типа с одинаковыми именами, например, два класса, то из них образуется один элемент, содержащий все методы и атрибуты обоих классов.

После установления отношений использования, зависимости, импорта и объединения элемент отношения появляется в пакете, из которого исходит стрелка отношения. Например, после установления отношения зависимости в виде Model Explorer появится элемент этого отношения (рис. 6.18).

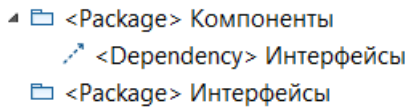


Рис. 6.18. Отношение зависимости в Model Explorer

7. Диаграмма классов (Class Diagram)

Диаграмма классов служит для описания классов и их отношений. Она относится к структурным диаграммам языка UML.

Класс представляет собой описание группы объектов реального мира, обладающих общим набором свойств (характеристик), поведением и общими отношениями с другими объектами.

Диаграмма классов описывает, как данные, хранящиеся в программной системе и базе данных, так и классы, используемые для реализации программной системы, например, формы ввода данных, классы, реализующие функции системы.

7.1. Создание диаграммы классов

Диаграмму классов можно создать с помощью панели инструментов (Dashboard) (раздел 12. Панель инструментов) или через меню, открываемому по правой кнопке мышки на элементе **<Model> Internet Provider** в виде Model Explorer, и его подменю (рис. 7.1).

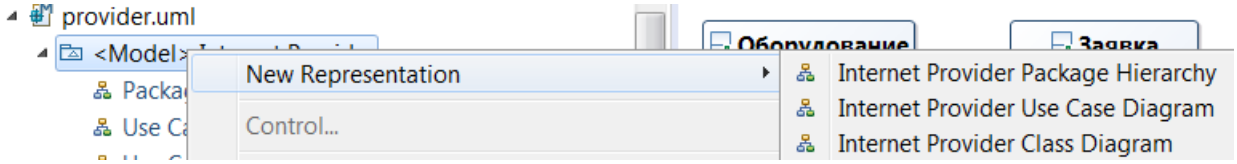


Рис. 7.1. Создание диаграммы классов

После выбора пункта **Internet Provider Class Diagram** в окне **New Class Diagram** можем изменить стандартное имя на имя **Классы** (рис. 7.2).

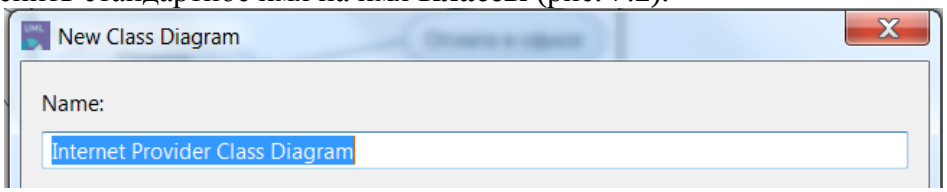


Рис. 7.2. Окно создания диаграммы классов

В Model Explorer увидим появившуюся диаграмму **Классы** (рис. 7.3).

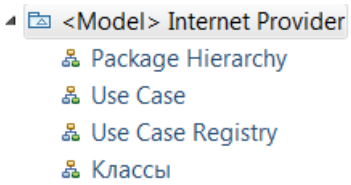


Рис. 7.3. Диаграмма классов в Model Explorer

В дальнейшем имя диаграммы можно всегда поменять, используя пункт **Rename** меню, открываемому по правой кнопке мышки на элементе **Классы** в виде Model Explorer (рис. 7.4).

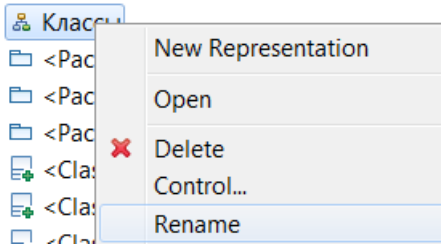


Рис. 7.4. Меню элемента

Проект системы может содержать несколько диаграмм классов.

7.2. Элементы диаграммы классов

Диаграмма классов имеет палитру инструментов, состоящую из четырех разделов, показанных на рис. 7.5.

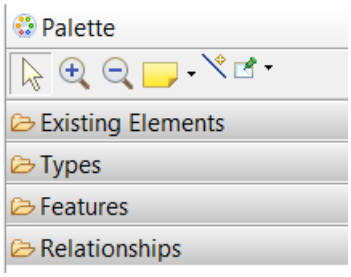


Рис. 7.5. Разделы палитра инструментов диаграммы классов

Первый раздел позволяет использовать на диаграмме существующие элементы (рис. 7.6).

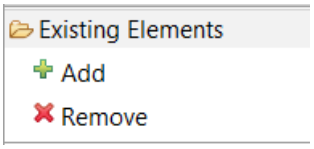


Рис. 7.6. Первый раздел палитры инструментов

Элемент добавить (**Add**) используется для добавления существующих элементов (компонентов, пакетов, классов) также как при добавлении существующих элементов на диаграмму компонентов (подраздел 21.2 Элементы диаграммы компонентов).

Элемент удаления (**Remove**) используется для удаления элемента диаграммы. Кликните на этот элемент, а потом на элемент диаграммы. Элемент будет удален с диаграммы, но не из модели, поэтому его можно будет вернуть на диаграмму с помощью элемента Add.

Второй раздел содержит элементы, описывающие различные типы данных (рис. 7.7).

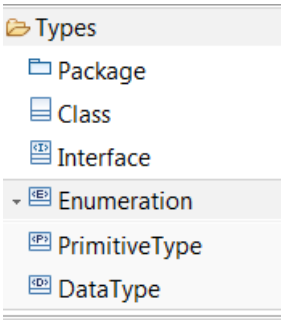


Рис. 7.7. Второй раздел палитры инструментов

Размещение этих элементов данных осуществляется на диаграмме, либо с помощью выбора соответствующего элемента в палитре инструментов (нужно кликнуть на элемент, а затем на диаграмму), либо с помощью всплывающего меню с иконками элементов, которое появляется, если навести мышку на пустое место области редактирования диаграммы и подождать одну-две секунды (рис. 7.8).



Рис. 7.8. Всплывающее меню

При помещении элемента на диаграмму появляется окно создания элемента (**Create**), в котором можно изменить задаваемые по умолчанию имя элемента, его видимость и указать является ли элемент абстрактным, т.е. элементом, для которого не создается экземпляров. Например, при создании нового класса появляется окно, показанное на рис. 7.9.

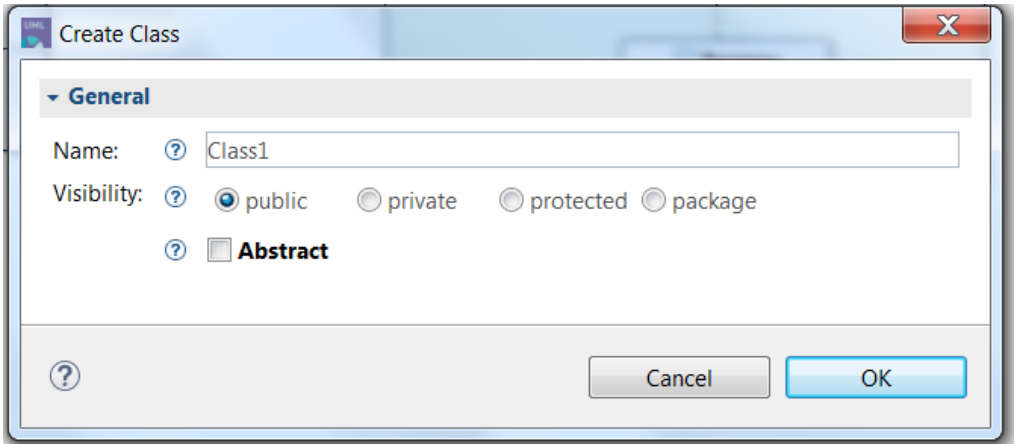


Рис. 7.9. Окно создания класса

Элемент класс (**Class**) используется для показа классов на диаграмме.

Элемент пакет (**Package**) используется для группировки других элементов, например, классов. Он позволяет добавить на диаграмму классов новый пакет, которого нет на диаграмме пакетов. Далее этот пакет может быть также добавлен на диаграмму пакетов, используеся элемент Add палитры инструментов диаграммы пакетов. Внутрь изображения пакета могут быть помещены другие элементы диаграммы. Например, так выглядит пакет **Таблицы**, внутри которого показаны классы, используемые для создания таблиц (рис. 7.10).

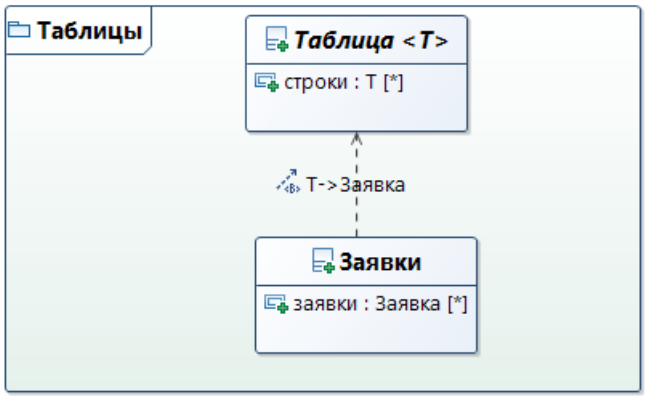


Рис. 7.10. Классы внутри пакета

Размещение классов внутри пакета будет отражено в Model Explorer (рис. 7.11).

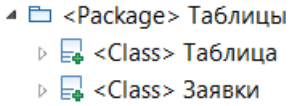


Рис. 7.11. Классы внутри пакета в Model Explorer

Более подробно эти два класса описаны в разделе «17.2 Отношения между элементами диаграммы классов».

Элемент может быть сразу размещен внутри пакета или же перемещен внутрь пакета с помощью мышки (нажать на элементе и удерживая переместить). Обратите внимание, что если вы переместите элемент из пакета наружу, то он исчезнет с диаграммы, но не из модели. Изображение элемента на диаграмме можно будет восстановить с помощью элемента добавления Add.

В пакет может быть добавлен не любой элемент. Если поместить мышку на изображение пакета и немного подождать, то появится меню с элементами, которые можно поместить в пакет (рис. 7.12).



Рис. 7.12. Всплывающее меню для пакета

Элемент интерфейс (**Interface**) используется для показа абстрактных классов, в которых определен набор операций, предоставляемых данным интерфейсом.

Элемент тип данных (**DataType**) – это класс, используемый для описания данных, с которыми работает программа. Например, можно описать тип данных **Адрес** или **Дата.** Два экземпляра типа данных считаются одинаковыми, если они имеют одинаковый набор значений свойств. Например, два адреса одинаковы, если у них совпадает город, улица, дом и квартира. Типы данных можно использовать также для описания сущностей реального мира, например **Клиент**, **Счет**, **Заявка**. Экземпляры этих данных используются в программе и хранятся, как правило, в базе данных.

Элемент перечисление (**Enumeration**) являет частным случает типа данных и служит для определения набора возможных значений. Например, можно определить тип **Оценка**, со значениями: 2, 3, 4, 5, или тип **Врямя года**, со значениями: зима, весна, лето, осень.

Элемент примитивный тип (**PrimitiveType**) также является частным случаем типа данных. Он служит для создания типов данных, не имеющих внутренней структуры. Например, числа и строки являются примитивными типами данных. Можно создать, например, примитивный тип **Фамилия**. Примитивный тип данных может иметь операции, определенные вне модели UML. Например, математические операции над числами.

7.3. Описание структуры элементов

Третий раздел палитры инструментов содержит элементы, являющиеся внутренней частью элементов, описанных во втором разделе, кроме пакетов (рис. 7.13).

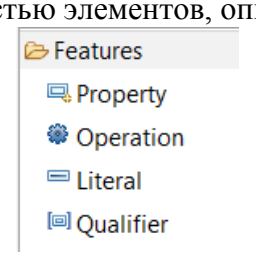


Рис. 7.13. Третий раздел палитры инструментов

Размещение одного элемента внутри другого элемента осуществляется с помощью клика на элемент из третьего раздела палитры инструментов, а затем на изображение элемента диаграммы, в который мы хотим поместить выбранный элемент.

Не всякий элемент из третьего раздела может быть помещен в элемент из второго раздела. Увидеть какие элементы можно вставить в элемент очень легко. Наведите мышку на элемент диаграммы, например, класс, и подождите одну-две секунды. Появится всплывающее меню с иконками элементов, которые могут быть размещены в классе. Видно, что класс может содержать внутри себя классы, свойства и операции (рис. 7.14).

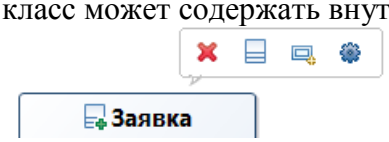


Рис. 7.14. Всплывающее меню для класса

Если навести мышку, например, на элемент перечисления (Enumeration), то увидим, что в него могут быть помещены свойства, операции и литералы (рис. 7.15).

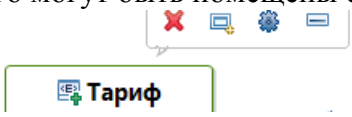


Рис. 7.15. Всплывающее меню для перечисления

Выбор элемента из всплывающего меню – это второй способ размещения элемента внутри другого элемента.

При размещении элемента внутри другого элемента он показываетс на диаграмме, а также внутри элемента в Model Explorer. Исключением являются свойства (**Property**) и операции (**Operation**), которые показываются на диаграмме только внутри классов и интерфейсов. Факт размещения свойств и операций внутри элементов данных можно увидеть только в Model Explorer.

Элемент литерал (**Literal**) используется для задания значения внутри перечислимого типа данных. Например, разместив литералы на элементе перечисления **Тариф**, можно указать какие бывают тарифы на услуги связи (рис. 7.16).

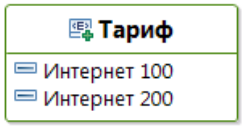


Рис. 7.16. Изображение литералов на элементе перечисления

Название тарифа указывается в данном случае в качестве имени элемента литерал (рис. 7.17).

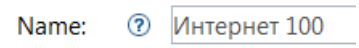


Рис. 7.17. Имя литерала

Элемент квалификатор (**Qualifier**) используется для выделения конкретного экземпляра среди некоторого набора экземпляров при описании отношения ассоциации (см. раздел 17.2 Отношения между элементами диаграммы классов).

7.3.1. Свойства элемента

Элемент свойство (**Property**) используется для описания структуры элемента. Свойства, используемые для описания класса, называют атрибутами. Свойство может быть задано у любого элемента из второго раздела палитры инструментов, кроме пакета. Свойства также есть у квалификатора (**Qualifier**).

Каждое свойство описывается с помощью набора полей, которые могут быть определены во время размещения свойства на элемента в окне **Create** или позже в виде **Properties** или окне редактирования **Edit**, которое открывается по двойному клику на изображении свойства на диаграмме (рис. 7.18).

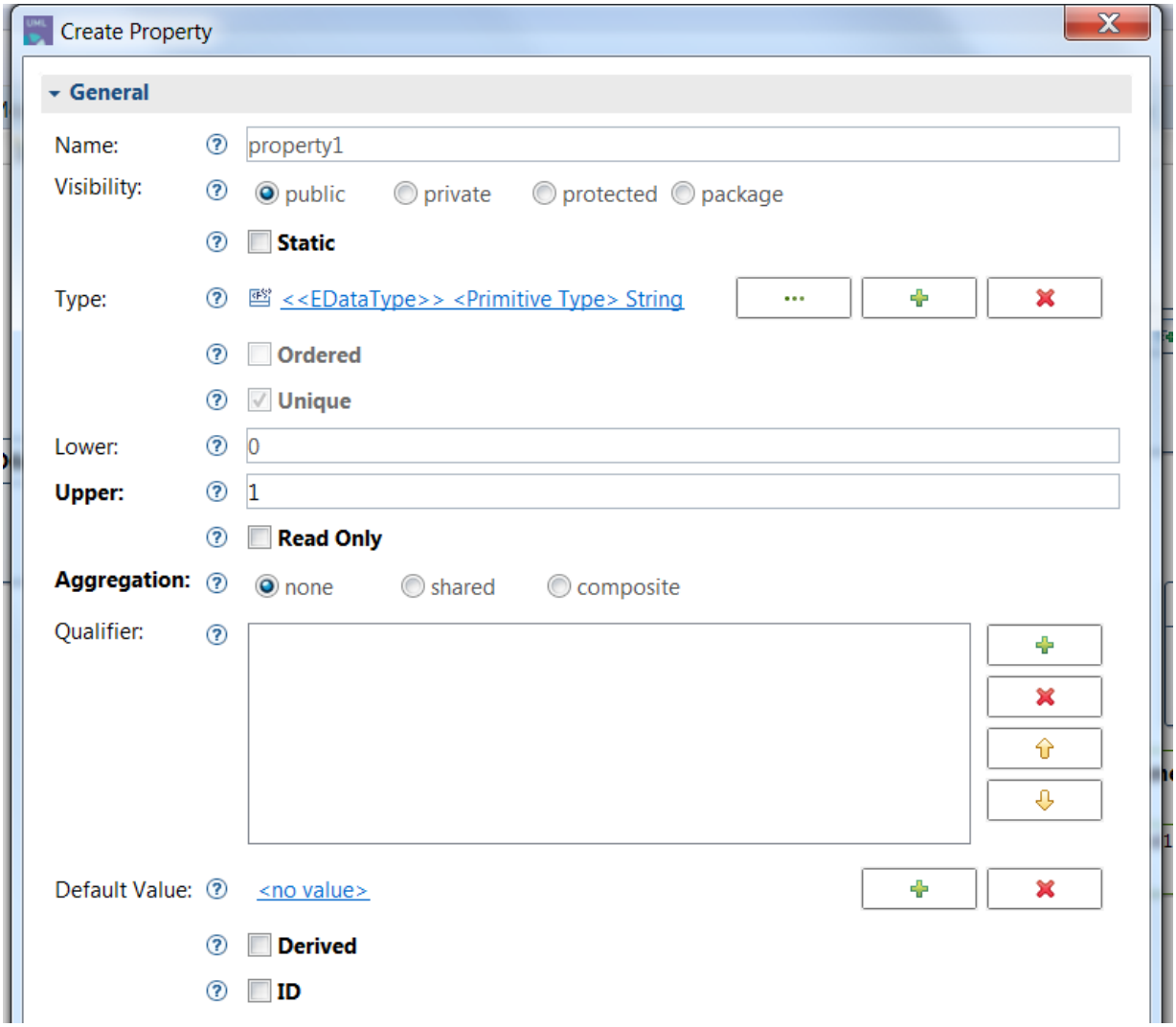


Рис. 7.18. Окно создания свойства

Имя свойства является обязательным и задается в поле **Name**.

Поле **Visibility** определяет видимость свойства. Общедоступное (**public**) свойство может быть использовано для чтения или записи в операциях данного элемента и любым другим элементом.

Закрытое (**private**) свойство недоступно для других элементов и может быть использовано только операциями данного элемента.

Защищенное (**protected**) свойство может использоваться в операциях данного элемента, оно недоступно для других элементов, но может использоваться элементами, которые связаны с данным элементом отношением обобщения (**Generalization**) и являются потомками данного элемента.

Пакетное (**package**) свойство доступно элементам, которые находятся в одном пакете с данным элементом или во вложенном подпакете.

Статическое (**Static**) свойство имеет единственное значение для всех экземпляров элемента. К статическому свойству обращаются с помощью указания имени элемента, а не конкретного экземпляра.

Тип свойства задается в поле **Type**. Тип свойства определяет множество допустимых значений свойства.

Например, тип **string** определяет, что в качестве значения свойства могут использоваться только строки символов.

Имеется достаточно большой набор примитивных типов для задания типа свойства, которые можно выбрать в окне **Select** в разделе **Ecore Primitive Types**. Окно **Select** открывается по клику на кнопку  в строке **Type** (рис. 7.19).

Рис. 7.19. Окно выбора примитивных типов

Можно воспользоваться также примитивными типами из раздела **Java Primitive Types** (рис. 7.20).

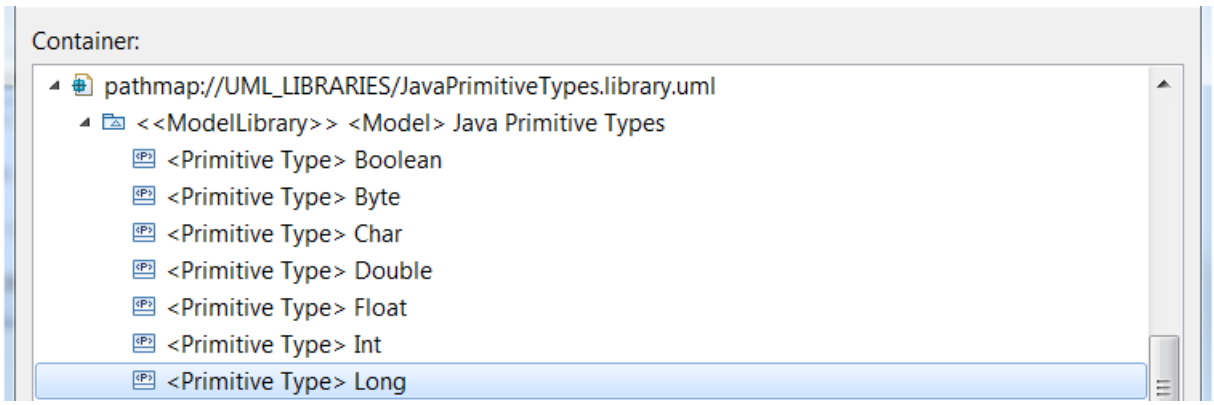


Рис. 7.20. Выбор примитивных типов Java

Предположим мы создали свой примитивный тип, например, **Фамилия** (рис. 7.21).

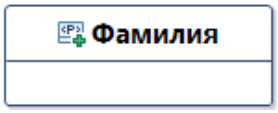


Рис. 7.21. Свой примитивный тип

Тогда при описании типа свойства **фамилия** вы можете в окне **Select** выбрать этот тип (рис. 7.22).

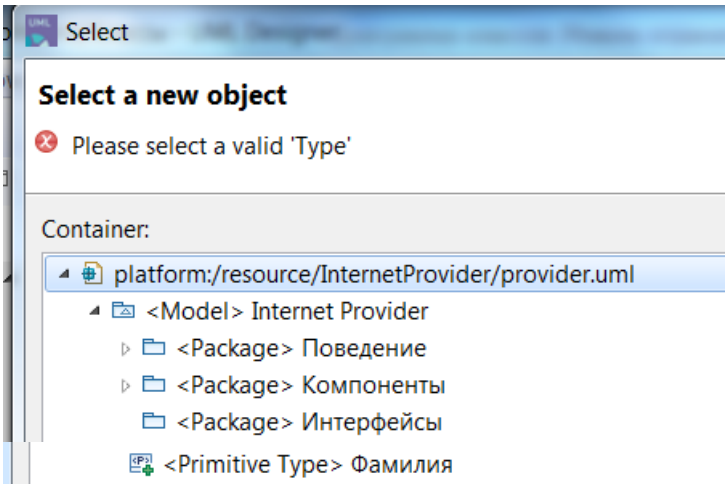


Рис. 7.22. Выбор своего примитивного типа

В качестве типа свойства можно также использовать созданные типы данных (**DataType, Enumeration**) и классы. Например, пусть мы создали типы данных **ФИО** и **Адрес**, а также перечислимый тип **Тариф** (рис. 7.23).

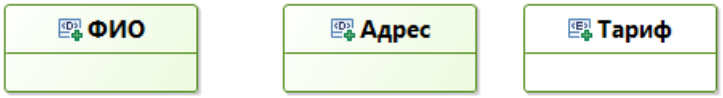


Рис. 7.23. Созданные типы

Эти типы данных можно использовать для задатия типов свойств (атрибутов) в классе **Заявка** (рис. 7.24).

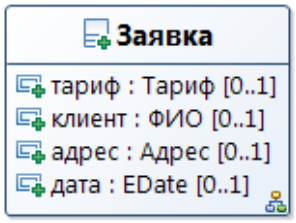


Рис. 7.24. Использование типов для атрибутов класса

Для свойства **дата** выбран примитивный тип **Data** из раздела **Ecore Primitive Types.**

Свойство имеет кратность (множественность) (multiplicity). Кратность определяет количество значений свойства. Свойство может иметь одно значение, может представлять набор значений, а может и вовсе не иметь значений. Возможное количество значений задается интервалом, для которого указывается нижняя (**Lower**) и верхняя (**Upper**) граница. Указание нуля в качестве нижней границы означает, что свойство может не иметь значений. Указание в качестве значения -1 означает, что свойство может иметь произвольное количество значений. На диаграмме и в Model Explorer вместо -1 появляется символ *.

Если вы посмотрите на описание некоторого свойства в Model Explorer, то увидите два дополнительных элемента **<Litel Integer> 0** и **<Literal Unlimited Natural> 1** (рис. 7.25).

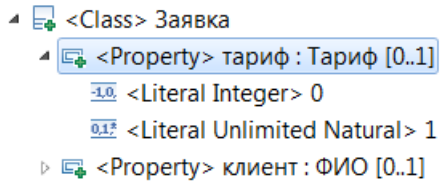


Рис. 7.25. Элементы свойства для задания Lower и Upper

Эти элементы не имеют имен, но имеют в данном примере значения 0 и 1, которые равны значениям **Lower** и **Upper** свойства **тариф**, соответственно. Это можно увидеть в Properties (рис. 7.26).

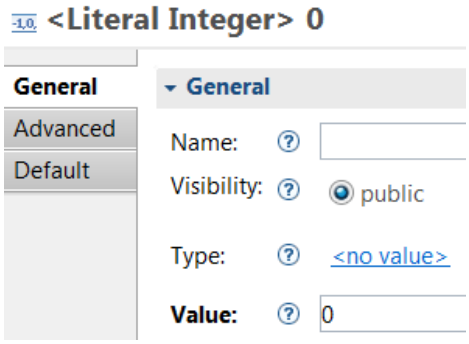


Рис. 7.26. Описание Lower

Набор значений является множеством, если все значения уникальны. Уникальность задается в поле **Unique**.

Поле **Ordered** указывает, что значения в наборе упорядочены.

Поле **Read Only** означает, что свойство доступно только для чтения и его значение не может быть изменено.

Поле агрегации (**Aggregation**) указывает, что данное свойство используется для агрегации экземпляров элементов. Например, для выполнения заявки клиента потребуется различное оборудование (например, wi-fi маршрутизатор, кабели, розетки). Тогда можно создать свойство, в котором будет хранится множество идентификаторов объектов класса **Оборудование.** Понятно, что при уничтожении заявки само оборудование не уничтожается. В этом случае значение поля агрегации равно **shared.** А теперь предположим, что для выполнения заявки требуется закупить некоторое оборудование. Список закупаемого оборудования можно хранить как объекты класса **Закупаемое оборудование.** В самом объекте заявка можно создать свойство агрегации, в котором будем хранить идентификаторы объектов закупаемого оборудования. В данном случае, если заявка отменяется, то нужно уничтожить не только заявку, но и все объекты из списка закупаемого оборудования. В этой ситуации для поля агрегации укажем значение **composite**.

Поле квалификации (**Qualifier**) используется для показа квалификаторов (использование квалификатора для отношения ассоциации в разделе 17.2 Отношения между элементами диаграммы классов).

Поле значение по умолчанию (**Default Value**) позволяет задать начальное значение элемента, которое призваивается свойству элемента при создании экземпляра элемента.

Поле производный (**Derived)** определяет, что значение свойства может быть вычислено на основе другой информации. Такие свойства обычно доступны только для чтения.

Например, можно указать в заявке свойство **Дата подключения** и свойство **Время задержки**. Предположим, что в назначенную дату клиенту не было установлено оборудование связи, тогда **Время задержки** будет определять количество дней, которые прошли с планируемой даты подключения клиента. Понятно, что его можно вычислить по дате подключения и текущей дате.

Поле идентификатор (**ID)** определяет, что данное свойство выступает в качестве идентификатора или его части, т.е. по значению этого свойства (или набору значений всех

свойств, определенных как ID) можно однозначно найти экземпляр элемента. Например, в классе **Заявка** можно определить свойство **Номер заявки**, которое будет служить идентификатором заявки. Идентификатор элемента называют также ключом. Хотя обычно идентификатором, в отличие от ключа, называют одно уникальное свойство, которое в качестве значения хранит некоторое число.

7.3.2. Операции элемента

Элемент операция (**Operation**) используется для описания действий, которые выполняют сами экземпляры элементов и которые можно выполнить с экземплярамии элементов. Операции могут иметь классы, интерфейсы, элементы данных, перечисления и примитивные типы.

Каждая операция описывается с помощью набора полей. При размещении операции на элементе появляется окно **Create Operation** (рис. 7.27).

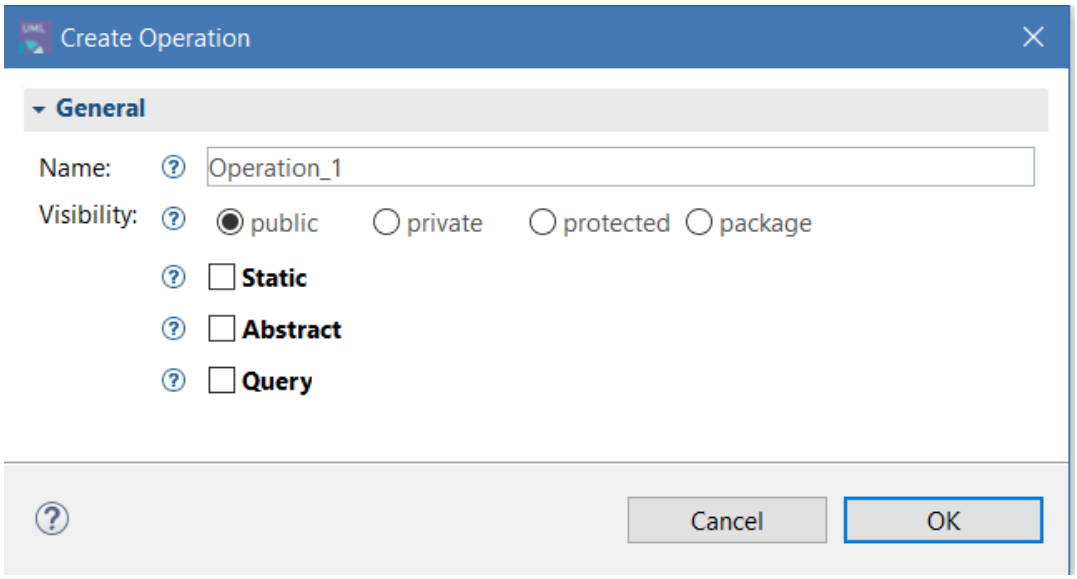


Рис. 7.27. Окно создания операции

Имя операции (**Name**) является обязательным для заполнения полем.

Поле видимости (**Visibility**) определяет возможность доступа к операции из других элементов аналогично определению видимости для свойств элементов.

Статическая операция (**Static**) вызывается для всего элемента, а не для его экземпляра. Она работает со статическим свойствами элемента. В частном случае ей можно передать указатель конкретного экземпляра для работы с его свойствами.

Абстрактная операция (**Abstract**) не имеет реализации в данном элементе. Она реализуется в потомках данного элемента.

Операция вида запрос (**Query**) не изменяет свойства самого элемента и других элементов, т.е. ее вызов не влияет на изменение состояния элементов и всей системы в целом.

После создания операции для нее могут быть указаны дополнительные поля (параметры, стереотип и комментарий) в виде **Properties** или окне редактирования **Edit**, которое открывается по двойному клику на изображении операции на диаграмме.

Параметры добавляются в поле **Parameters** с помощью кнопки (рис. 7.28).



Рис. 7.28. Добавление параметров

В появляющемся окне **Create** доступен только один тип для параметра (рис. 7.29).

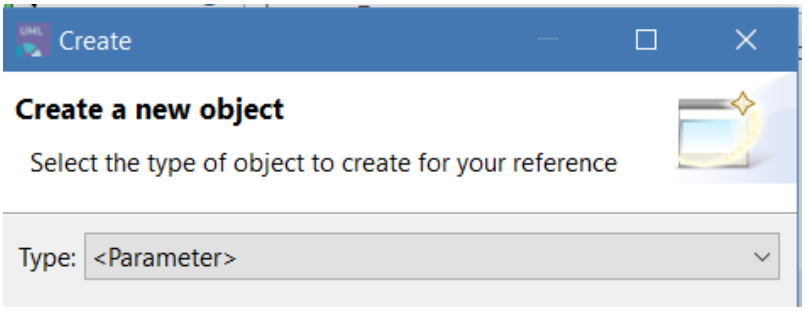


Рис. 7.29. Выбор типа параметра

После нажатия кнопки Finish в окне Create параметр можно описать подробно. Переход к описанию параметра осуществляется по двойному клику на строку <Parameter> в окне Edit или виде Properties операции (рис. 7.30).



Рис. 7.30. Показ параметров в Properties и Edit

Можно также перейти к описанию параметра, кликнув на строку <Parameter> в Model Explorer (рис. 7.31).

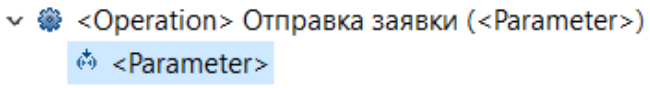


Рис. 7.31. Показ параметра в Model Explorer

Поля описания параметра имеют тоже назначение, что и поля свойства элемента (рис. 7.32).

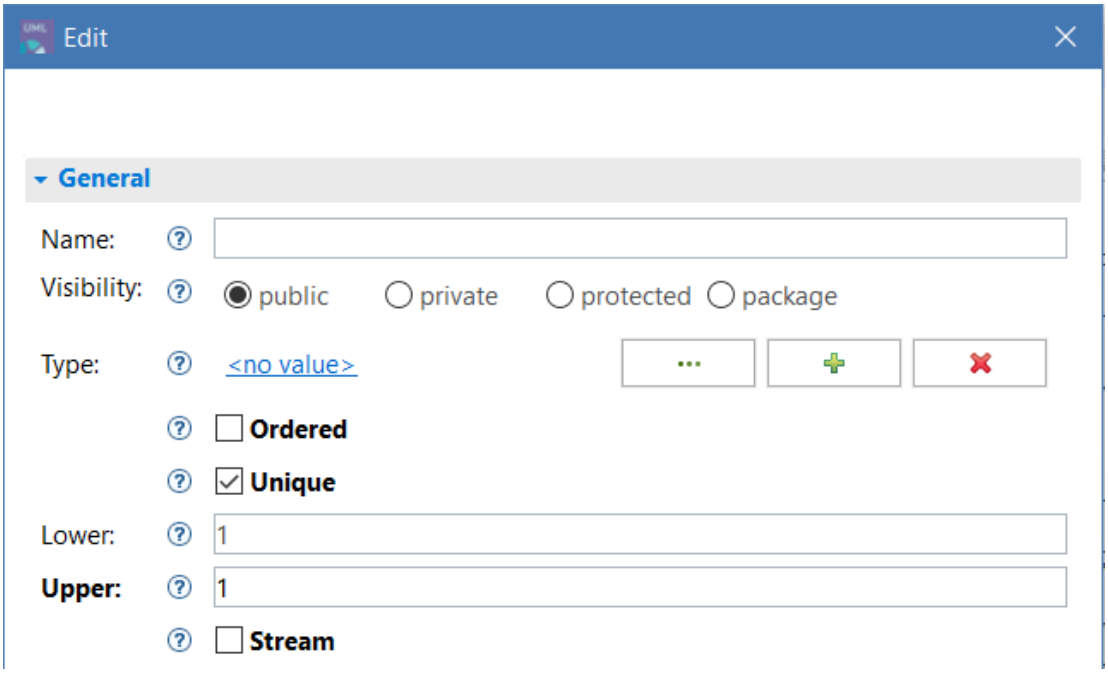


Рис. 7.32. Поля параметра в окне редактирования

Новым является поле **Stream**, которое говорит, что операция может принимать входной параметр в процессе ее выполнения, а не только при ее вызове, и может выдать выходной параметр также в процессе выполнения операции, а не только по ее завершению.

В качестве стереотипа операции могут быть указаны, например, стереотип **Create** для операции, создающей экземпляр элемента, и стереотип **Destroy** для операции, уничтожающей экземпляра элемента.

У параметра есть еще ряд полей, которые задаются в разделе Advanced вида Properties.

Направление параметра (**Direction**) определяет передается ли параметр в операцию (**in**), из операции (**out**), является ли одновременно входным и выходным (**inout**), возвращаемым (**return**) (рис. 7.33).



Рис. 7.33. Задание направления параметра

Операция, являющаяся функцией, возвращает обычно одно значение. Но в некоторых языках программирования (например, Python) функция может возвращать несколько значений. Результат работы функции описывается в UML как набор возвращаемых параметров.

7.4. Отношения между элементами диаграммы классов

Отношения между элементами диаграммы устанавливаются с помощью элементов, представленных в четвертом разделе палитры инструментов (рис. 7.34).

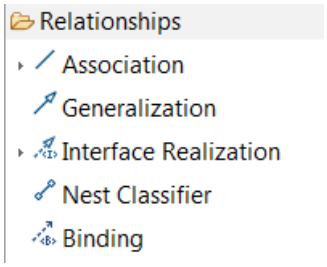


Рис. 7.34. Четвертый раздел палитры инструментов

В разделе часть отношений сгруппированы и поэтому сразу не видны в палитре инструментов.

Первая группа отношений – это отношение ассоциации (Association) и различные частные случаи ассоциации. Отношения этой группы показываются по клику на значок треугольника слева от элемента Association (рис. 7.35).

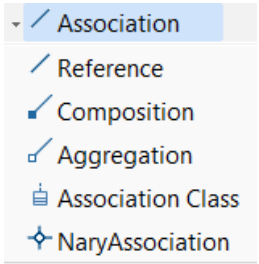


Рис. 7.35 Отношения ассоциации

Если выбрать одно из отношений этой группы, то список отношений будет свернут и первым отношением в группе станет выбранное отношение. Например, при выборе Reference мы увидим (рис. 7.36).

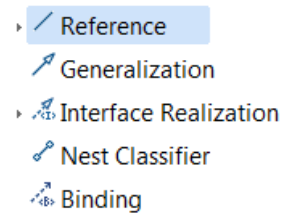


Рис. 7.36. Изменение имени отношения для группы отношений

Вторая группа отношений – это отношение реализации интерфейса (Interface Realization) и другие виды зависимостей между классами. Отношения этой группы показываются по клику на значок треугольника слева от элемента Interface Realization (рис. 7.37).

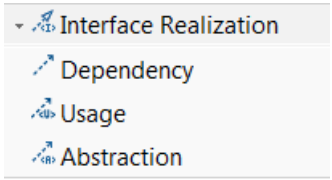


Рис. 7.37. Отношения зависимости

Отношения можно устанавливать, между классами, интерфейсами и элементами данных. В объектно-ориентированных языках программирования элементы данных обычно также реализуются как классы, поэтому далее будем рассматривать отношения на примере отношений между классами. Интерфейс – это тоже класс.

Большинство отношений являются бинарными, т.е. связывают два класса. Для установки бинарного отношения между классами кликните на элемент отношения в палитре инструментов, а затем на классы. Для некоторых отношений, иконки которых содержат линию со стрелкой или специальным значком на одной стороне линии, важен порядок в котором выбираются классы. Стрелка отношения направлена от первого выбранного класса ко второму выбранному классу. А значок, наоборот, устанавливается на стороне первого выбранного класса.

7.4.1. Отношения ассоциации

Отношение ассоциации (**Association**) является наиболее общим отношением между классами. Оно говорит о том, что есть какая-то связь между объектами этих классов. Например, для выполнения заявки требуется оборудование, клиент имеет счет в банке, с помощью которого он оплачивает подключение к услугам связи.

Ассоциация между двумя классами, например, **Заявка** и **Оборудование** по умолчанию будет иметь вид (рис. 7.38).

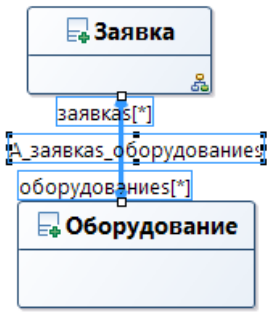


Рис. 7.38. Показ отношения ассоциации на диаграмме

Изменить описание отношения можно в виде Properties или окне Edit, которое открывается по двойному клику на линию, соединяющую классы, или на одно из имен описания отношения на диаграмме (рис. 7.39).

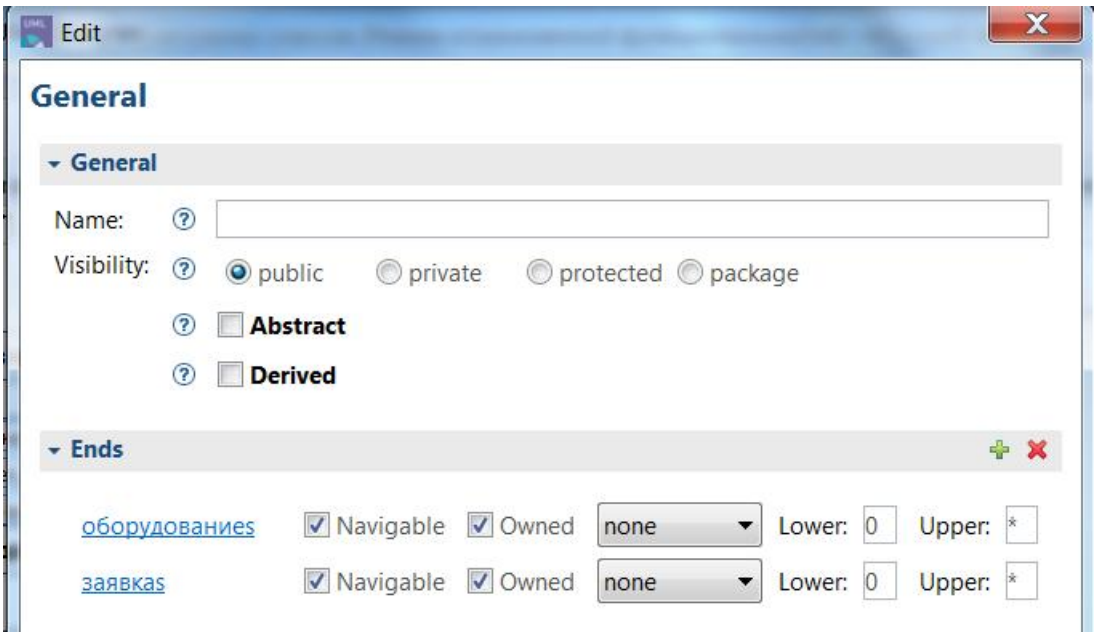


Рис. 7.39. Окно редактирования отношения ассоциации

Имя отношения (**Name)** – это глагол в предложении, которое описывает данное отношение. Например, имя **Требует установки** будет означать, что заявка требует установки оборудования. Можно прочитать это предложение в обратном порядке: оборудование требуется для установки. В этом случае имя будет **Требуется для установки**. Обратите внимание, что имя, которое показывается на диаграмме по умолчанию, не видно в поле **Name.** Если вы хотите, чтобы отношение не имело имени, то введите пробел в поле Name.

На концах линии отношения указывают роли, которые выполняют объекты классов в данном отношении. Для изменения имени роли кликните на одно из имен ролей в разделе **Ends** окна **Edit**, например, **оборудованиеs.** В появившемся окне укажите имя роли (рис. 7.40).

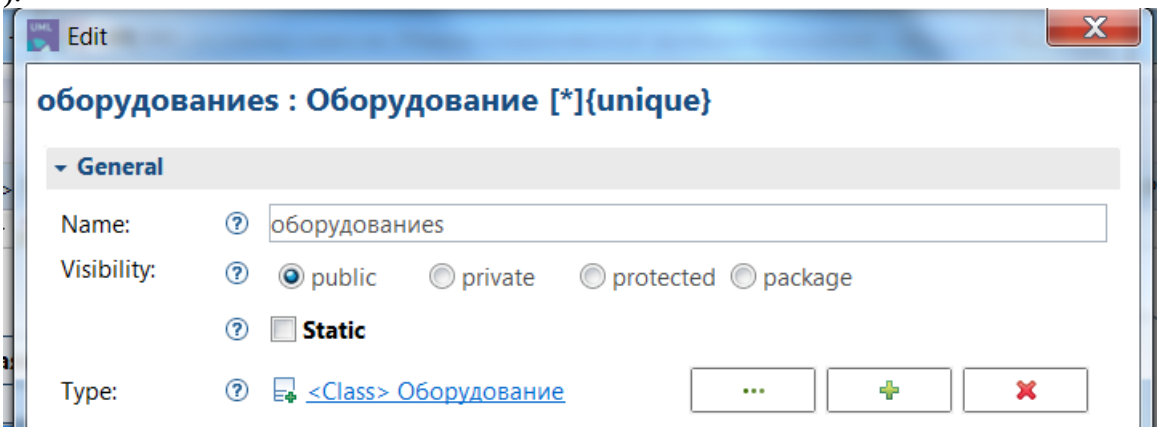


Рис. 7.40. Указание имени роли

Так будет выглядеть ассоциация после указания имен ролей на диаграмме (рис. 7.41).

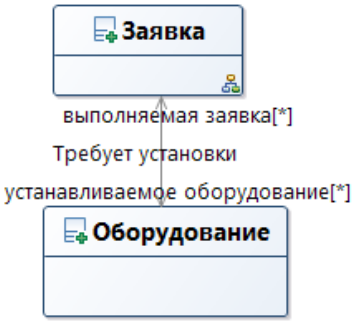


Рис. 7.41. Отношение ассоциации с ролями

В Model Explorer увидим описание ассоциации (рис. 7.42).

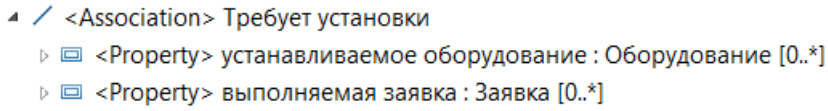


Рис. 7.42. Описание ассоциации в Model Explorer

Как правило, одновременно имена ролей и имя отношения не указываются на диаграмме. Если удалить имя роли в окне **Edit** описания роли, то вновь указать имя роли можно будет только в виде Properties, кликнув на строку **<Property>** с нужной ролью в Model Explorer.

Для каждой роли в полях **Lowe**r и **Upper** указывается нижня и верхняя границы количества объектов, которые участвуют в экземпляре отношении. В данном случае при выполнении заявки устанавливается несколько экземпляров оборудования (Upper = * или -1) или не устанавливается вовсе (Lower = 0). Одно и тоже оборудование может использоваться при выполнении разных заявок или не использоваться вовсе.

Навигационный признак (**Navigable),** указанный для роли некоторого объекта, означает, что объекты данного класса знают о существовании объектов другого класса, участвующего в ассоциации.

Например, уберем это признак для роли **выполняемая заявка** (рис. 7.43).

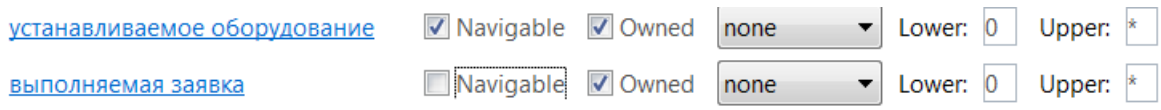


Рис. 7.43. Изменение признака Navigable

Это будет означать, что объект заявка знает об устанавливаемом оборудовании, а объект оборудования – нет. Говорят, что существует навигация между заявкой и ее оборудованием. В объекте класса **Заявка** будет храниться ссылка на устанавливаемое оборудование в виде набора идентификаторов объектов класса **Оборудование**. На том конце, где установлен признак **Navigable**, рисуется стрелка. На диаграмме стрелка отношения будет направлена от класса **Заявка** в сторону класса **Оборудование** (рис. 7.44).

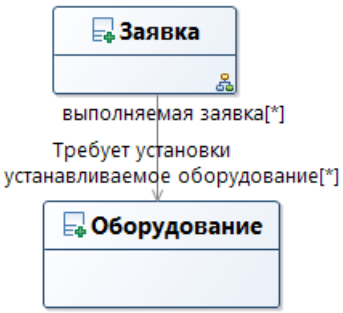


Рис. 7.44. Однонаправленная навигация

Также навигация означает, что объекты одного класса могут посылать сообщения объектам другого класса или вызывать их операции. Отправка сообщения выполняется в направлении стрелки.

Для того, чтобы UML Designer записал в класс **Заявка** ссылку на **Оборудование** уберите галочку в поле **Owned** для оборудования (рис. 7.45).

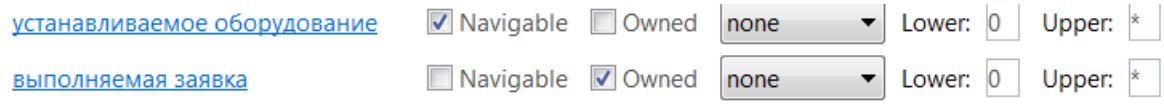


Рис. 7.45. Добавление ссылки в класс

В Model Explorer в классе **Заявка** увидим новое свойство **устанавливаемое оборудование** (рис. 7.46).

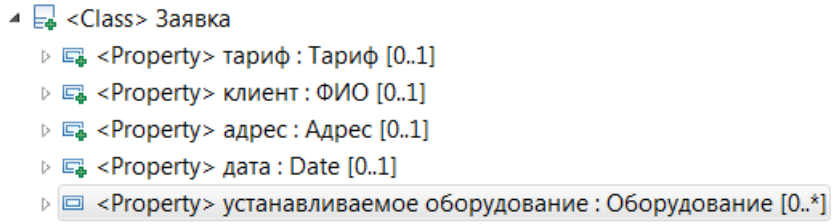


Рис. 7.46. Ссылка на **Оборудование**

Если мы хотим сразу установить навигацию только в одну сторону, то вместо отношения ассоциация в палитре инструментов можно выбрать отношение ссылки (**Reference**). При установке отношения ссылки сначала нужно кликнуть на класс, из которого исходит стрелка, а потом на класс, в который входит стрелка. В модели UML элемент ссылки будет иметь как и отношение ассоциации класс <Association>, но признак в поле **Navigable** будет установлен только для одного конца отношения, т.е. для одной роли. В предыдущем примере вместо установки отношения ассоциации между заявкой и ее оборудованием можно было бы использовать отношение ссылки.

Ассоциация может быть как бинарной, так и n-арной, если она связывает более двух классов. Например, тернарная ассоциацию между классами **Оборудование**, **Счет**, **Заявка** означает, что оборудование, устанавливаемое для выполнения некоторой заявки, оплачивается по счету.

Для создания тернарной ассоциации нужно установить сначала бинарную ассоциацию между любой из пары классов, участвующих в ассоциации, потом снова выбрать элемент ассоциации в палитре инструментов, а затем выбрать класс и установленное отношение на диаграмме (в любом порядке). В результате увидим следующее на рис. 7.47.

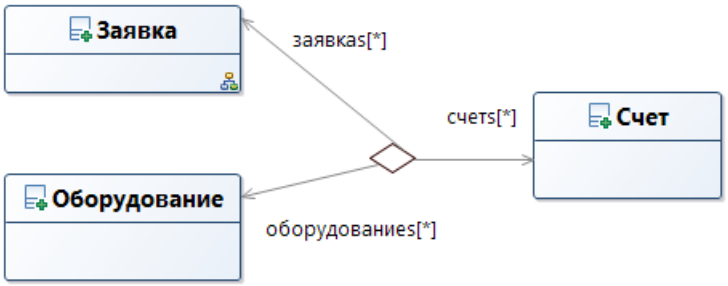


Рис. 7.47. Тернарная ассоциация

Можно создать N-арную ассоциацию с помощью отношения N-арная ассоциация (**NaryAssociation**). Кликните на этот элемент в палитре инструментов, затем на любую пару классов. После клика на второй класс появится окно, в котором можно выбрать еще произвольное количество классов, участвующих в данной ассоциации (рис. 7.48).

Ассоциация (в общем случае N-арная) может иметь атрибуты. Например, для приведенной тернарной ассоциации нужно указать сумму и дату оплаты. Эти атрибуты нельзя указать непосредственно для ассоциации. Для этой цели используется отношение ассоциативный класс (**Association Class**).

Удалим предыдущую тернарную ассоциацию. Кликнем на одну из линий и удалим ее через меню, открывающееся по правой кнопке мышки. Тернарная ассоциация станет бинарной. Далее удалим оставшуюся ассоциацию.

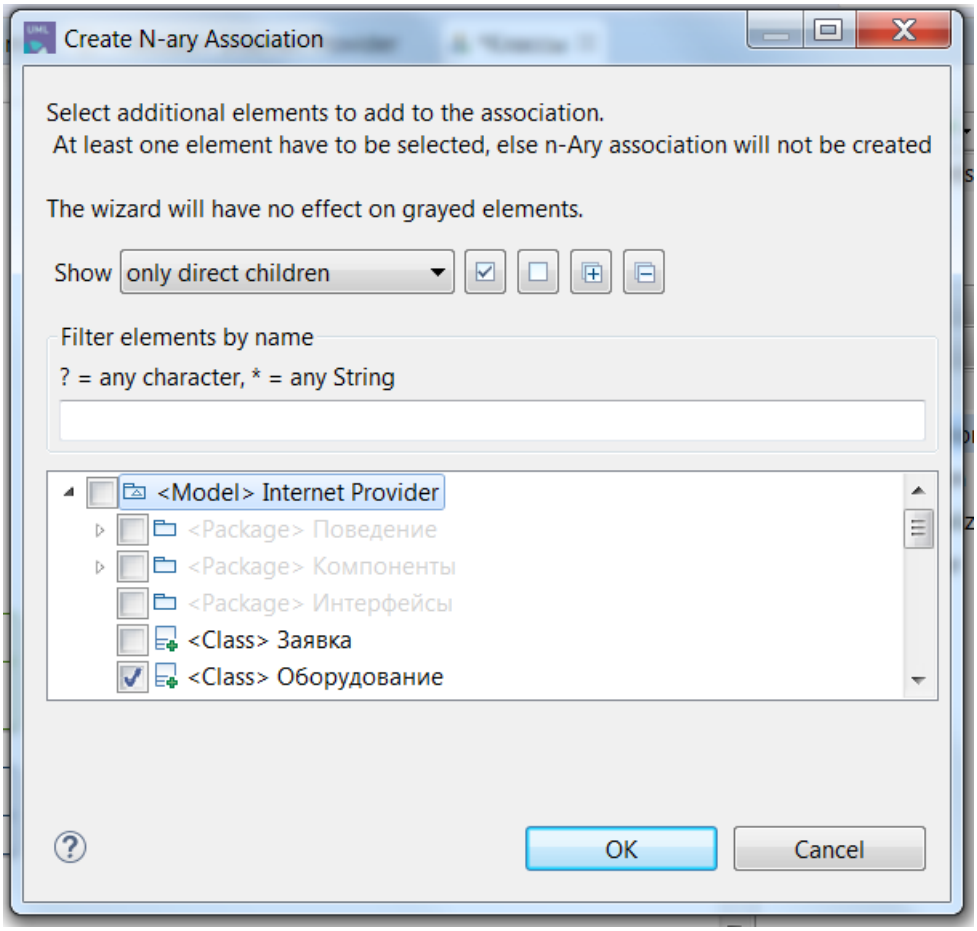


Рис. 7.48. Окно создания N-арной ассоциации

Теперь создадим ассоциацию заново, выбрав в палитре инструментов элемент **Association Class** и последовательно кликнем на пару из трех классов, например, **Оборудование и Счет** (рис. 7.49).

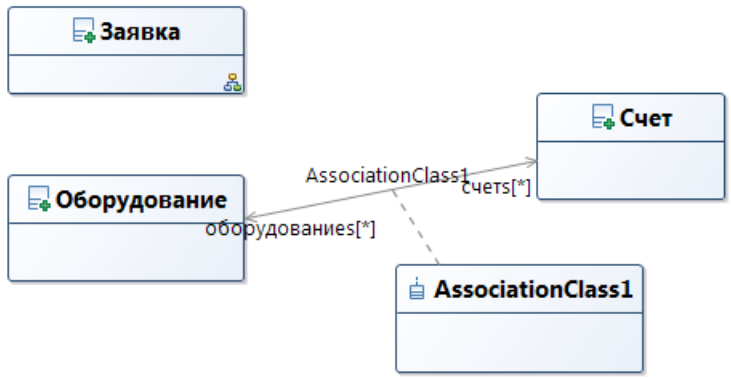


Рис. 7.49. Ассоциативный класс для бинарной ассоциации

Класс **AssociatinClass1** создается автоматически. Переименуем его в **Оплата** и добавим в него атрибуты **Сумма** и **Дата**. Далее соединим отношением ассоциации класс **Заявка** и созданное отношение **AssociationClass1** (рис. 7.50).

Если в отношении ассоциации на одной стороне участвует 0 или один объект, а на другой множество объектов (*), то говорят, что это отношение вида «один ко многим». Если же на обоих сторонах отношения может быть произвольное множество объектов, то говорят, что это отношение вида «многие ко многим». В рассмотренном примере отношение **Требует установки** является отношением «многие ко многим». Следует отметить, что таким оно является в силу предположения, что указывается некоторый тип оборудования (например,

wi-fi маршрутизатор фирмы Cisco), а не конкретный экземпляр этого оборудования. Понятно, что оборудование некоторого типа может быть установлено у разных клиентов.

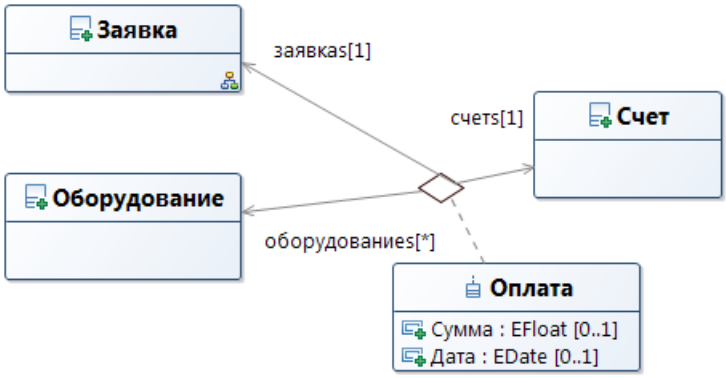


Рис. 7.50. Ассоциативный класс для тернарной ассоциации

В примере тернарной ассоциации оплачивается некоторый набор оборудования для одной заявки по одному счету, поэтому на стороне оборудования указывается символ *, а на стороне заявки и счета значение 1.

Предположим, что для выполнения заявки было закуплено некоторое оборудование и мы хотим хранить информацию о нем с точностью до конкретного экземпляра оборудования с указанием его идентификационного номера. В этом случае мы получим отношение вида «один ко многим», т.к. конкретный экземпляр оборудования был куплен для выполнения одной заявки. Хотя ранее было отмечено, что имена ролей и имя ассоциации обычно одновременно не указываются, но в данном случае они нам понадобятся, чтобы понимать с каким отношением и его окончанием мы работаем (рис. 7.51).

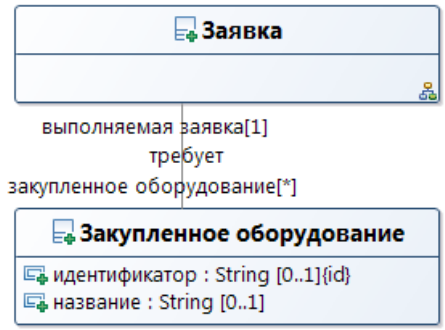


Рис. 7.51. Отношение «один ко многим»

Конкретная заявка использует множество объектов класса **Закупленное оборудование**. Для того, чтобы выделить из этого подмножества только один объект оборудования используется квалификатор (**Qualifier**). Таким квалификатором является ключ объекта класса **Закупаемое оборудование**, т.е. атрибут **идентификатор**. Ключ – это набор атрибутов, по значениям которых можно однозначно найти объект класса.

При размещении квалификатора на отношении (нужно кликнуть на элемент **Qualifier** в палитре, а затем на линию отношения) появляется окно, в котором выбираем роль объекта, на стороне которого мы хотим установить квалификатор (в данном случае - это **выполняемая заявка**), и нажимаем кнопку **OK** (рис. 7.52).

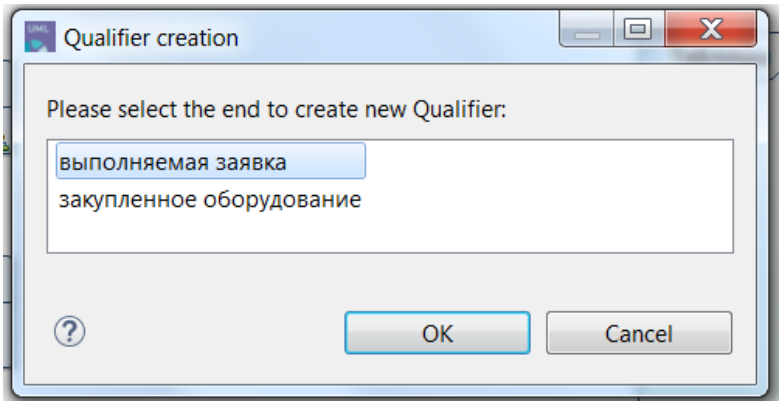


Рис. 7.52. Создание квалификатора

Появляется окно в котором указано имя ассоциации (рис. 7.53). Ничего не меняя в этом окне, нажимаем кнопку **OK**.

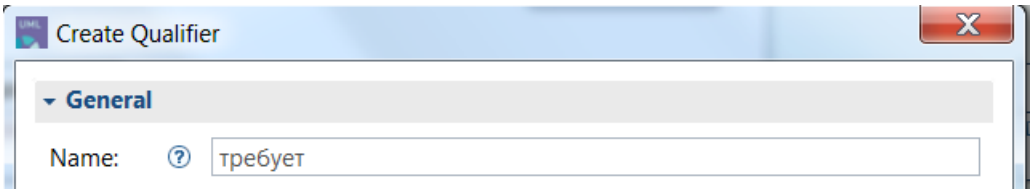


Рис. 7.53. Имя ассоциации при создании квалификатора

На диаграмме появляется квалификатор с именем qualifier1 (рис. 7.54).

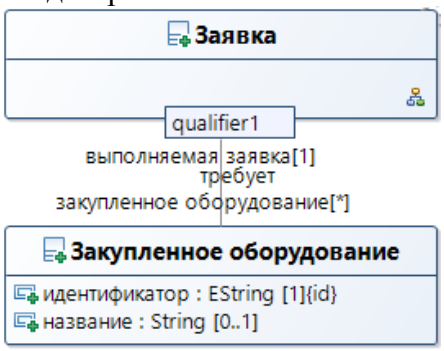


Рис. 7.54. Изображение квалификатора на диаграмме

В описании роли **выполняемая заявка** этот квалификатор появляется в поле **Qualifier** (рис. 7.55).

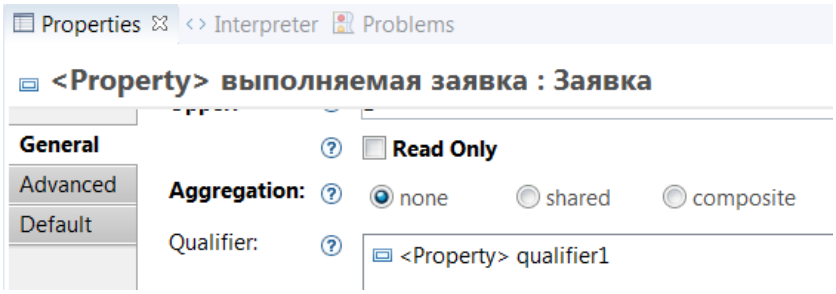


Рис. 7.55. Описание квалификатора в виде Properties

Данный квалификатор относится не к классу **Заявка**, а к концу ассоциации, который задается с помощью роли объекта класса **Заявка** при его участии в ассоциации. Это хорошо видно в Model Explorer (рис. 7.56).

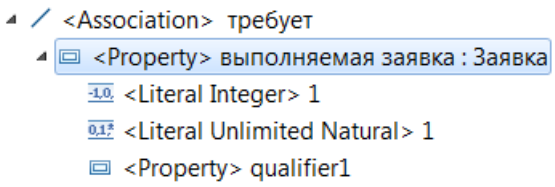


Рис. 7.56. Описание квалификатора в Model Explorer

Далее нужно описать квалификатор. Кликнем на его имя в Model Explorer и виде Properties укажем его имя, равное имени атрибута **идентификатор** класса **Закупленное оборудование** (рис. 7.57).

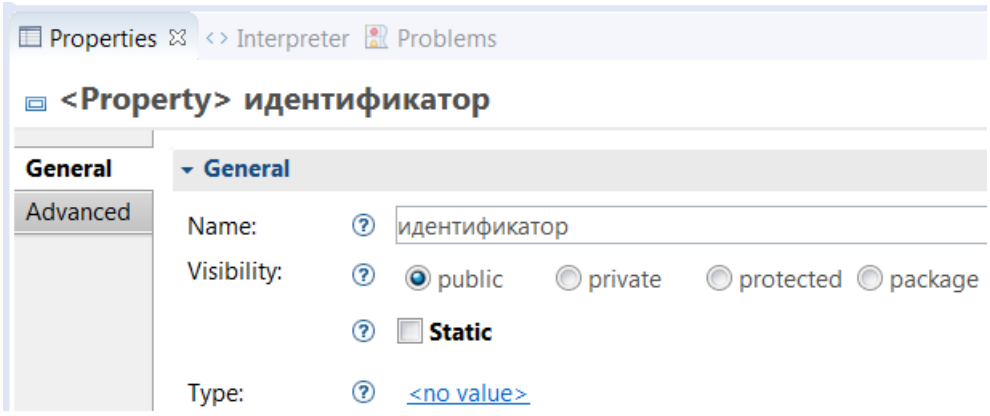


Рис. 7.57. Изменение имени квалификатора

При описании квалификатора можно указать и другие поля, кроме имени, например, тип. В этом случае, правда, нужно следить, чтобы описание квалификатора совпадало с описанием атрибута **идентификатор**.

На диаграмме мы увидим показанное на рис. 7.58.

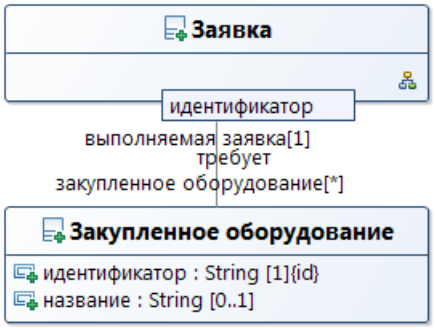


Рис. 7.58. Квалификатор **идентификатор** на диаграмме

Если бы ключ в классе **Закупленное оборудование** состоял из нескольких атрибутов, то тогда нужно было бы добавить еще квалификаторы, чтобы набор квалификаторов соответствовал набору атрибутов ключа.

Следует отметить, что изображение квалификаторов на диаграмме не обязательно. В качестве квалификатора выступает ключ класса. В некоторых случаях для сокращения изображения мы можем скрыть атрибуты класса. Тогда, чтобы показать по какому ключу выбираются объекты из множества объектов, может быть использован квалификатор.

Отношение агрегации (**Aggregation**) и композиции (**Composition**) являются частными случаями отношения ассоциации и используются для показа того факта, что объект некоторого класса состоит из некоторых частей, которые являются объектами другого класса. Отличием композиции от агрегации является то, что при удалении агрегированного объекта его части не удаляются, а при удалении объекта, части которого связаны с ним с помощью композиции, эти части удаляются. В модели UML, и агрегация, и композиция будут иметь, как и отношение ассоциации, класс <Association>. Вместо значения **None** в

описании роли на стороне класса, объекты которого являются частями другого объекта, мы увидим в случае агрегации значение **shared**, а вслучае композиции – **composite**. При установке отношения сначала нужно кликнуть на класс, объекты которого состоят из частей, а потом на класс, объекты которого являются частями.

Например, форма ввода данных для подачи заявки может содержать несколько полей для ввода информации и кнопки подачи заявки и отказа от заявки. Эти классы можно связать с помощью отношений композиции, так как при удалении формы будут удалены и составляющие ее объекты (рис. 7.59).

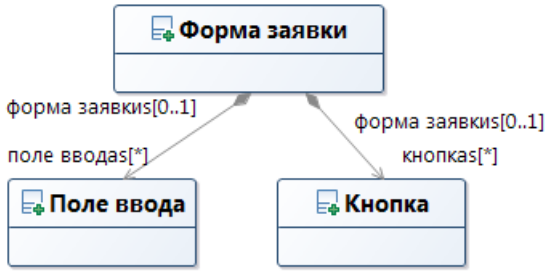


Рис. 7.59. Отношения композиции

В Properties композиция между формой и полем описывается с помощью двух ролей, одна из которых имеет признак composite (рис. 7.60).

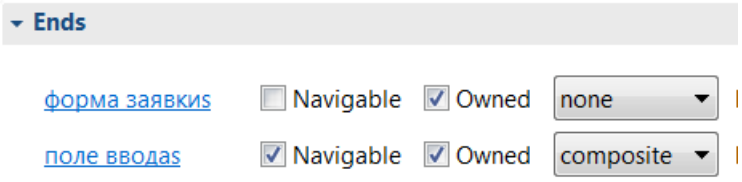


Рис. 7.60. Описание композиции между классами **Форма заявки** и **Поле ввода**

Обратите внимание, что существует навигация (стрелка) в направлении от формы к полю. Форма знает о том, какие поля на ней размещены, а сами поля – нет.

Примером агрегации является группа техников, которые выезжают к клиенту для установки оборудования. Каждый техник является частью группы (рис. 7.61).

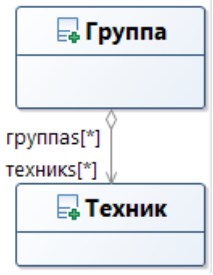


Рис. 7.61. Отношение агрегации

В Properties агрегация между группой и техником описывается с помощью двух ролей, одна из которых имеет признак shared (рис. 7.62).



Рис. 7.62. Описание агрегации между классами **Группа** и **Техник**

Обратите внимание, что по умолчанию на диаграмме поле может быть связано не более чем с одной формой (в роли **форма заявкиs** указано [0..1]), т.к. без формы его не существует, а вот техник может быть задействован в нескольких группах (в роли **группаs** указано [*]).

7.4.2. Отношение обобщения

Отношение обобщения (**Generalization**) используется для связи класса и наследуемого от него класса. Класс потомок (дочерний класс) наследует все атрибуты и операции класса предка (родителя). Для установки отношения сначала кликните на класс потомок, а потом на его предка. Стрелка отношения будет направлена в сторону предка.

Мы уже применяли отношение обобщения на диаграмме вариантов использования, например, для связи актеров. Если сведения об актерах хранятся в системе, то им соответствуют классы, которые также будут связаны между собой отношением обобщения.

Например, клиент, менеджер и техник являются пользователями системы. Их общие атрибуты и операции можно поместить в родительский класс **Пользователь** (рис. 7.63).

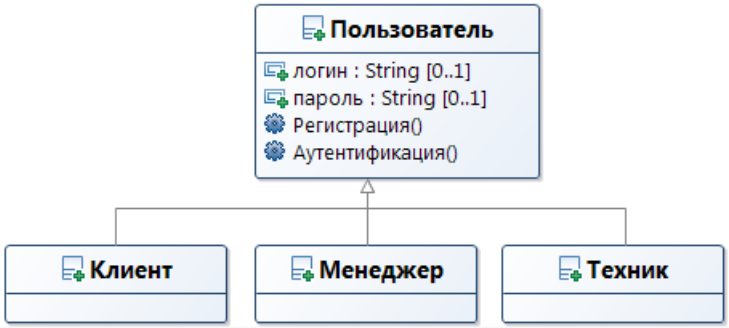


Рис. 7.63. Отношения обобщения

7.4.3. Отношение реализации интерфейса

Отношение реализация интерфейса (**Interface Realization**) предназначено для связи между интерфейсом и классом, в котором реализованы операции, перечисленные в интерфейсе. В общем случае для реализации интерфейса может потребоваться несколько классов. Для установки отношения нужно сначала кликнуть на класс, а потом на интерфейс.

Например, можно создать интерфейс и класс для работы с таблицами (рис. 7.64).

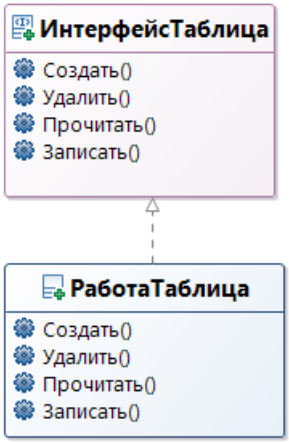


Рис. 7.64. Отношение реализации интерфейса

Описание операций отношения появляется в Model Explorer внутри класса (рис. 7.65).

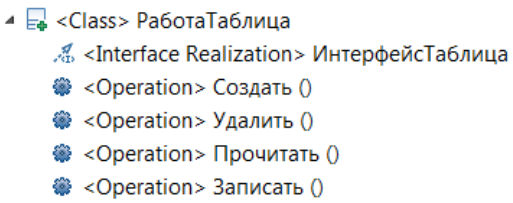


Рис. 7.65. Описание операций в Model Explorer

7.4.4. Отношения зависимости

Отношение зависимости (**Dependency**) показывает, что один класс (клиент) зависит от изменений, происходящих в другом классе, называемом поставщиком или источником зависимости.

Например, при описании класса Заявка используются типы данных **ФИО**, **Адрес** и перечисление **Тариф** (рис. 7.66).

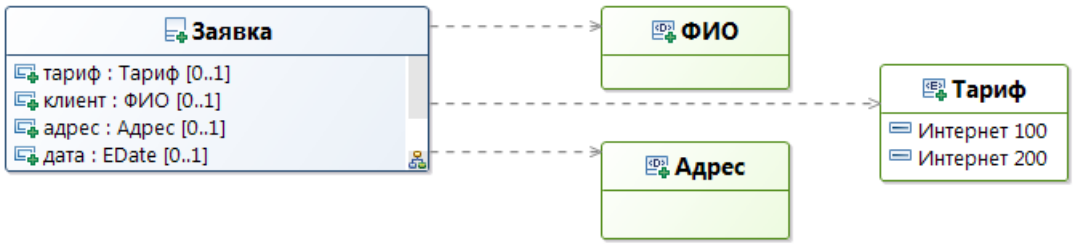


Рис. 7.66. Отношения зависимости между классом и типами данных

На диаграмме не видны свойства элементов данных **ФИО** и **Адрес**, но их всегда можно увидеть в Model Explorer. Например, тип данных ФИО имеет свойства, показанные на рис. 7.67.

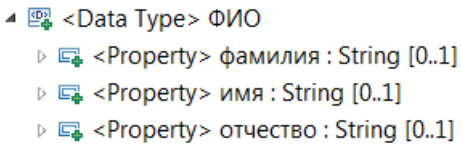


Рис. 7.67. Свойства элемента данных

Отношение зависимости также может показывать использование клиентом данных и операций поставщика. В этом случае используется частный случай отношения зависимости, которое называется использованием (**Usage**). Так как, обычно операции предоставляются через интерфейс, то с помощью этого отношения можно показать, например, что таблица **Заявки** обращается к операциям интерфейса **ИнтерфейсТаблица** для выполнения общих операций с таблицей (рис. 7.68).

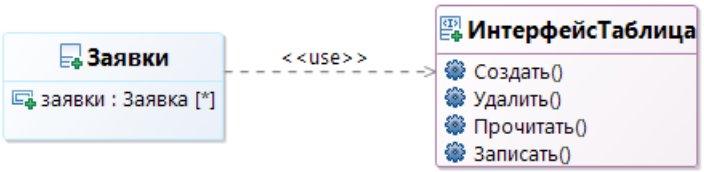


Рис. 7.68. Отношение зависимости между классом и интерфейсом

Отношение абстракции (**Abstraction**) также является частным случаем отношения зависимости и показывает, что поставщик зависимости является более абстрактным представлением клиента зависимости. Например, на начальном этапе проектирования системы был создан класс **Заявка1**, который в ходе дальнейшего проектирования был разработан более подробно (рис. 7.69).

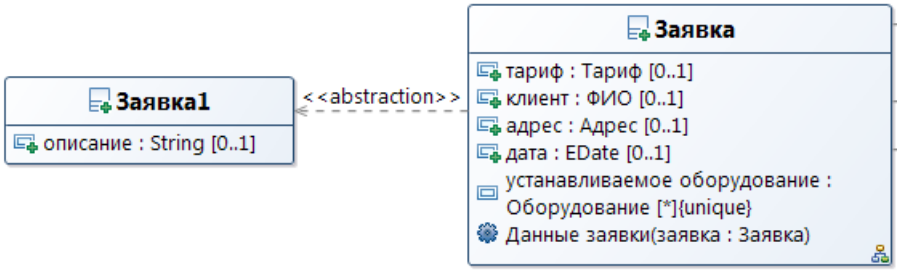


Рис. 7.69. Отношение абстракции

Другим примером абстракции является отношение между некоторым классом и разными его реализациями. Например, одна реализация отличается от другой тем, что были выполнены изменения в коде операций с целью их оптимизации, например, по времени выполнения.

Отношения зависимости, использования и абстракции в модели находятся на одном уровне с классами, которые они соединяют. Это видно в Model Explorer (рис. 7.70)

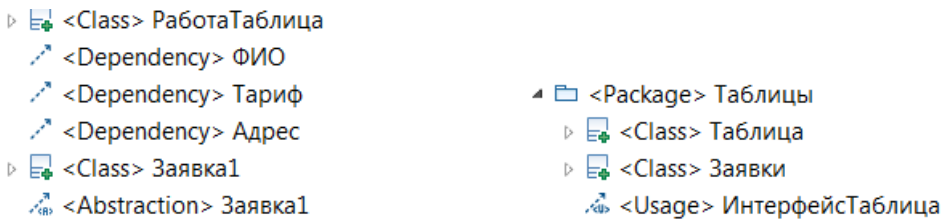


Рис. 7.70. Отношения зависимости, использования и абстракции

7.4.5. Отношение вложенности

Отношение вложенности (**Nest Classifier**) используется для показа того факта, что один класс (интерфейс, перечисление) описан внутри другого класса. При создании отношения сначала кликните на вложенный класс, а потом на класс, который его содержит.

Отношение вложенности похоже на отношение композиции, когда мы хотим описать, что объект некоторого класса состоит из отдельных частей. Но, в отличие от отношения композиции, классы, описанные внутри другого класса, используются только в данном классе.

Предположим, что класс **Пользователь** содержит более подробную информацию о пользователе, чем логин и пароль. Сам логин пароль, а также операции с ними могут быть объединены в отдельный класс **Учетная запись**, который вложен в класс **Пользователь**. При создании связи появится окно Deletion, в котором нужно кликнуть на OK (рис. 7.71).

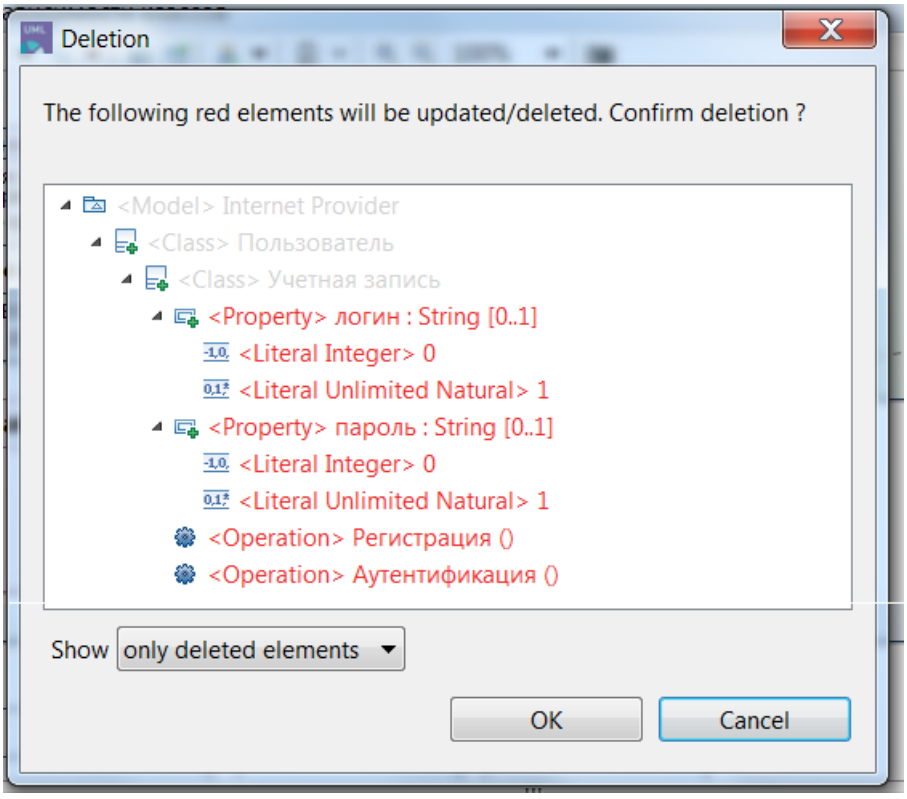


Рис. 7.71. Удаление элементов при создании отношения вложенности

Показанные в окне элементы не удаляются, а только меняют свое местоположение в модели. В результате на диаграмме увидим следующее (рис. 7.72).

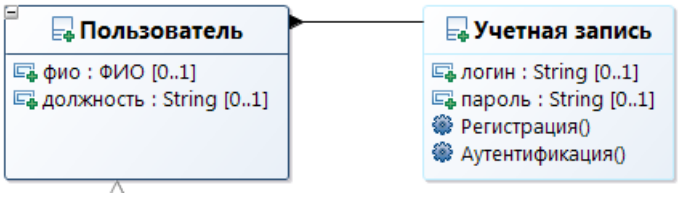


Рис. 7.72. Отношение вложенности на диаграмме

В Model Explorer видно, что класс **Учетная запись** вложен в класс **Пользователь** (рис. 7.73).

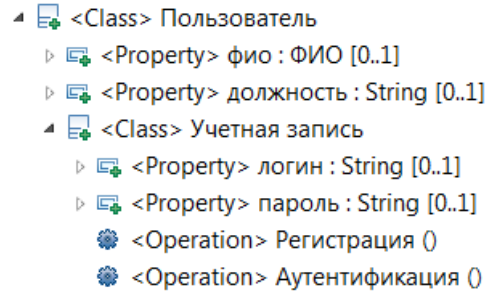


Рис. 7.73. Отношение вложенности в Model Explorer

7.4.6. Отношение связывания

Отношение связывания (**Bind)** устанавливается между абстрактным параметризованным классом (шаблоном) и классом, который создается на основе параметризованного класса путем указания значения параметра. В качестве значения параметра используется некоторый тип. Это может быть простой тип, например, числовой (int), строковый (string), или тип, являющийся классом. Параметризованный класс не может иметь объектов. Он используется для определения других классов. Объекты создаются для классов, построенных на основе параметризованного класса.

Предположим, что мы хотим работать с несколькими таблицами, например, таблицей заявок, таблицей оборудования и таблицей тарифов. Каждая из таблиц – это множество строк. Для всех таблиц есть общие операции, например, добавить строку, удалить строку. В этом случае можно создать абстрактный класс **Таблица** с параметром **T,** который будет принимать значения конкретных классов в зависимости от того какие строки в таблице мы хотим хранить. Например, используя в качестве значения параметра **T** класс **Заявка**, можно создать класс **Заявки**, в котором каждая строка таблицы будет представлять данный заявки, описанные в классе **Заявка.**

Создадим класс с именем, например, **Class5**. В свойствах класса укажем, что это абстрактный класс, т.е. класс для которого не будут создаваться объекты (рис. 7.74).

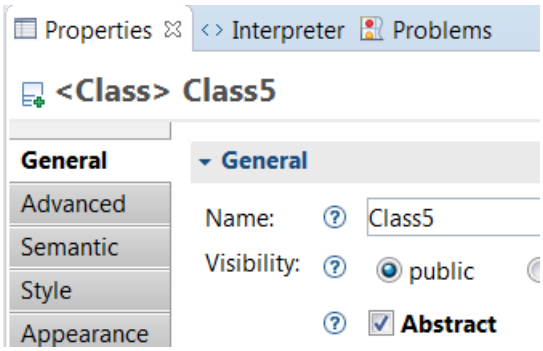


Рис. 7.74. Создание абстрактного класса

На диаграмме имена абстрактных классов показываются курсивом.

Далее на диаграмме кликнем на имя класса **Class5** для его выделения и еще раз для перехода в режим редактирования. После имени в угловых скобках запишем имя параметра, например, **T** (рис. 7.75).

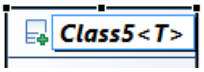


Рис. 7.75. Создание параметризованного класса

После этого нажмем клавишу Enter. На диаграмме между именем класса и его типом появится пробел (рис. 7.76).

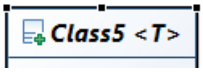


Рис. 7.76. Описание параметризованного класса

В Model Explorer мы увидим, что параметр T является произвольным классом (рис. 7.77).

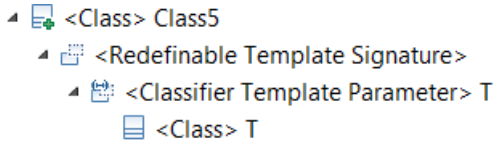


Рис. 7.77. Описание параметра класса в Model Explorer

После этого переименуем класс **Class5** в класс **Таблица**. Если сначала назвать класс по-русски, то не удается вышеописанным способом определить параметр.

Для установки отношения связывания между параметризованным классом **Таблица** и классом **Заявки**, кликнем на элемент **Bind** в палитре инструментов, затем на класс **Заявки**, а потом на класс **Таблица** (рис. 7.78).

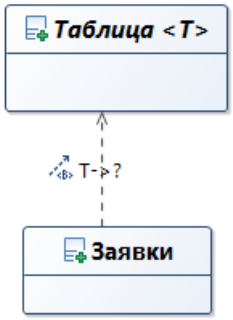


Рис. 7.78. Отношение связывания

В случае использования стандартных типов, например, **int**, указание типа параметра может быть выполнено с помощью его ввода в строке **T->?** вместо знака вопроса непосредственно на диаграмме (рис. 7.79).

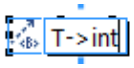


Рис. 7.79. Конкретизация типа

В нашем случае указать тип **Заявка** так не получится. Поэтому в Model Explorer найдем класс **Заявки** и перейдем на элемент **<Template Parameter Substitution>** (рис. 7.80).

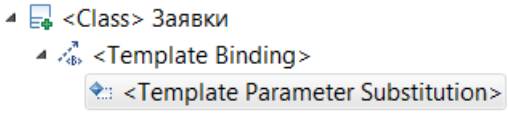


Рис. 7.80. Конкретизация типа в Model Explorer

В виде Properties для этого элемента изменим свойство **Actual** (рис. 7.81).

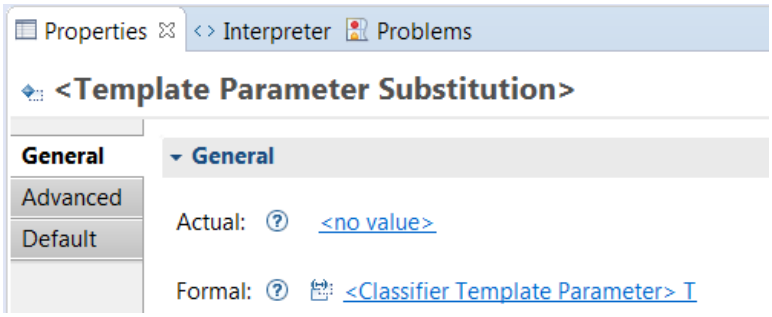


Рис. 7.81. Свойство Actual для задания типа

В строке **Actual** кликнем на кнопку  и в открывшемся окне выберем класс **Заявка** и далее кликнем на кнопку **Finish** (рис. 7.82).

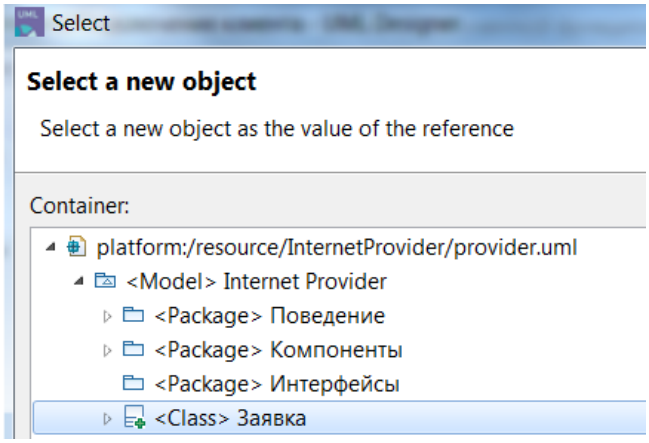


Рис. 7.82. Выбор типа

В виде Properties увидим новое значение свойства **Actua**l (рис. 7.83).



Рис. 7.83. Значение свойства Actual

На диаграмме появится описание отношения связывания с типом **Заявка** (рис. 7.84).

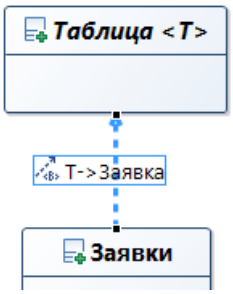


Рис. 7.84. Описание отношения связывания с типом **Заявка**

Параметризованный класс **Таблица** содержит строки типа **T**, а класс **Заявка** будет содержать заявки класса **Заявка**. Добавим свойство с именем **строки** в класс **Таблица** и свойство с именем **заявки** в класс **Заявки**. В окне редактирования **Edit** свойств элемента или виде Properties определим их свойства. Для элемента **строки** выберем тип **<Class> T**, а для элемента **заявки** - тип **<Class> Заявка**. Для создания массивов укажем в поле **Upper** значение -1. На диаграмме увидим следующее изображение классов (рис. 7.85).

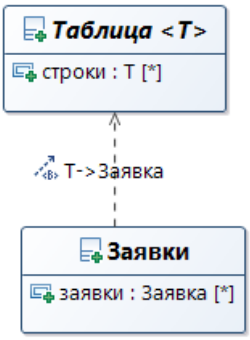


Рис. 7.85. Описание атрибутов

8. Диаграмма деятельности (Activity Diagram)

На диаграмме деятельности показывается последовательность выполнения определенных действий, передача данных и изменение состояния объектов данных.

Диаграмма деятельности используется для моделирования потоков управления деятельностью и потоков объектов деятельности.

Поток управления деятельности представляет собой последовательность выполнения определенных действий. В общем случае действия на диаграмме деятельности могут выполняться параллельно. Это означает, что во время деятельности может существовать несколько потоков управления деятельностью, каждый из которых соответствует некоторому параллельному процессу, выполняемому для реализации деятельности.

Поток объектов деятельности представляет собой передачу объектов данных в процессе выполнения деятельности.

Диаграмма деятельности – это граф, состоящий из узлов (ActivityNodes), соединенных направленными ребрами (ActivityEdges).

На диаграмме деятельности существует три типа узлов:

1. Исполняемые узлы (ExecutableNodes) показывают выполняемые действия в процессе осуществления деятельности.
2. Управляющие узлы (ControlNodes) позволяют описывать переходы от одного действия к другим действиям в зависимости от определенных условий; осуществлять распараллеливание деятельности; проводить синхронизацию параллельных потоков деятельности.
3. Объектные узлы (ObjectNodes) описывают данные, используемые в процессе выполнения деятельности.

На диаграмме деятельности существует два типа ребер:

1. Ребра потоков управления (ControlFlow) показывают переходы от одного действия к другому действию.
2. Ребра потоков объектов (ObjectFlow) показывают передачу данных от одного объекта данных к другому.

Путь в графе, проходящий через исполняемые узлы, представляет собой поток управления деятельностью, а путь проходящий через объектные узлы - поток объектов деятельности.

Диаграмма деятельности описывает некоторое поведение в моделируемой системе. Она применяется на разных этапах разработки программного обеспечения и может быть использована для разных целей:

- описания бизнесс-процессов;
- описания вариантов использования;
- описания алгоритмов программ.

8.1. Создание диаграммы деятельности

В зависимости от цели использования диаграммы деятельности она может быть создана разными способами.

Если диаграмма деятельности описывает бизнес-процесс, т.е. показывает некоторый процесс, состоящий из непоследовательности действий, которые выполняются разными сотрудниками или подразделениями предприятия, например, процесс подключения услуги связи клиенту, то такую диаграмму деятельности мы можем поместить непосредственно в пакет **Поведение**.

Для этого создадим диаграмму деятельности с помощью меню, которое откроем по правой кнопке мышки, встав на строку пакета **<Package>Поведение** в виде Model Explorer. Выберем пункт **New Representation** и далее в открывшемся подменю пункт **Activity Diagram** (как описано в разделе **Диаграмма пакетов**).

По умолчанию в пакете **Поведение** будет создан элемент **<Activity> Поведение** и в нем диаграмма с именем **Поведение Activity Diagram** (рис. 8.1).

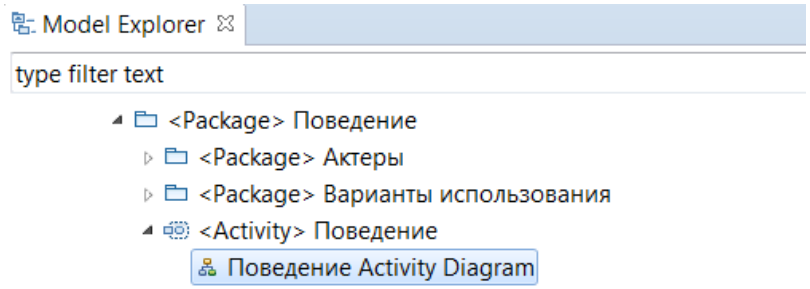


Рис. 8.1. Создание диаграммы деятельности внутри пакета

Имя элемента класса <Activity> и имя диаграммы можно изменить с помощью пункта **Rename** в меню, открывающемся по правой кнопке мышки на имени в виде Model Explorer. В общем случае эти имена могут совпадать. В элементе **<Activity>Поведение** будут находится также все элементы диаграммы деятельности.

Включение диаграммы в пакет отражается на диаграмме пакетов с помощью значка  в правом нижнем углу пакета (рис. 8.2).

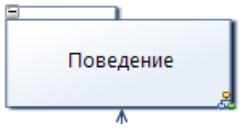


Рис. 8.2. Изображение пакета с включенной диаграммой

Диаграмму, включенную в пакет теперь можно открыть с помощью пункта **Open** в меню, открывающемся по нажатию правой кнопке мышки на изображении пакета (рис. 8.3).

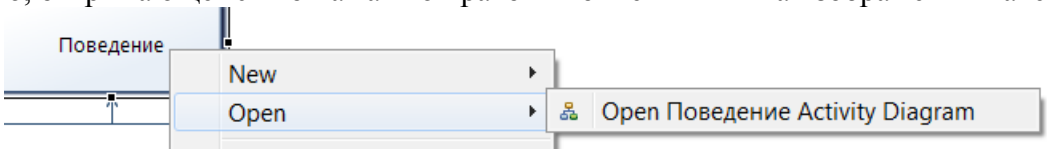


Рис. 8.3. Открытие диаграммы, включенной в пакет

К сожалению, в существующей версии после переименования диаграммы ее имя изменится в виде Model Explorer, но не изменится на предыдущем рисунке.

Диаграмму деятельности можно создать внутри пакета и другим способом. Для этого на диаграмме пакетов нужно нажать правую кнопку мышки на пакете **Поведение**, выбрать в меню пункт **New** и в открывшемся подменю выбрать пункт **Activity diagram** (рис. 8.4).

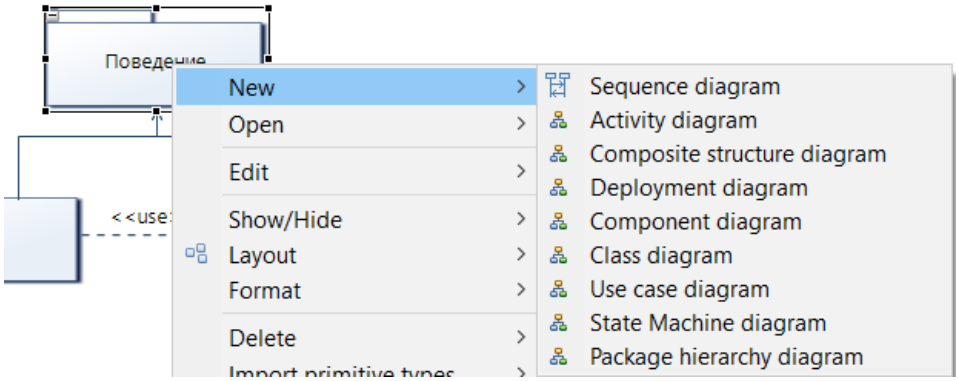


Рис. 8.4. Создание диаграммы деятельности внутри пакета на диаграмме пакетов

Как видно из рисунка, таким способом можно создать и другие диаграммы внутри пакета.

При попытке создания таким способом диаграммы некоторого типа, для которой в пакете уже есть диаграмма данного типа, новая диаграмма использует данные существующей диаграммы и создается в одном с нею элементе класса **<Activity>**. Например, при создании диаграммы **Поведение2 Activity Diagram** она появится в элементе **<Activity> Поведение.** Диаграмма **Поведение Activity Diagram** и диаграмма **Поведение2 Activity Diagram** будут содержать одни и теже элементы, но их можно по разному расположить на диаграммах. Тоже самое произойдет, если создать диаграмму **Поведение2 Activity Diagram**, нажав правую кнопку мышки на строке **<Activity> Поведение** в виде **Model Explorer**.

Данный способ можно использовать только в целях показа разных видов одной и той же диаграммы некоторого типа. Данный способ нельзя использовать, если необходимо создать новую диаграмму некоторого типа при наличии диаграммы такого же типа.

Если создается диаграмма деятельности, которая описывает некоторый вариант использования, то данную диаграмму лучше привязать к этому варианту использования.

Откроем, например, диаграмму **Use Case** нашего примера и нажмем правую кнопку мышки на варианте использования **Заявка на подключение**, выберем пункт **New** и далее **Activity diagram** (рис. 8.5).

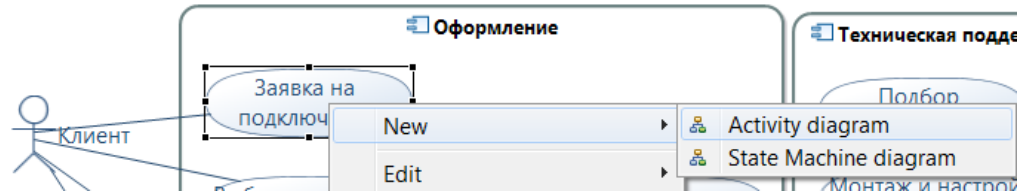


Рис. 8.5. Создание диаграммы деятельности для варианта использования

В открывшемся окне можно поменять имя диаграммы (рис. 8.6).

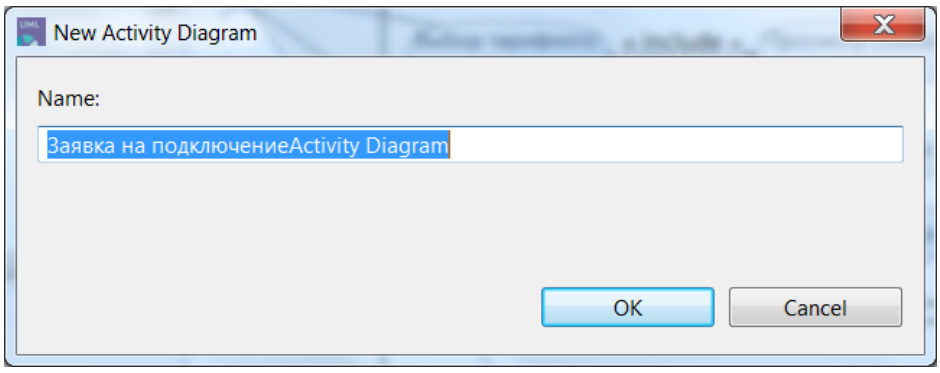


Рис. 8.6. Окно создания диаграммы деятельности

В результате диаграмма появится в элементе варианта использования **<Use Case> Заявка на подключение** в модели, а на диаграмме **Use Case** рядом с вариантом использования появится значок , обозначающий наличие диаграммы, описывающей данный вариант использования (рис. 8.7).

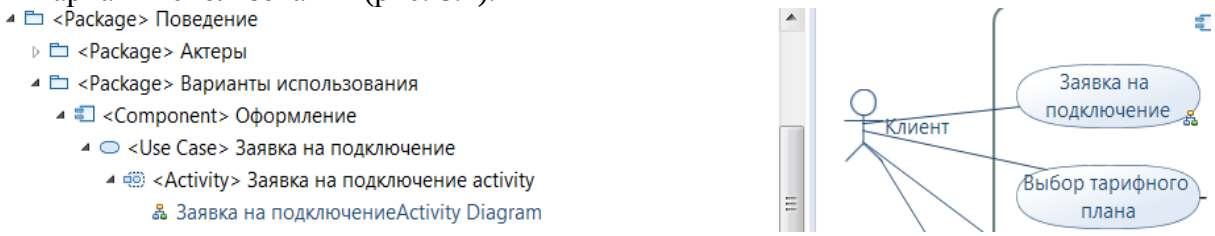


Рис. 8.7. Диаграмма деятельности внутри варианта использования

Теперь меню, открывающееся по правой кнопке мышки на изображении варианта использования, позволит перейти по пункту **Open** к диаграме деятельности **Заявка на подключение Activity Diagram** (рис. 8.8).



Рис. 8.8. Открытие диаграммы деятельности для варианта использования

К сожалению, в существующей версии после переименования диаграммы ее имя изменится в виде Model Explorer, но не изменится на предыдущем рисунке.

Как и в случае с созданием двух диаграмм одного типа с помощью меню, открывающемся по правой кнопке мышки на пакете, создание двух диаграмм одного типа для одного варианта использования будет означать, что эти диаграммы будут использовать одни и теже элементы, но их можно будет по разному расположить на диаграммах. Т.е. таким способом мы получим несколько видов одной и той же диаграммы.

Созданные диаграммы деятельности также видны во взгляде **Reused** модели, откуда их можно открыть двойным кликом (рис. 8.9).

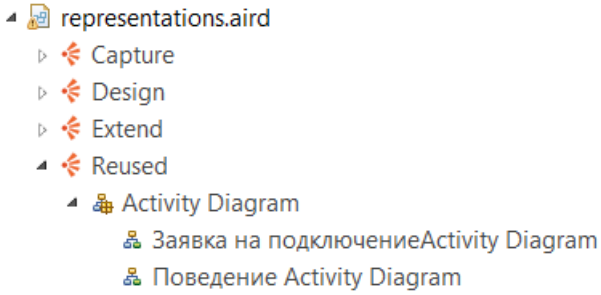


Рис. 8.9. Диаграммы деятельности во взгляде Reused

8.2. Элементы диаграммы деятельности

Диаграмма деятельности имеет палитру инструментов, показанную на рис. 8.10.

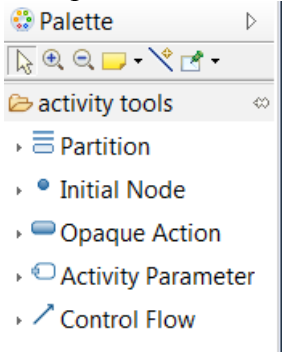


Рис. 8.10. Палитра инструментов диаграммы деятельности

Элементы палитры инструментов объединены в группы. На палитре показывается один из элементов каждой группы. Для размещения элемента на диаграмме кликните на элемент, а затем на диаграмму. Список других элементов данной группы можно увидеть, нажав на значок «треугольник» слева от имени элемента.

Например, нажав на «треугольник» рядом с элементом **Partition**, мы увидим на рис. 8.11 еще два элемента.

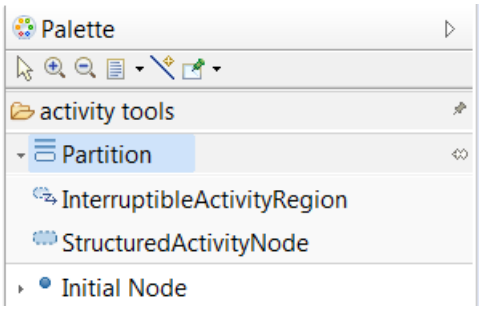


Рис. 8.11. Элементы палитры группы Partition

Выбор другого элемента из группы по клику мышкой, например, **InterruptibleActivityRegion**, приведет к закрытию списка элементов группы. Теперь уже этот выбранный элемент будет показан как заголовок группы элементов (рис. 8.12).

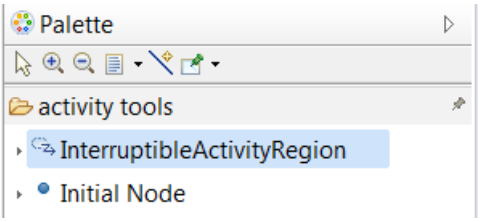


Рис. 8.12. Изменение элемента группы элементов

Элемент раздел (**Partition)** позволяет разделить диаграмму на несколько разделов, которые еще называют дорожками (по аналогии с дорожками в плавательном бассейне).

Каждая дорожка соответствует описанию деятельности некоторого подразделения предприятия, или некоторого актера, или некоторого класса.

Например, на рис. 8.13 показано, как выглядят дорожки, соответствующие деятельности трех актеров.

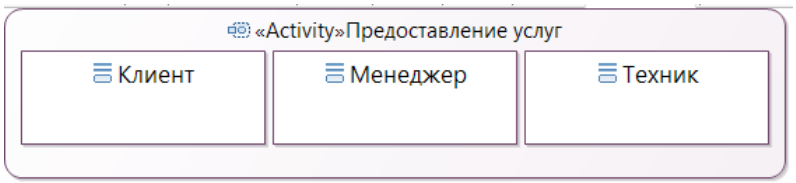


Рис. 8.13. Разделы и дорожки

Сама диаграмма и ее элемент класса <Activity>, по умолчанию названные **Поведение**, теперь переименованы в **Предоставление услуг**. На диаграмме показывается имя элемента класса <Activity>, а не самой диаграммы. Имя диаграммы показывается на закладке области редактирования.

Некоторое действие может быть достаточно сложным и для его описания потребуется нарисовать несколько связанных друг с другом действий, т.е. действие является составным. В этом случае с помощью элемента структурный узел деятельности **(StructuredActivityNode)** рисуется область, в которой располагается часть диаграммы деятельности.

Элемент область с прерыванием **(InterruptibleActivityRegion)** также предназначен для создания некоторой области, в которой размещается часть диаграммы деятельности.

Отличием этой области от области, создаваемой с помощью элемента **StructuredActivityNode,** является то, что изображенная в нем последовательность действий может быть прервана по некоторому событию. В результате прерывания происходит переход к некоторому другому действию, расположенному вне этой области.

8.3. Исполняемые узлы

Основным элементом диаграммы деятельности является действие (**Action**). Действие изображается с помощью одного из элементов, показанных на рис. 8.14.

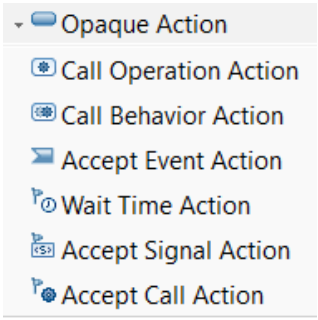


Рис. 8.14. Элементы действия

Для изображения произвольного действия используется элемент **Opaque Action** (рис. 8.15).

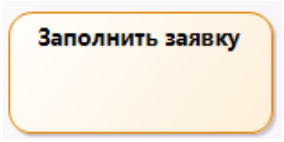


Рис. 8.15. Произвольное действие

Как правило, действие, изображаемое с помощью элемента **Opaque Action**, является достаточно элементарным. Например, это некоторое вычисление, или ввод некоторых данных, или выдача некоторой информации. Если же необходимо показать сложное действие, которое представляет собой несколько элементарных действий, т.е. является составным, то используется элемент вызова деятельности (**Call Behavior Action).** Составное действие описывается, либо с помощью другой диаграммы деятельности, либо с помощью диаграммы состояний (State Machine Diagram).

При помещении элемента **Call Behavior Action** на диаграмму открывается окно, показанное на рис. 8.16. В этом окне нужно выбрать диаграмму деятельности или диаграмму состояний.

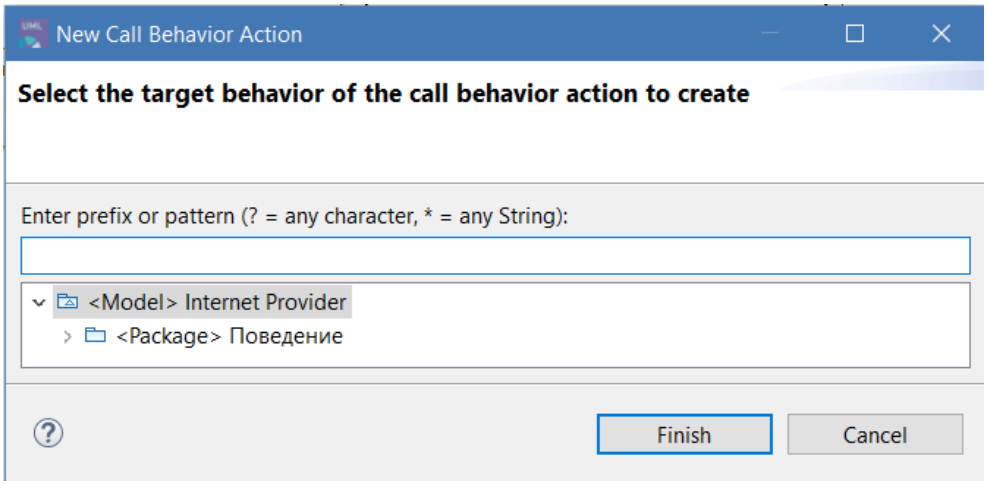


Рис. 8.16. Окно выбора диаграммы деятельности или диаграммы состояний

Например, выберем диаграмму деятельности **<Activity> Заявка на подключение** (рис. 8.17).

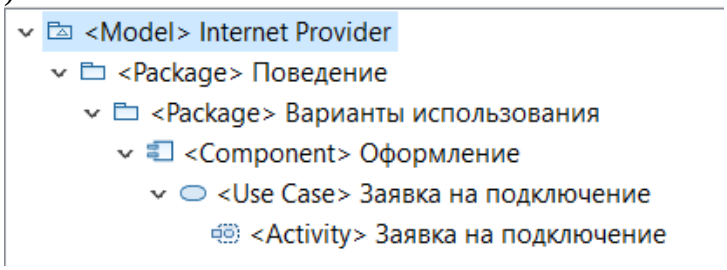


Рис. 8.17. Выбор диаграммы деятельности

На диаграмме появится элемент обозначающий составную деятельность, которая описана на диаграмме деятельности **Заявка на подключение** (рис. 8.18)**.** Первоначально эта диаграмма называлась **Заявка на подключение Activity Diagram**.

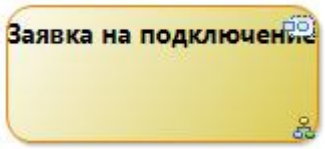


Рис. 8.18. Составная деятельность

Значок в правом нижнем углу означает, как мы уже знаем, что этот элемент описывается с помощью другой диаграммы, которую можно открыть по пункту **Open** в меню, которое появляется при нажатии правой кнопки мышки на элементе. Значок в правом верхнем углу означает, что связанная с данным элементом диаграмма является диаграммой деятельности.

Элемент вызова операции (**Call Operation Action)** используется для показа на диаграмме действия, связанного с вызовом некоторой операции (метода) класса. При помещении на диаграмму этого элемента появляется окно **New Call Operation Action** (рис. 8.19).

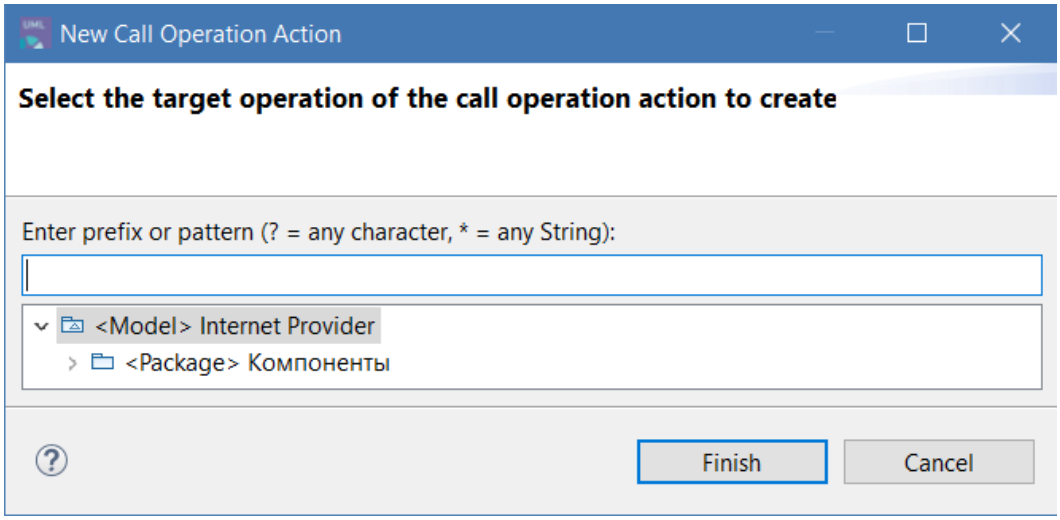


Рис. 8.19. Выбор операции класса

В этом окне нужно найти соответствующий класс и его операцию, вызов которой мы хотим изобразить на диаграмме деятельности.

Вызов операции без параметров показан на рис. 8.20.

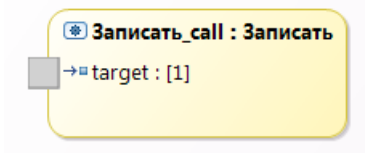


Рис. 8.20. Операция

Вызов операции с передаваемым параметром **дата** и возвращаемым значением **заявки** показан на рис. 8.21.

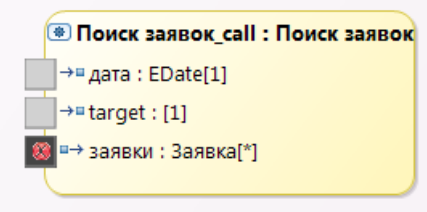


Рис. 8.21. Операция с параметрами

Каждый параметр операции показывается как входной контакт, а результат операции – как выходной контакт. Кроме этого показывается контакт target. Данный контакт используется для передачи в него объекта, для которого выполняется данная операция.

Понятно, что использование операций классов на диаграмме деятельности предполагается, что мы уже нарисовали диаграмму классов. Если мы используем диаграмму деятельности для описания бизнес-процесса или варианта использования, то, как правило, диаграммы классов у нас еще нет. А вот к моменту, когда нам нужно будет описать некоторый алгоритм программы у нас уже будет диаграмма классов и мы сможем описать этот алгоритм, используя диаграмму деятельности и показывая на ней вызовы методов.

Для показа ожидания некоторого события используется элемент принятия события (**Accept Event Action)** (рис. 8.22).

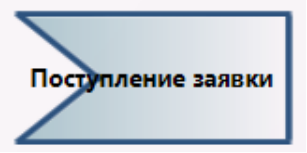


Рис. 8.22. Принятие события

В случае, если событием, которого мы ждем, является вызов некоторой операции класса, то используется элемент принятия вызова (**Accept Call Action)**.

В случае, если событием, которого мы ждем, является некоторый сигнал, то используется элемент принятия сигнала (**Accept Signal Action)**. Сигнал представляет собой класс, объекты которого используются для передачи информации.

В случае, если событием, которого мы ждем, является наступление некоторого момента времени или прохождение некоторого интервала времени, то используется элемент ожидания события времени (**Wait Time Action)** (рис. 8.23).



Рис. 8.23. События времени

После размещения события на диаграмму в модели создается элемент класса **<Accept Event Action**>, например, элемент **<Accept Event Action> Поступление заявки**, а внутри него элемент **<Trigger> Trigger1** (рис. 8.24).

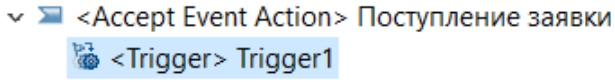


Рис. 8.24. Событие принятия вызова в Model Explorer

По двойному клику на элементе **<Trigger> Trigger1** в виде Properties, в разделе Advanced можно увидеть, что ожидается поступление события с именем **ChangeEvent2** класса **<Change Event>** (рис. 8.25).

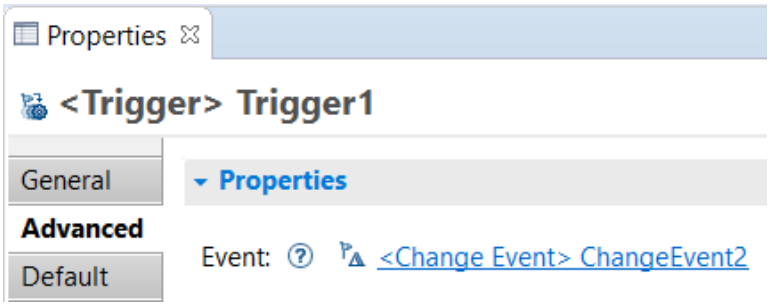


Рис. 8.25. Описание события

В модели также создается элемент **<Change Event> ChangeEvent2**, который виден в Model Explorer (рис. 8.26).

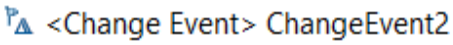


Рис. 8.26. Событие класса <Change Event> в Model Explorer

Особенностью является то, что все элементы диаграммы деятельности **Поведение Activity Diagram** в виде Model Explorer располагаются внутри элемента **<Activity>Поведение**, а элемент **<Change Event> ChangeEvent2** располагается на одном уровне с элементом **<Activity>Поведение** внутри элемента **<Package> Поведение.**

При создании элемента ожидания события времени (**Wait Time Action**) также создается элемент класса **<Accept Event Action**>, но подчиненный ему триггер будет ждать события класса **<Time Event>**. В модели при этом создается элемент, показанный на рис. 8.27.

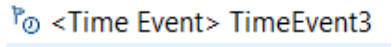


Рис. 8.27. Событие класса <Time Event> в Model Explorer

8.4. Объектные узлы

Для изображения объектных узлов на диаграмме деятельности используются элементы из группы элементов **ActivityParameter**, показанные на рис. 8.28.

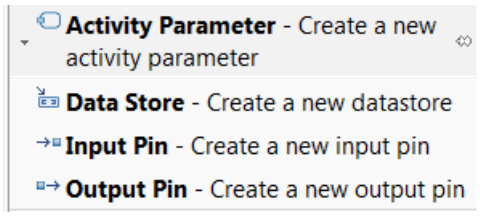


Рис. 8.28. Объектные узлы

Объектный узел может представлять данные простых типов, например, строка, число, или же данные, хранящиеся в объекте некоторого класса.

Элемент параметр деятельности (**Activity Parameter**) служит для описания входных и выходных данных деятельности. Для выполнения деятельности необходимо наличие всех входных параметров. Они определяют предусловие выполнения деятельности. По завершению деятельности формируются выходные параметры. Они определяют постусловие выполнения деятельности.

Параметры показываются на границе области, изображающей деятельность. UML Designer сам размещает параметры на границе диаграммы деятельности, когда элемент **Activity Parameter** помещается внутрь диаграммы деятельности. Можно перетащить мышкой изображение параметра в нужное место на границе деятельности.

Например, параметром для выполнения деятельности по оформлению заявки должна быть поданная заявка (рис. 8.29).

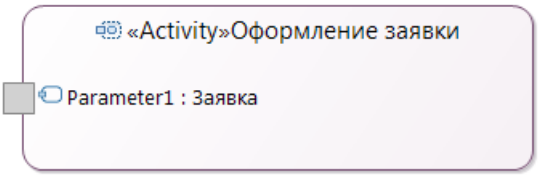


Рис. 8.29. Параметр деятельности

На диаграмме видно имя параметра Parameter1 и его тип Заявка, указанный после двоеточия.

При помещении элемента **Activity Parameter** на диаграмму открывается окно создания параметра (рис. 8.30).

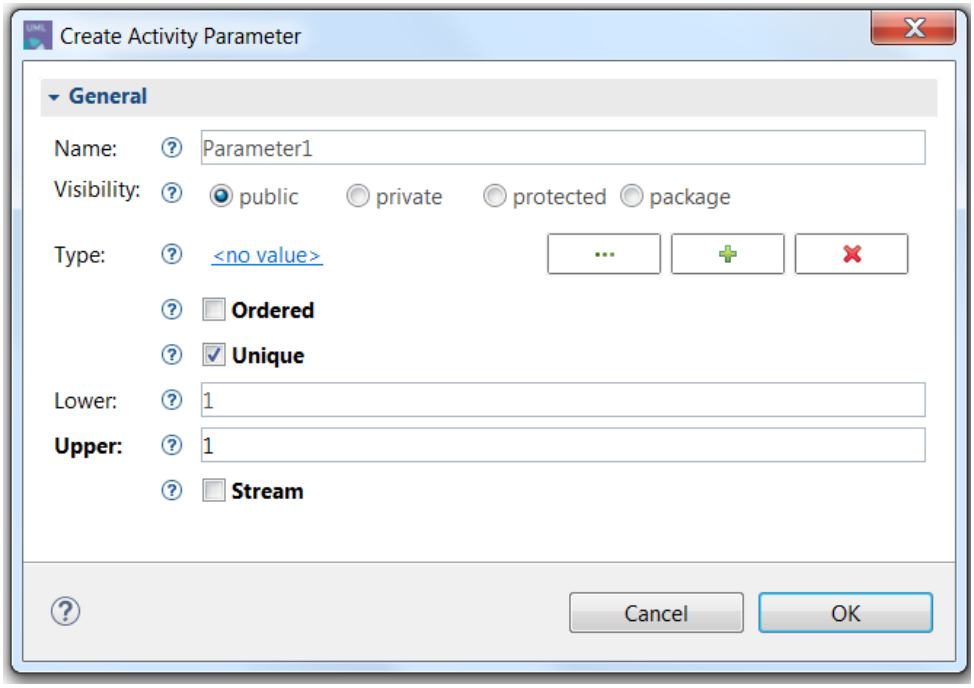


Рис. 8.30. Окно создания параметра

В этом окне можно поменять имя параметра и его тип. Тип параметра выбирается по клику мышкой на значок  .

В качестве типа параметра можно, например, выбрать класс <Class> Заявка из папки <Model> Internet Provider (этот класс должен быть заранее создан на диаграмме классов) (рис. 8.31).

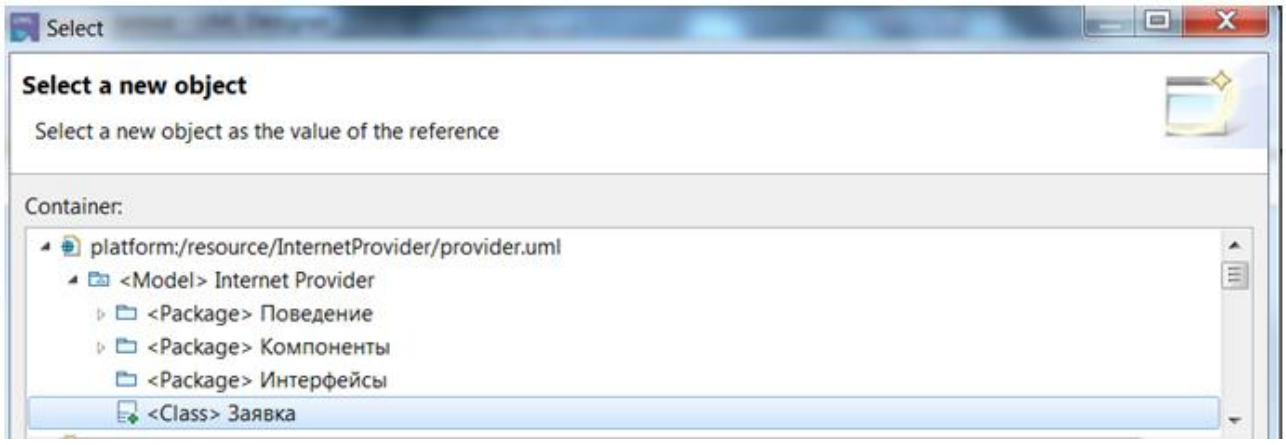


Рис. 8.31. Выбор типа параметра

Если параметр имеет простой тип данных, то его можно выбрать в одной из папок, показанных на рис. 8.32.

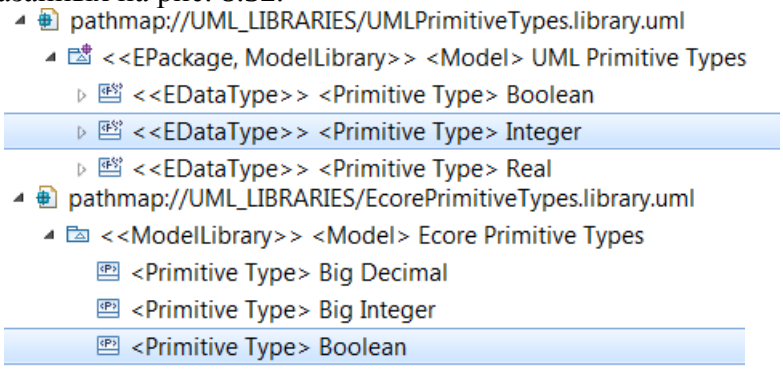


Рис. 8.32. Выбор примитивного типа параметра

После размещения параметра на диаграмму в модели создаются два элемента: сам параметр (<Parameter> Parameter1: Заявка) и графический элемент, изображающий объектный узел параметр (<Activity Parameter Node> Parameter1 (рис. 8.33).

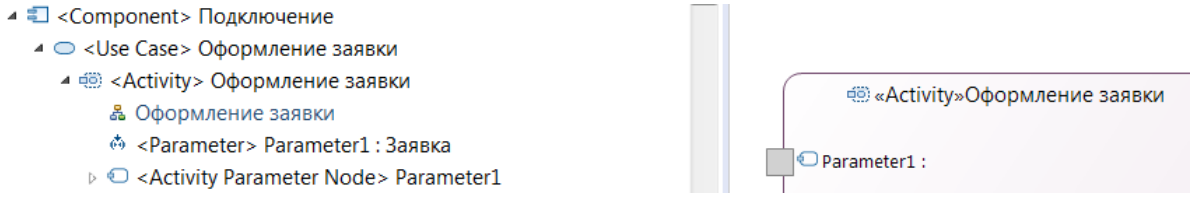


Рис. 8.33. Параметр и его графический элемент

Видно, что для самого параметра мы выбрали тип **Заявка**, а для объектного узла тип еще не выбран. По умолчанию имя параметра и имя узла параметра совпадают. Имя узла и его тип можно поменять в окне, открываемом двойным кликом по изображению объектного узла параметра на диаграмме (рис. 8.34).

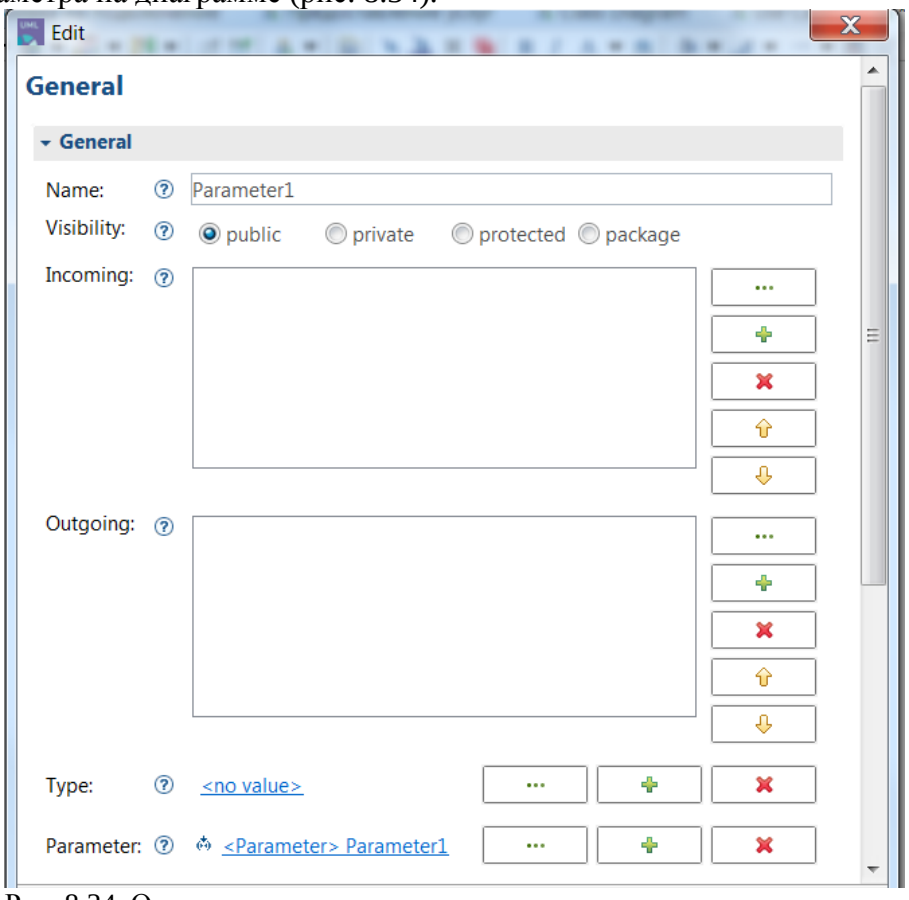


Рис. 8.34. Окно редактирования параметра

Тип объектного узла параметра в поле Type нужно установить таким же, что и тип параметра в поле Parameter. В данном примере для типа выберем класс **Заявка** и переименуем объектный узел в **Заявка [отправлена]**. Результат показан на рис. 8.35.

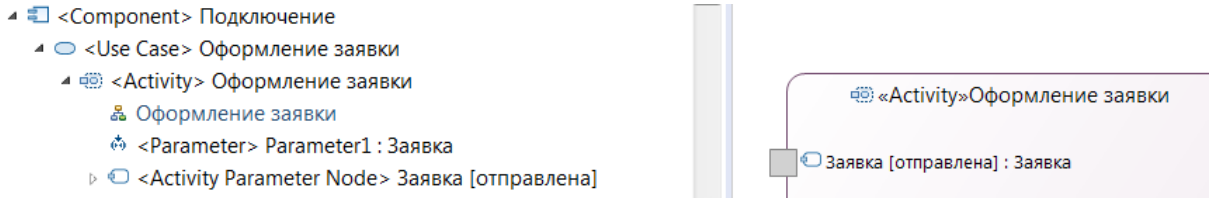


Рис. 8.35. Изменение типа параметра и имени объектного узла

В квадратных скобках, как принято в языке UML, указывается состояние объекта, передаваемого в качестве параметра. Таким образом, мы показываем, что для начала выполнения деятельности **Оформление заявки** в качестве параметра должен поступить объект **Заявка** класса **Заявка** в состоянии **отправлена**. К сожалению, UML Designer не дает возможности определить состояние объекта в явном виде и приходится это делать с помощью указания состояния в имени объектного узла.

Появление в модели параметра и объектного узла для этого параметра позволяет в дальнейшем использовать данный параметр для другого объектного узла, являющегося параметром. Например, выходной параметр одной диаграммы деятельности может поступать в другую диаграмму деятельности в качестве входного параметра. Один и тот же параметр не должен быть указан более чем для одного входного объектного узла и одного выходного объектного узла. Для выбора параметра объектного узла нужно кликнуть в окне Edit редактирования свойств объектного узла на значок  в строке **Parameter** и в открывшемся окне выбрать уже имеющийся параметр (рис. 8.36).

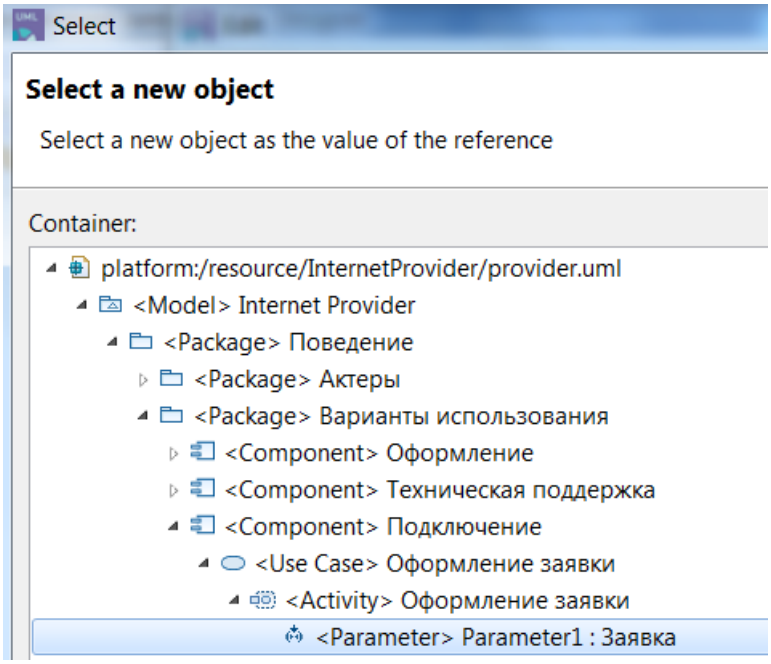


Рис. 8.36. Выбор параметра

В стандарте UML существует два класса объектных узлов, используемых для хранения данных: центральный буфер (**CentralBufferNode**) и хранилище данных (**DataStoreNode**).

Центральный буфер используется для временного хранения входящего в него потока данных. Данные из буфера поступают в выходной поток данных по запросам.

Хранилище данных отличается от буфера данных тем, что поступившие в него данные, постоянно в нем хранятся и не удаляются из него, если они поступают в выходной поток данных.

В UML Designer для обоих вариантов хранения данных используется элемент хранилище данных (**Data Store**). Например, список оборудования, реализуемый в виде некоторого файла или таблицы базы данных, может быть представлен на диаграмме, как показано на рис. 8.37.

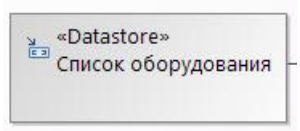


Рис. 8.37. Хранилище данных

При выполнении действия может потребоваться наличие некоторых данных. Для этой цели используется элемент входной контакт (**Input Pin**). Входных контактов может быть несколько. Входной контакт имеет имя и тип, которые задаются в окне **Create Input Pin**, появляющемся при помещении контакта на элемент деятельности (рис. 8.38).

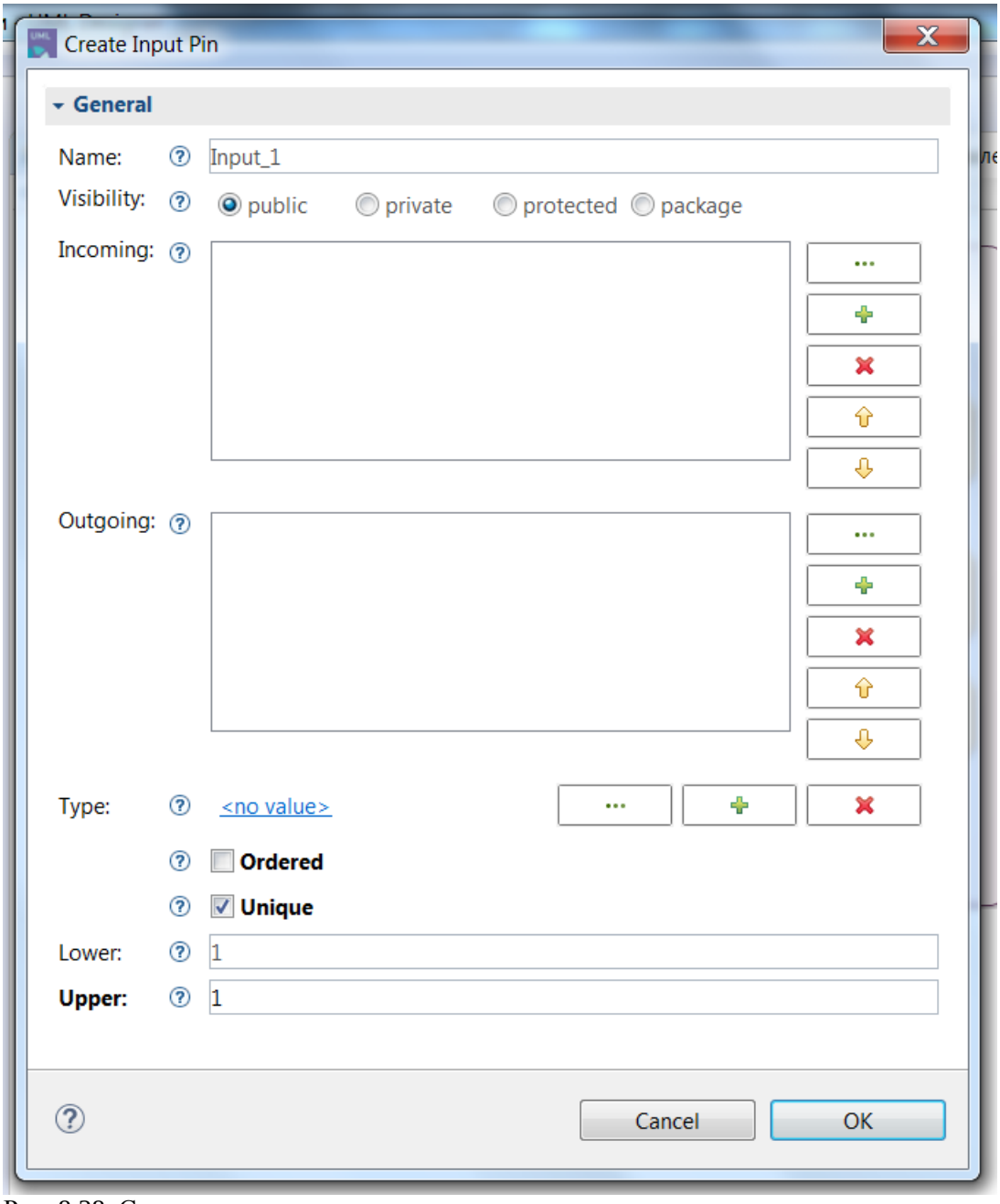


Рис. 8.38. Создание входного контакта

В данном окне также можно указать количество поступающих объектов с помощью задания нижней (Lower) и верхней (Upper) границы. Если граница может быть произвольной, то ставим * или -1. В окне * заменяется автоматически на -1, а на диаграмме показывается *.

Например, так будет выглядеть входной контакт по имени **Список возможного оборудования**, для типа которого выбран класс **Оборудование** (рис. 8.39).

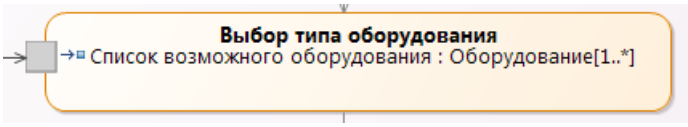


Рис. 8.39. Входной контакт

Действие может начаться только тогда, когда поступили данные на все входные контакты.

После выполнения действия могут быть сформированы некоторые данные. Для этой цели используется элемент выходной контакт (**Outut Pin**). Выходных контактов может быть несколько. После выполнения действия данные поступают на все его выходные контакты.

Например, так будет выглядеть выходной контакт по имени **Output_0**, для типа которого выбран класс **Заявка** (рис. 8.40).

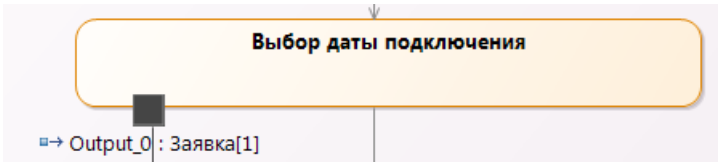


Рис. 8.40. Выходной контакт

8.5. Управляющие узлы

На диаграмме деятельности используются управляющие узлы (Node) из группы элементов InitialNode, которая содержит элементы, показанные на рис. 8.41.

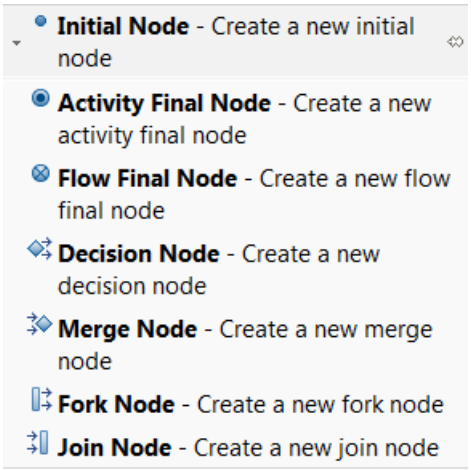


Рис. 8.41. Управляющие узлы

Элемент начальный узел (**Initial Node)** – это начальный элемент диаграммы деятельности (рис. 8.42).



Рис. 8.42. Начальный узел

Элемент конечный узел деятельности (**Activity Final Node)** – это последний элемент диаграммы деятельности, показывающий, что деятельность завершена (рис. 8.43).



Рис. 8.43. Конечный узел

Элемент конечный узел потока (**Flow Final Node)** – это последний элемент некоторого потока управления, показывающий, что данный поток завершен (рис. 8.44).

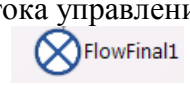


Рис. 8.44. Конечный узел потока

Элемент решения (**Decision Node)** используется для показа узла, в котором принимается решения о переходе к одному из следующих узлов в зависимости от того, какое условие выполняется (рис. 8.45).

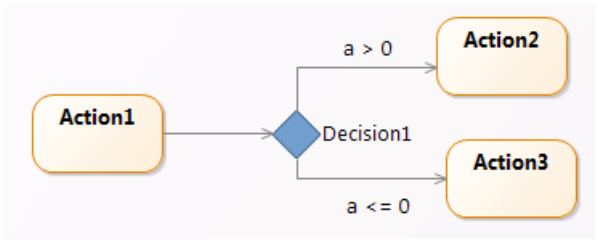


Рис. 8.45. Решение

Узел рещения имеет одно входящее ребро и два или более исходящих ребер, каждое из которых определяет некоторый переход в зависимости от выполнения условия, соответствующего этому ребру (это условие называется называется сторожевым). Одновременно должно выполняться только одно условие, чтобы определить переход по одинственному из исходящих ребер.

Элемент слияния (**Merge Node)** предсталяет собой узел, в который входят два или более ребер и выходит всего одно ребро (рис. 8.46).

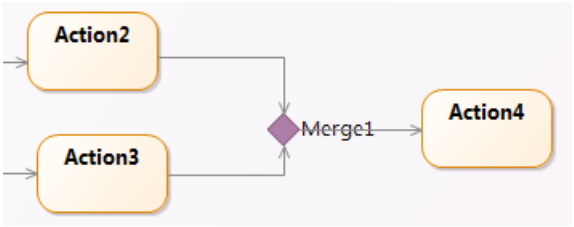


Рис. 8.46. Слияние

Элемент ветвления (**Fork Node)** используется для распараллеливания потока управления. Один поток управления, входящий в этот узел, разделяется на два или более параллельно исполняемых потоков управления (рис. 8.47).

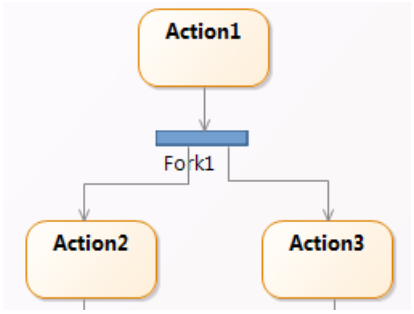


Рис. 8.47. Ветвление

Элемент объединения (**Join Node)** используется для объединения нескольких параллельно выполняемых потоков управления в один поток (рис. 8.48).

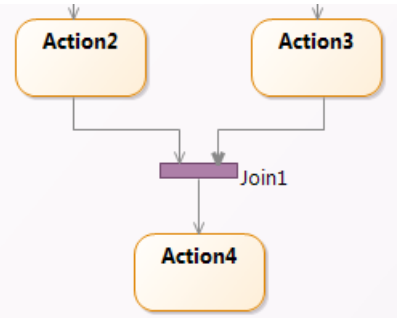


Рис. 8.48. Объединение

Переход к узлу, в который входит ребро, исходящее из узла объединения, происходит только после того, как все входящие параллельно исполняемые потоки достигнут узла объединения. Т.е. в узле объединения происходит ожидание завершения действий, выполняемых параллельными потоками управления.

Управляющие узлы не удается поместить непосредственно в области **Partition, StructuredActivityNode** и **InterruptibleActivityRegion**. Их можно поместить на диаграмму вне этих областей, а потом перетащить мышкой в нужную область.

8.6. Отношения между элементами диаграммы деятельности

Между элементами диаграммы деятельности могут существовать отношения, показанные на рис. 8.49, которые на графе деятельности, изображаются с помощью ориентированных ребер (дуг).

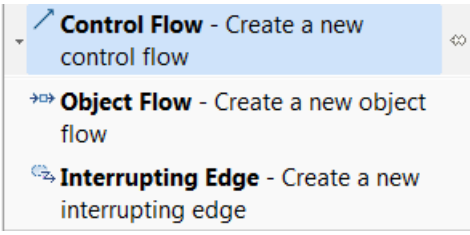


Рис. 8.49. Отношения между элементами

Для показа перехода от одного действия к другому действию используется элемент поток управления (**Control Flow)** (рис. 8.50).



Рис. 8.50. Переход

Последовательность ребер потока управления, связывающих между собою узлы действий, показывает поток управления деятельности.

Элемент потока управления может иметь описание, как показано на рис. 8.51. Например, описание ребра потока управления необходимо, когда используется элемент ветвления (**Decision**). Для ввода описания кликните мышкой на ребро, а затем, после выделения ребра, начните набирать текст. Этот текст можно в дальнейшем исправить, кликнув на него мышкой. Текст также с помощью мышки можно перетащить на другое место.

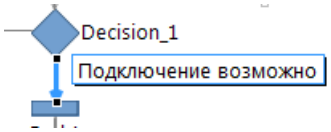


Рис. 8.51. Ветвление

Каждая диаграмма деятельности имеет один начальный узел, который соединяется потоком управления с первым действием (рис. 8.52).

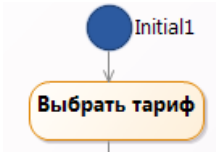


Рис. 8.52. Начальный узел на диаграмме

Каждая диаграмма деятельности имеет один (или несколько) конечных узлов **Activity Final Node**, соединяемых с последними действиями, выполняемыми в процессе деятельности (рис. 8.53).



Рис. 8.53. Конечный узел на диаграмме

Использование нескольких конечных узлов позволяет нарисовать диаграмму без пересечения ребер, ведущих к одному конечному узлу.

Начальные и конечные узлы можно (но необязательно) также использовать при рисовании части диаграммы деятельности внутри областей, создаваемых с помощью элементов **StructuredActivityNode** и **InterruptibleActivityRegion**.

Для прерывания потока управления внутри области с прерыванием используется событие прерывания, которое соединяется с узлом, расположенного вне области, с помощью ребра прерывания (**InterruptingEdge)**. Узел действия, которому передается управление по прерыванию, начинает новый поток управления, отличающийся от основного потока управления деятельностью. Поэтому для завершения нового потока используется конечный узел потока (рис. 8.54).

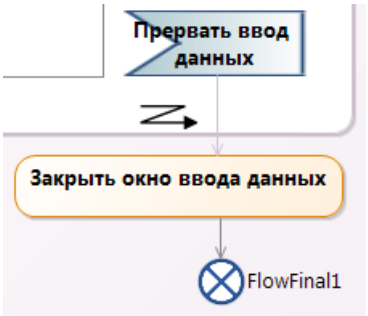


Рис. 8.54. Прерывание

Передача данных изображается с помощью элемента потока объектов (**Object Flow**). Элемент поток объектов может соединять между собой только объектные узлы. Например, входной параметр деятельности и входной контакт действия, выходной контакт одного действия и входной контакт другого действия, контакт и хранилище данных, выходной контакт действия и выходной параметр деятельности.

На рис. 8.55 показана передача данных из последнего действия деятельности наружу деятельности.



Рис. 8.55. Передача данных наружу деятельности

8.7. Примеры диаграмм деятельности

Рассмотрим диаграмму деятельности, показанную на рис. 8.56, которая описывает вариант использования **Заявка на подключение**.

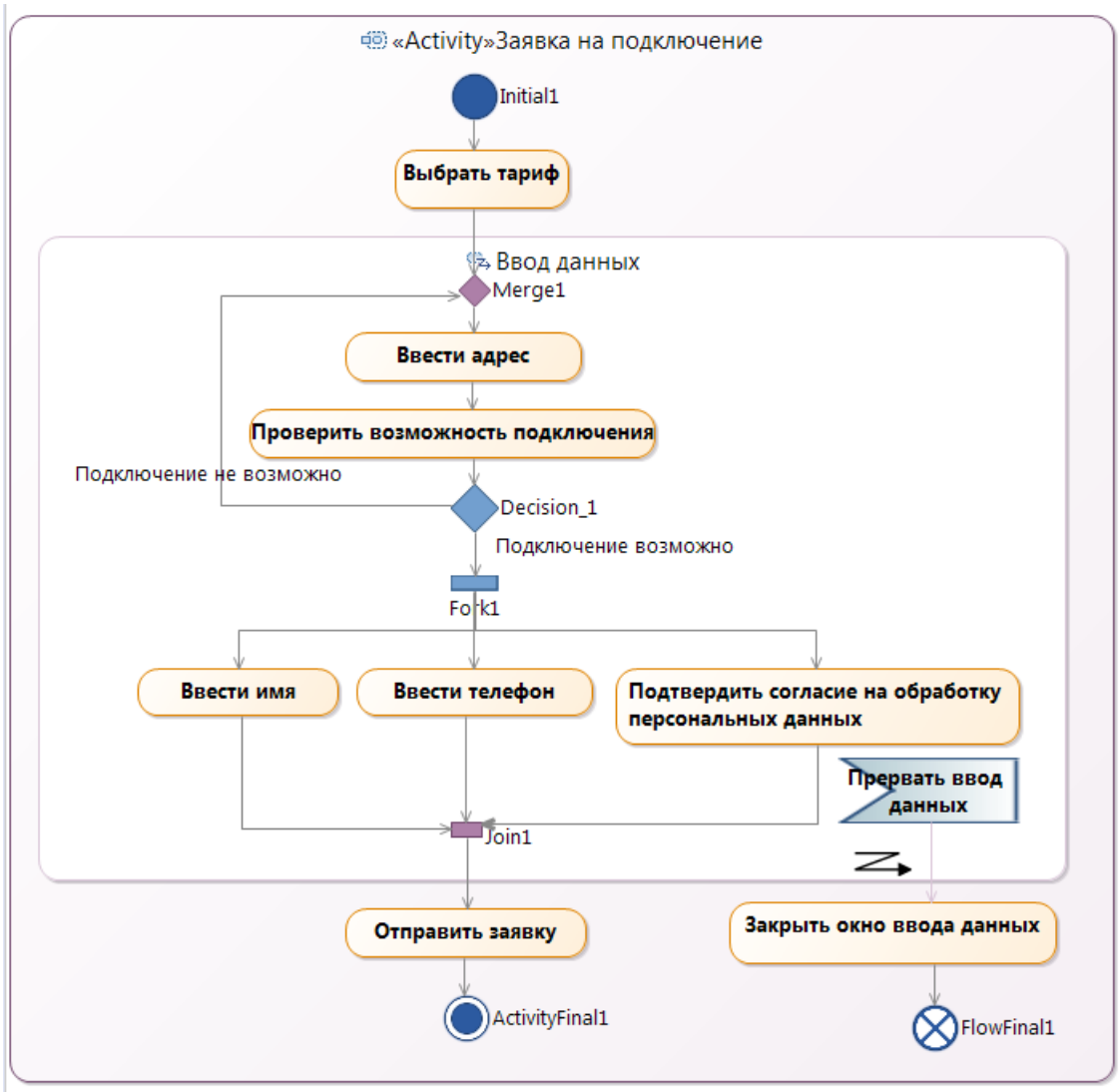


Рис. 8.56. Заявка на подключение

Данная диаграмма была создана с помощью меню, открываемого по правой кнопке мышки на варианте использования, поэтому в модели мы видим элемент **<Activity> Заявка на подключение** внутри элемента **<Use Case> Заявка на подключение**. Сама диаграмма и все ее элементы расположены внутри элемента **<Activity> Заявка на подключение** (рис. 8.57).

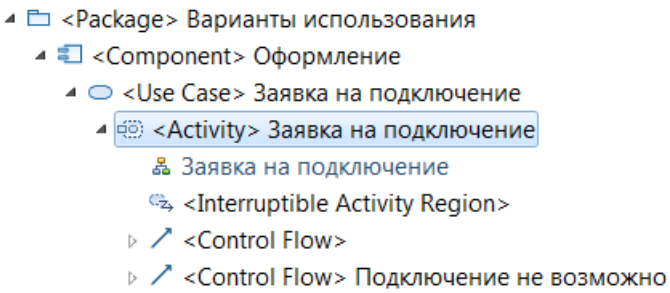


Рис. 8.57. Расположение диаграммы в Model Explorer

Как видно, все узлы на диаграмме поименованы. Это удобно при поиске данных узлов в описании модели в виде Model Explorer. С другой стороны, можно убрать имена начального и конечных узлов, узлов ветвления и слияния с диаграммы, т.к. на данной диаграмме они не несут смысловой нагрузки. Имя узла можно стереть в поле Name вида Properties или окне Edit со свойствами элемента, открываемому по двойному клику на элементе.

Например, окно Edit для узла Join1 показано на рис. 8.58.

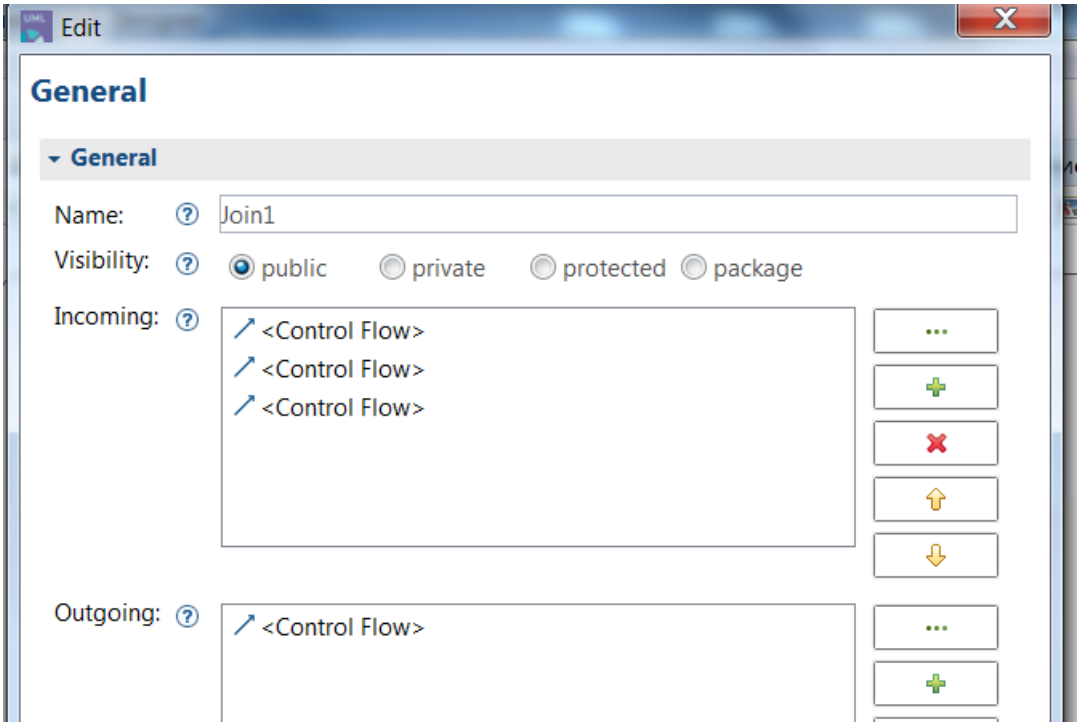


Рис. 8.58. Потоки управления

В этом окне также видны все входящие (Incoming) и исходящие (Outgoing) потоки узла.

Вторым примером является диаграмма деятельности, показанная на рис. 8.59. Она описывает вариант использования **Оформление заявки**, выполняемый менеджером во время разговора с клиентом по телефону.

На диаграмме показан поток управления, состоящий из трех действий, и поток объектов. Для выполнения действия **Согласование услуг** необходимо наличие поступившей заявки. При поступлении заявка находится в состоянии **отправлена**. При выполнении действия **Выбор типа оборудования** выполняется запрос к списку оборудования в хранилище данных. Клиент и менеджер договариваются об использовании, например, стандартного оборудования провайдера или оборудования определенной фирмы. После выполнения действия **Выбор даты подключения**, которое является последним в потоке управления, выдается заявка, передаваемая в качестве выходного параметра деятельности **Оформление заявки**. Заявка меняет свое состояние на состояние **оформлена**.

Если надпись рядом с контактом залезает на другой элемент диаграммы, то ее можно переместить мышкой, разместив слева, справа, сверху или под контактом.

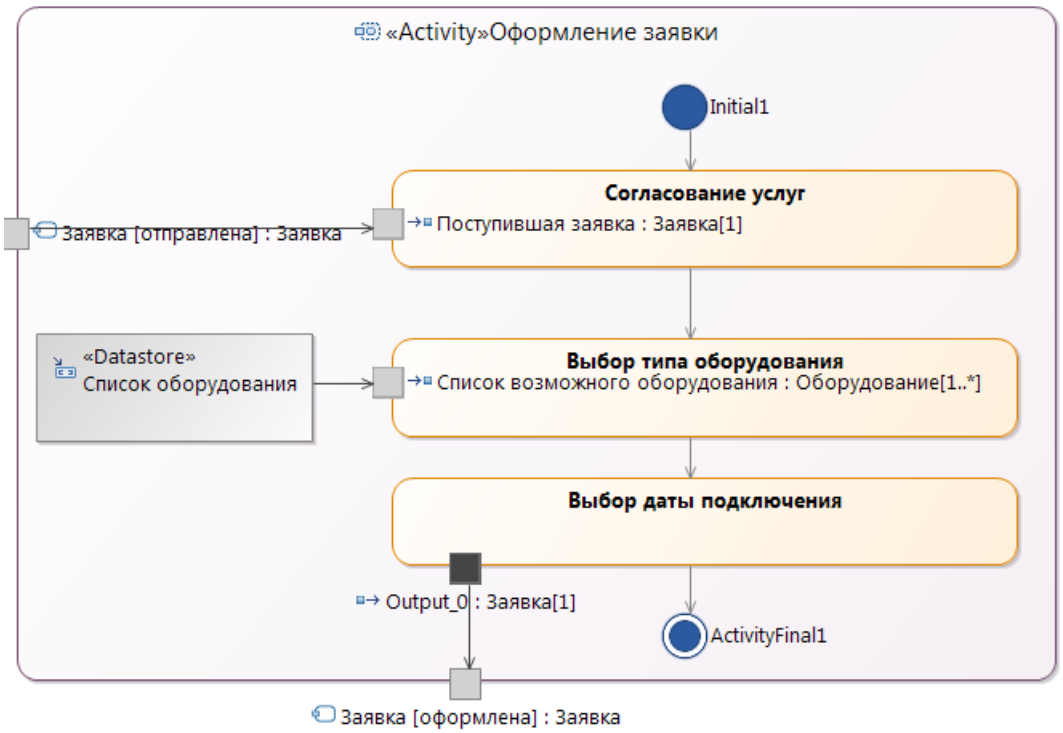


Рис. 8.59. Оформление заявки

Третий пример описывает бизнес-процесс предоставления услуг клиенту по подключению его к интернету. Диаграмма приведена на рис. 8.60.

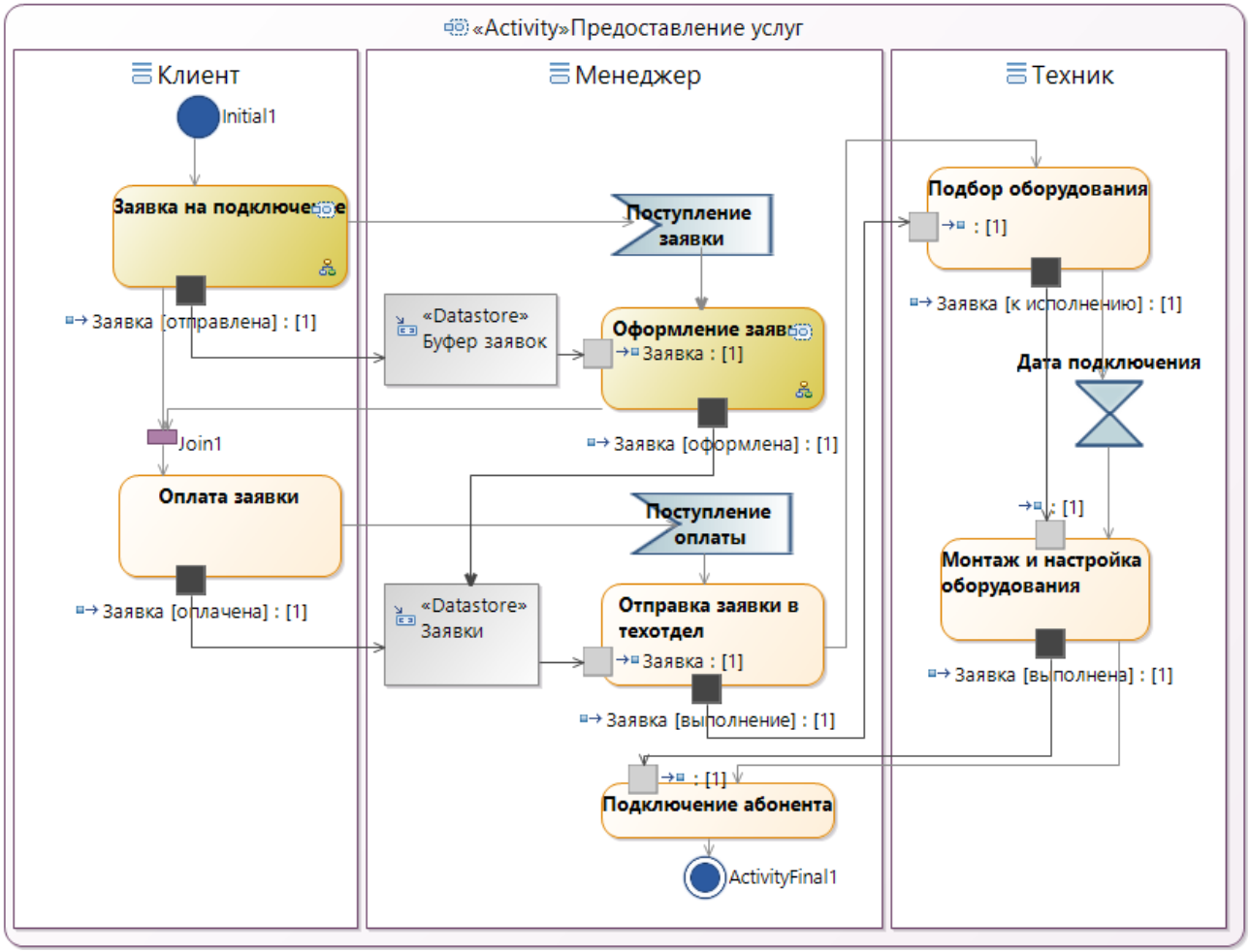


Рис. 8.60. Предоставление услуг

Диаграмма разделена на три раздела по количеству участников в бизнес-процессе. Действия на данной диаграмме показаны достаточно укрупненные – они соответствуют вариантам использования. Часть из действий описаны на других диаграммах деятельности, показанных выше. На диаграмме виден как поток управления, так и поток объектов деятельности. Для объекта **Заявка** показано изменение состояния заявки после выполнения каждого действия. Для сокращения диаграммы не показан класс объекта **Заявка**, т.к. он имеет тоже имя. Также для входящих контактов, для которых в связанных с ними исходящих контактах указан объект **Заявка**, не задано имя и тип контакта.

Элемент **Datastore** на диаграмме используется для двух целей. В первом случае, как буфер, в который временно помещаются заявки. Во втором случае – это база данных, в которую поступают оформленные менеджером заявки и постоянно хранятся там. При этом после оплаты заявка, находящаяся в хранилище данных **Заявки**, меняет свое состояние на **оплачена**.

Элементы принятия события используются для показа асинхронной работы. Когда клиент завершает выполнение действия **Заявка на подключение**, то посылается событие менеджеру о том, что поступила заявка. При этом клиент может продолжить свою деятельность (в данном примере эта деятельность на диаграмме не отражена). Параллельно с действиями клиента менеджер может приступить к оформлению заявки. Обратите внимание, что здесь не используется элемент ветвления (**Fork Node**), так как посылка сигнала уже означает распараллеливание потока управления.

Для начала выполнения действия в узле **Оплата заявки** должно произойти завершение выполнения действий двух параллельных потоков, поэтому используется узел объединения (**Join Node**). На практике это означает, что оплата заявки может быть выполнена клиентом только после ее отправки и завершения оформления заявки менеджером, так как в процессе оформления заявки менеджер общается с клиентом и может поменять некоторые параметры заявки, например, по желанию клиента установить не стандартное оборудование.

Так как менеджер выбирает заявки на оформление из буфера, то тем самым показывается, что он может обработать несколько поступивших заявок.

Событие времени **Дата подключения** используется на диаграмме для показа того, что техник выезжает к клиенту для монтажа и настройки оборудования, когда наступает дата, согласованная с клиентом.

9. Диаграмма состояний (State Machine Diagram)

Диаграмма состояний служит для описания состояний, в которых могут находится объекты системы. Диаграмма состояний, как и диаграмма деятельности, относится к динамическим диаграммам, которые показывают что происходит в системе со временем. На диаграмме изображаются состояния и переходы от одного состояния в другое состояние при возникновении событий. Она представляет собой некоторый конечный автомат.

9.1. Создание диаграммы состояний

Диаграмма состояний описывает изменения, происходящие с объектами системы с течением времени, поэтому ее можно поместить в пакет **Поведение**.

Для этого создадим диаграмму состояний с помощью меню, которое откроем по правой кнопке мышки, встав на строку пакета **<Package> Поведение** в виде Model Explorer. Выберем пункт **New Representation** и далее в открывшемся подменю пункт **State Machine Diagram**.

По умолчанию в пакете **Поведение** будет создан элемент **<State Machine > Поведение** и в ней диаграмма с именем **Поведение State Machine Diagram** (рис. 9.1).

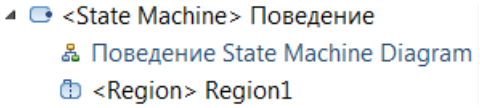


Рис. 9.1. Создание диаграммы состояний

Имя элемента и имя диаграммы можно изменить с помощью пункта **Rename** в меню, открывающемся по правой кнопке мышки на имени в виде Model Explorer. В общем случае эти имена могут совпадать. В папке **< State Machine >Поведение** будут находится также все элементы диаграммы состояний. По умолчанию при создании диаграммы в ней появляется элемент области (Region) с именем Region1.

В области редактирования появляется область для рисования диаграммы (рис. 9.2).

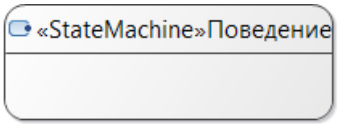


Рис. 9.2. Область рисования диаграммы состояний

Обратите внимание, что показано имя элемента класса <StateMachine>, а не диаграммы.

Чтобы увеличить область для рисования диаграммы кликните на строку с именем элемента, чтобы появились точки, за которые ее можно растянуть. Если кликнуть в нижнюю часть картинки, то область можно будет растянуть только по ширине.

Если диаграмма состояний описывает изменение состояний, в процессе выполнения некоторого варианта использования, то ее можно привязать к варианту использования. Откроем диаграмму **Use Case** и нажмем правую кнопку мышки, например, на варианте использования **Подключение абонента**, выберем пункт **New** и далее **State Machine diagram** (рис. 9.3).

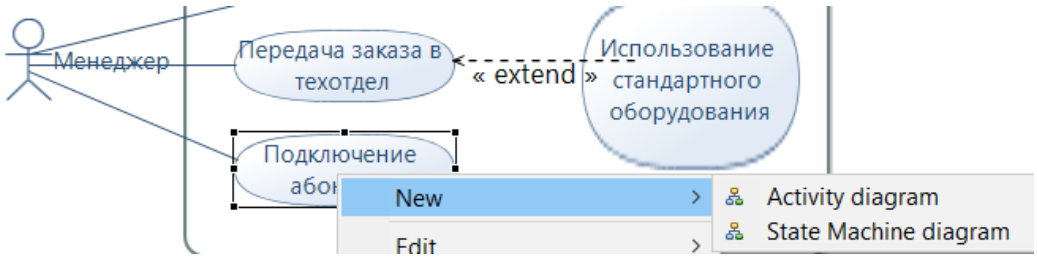


Рис. 9.3. Создание диаграммы состояний для варианта использования

В общем случае диаграмма состояний описывает изменение состояний объекта класса, поэтому ее можно привязать к этому классу. Для этого надо выбрать соответствующий класс, например, **Заявка** на диаграмме классов и нажать правую кнопку мышки. В появившемся меню выбрать пункт **New** и далее **State Machine diagram** (рис. 9.4).

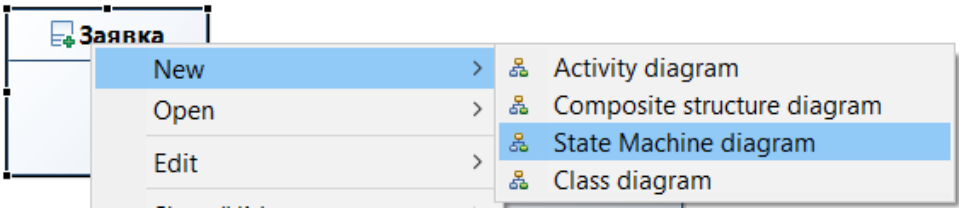


Рис. 9.4. Создание диаграммы состояний для класса

Такую диаграмму можно будет открывать, используя пункт меню Open (рис. 9.5).

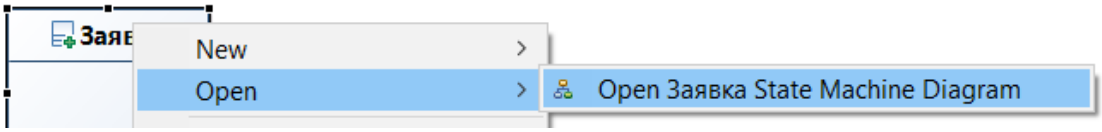


Рис. 9.5. Открытие диаграммы состояний для класса

Нужно помнить (раздел 18. Диаграмма деятельности), что во-первых, имя диаграммы, показываемое через пункт Open, не будет меняться при его изменении, и, во-вторых, создание двух диаграмм состояний для одного варианта использования или одного класса

будет означать, что эти диаграммы будут использовать одни и теже элементы, но их можно будет по разному расположить на диаграммах.

После создания диаграммы состояний для класса Заявка в элементе класса появляется элемент **<State Machine> Заявка statemachine** (имя по умолчанию), а внутри него элемент области **<Region> Region1**, в которой будут располагаться все остальные элементы диаграммы (рис. 9.6).

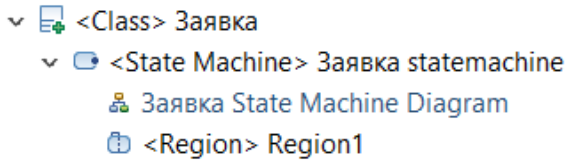


Рис. 9.6. Диаграмма состояний в Model Explorer

9.2. Элементы диаграммы состояний

Диаграмма состояний имеет палитру инструментов, показанную на рис. 9.7.

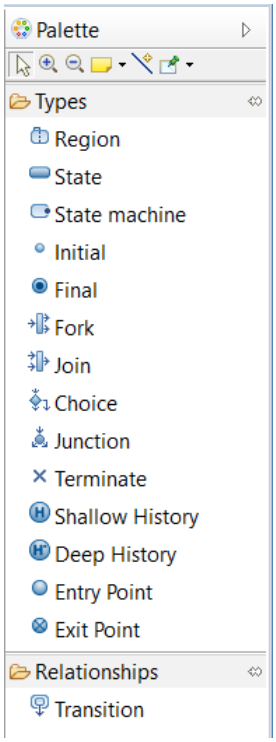


Рис. 9.7. Палитра инструментов диаграммы состояний

Элемент состояние (**State**) показывает некоторое состояние, в котором находится объект в определенный период времени своего существования. Состояние объекта определяется некоторым набором значений атрибутов объекта.

Элемент область (**Region**) используется для разделения диаграммы на несколько областей. В каждой области рисуется отдельная диаграмма состояний для разных объектов.

Состояние может быть составным, т.е. внутри состояния рисуется диаграмма состояний. Состояния внутренней диаграммы называются подсостояниями, а сама диаграмма описывает один или несколько конечных подавтоматов.

Для показа составного состояния используется элемент **State machine**. Как видно в Model Explorer этот элемент содержит внутри себя область, поэтому внутри этой области можно нарисовать отдельную диаграмму.

Например, так выглядит диаграмма с двумя областями, в первой области которой находится составное состояние StateMachine1, а во второй – состояние State2. Внутри составного состояние StateMachine1 находится состояние State1 (рис. 9.8).

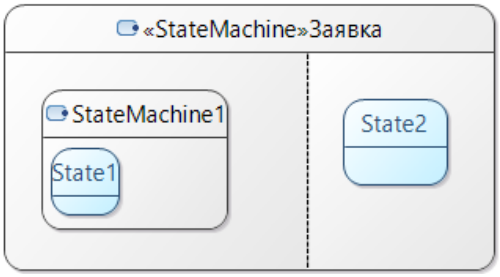


Рис. 9.8. Области и составное состояние

Составное состояние также может содержать несколько областей, в каждой из которых рисуется свой конечный автомат. Имена состояний в разных областях могут совпадать (рис. 9.9).

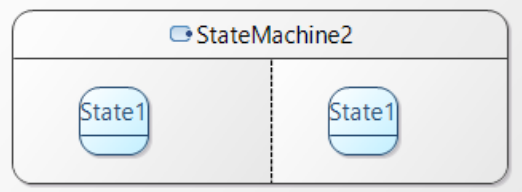


Рис. 9.9. Одноименные состояния в разных областях

Если кликнуть мышкой на некоторую область диаграммы, то левом верхнем углу области появится символ “-“ (рис. 9.10).

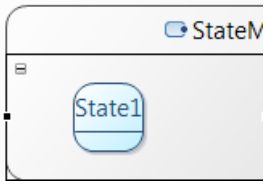


Рис. 9.10. Символ скрытия диаграммы

Кликнув на этот символ, можно скрыть диаграмму внутри области. При этом в верхнем левом углу появится символ “+“, который используется для показа диаграммы внутри области. Если вы видите пустую область на диаграмме, то это еще не значит, что на ней нет диаграммы. Кликните на область. Если при этом появляется значок “+“, то это означает, что область не пуста, а скрыта.

Составное состояние, разделенное на несколько областей, называется ортогональным состоянием. Подавтоматы каждой из областей составного состояния выполняются параллельно.

Как элемент состояние, так и элемент составное состояние в UML модели имеют класс State. Напомним, что в данном случае речь идет о классе элемента в языке UML, а не о классе, который представлен на диаграмме классов. Это видно в виде Model Explorer (рис. 9.11).

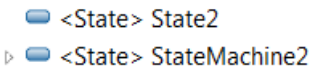


Рис. 9.11. Класс <State>

Отличием является наличие у составного состояния поля **Submachine**, которое видно в виде Properties (рис. 9.12).

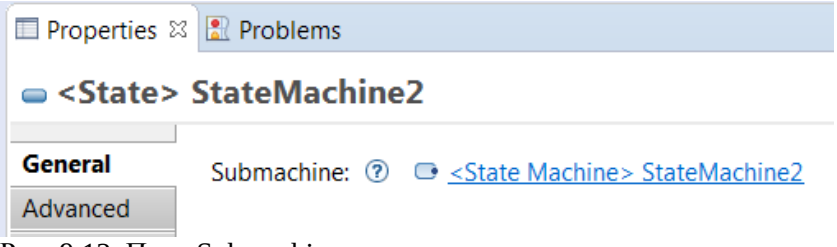


Рис. 9.12. Поле Submachine у составного состояния

Состояния соединены между собой переходами (**Transition**). Переход от одного состояния к другому состоянию происходит при возникновении некоторого события или по завершению деятельности в состоянии.

Для рисования перехода от состояния State2 в состояние State3 кликните на элемент Transition в палитре инструментов, затем на состояние State2, а потом на состояние State3 (рис. 9.13).



Рис. 9.13. Переход между состояниями

Нельзя соединить состояния из разных областей, например, состояние State1 и State2.

Для указания события, вызывающего переход, сторожевого условия, которое должно быть выполнено, чтобы переход осуществился, а также действия, вополняемого при переходе, кликните на стрелку и наберите текст (рис. 9.14).

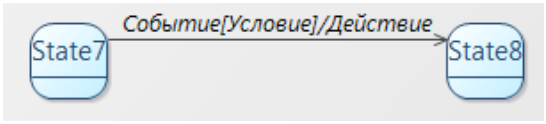


Рис. 9.14. Событие, условие и действие перехода

Диаграмма начинается с начального состояния (**Initial**):  и заканчивается финальным состоянием (**Final**):  .

Можно прервать работу конечного автомата, перейдя в завершающее состояние (**Terminate**):  .

Диаграмма, нарисованная внутри составного состояния, имеет, как и основная диаграмма, начальное и конечное состояние (рис. 9.15).

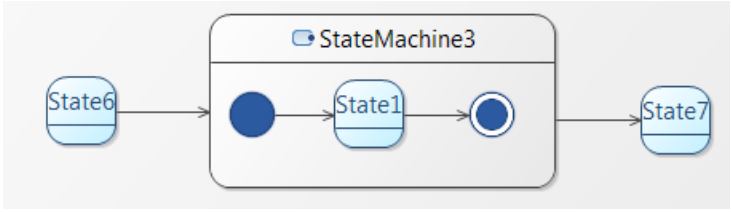


Рис. 9.15. Диаграмма составного состояния

Переход в составное состояние означает переход в начальное состояние диаграммы составного состояния. Попадание в конечное состояние диаграммы составного состояния означает выход из составного состояния. Если диаграмма составного состояния не имеет финального состояния, то выход из составного состояния может произойти, когда объект находится в любом состоянии составного состояния.

Для показа входа в составное состояние используется элемент точка входа (**Entry Point**), а для выхода из составного состояния – точка выхода (**Entry Exit**) (рис. 9.16).

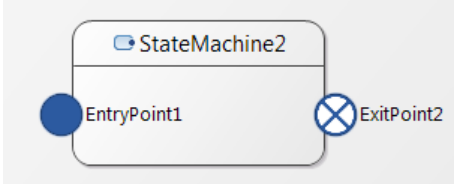


Рис. 9.16. Вход и выход из составного состояния

В стандарте UML точка входа используется, когда нужно показать вход в определенное подсостояние составного состояния, которое отличается от начального состояния. Точка выхода используется, когда нужно показать выход из составного состояния через определенное подсостояние составного состояния, отличающееся от финального состояния.

К сожалению, в UML Designer не удается соединить точку входа и точку выхода с другими подсостояниями и с состояниями вне составного состояния с помощью переходов.

Элемент выбора (**Choice**) используется в случае, когда одно событие в зависимости от сторожевого условия может привести к переходу из одного состояния в разные состояния (рис. 9.17).

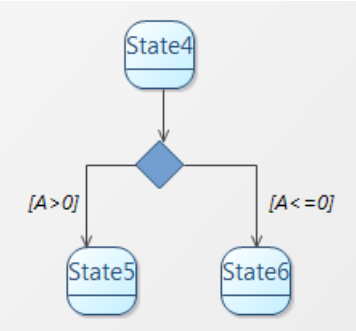


Рис. 9.17. Выбор

Элемент соединение (**Junction**) используется для тех же целей, что и Choice, но позволяет, как соединить несколько потоков в один, так и показать несколько выходов при разных сторожевых условиях. Например, по событию e1 из состояния State6 произойдет переход в состояние State9, если выполняются два условия: g > 5 и a = 2 (рис. 9.18).

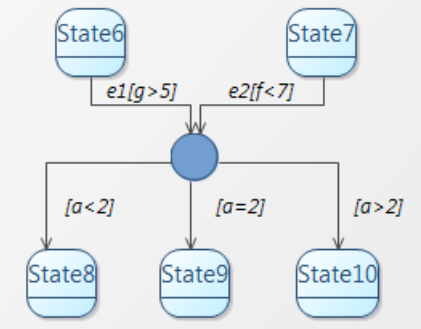


Рис. 9.18. Соединение

Элемент глубокой предистории (**Deep History**) при выходе из составного состояния запоминает подсостояние, расположенное на любом уровне вложенности состояний в составное состояние. Переход в этот элемент из состояния, расположенного вне составного состояния, позволяет вернуться в это подсостояние после выхода из составного состояния.

Например, если объект находился в состоянии State2 и произошел переход в состояние State4, то при переходе обратно из состояния State4 в составное состояние StateMachine5 через элемент глубокой предистории произойдет возврат к состоянию State2 (рис. 9.19).

Элемент неглубокой предистории (**Shallow History**) при выходе из составного состояния запоминает подсостояние, расположенное на первом уровне вложенности состояний в составное состояние. Переход в этот элемент из состояния, расположенного вне составного состояния, позволяет вернуться в это подсостояние после выхода из составного состояния.

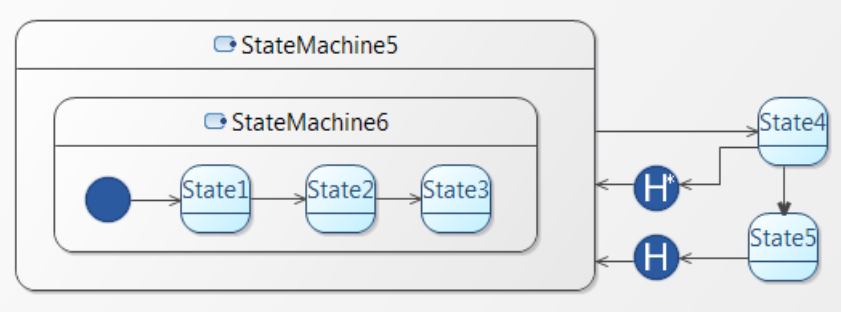


Рис. 9.19. Глубокая и неглубокая предистории

После попадания в состояние State4 может произойти переход в состояние State5. Если далее из этого состояния будет выполнен переход в составное состояние StateMachine5 через элемент неглубокой предистории, то произойдет возврат к началу конечного автомата, описанного в составном состоянии StateMachine6, т.е. выполнится переход в состояние State1.

Переход в составное состояние может запустить работу сразу нескольких подавтоматов, которые работают параллельно. В этом случае используется элемент ветвления (**Fork**). Для показа выхода из такого составного состояния, которое может произойти только в случае завершения работы всех подавтоматов составного состояния, используется элемент соединения (**Join**) (рис. 9.20).

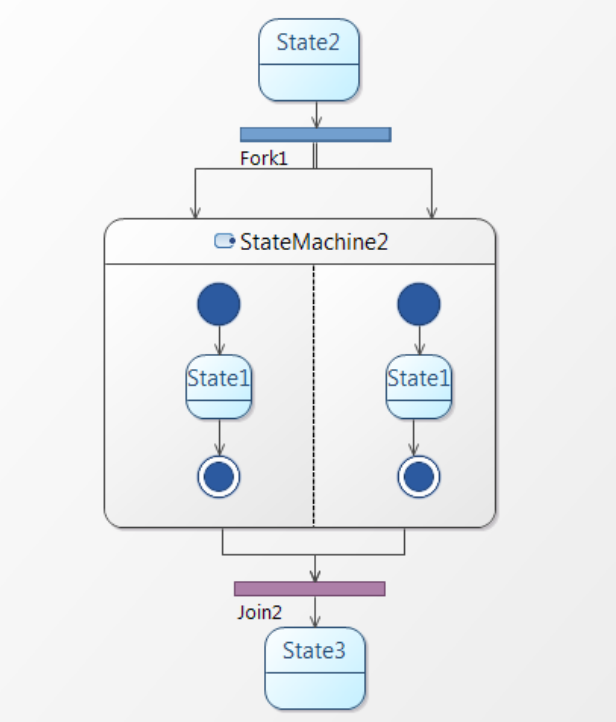


Рис. 9.20. Ветвление и соединение

9.3. Переход между состояниями

Для показа перехода от одного состояния в другое состояние используется элемент **Transition**. Его краткое описание было дано в предыдущем разделе. Для рисования диаграммы и задания события, сторожевого условия и действия для перехода вполне достаточно этого описания. Если же вы хотите более подробно разобраться со свойствами перехода и тем как описание перехода, записанное непосредственно на диаграмме, представлено в свойствах перехода, то читайте дальше.

Описание перехода можно изменить, либо в виде Properties, либо в окне Edit, которое открывается по двойному клику на линии перехода (рис. 9.21).

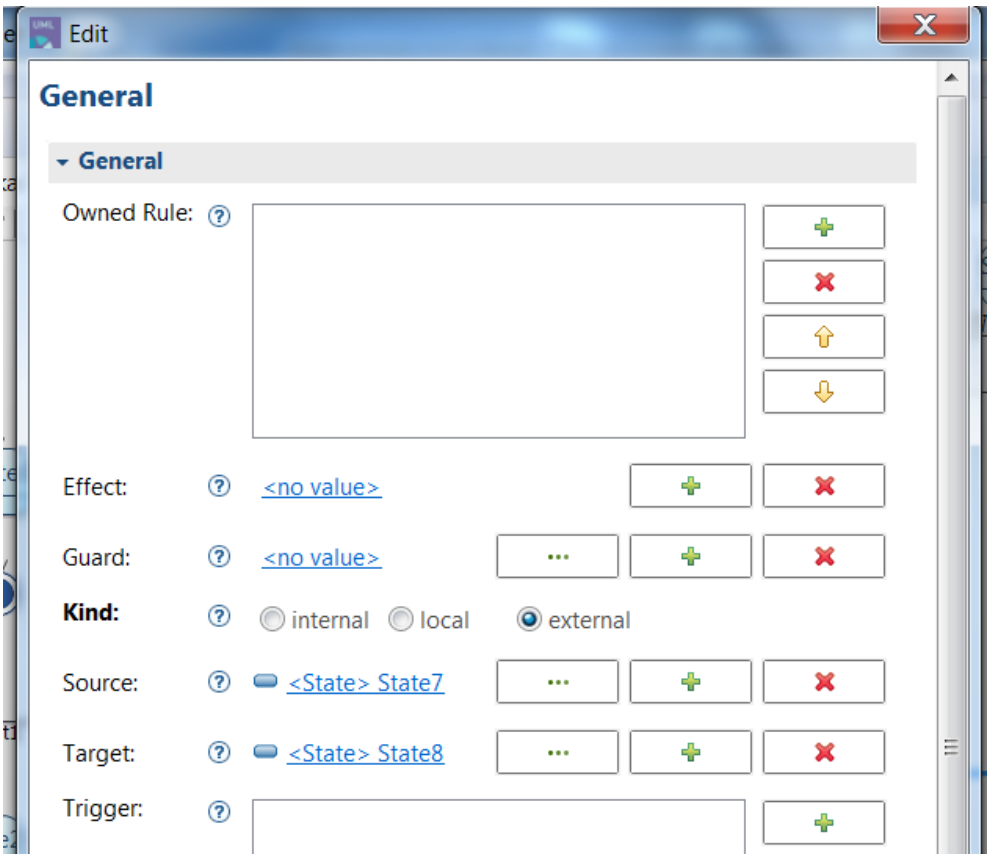


Рис. 9.21. Описание перехода в окне редактирования

Если вы решили описать событие, сторожевое условие и действие, используя вид Properties или окно Edit, то не надо их задавать непосредственно на диаграмме, как это было описано в предыдущем разделе. Можно, конечно, задать сначала описание перехода непосредственно на диаграмме, а потом изменить его, используя Properties и окно Edit.

Для указания события кликнете на кнопку  в поле **Trigger** и далее на кнопку **Finish** в появившемся окне **Create** (рис. 9.22).

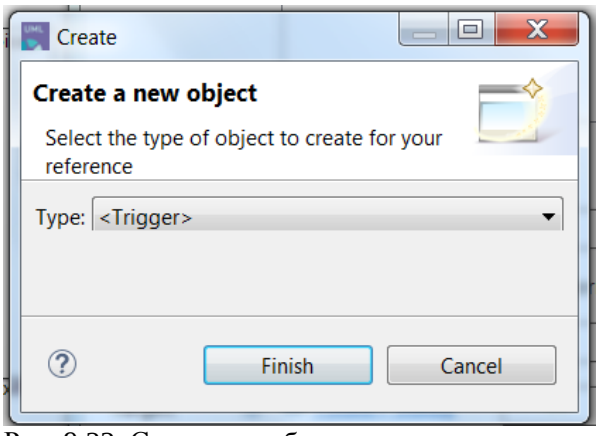


Рис. 9.22. Создание события

Поле **Trigger** примет значение элемента класса <Trigger> (рис. 9.23).



Рис. 9.23. Событие в поле Trigger

По клику на строке <Trigger> в поле **Trigger** окна **Edit** открывается окно редактирования события **Edit Many**, в котором укажем имя события (рис. 9.24).

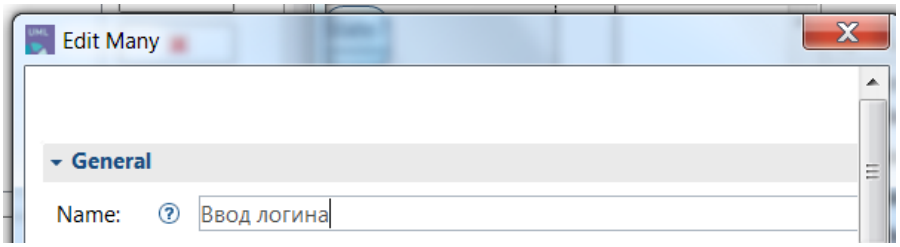


Рис. 9.24. Изменение имени события

После закрытия окна **Edit Many** и окна **Edit** переход на диаграмме состояний будет иметь имя события (рис. 9.25).

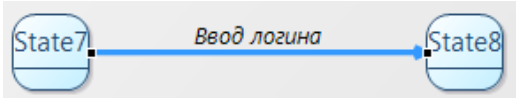


Рис. 9.25. Переход с указанием события

Надпись, описывающая переход, может быть отредактирована прямо на диаграмме. Кликните на нее мышкой, а потом еще раз непосредственно на надписи. Надпись также можно перетащить мышкой на другое место. Кликните, потом нажмите мышку и тащите.

Для удобства работы с конкретным переходом лучше задать для него имя в поле **Name** раздела **Advanced** вида Properties (рис. 9.26).

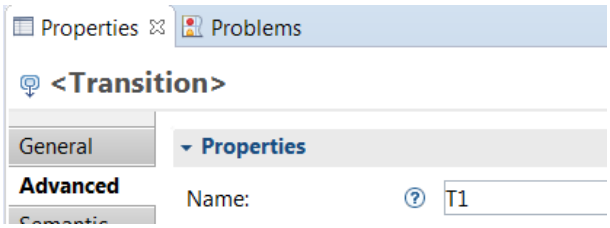


Рис. 9.26. Задание имени перехода

В этом случае элемент перехода легко найти в Model Explorer по имени (рис. 9.27).

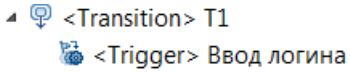


Рис. 9.27. Переход и событие в Model Explorer

Для определения сторожевого условия перехода кликните на кнопку  в поле **Guard** окна **Edit**, в котором показаны свойства перехода, и в появившемся окне **Create** выберите строку с моделью **<Model> Internet Provider** и нажмите кнопку **Finish** (рис. 9.28).

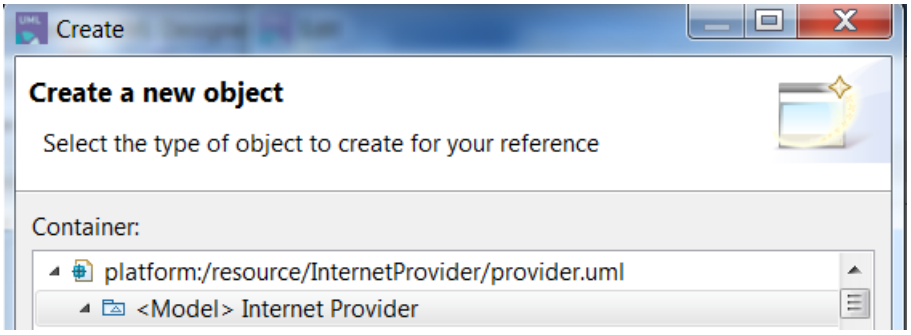


Рис. 9.28. Создание сторожевого условия

По умолчанию в поле **Guard** и в поле **Owned Rule** появятся элементы класса <Constraint> (рис. 9.29).

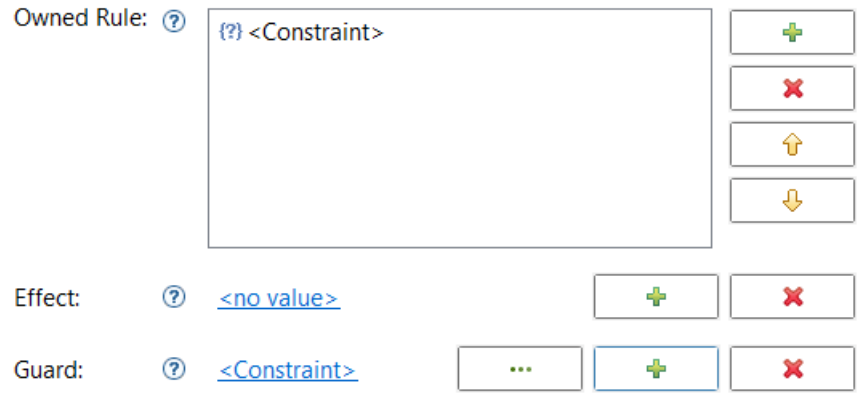


Рис. 9.29. Показ сторожевого условия

В окне **Create** можно выбрать класс сторожевого условия в поле **Type** (рис. 9.30).

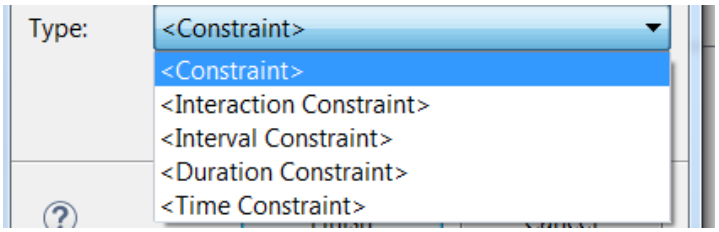


Рис. 9.30. Выбор класса сторожевого условия

Например, если при возникновении события нужно проверить момент времени наступления этого события, то выберите класс <Time Constraint>.

Если нужно поменять класс условия, то необходимо удалить элементы из поля **Guard** и **Owned Rule** с помощью кнопки  и повторить процедуру определения условия для поля **Guard**.

Далее по двойному клику на строке <Constraint> в поле **Owned Rule** или поле **Guard** открывается окно редактирования условия (рис. 9.31).

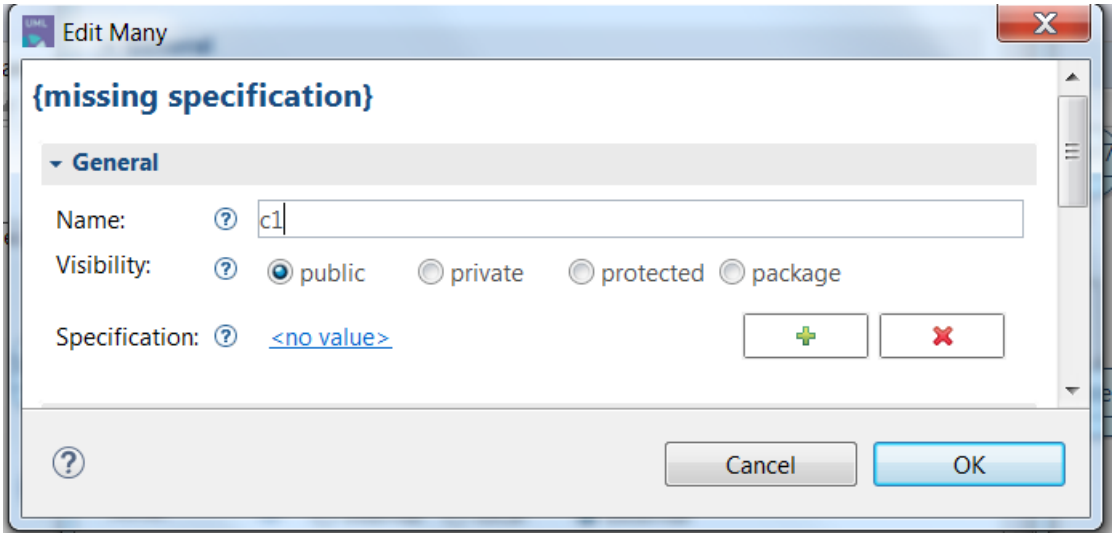


Рис. 9.31. Указание имени условия

В этом окне укажите имя условия в поле **Name** и добавьте описание спецификации, кликнув на кнопку  в поле **Specification**.

В открывшемся окне **Create** выберите в поле **Type** класс выражения для задания условия. В простейшем случае в поле **Type** выберем класс <String Expression> (рис. 9.32).

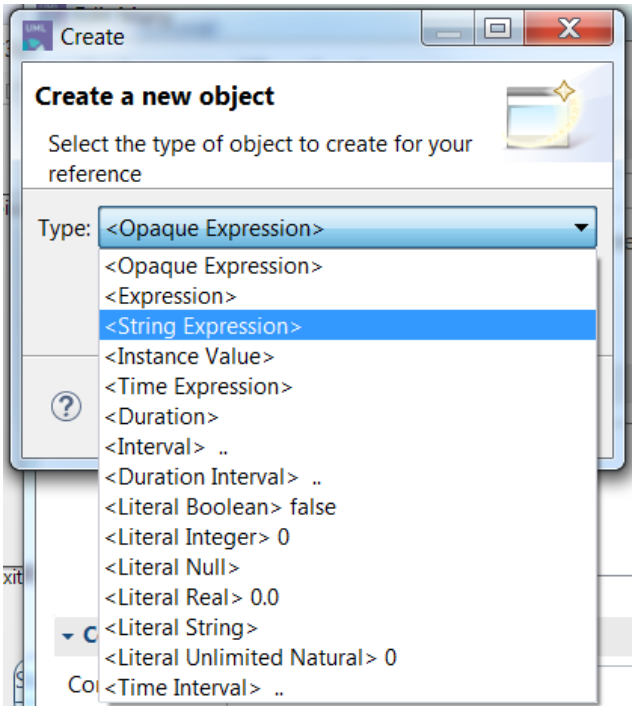


Рис. 9.32. Определение типа условия

В окне **Edit Many** для поля **Specification** появится элемент этого класса (рис. 9.33).



Рис. 9.33. Описание спецификации условия

По клику на строке <String Expression> в поле **Specification** открывается окно редактирования выражения, определяющего условие наступления события. В поле **Name** напишем условие (рис. 9.34).

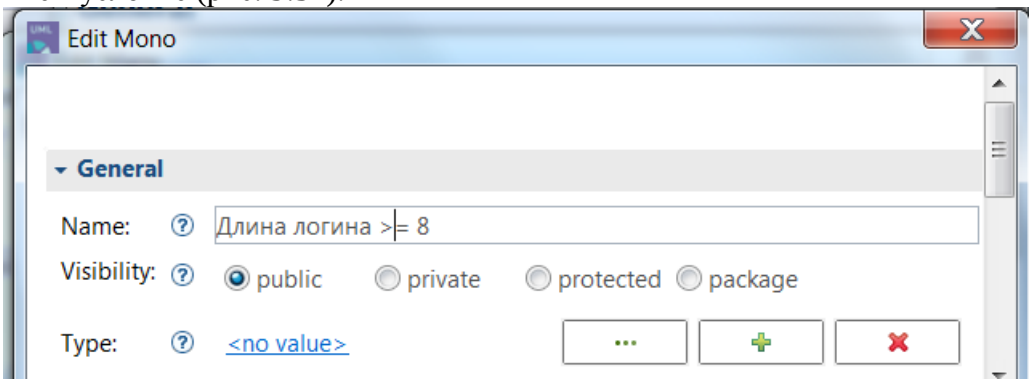


Рис. 9.34. Задание условия

Закрыв все окна редактирования, в Model Explorer увидим описание условия (рис. 9.35).

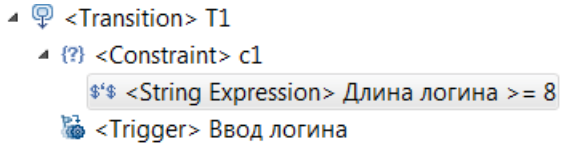


Рис. 9.35. Показ условия в Model Explorer

На диаграмме появится описание условия перехода (рис. 9.36).

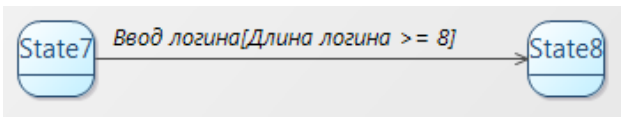


Рис. 9.36. Переход с событием и строжевым условием

Для указания действия, выполняемого при переходе из одного состояния в другое состояние, кликните на кнопку  в поле **Effect** окна редактирования **Edit** или виде Properties для элемента перехода. В появившемся окне **Create** выберите в поле **Type** класс действия и нажмите кнопку **Finish** (рис. 9.37).

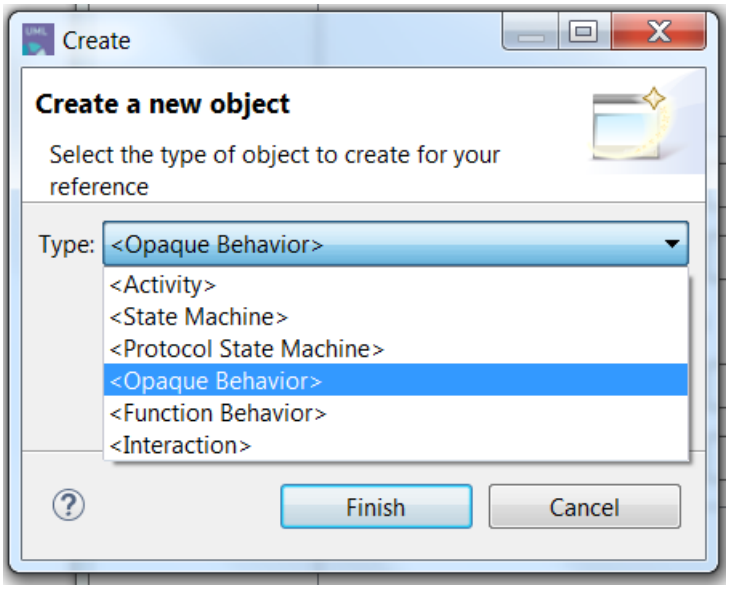


Рис. 9.37. Выбор класса действия

Класс <Activity> позволяет описать действие в форме диаграммы деятельности.

Класс <State Machine> позволяет описать действие в форме диаграммы состояний.

Класс <Protocol State Machine> позволяет описать действие в форме диаграммы состояний описания некоторого протокола использования объекта. Протокол также описывает состояния, события и переходы, но не описывает действия, т.е. поведение объекта. Классы <State Machine> и <Protocol State Machine> используют в UML Designer одну и туже палитру инструментов диаграммы состояний.

Класс <Opaque Behavior> позволяет описать в текстовом виде действие на любом языке.

Класс <Function Behavior> позволяет также описать в текстовом виде действие на любом языке. В стандарте UML этот класс является подклассом класса <Opaque Behavior> и отличается от него тем, что не получает доступ и не изменяет какие-либо объекты или другие внешние данные, т.е. с точки зрения некоторого языка программирования является описанием функции.

Класс <Interaction> позволяет описать действие в форме диаграммы последовательности.

Выберем класс <Opaque Behavior>. В окне Edit в поле Effect появится элемент класса <Opaque Behavior> (рис. 9.38).



Рис. 9.38. Описание типа действия

По клику на строке <Opaque Behavior> в поле **Effect** открывается окно редактирования действия, выполняемого при переходе. В поле **Name** напишем имя действия (рис. 9.39).

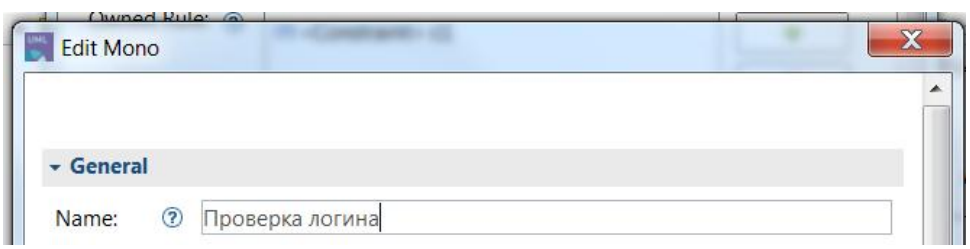


Рис. 9.39. Задание имени действия

Закрыв окна редактирования **Edit Mono** и **Edit** по кнопке **OK**, в Model Explorer увидим, что переход **<Transition> T1** содержит элементы, описывающие событие класса <Trigger>, условие класса <Constraint> с выражением класса <String Expression> и действие класса <Opaque Behavior> (рис. 9.40).

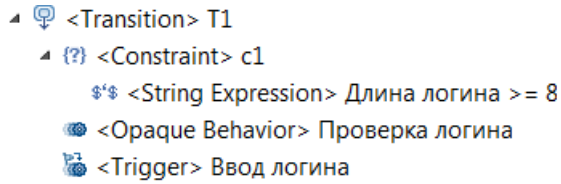


Рис. 9.40. Описание перехода в Model Explorer

На диаграмме появится имя действия, выполняемого при переходе (рис. 9.41).

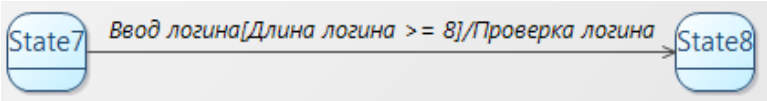


Рис. 9.41. Изображение события, условия и действия, связанных с переходом

Для подробного описания действия на некотором языке программирования или на псевдокоде кликните на строку <Opaque Behavior> в Model Explorer. Вид Properties в разделе **Advanced** содержит поле **Bod**y, в котором можно написать код программы или любой текст описания действия, и поле **Lanquage** для указания языка программирования (рис. 9.42).

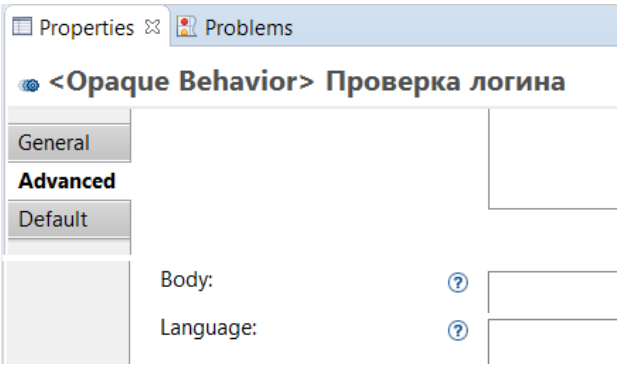


Рис. 9.42. Подробное описание действия

В предыдущем разделе было создано описание перехода с помощью ввода текста на диаграмме (рис. 9.43).

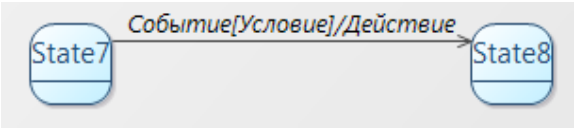


Рис. 9.43. Описание перехода на диаграмме

Это описание представлено в Model Explorer элементами, показанными на рис. 9.44.

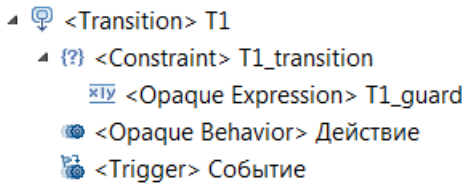


Рис. 9.44. Описание перехода в Model Explorer

Отличием от выше приведенного способа описания условия является задание условия в элементе **<Opaque Expression> T1_quard**. Кликнув на этот элемент в виде Properties увидим **Условие** в поле Body (рис. 9.45).

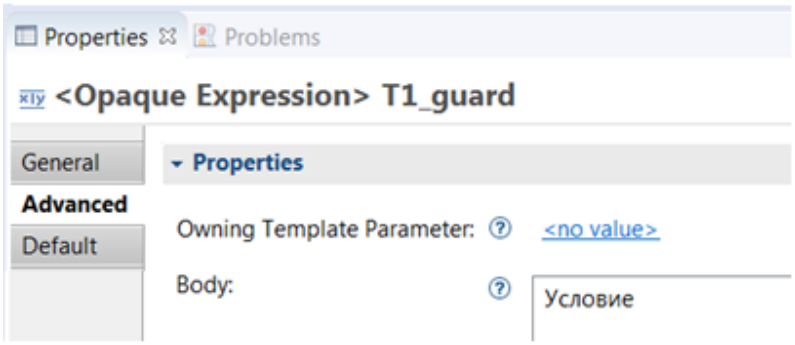


Рис. 9.45. Показ условия в Properties

В качестве класса действия можно выбрать один из классов **Activity**, **State Machine**, **Protocol State Machine** или **Interaction**, позволяющий описать другую диаграмму. Выберем, например класс Activity и присвоим имя **Действие** элементу этого класса, как описано выше. Тогда в Model Explorer, нажав правую кнопку мышки на элементе **<Activity> Действие**, откроется меню, с помощью которого можно создать диаграмму деятельности (рис. 9.46).

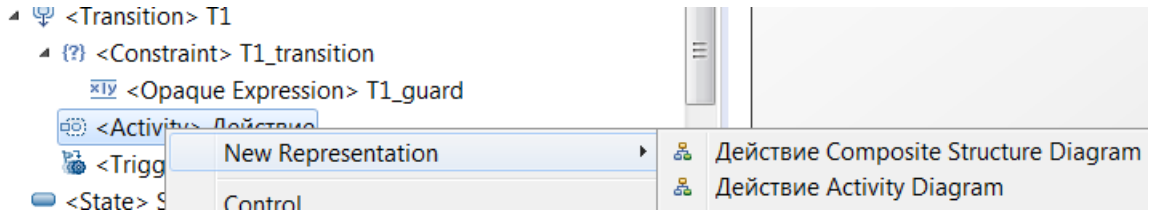


Рис. 9.46. Создание диаграммы деятельности для описания действия перехода

9.4. Действия в состоянии

Выполнение действий на диаграмме состояний возможно не только при переходе из одного состояния в другое состояние, но также при нахождении объекта в состоянии.

Действия могут быть описаны во время создания состояния в окне **Create State** (рис. 9.47).

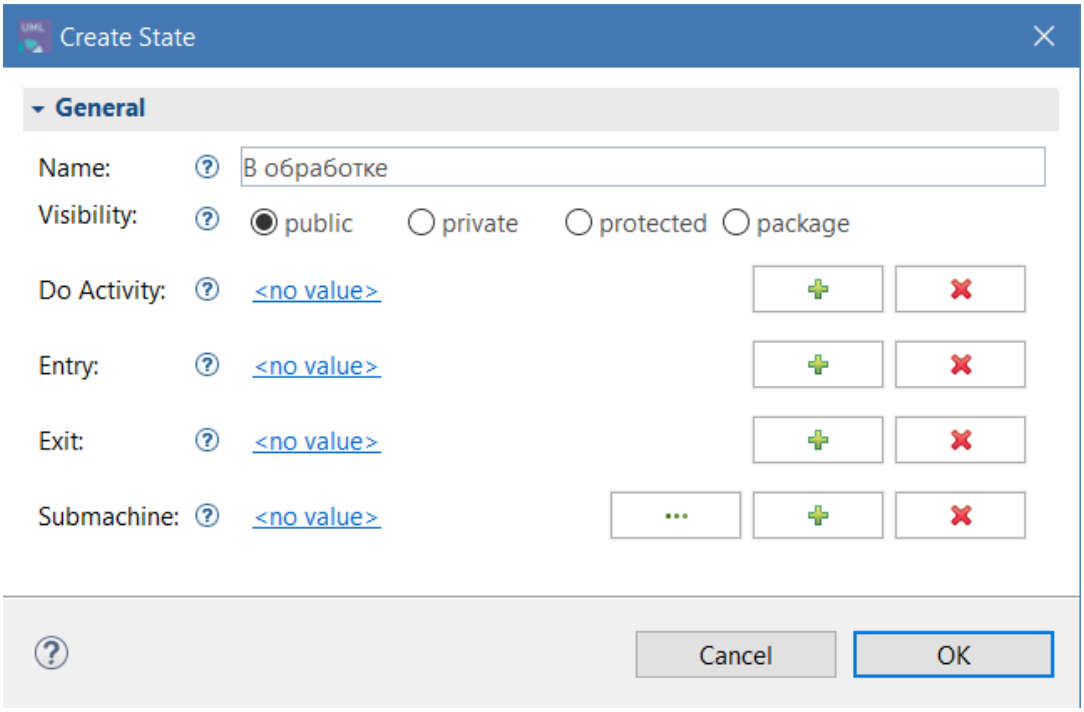


Рис. 9.47. Описание действий в состоянии при его создании

Если состояние уже создано, то действия объекта могут быть описаны в виде Properties или в окне редактирования состояния Edit, открываемого по двойному клику мышки на состоянии.

В состоянии могут быть выполнены три типа действий:

- **Entry** действие выполняется при входе в состояние. При этом не важно по какому переходу произошел вход в состояние;
- **Do Activity** действие выполняется во время нахождения объекта в состоянии;
- **Exit** действие выполняется при выходе из состояния. При этом не важно по какому переходу произойдет выход из состояния.

Для описания действия кликните на кнопку  в строке одного из действий в окне Create State, или виде Properties, или окне Edit. Появится такое же окно Create, как в случае описания действия при переходе. Дальнейшее описание действия аналогично описанию действия для перехода (предыдущий раздел).

Например, так будет выглядеть состояние объекта Заявка, в котором определены действия класса < Opaque Behavior > (рис. 9.48).

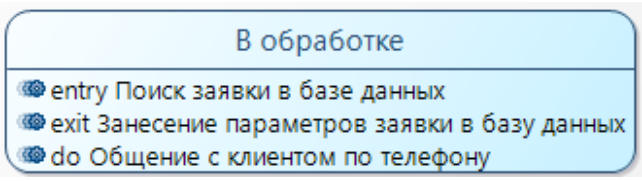


Рис. 9.48. Действия в состоянии

На диаграмме показаны имена действий. Сами действия могут быть подробно описаны в полях **Body** каждого из действий. Свойства каждого действия могут быть отредактированы в виде Properties по клику на строку элемента действия в Model Explorer (рис. 9.49).

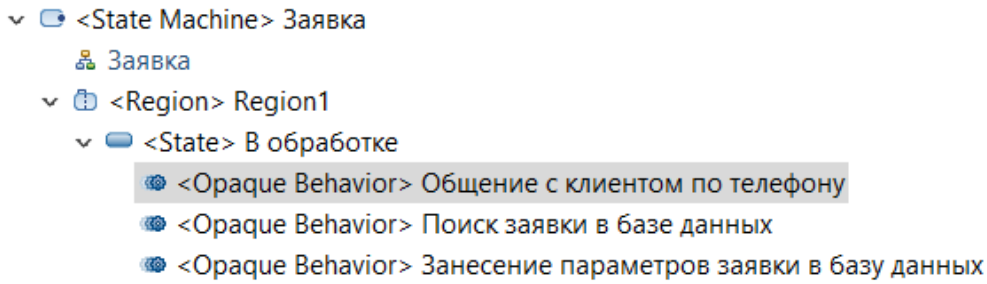


Рис. 9.49. Действия в состоянии в Model Explorer

Если в состоянии показаны действия, то при задании перехода мышку надо навести, либо на имя состояния, либо на пустое место внутри состояния, но не на имя действия.

9.5. Пример диаграммы состояний

Рассмотрим диаграмму состояний для объекта класса Заявка, показанную на рис. 9.50. Основные состояния объекта уже были определены на диаграмме деятельности **Предоставление услуг**. Данная диаграмма привязана к классу **Заявка** на диаграмме классов (раздел 18.1. Создание диаграммы состояний).

Диаграмма начинается с состояния **Создание**, в котором происходит ввод данных о клиенте.

После отправки заявки клиентом происходит переход в состояние **Отправлена**, при входе в которое формируется дата отправки. Когда менеджер берет заявку на обработку, она переходит в состояние **В обработке**. При входе в это состояние менеджер выбирает из базы данных информацию о заявке. Далее мееджер общается с клиентом по телефону и уточняет данные по заявке. Когда обработка закончена, то параметры заявки сохраняются в базу

данных и происходит переход в состояние **Оформлена**. Из этого состояния при поступлении оплаты происходит переход в состояние **Оплачена** и сохраняются данные об оплате. Когда менеджер отправляет заявку в техотдел она получает состояние **Выполнение**. В нем происходит подбор оборудования для установки клиенту. По окончанию подбора оборудования заявка переходит в состояние **К исполнению**. Когда наступает дата установки, если клиент находится дома, то заявка переходит в состояние **Монтаж оборудования**, в котором техник выезжает к клиенту и выполняет монтаж и настройку оборудования. По завершению работ заявка переходит в состояние **Выполнена**. В этом состоянии менеджер подключает абонента и сохраняет информацию о выполненной заявке и новом клиенте в базе данных. На этом диаграмма заканчивается.

Диаграмма деятельности и диаграмма состояний дополняют друг друга, хотя некоторая информация на них дублируется.

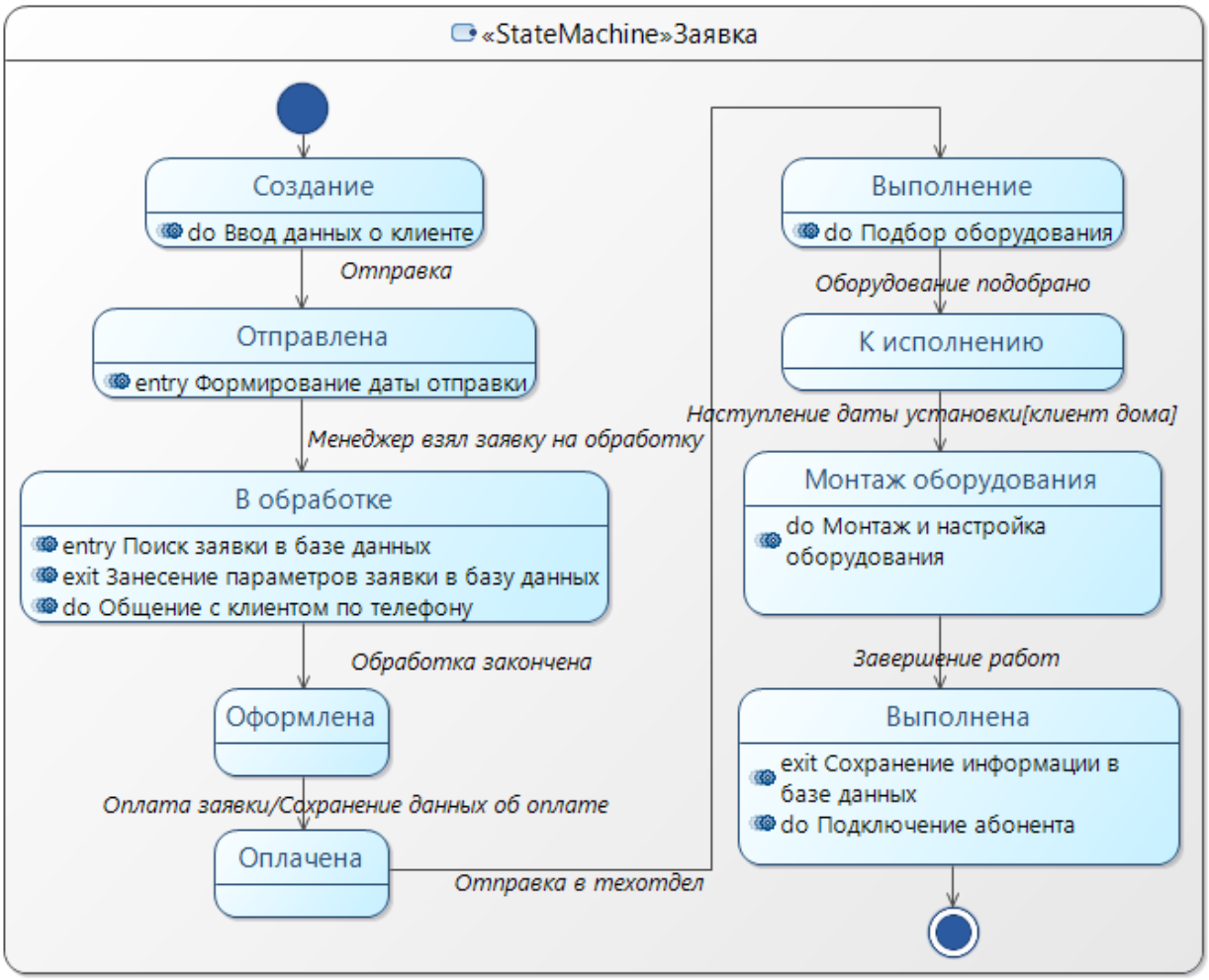


Рис. 9.50. Пример диаграммы состояний

10. Диаграмма последовательности (Sequence Diagram)

Диаграмма последовательности служит для показа обмена сообщениями между актерами и объектами классов. Вместе с диаграммой коммуникации (Communication Diagram), которая не реализованиа в UML Designer, она относится к диаграммам взаимодействия (Interaction Diagram). Взаимодействие между актерами и объектами показывается во времени, поэтому данная диаграмма относится к диаграммам поведения или динамическим диаграммам. На диаграмме показывается обмен сообщениями во время выполнения некоторого потока событий (сценария) варианта использования. Диаграмма может отражать полный поток событий или некоторые его подпотоки. В результате для описания потока событий варианта использования может потребоваться несколько диаграмм последовательности.

10.1. Создание диаграммы последовательности

Диаграмма последовательности описывает выполнение некоторого варианта использования, поэтому логично было бы привязать ее к некоторому варианту использования, как это было сделано для диаграмм деятельности. Но в UML Designer нет такой возможности. Поэтому поместим диаграмму последовательности в пакет **Поведение**.

Для этого создадим диаграмму последовательности с помощью меню, которое откроем по правой кнопке мышки, встав на строку пакета **<Package> Поведение** в виде Model Explorer. Выберем пункт **New Representation** и далее в открывшемся подменю пункт **Sequence Diagram**. В результате будет создан элемент **Поведение** класса **Interaction** и внутри него диаграмма, имя которой взято по умолчанию (рис. 10.1).

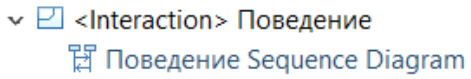


Рис. 10.1. Диаграмма последовательности в Model Explorer

Все элементы диаграммы будут располагаться внутри элемента **<Interaction> Поведение**.

10.2. Элементы диаграммы последовательности

Диаграмма последовательности имеет палитру инструментов, показанную на рис. 10.2.

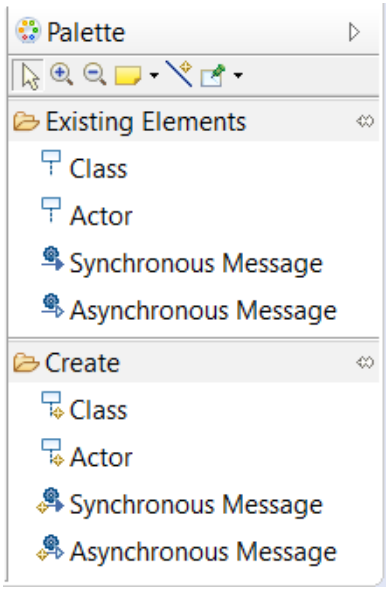


Рис. 10.2. Палитра инструментов диаграммы последовательности

На диаграмме последовательности можно выбирать существующие элементы (что предпочтительно) с помощью элементов раздела **Existing Elements** или создавать новые элементы с помощью элементов раздела **Create.**

10.2.1. Актеры и объекты

Элемент актер (**Actor**) используется для показа актеров, участвующих в варианте использования, который описывается с помощью диаграммы последовательности. При этом, показываются только те актеры, которые непосредственно взаимодействуют с системой.

Например, в варианте использования **Оформление заявки** принимают участие актеры **Менеджер** и **Клиент**, но если с системой взаимодействует только менеджер, а общение менеджера с клиентом происходит по телефону, то нет необходимости показывать актера **Клиент** на диаграмме. Для использования существующего актера нужно кликнуть на элемент актер в разделе существующие элементы, а затем на диаграмму. Появится окно Selection Wizard, в котором нужно выбрать актера (рис. 10.3).

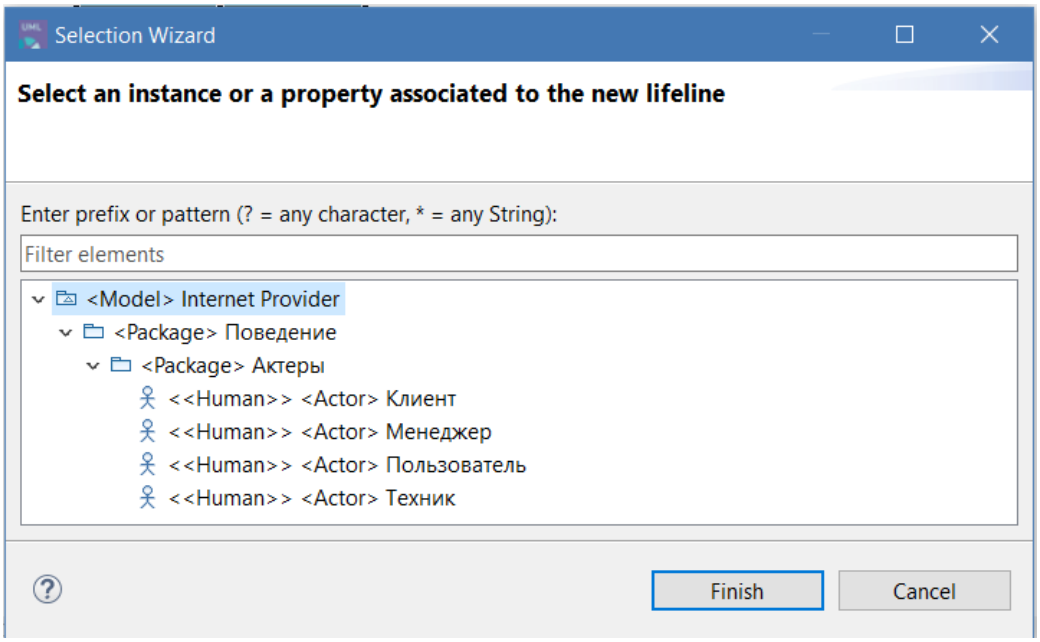


Рис. 10.3. Выбор актера

При создании нового актера с помощью элемента из раздела **Create** палитры инструментов появится окно, в котором можно указать имя нового актера.

На диаграмме актер изображается с помощью прямоугольника с линией жизни (пунктирная линия) и его имя дублируется в верхней строке диаграммы (рис. 10.4).

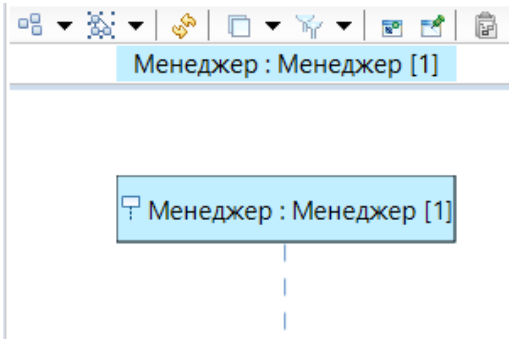


Рис. 10.4. Изображение актера

В виде Model Explorer появится элемент класса **Property**, описывающий актера, и элемент линии жизни актера класса **Lifeline** (рис. 10.5).

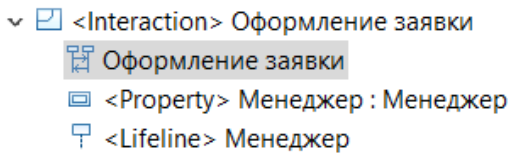


Рис. 10.5. Актер и его линия жизни в Model Explorer

Кликнув на строку **<Property> Менеджер:Менеджер**, в виде Properties в поле **Name** можно указать имя конкретного менеджера (рис. 10.6).

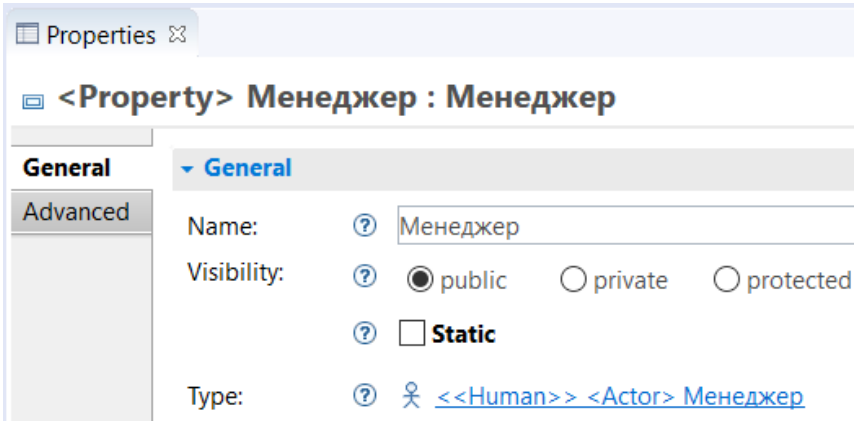


Рис. 10.6. Описание актера в виде Properties

Обычно на диаграмме последовательности не указывают конкретных имен. Если имя актера в поле Name совпадает с его именем в поле Type, то считается что это произвольный актер, который выполняет данную роль. В стандарте UML произвольного актера принято изображать без конкретного имени, например **:Менеджер**, поэтому его называют анонимным.

Имя актера может понадобиться, если на диаграмме изображаются два актера, выполняющих одну и туже роль. Например, если бы в оформлении заявки принимали участие два менеджера.

Элемент класс (**Class**) используется для показа объектов классов, которые участвуют во взаимодействии с актером и между собой во время выполнения варианта использования. При помещении существующего класса на диграмму открывается окно выбора класса (рис. 10.7).

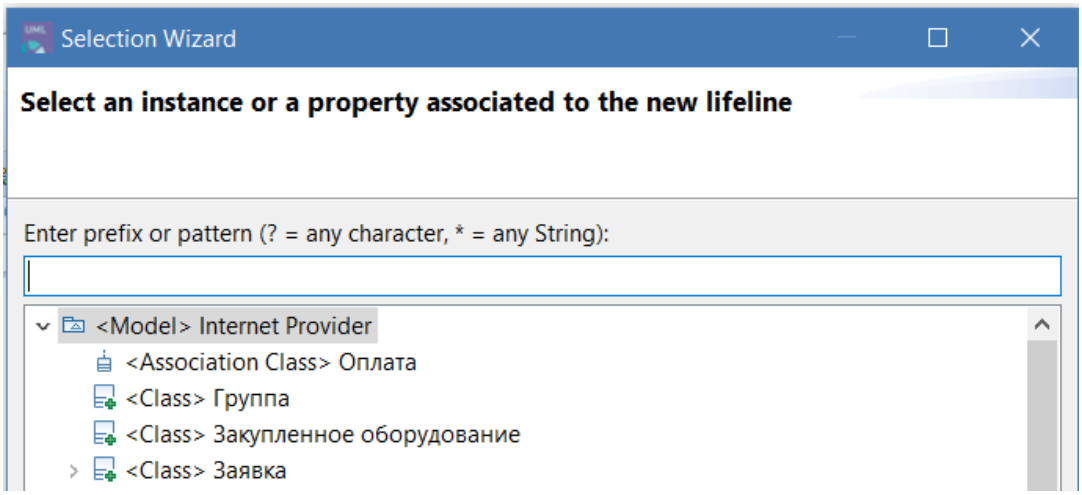


Рис. 10.7. Выбор существующего класса

Можно также создать новый класс с помощью элемента из раздела **Create** палитры инструментов.

На диаграмме класс изображается аналогично актеру (рис. 10.8).

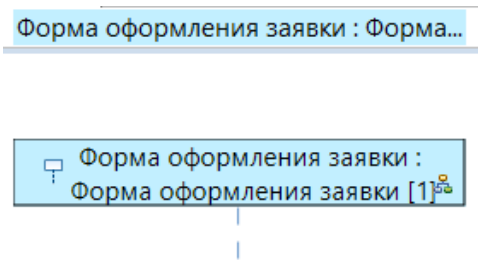


Рис. 10.8. Изображение класса

В данном случае использовался уже имеющийся на диаграмме классов класс, поэтому на его изображении на диаграмме последовательности в правом нижнем углу появляется иконка  . Она означает, что по нажатию на этом элементе правой кнопки мышки можно через пункт меню **Open** открыть диаграмму классов (рис. 10.9).



Рис. 10.9. Открытие диаграммы классов, содержащей класс, показанный на диаграмме последовательности

В Model Explorer можно увидеть, что появился объект и линия его жизни (рис. 10.10).

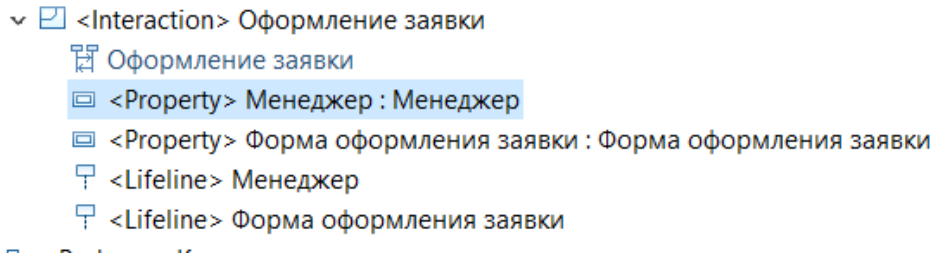


Рис. 10.10. Объект и его линия жизни в Model Explorer

Актеры и объекты классов, размещенные на диаграмме, видны также в качестве атрибутов элемента **<Interaction> Оформление заявки** в виде Properties (рис. 10.11).

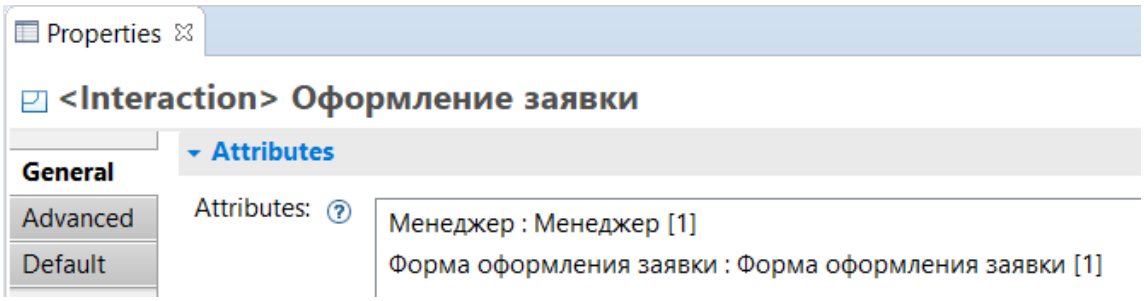


Рис. 10.11. Актеры и объекты в виде Properties

По умолчанию имя объекта совпадает с именем класса, что означает, что это произвольный объект. В стандарте UML принято изображать объект без имени (анонимный объект), например **:Форма оформления заявки**. Как и для актера, имя объекта на диаграмме последовательности имеет значение в случае изображения нескольких объектов одного класса. Для того, чтобы изменить имя объекта надо кликнуть на него на диаграмме. После этого в виде Properties покажется описание линии жизни объекта и нужно кликнуть на описание объекта в поле **Represents** (рис. 10.12).

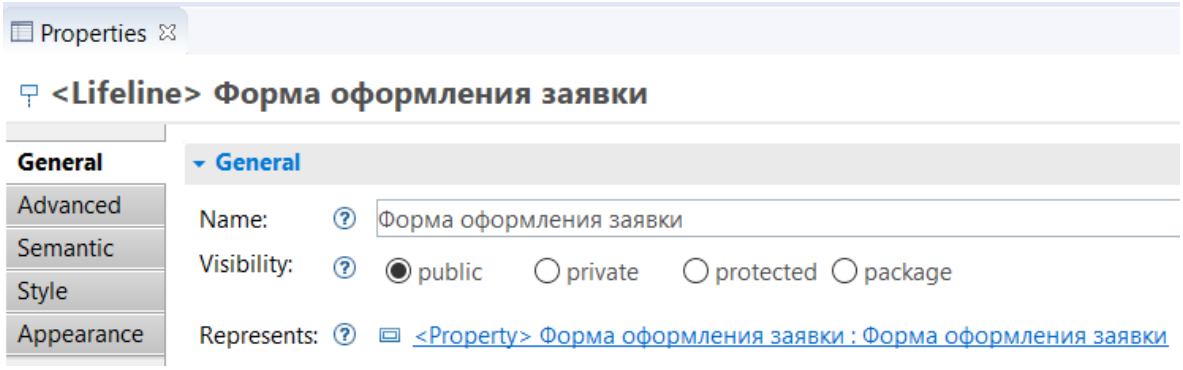


Рис. 10.12. Описание линии жизни объекта

Появится окно, в котором можно изменить имя объекта (рис. 10.13).

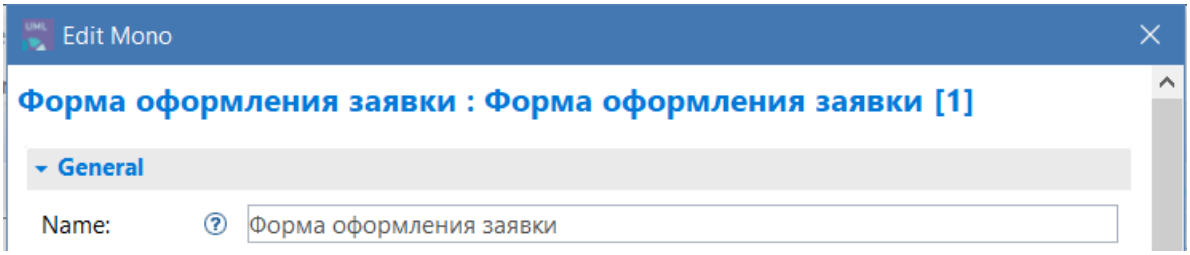


Рис. 10.13. Изменение имени объекта

Также можно сразу перейти к описанию объекта в виде Properties, если кликнуть на описание объекта в Model Explorer (рис. 10.14).

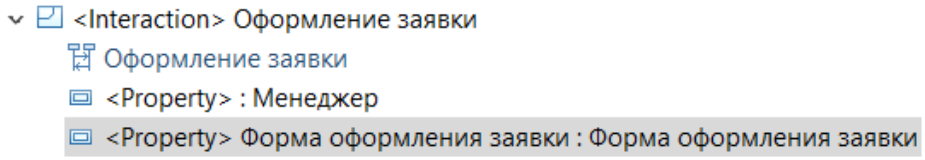


Рис. 10.14. Описание объекта в Model Explorer

Для каждого объекта на диаграмме последовательности можно также указать его стереотип в виде Properties (рис. 10.15).

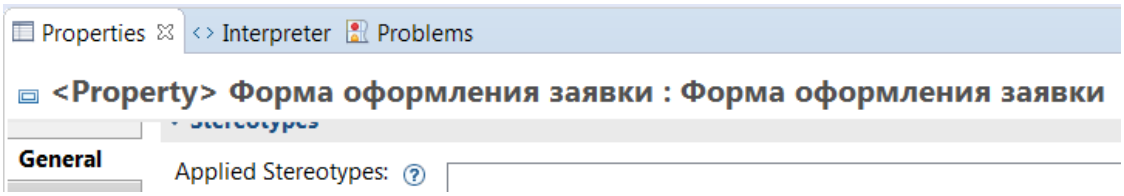


Рис. 10.15. Описание стереотипа объекта

Компанией Rational Software была разработана методология проектирования, названная Rational Unified Process (RUP). Согласно этой методологии все классы можно разбить на три группы, присвоив им следующие стереотипы:

- **boundary** - граничные классы, реализующие интерфейс взаимодействия пользователя с системой;
- **control** – управляющие классы, реализующие логику работы системы при реализации вариантов использования;
- **entity** – классы-сущности, описывающие сущности реального мира.

UML Designer не позволяет явно указать эти стереотипы. Если мы хотим показать стереотип объекта, на диаграмме последовательности, которого нет среди стандартных стереотипов, то его можно записать в кавычках перед именем объекта. Например, можно для анонимного объекта класса **Форма оформления заявки** вместо имени объекта указать только стереотип, тогда на диаграмме объект будет иметь вид, показанный на рис. 10.16.

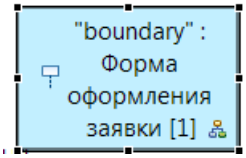


Рис. 10.16. Граничный объект

10.2.2. Сообщения

Актер общается с объектами и объекты общаются между собой с помощью передачи сообщений. Каждое сообщение представляет собой вызов некоторой операции класса.

На диаграмме сообщения изображаются с помощью элементов синхронное сообщение (**Synchronous Message**) и асинхронное сообщение (**Asynchronous Message**). Если

используются уже определенные операции классов, то эти элементы выбираются из раздела существующих элементов палитры инструментов. Новые операции могут быть созданы с помощью элементов из раздела Create.

Для показа сообщения сначала кликните на элемент сообщения в палитре инструментов, затем на линию жизни актера или объекта, который посылает сообщение (вызывает операцию), а потом на линию жизни объекта, которому отправляется сообщение (вызывается его операция).

В случае использования существующей операции появится окно выбора операции, в котором перечислены операции класса, принимающего сообщение.

Например, при передаче сообщения объекту класса **Форма оформления заявки** появится окно с его операциями (рис. 10.17).

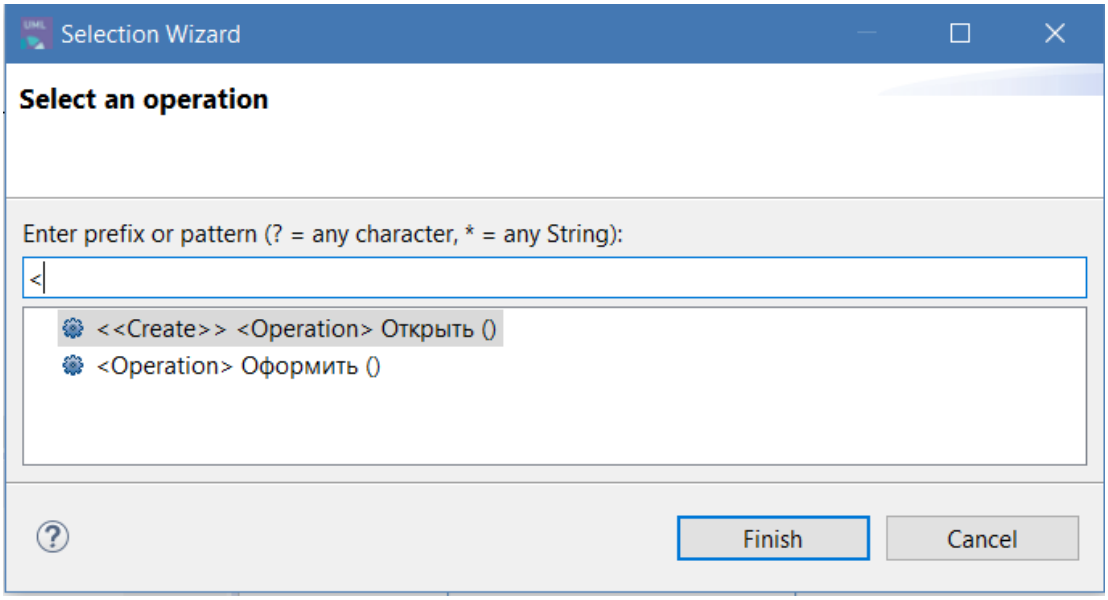


Рис. 10.17. Выбор операции при передаче сообщения

В случае использования новой операции UML Designer автоматически добавляет новую операцию в класс, принимающий сообщение и присваивает ей имя, которое затем может быть изменено. Новую операцию можно увидеть в классе на диаграмме классов.

Отличием синхронного сообщения от асинхронного является то, что при посылке синхронного сообщения отправитель должен дождаться ответа, а при посылке асинхронного сообщения он продолжает свою деятельность, не дожидаясь ответа на сообщение.

Например, менеджер должен сначала открыть форму оформления заявки на экране и для этого посылает синхронное сообщение **Открыть**. При получении этого сообщения объект класса **Форма оформления заявки** выполняет операцию **Открыть** и после ее завершения посылает обратно ответ **Открыть_reply**. Менеджер должен дождаться открытия формы, т.к. иначе он не сможет начать с нею работать.

После завершения оформления заявки менеджер посылает сообщение **Оформить**. В этом случае нет необходимости ждать окончания выполнения операции оформления, поэтому данное сообщение может быть послано в асинхронном режиме. На диаграмме видно, что после получения этого сообщения объект выполняет операцию **Оформить**, но не посылает ответного сообщения (рис. 10.18).

Последовательность передачи сообщений во времени определяется их расположением на диаграмме. Верхнее сообщение посылается раньше нижнего.

Процесс выполнения операции отражается с помощью серого прямоугольника на линии жизни, который означает, что объект находится в активном состоянии. Можно изменить размер этого прямоугольника, кликнув на него и потянув мышкой за его верхний или нижний край. Можно перетащить по линии жизни этот прямоугольник вместе с сообщениями, нажав на него мышкой и удерживая мышку.

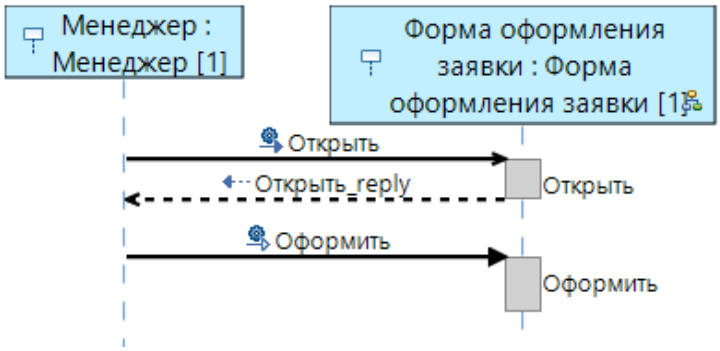


Рис. 10.18. Синхронное сообщение **Открыть** и асинхронное сообщение **Оформить**

Посмотрим на описание сообщения, например, **Открыть**. Для этого кликнем на него на диаграмме или в Model Explorer (рис. 10.19).

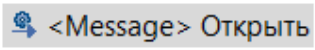


Рис. 10.19. Синхронное сообщение в Model Explorer

В Properties в поле **Message Sort** мы увидим тип сообщения, как показано на рис. 10.20.

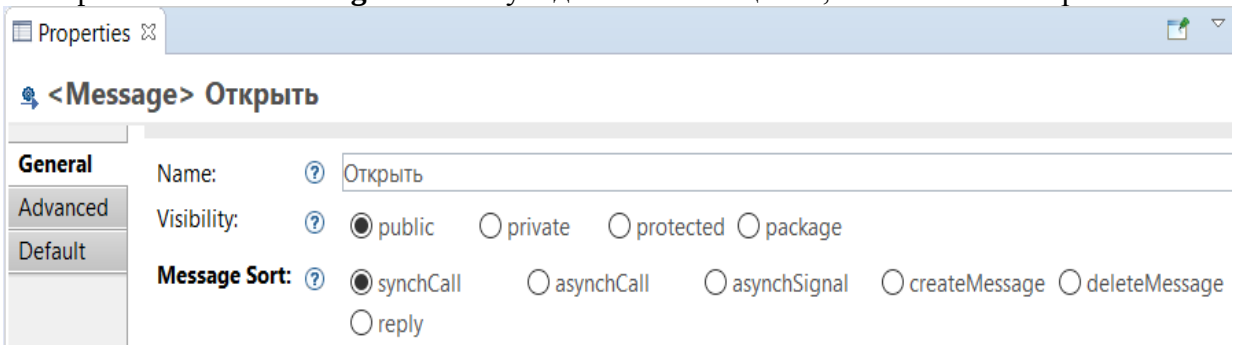


Рис. 10.20. Синхронное сообщение в Properties

Имеются пять типов сообщений:

- syncCall – синхронное сообщение, вызывающее операцию;
- asyncCall – асинхронное сообщение, вызывающее операцию;
- createMessage – сообщение, с помощью которого создается объект класса;
- deleteMessage – сообщение, с помощью которого уничтожается объект класса;
- reply – ответное сообщение.

В нашем примере сообщение **Открыть** создает новый объект класса **Форма оформления заявки**, поэтому для него нужно выбрать значение **createMessage**. На диаграмме увидим специальную иконку для сообщения создания объекта (рис. 10.21).

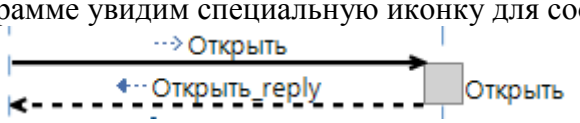


Рис. 10.21. Сообщение, создающее новый объект

Предположим, что для закрытия формы есть операция **Закрыть**. Эта операция уничтожает объект формы, поэтому выберем для нее значение **deleteMessage**. На диаграмме увидим специальную иконку для сообщения уничтожения объекта (рис. 10.22).

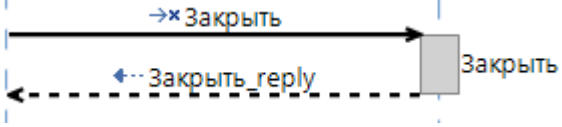


Рис. 10.22. Сообщение, уничтожающее объект

Для уничтожения сообщения кликните на сообщение, откройте меню по правой кнопке мышки, выберите пункт **Edit** и далее подпункт **Delete from Model**. Посмотрите в Model Explorer, что удалились все элементы, относящиеся к сообщению (операция и по два элемента для каждого сообщения) (рис. 10.23).

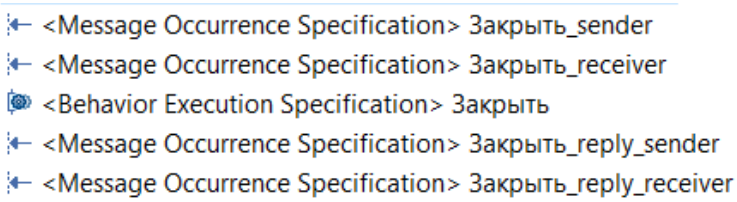


Рис. 10.23. Элементы описания сообщения в Model Explorer

Если какой-то элемент не удалился, то удалите его через меню, открываемому по правой кнопке мышки на этом элементе в Model Explorer.

Сообщение может посылаться во время выполнения операции объектом. Кликните на нужный элемент сообщения в палитре инструментов, затем на серый прямоугольник, соответствующий операции, а потом на линию жизни объекта, которому передается сообщение.

Например, операция **Выбор заявки** запрашивает список заявок у объекта класса **Оформление** с помощью вызова операции **Поиск заявок** (рис. 10.24).

Во время выполнения операции может потребоваться обращение к операции того же класса. Например, после получения списка заявок от объекта класса **Оформление** их нужно показать на экране в виде таблицы. Для этого нужно выполнить операцию **Показ** из класса **Оформление заявки.** Кликните на нужный элемент сообщения в палитре инструментов, затем на серый прямоугольник операции **Выбор заявки**, а потом на линию жизни под этим прямоугольником. На диаграмме появится так называемый рекурсивный вызов, когда объект вызывает свою же операцию (рис. 10.24).

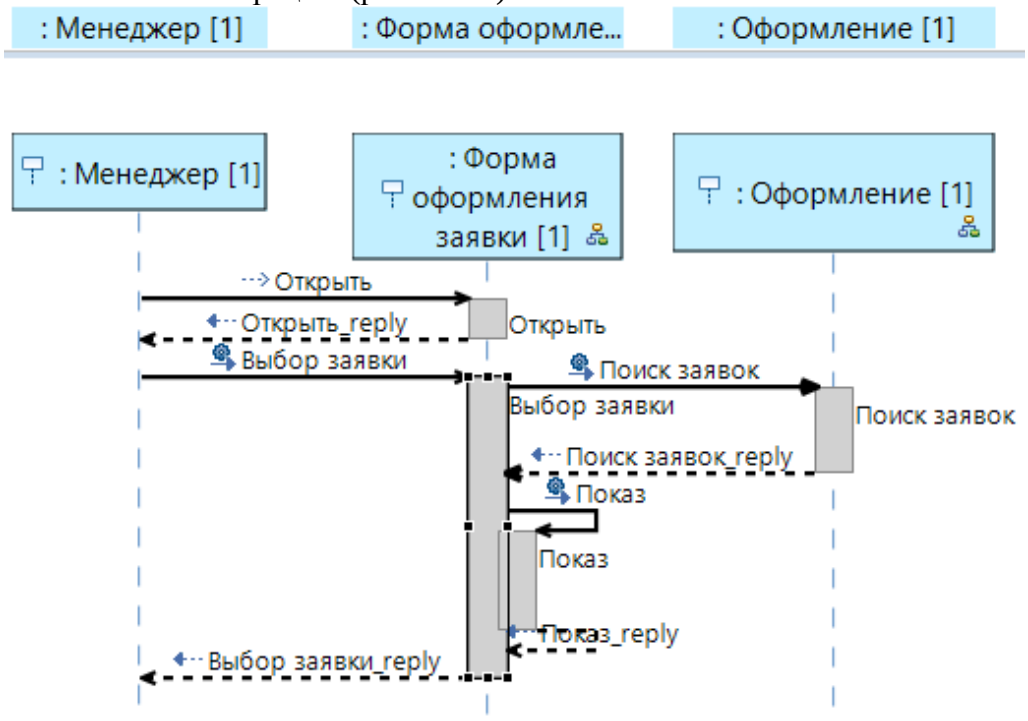


Рис. 10.24. Вызов операции во время выполнения другой операции и рекурсивный вызов

Кликнув на имя сообщения над стрелкой, в виде Properties можно увидеть подробное описание операции, соответствующей этому сообщению. Например, для сообщения **Поиск заявок** в поле **Signature** мы увидим, что при вызове операции передается параметр **дата**, а результатом выполнения операции является массив заявок (рис. 10.25).



Рис. 10.25. Параметры операции

Эту операцию можно также увидеть и на диаграмме классов (рис. 10.26).

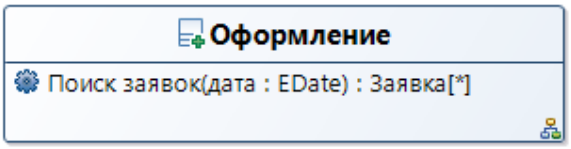


Рис. 10.26. Операция класса, выполняемая при передаче сообщения

Значок в правом нижнем углу изображения класса означает, что с этим классов связаны другие диаграммы. Мы уже видели, что с классом можно связать диаграмму состояний. В данном случае с классом связана диаграмма последовательности, которую можно открыть через меню, открываемому по правой кнопке мышки на изображении класса (рис. 10.27).

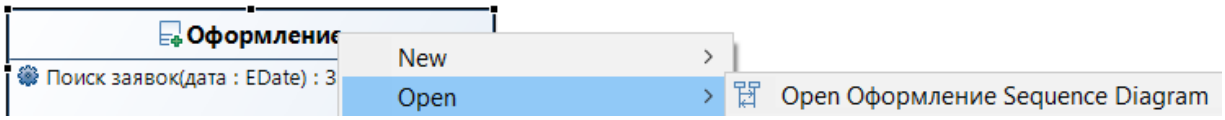


Рис. 10.27. Открытие диаграммы последовательности, связанной с классом

10.3. Пример диаграммы последовательности

На рис. 10.28 приведена более полная диаграмма последовательности. Диаграмма учитывает, как ранее рассмотренные классы, так и дополнительный класс **Заявки**, обеспечивающий работу с данными по заявкам. Также зададим для каждого объекта его стереотип. Четкое разделение классов на три стереотипа (boundary, control, entity), в общем случае, не является обязательным, но, по крайней мере, на начальном этапе проектирования системы позволяет четко определить чем занимаются эти классы. Такое разделение также упрощает дальнейшую разработку этих классов.

В предположении, что менеджер уточняет у клиента информацию по телефону, мы не изображаем клиента на диаграмме, а показываем сам процесс с помощью вызова операции **Работа**.

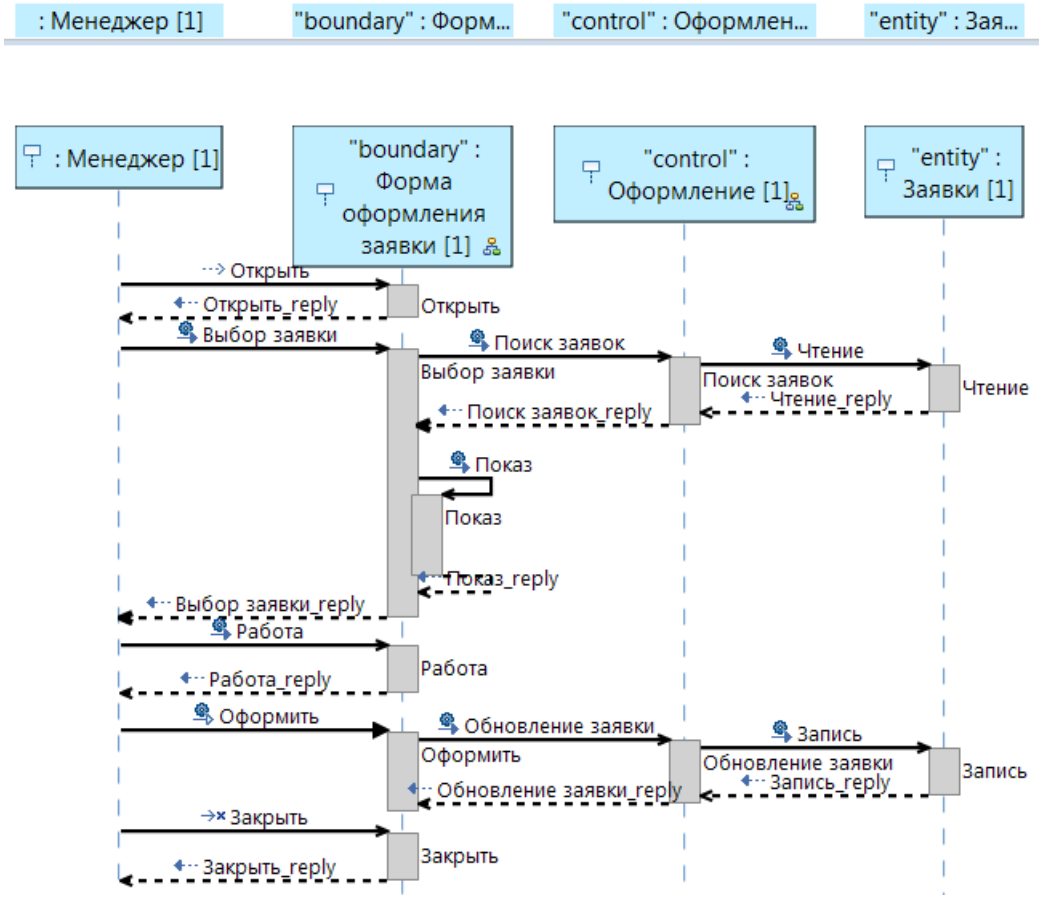


Рис. 10.28. Пример диаграммы последовательности

11. Диаграмма компонентов (Component Diagram)

Диаграмма компонентов служит для описания структуры системы. Вся система разбивается на отдельные модульные части – компоненты. Компонент представляет собой упаковку некоторого содержимого системы. Компонент предоставляет некоторые услуги с помощью интерфейсов другим компонентам. Для работы компонента требуется определеный набор услуг других компонентов. Компонент системы может быть легко заменен на другой компонент, который предоставляет и требует тот же набор интерфейсов, что и исходный компонент. Добавляя в систему новые компоненты можно расширить ее функциональные возможности.

Компонент может представлять из себя как логическую часть системы, например, подсистему, так и физическую ее часть, например, файл с кодом программы или данными.

11.1. Создание диаграммы компонентов

Диаграмма компонентов может быть размещена, например, непосредственно в элементе **<Model> Internet Provider** или в пакете **Компоненты**. Создадим ее в пакете **Компоненты** с помощью меню, которое откроем по правой кнопке мышки, встав на строку элемента **<Package> Компоненты** в виде Model Explorer. Выберем пункт **New Representation** и далее в открывшемся подменю пункт **Компоненты Component Diagram** (рис. 11.1).

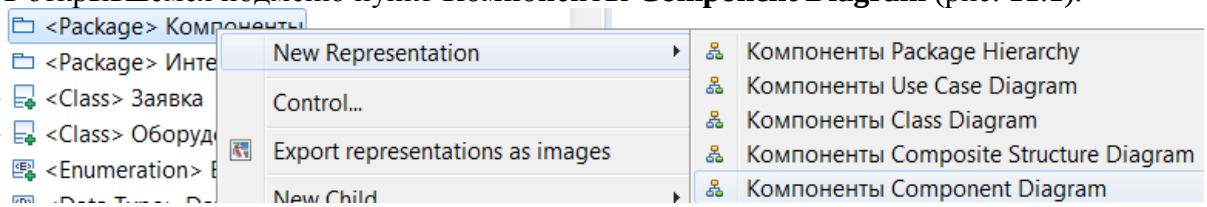


Рис. 11.1. Создание диаграммы компонентов

В открывшемся окне **New Component Diagram** изменим имя диаграммы на **Компоненты** (рис. 11.2).

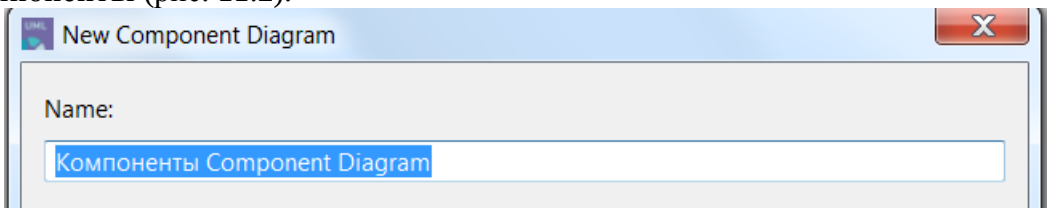


Рис. 11.2. Изменение имени диаграммы

В Model Explorer увидим появившуюся диаграмму (рис. 11.3).

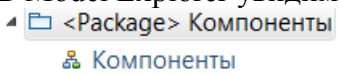


Рис. 11.3. Диаграмма компонентов в Model Explorer

11.2. Элементы диаграммы компонентов

Диаграмма компонентов имеет палитру инструментов, состоящую из трех разделов. Первый раздел позволяет использовать на диаграмме существующие элементы (рис. 11.4).

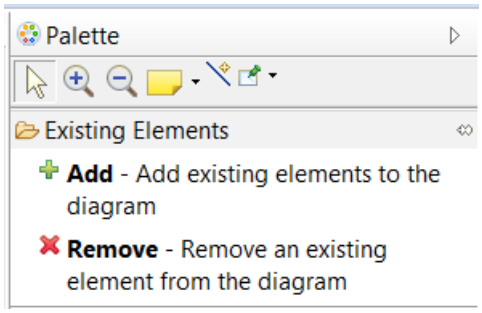


Рис. 11.4. Первый раздел палитры инструментов

Элемент добавить (**Add**) используется для добавления существующих элементов. Кликните на эемент Add, а затем на диаграмму. Появится окно, где в поле Show выберите какие группы элементов вы хотите добавить (потомки элемента размещения; не показанные на диаграмме элементы; все элементы) (рис. 11.5).

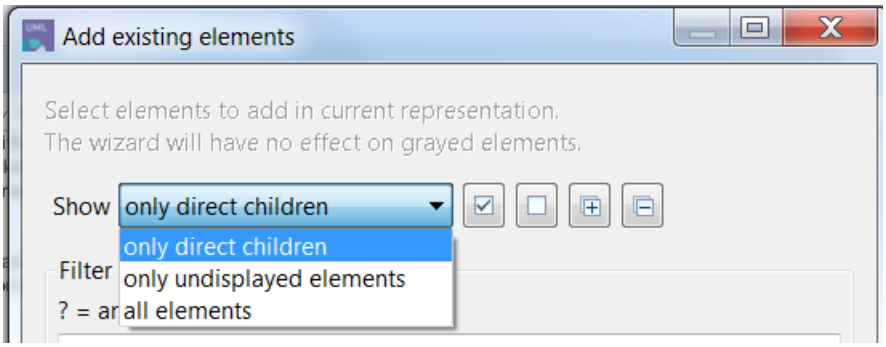


Рис. 11.5. Варианты показа существующих элементов

В нашем примере диаграмма создавалась внутри пакета **Компоненты,** поэтому при добавлении элемента на диаграмму и выборе **only direct children** вы увидите все элементы, подчиненные **<Package>Компоненты.** Если же добавить элемент внутрь некоторого компонента, то будут показаны только элементы, находящиеся внутри этого компонента.

Добавить на диаграмму можно будет только те элементы, которых нет на диаграмме.

Можно также указать в поле Filter условие для показа только некоторых элементов,, Например, фильтр *Class* можно использовать для показа только классов.

Предположим, мы хотим добавить элементы, которые не присутствуют на данной диаграмме (рис. 11.6).

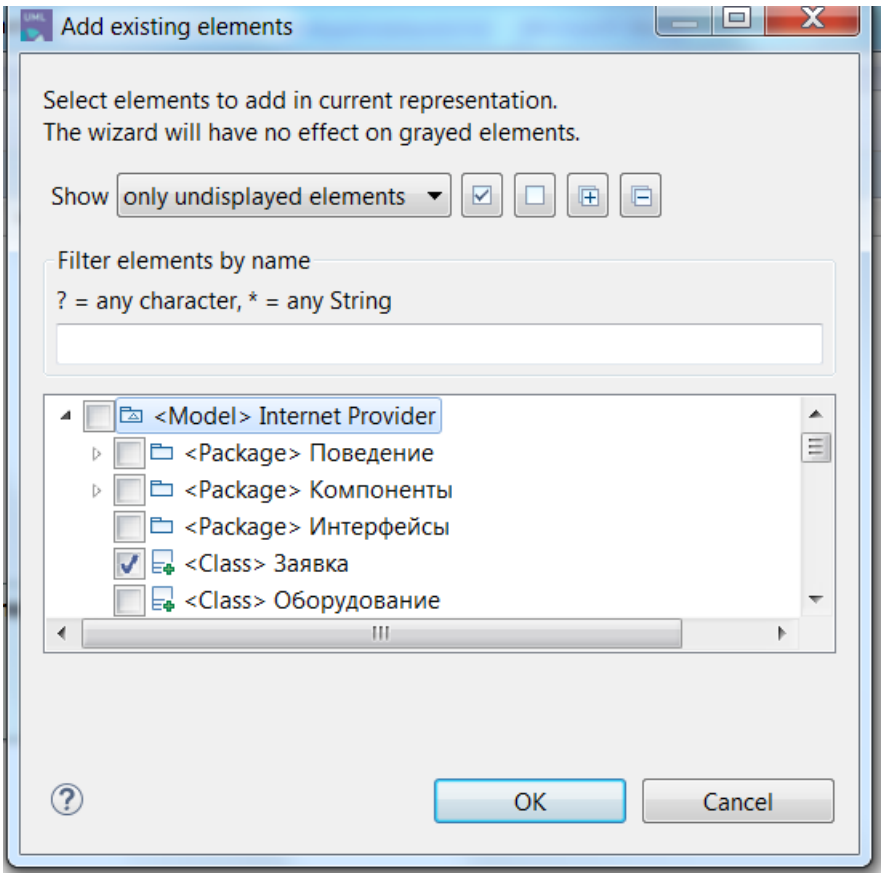


Рис. 11.6. Выбор элементов для показа на диаграмме

Выберите один или несколько элементов и они появятся на диаграмме. Например, если уже нарисована диаграмма классов, то классы и интерфейсы, показанные на этой диаграмме,

можно использовать на диаграмме пакетов. Также можно использовать компоненты, которые были описаны на диаграмме вариантов использования.

Элемент удаления (**Remove**) используется для удаления элемента диаграммы. Кликните на этот элемент, а потом на элемент диаграммы. Элемент будет удален с диаграммы, но не из модели, поэтому его можно будет вернуть на диаграмму с помощью элемента Add.

Второй раздел палитры инструментов содержит элементы диаграммы, показанные на рис. 11.7.

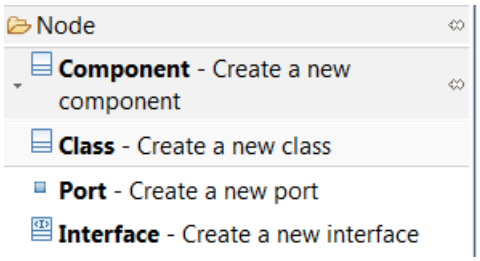


Рис. 11.7. Второй раздел палитры инструментов

Элемент компонент (**Component**) используется для показа компонента на диаграмме. Например, у нас есть подсистема оплаты (рис. 11.8).

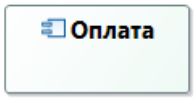


Рис. 11.8. Компонент

Имя компонента можно указать при создании компонента в окне **Create Component** (рис. 11.9).

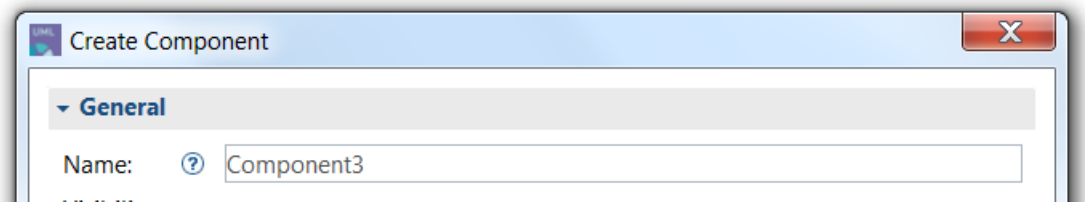


Рис. 11.9. Создание компонента

Имя компонента можно будет изменить в окне Edit, открываемому по двойному клику на элементе диаграммы, или поле Name элемента в виде Properties.

Для компонета может быть указан стереотип с помощью клика на кнопку  в поле Stereotipes в окне Edit или виде Properties (рис. 11.10).



Рис. 11.10. Определение стереотипа компонента

В открывшемся окне выберите стереотип и нажмите кнопку Add, а затем Finish (рис. 11.11).

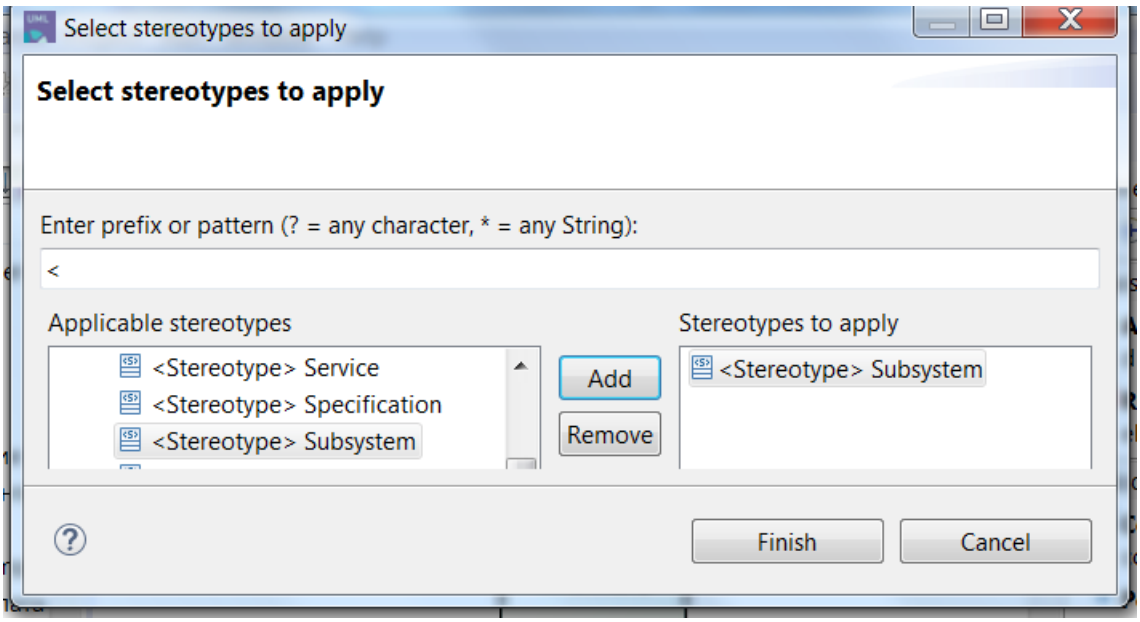


Рис. 11.11. Выбор стереотипа

На диаграмме для компонента появится описание его стереотипа (рис. 11.12).

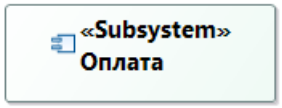


Рис. 11.12. Подсистема

Компонент может содержать внутри себя другие компоненты. Компонент, принадлежащий другому компоненту, располагается на диаграмме внутри компонента или связывается с ним с помощью отношения **Component Realization** (см. следующий раздел) (рис. 11.13).

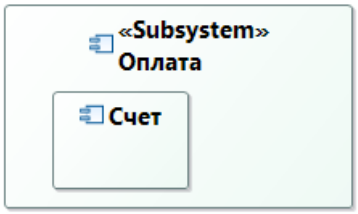


Рис. 11.13. Внутренний компонент

Элемент класс (**Class**) используется для показа классов, которые реализуют компоненты. Например, компонент **Счет** – это физический файл, в котором содержится описание класса **Счет** на некотором языке программирования. Класс, принадлежащий компоненту, располагается на диаграмме внутри компонента или связывается с ним с помощью отношения **Component Realization** (см. следующий раздел) (рис. 11.14).

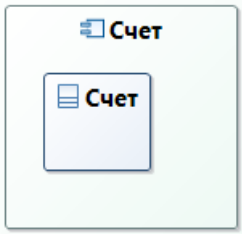


Рис. 11.14. Класс, реализующий компонент

При добавлении на диаграмму класса **Счет** с помощью инструмента добавления существующих элементов **Add** сначала поместите класс на свободное место диаграммы, а уже потом перетащите его мышкой на компонент или установите между классом и

компонентом отношение реализации. После этого в виде Model Explorer вы увидите, что элемент **<Class> Счет** переместился внутрь элемента **<Compoinent> Счет** (рис. 11.15).

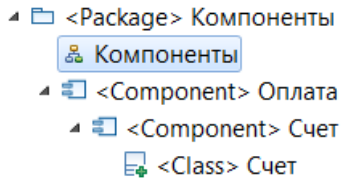


Рис. 11.15. Класс, реализующий компонент, в Model Explorer

Элемент интерфейс (**Interface**) используется для описания набора операций. Интерфейс только описывает операции, но не реализует их. На диаграмме интерфейс изображается как показано на рис. 11.16.

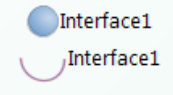


Рис. 11.16. Интерфейс

Элемент порт (Port) используется для группировки интерфейсов. На диаграмме порт изображается на границе компонента или класса, как показано на рис. 11.17.

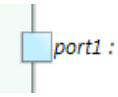


Рис. 11.17. Порт

11.3. Отношения между элементами диаграммы компонентов

Отношения между элементами диаграммы компонентов задаются с помощью элементов, представленных в третьем разделе палитры инструментов (рис. 11.18).

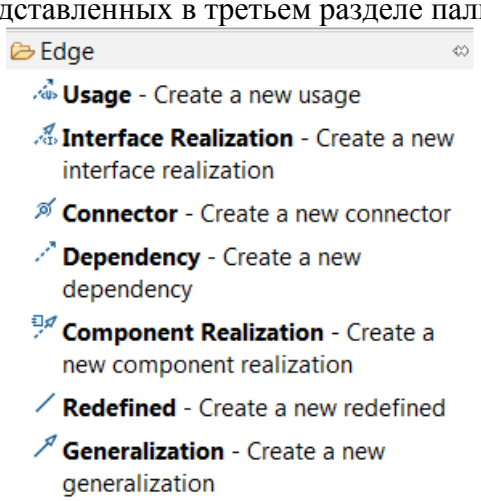


Рис. 11.18. Третий раздел палитры инструментов

Для установки любого отношения кликните на элемент отношения, затем на элемент, из которого исходит дуга отношения, а потом на элемент, в который входит дуга отношения.

Отношение реализации (**Component Realization**) показывает реализацию компонента с помощью класса или другого компонента (рис. 11.19).

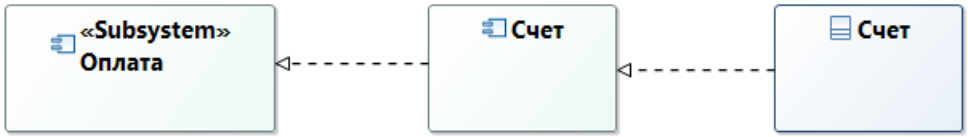


Рис. 11.19. Отношение реализации компонента

Для реализации интерфейсов используются классы. На диаграмме компонентом можно также показать, что интерфейс реализуется некоторым компонентом без указания конкретного класса, который его реализует. Реализуемый некоторым классом или компонентом интерфейс, называется предоставляемым или экспортируемым интерфейсом.

Отношение реализация интерфейса (**Interface Realization**) связывает интерфейс с компонентом или классом (рис. 11.20).

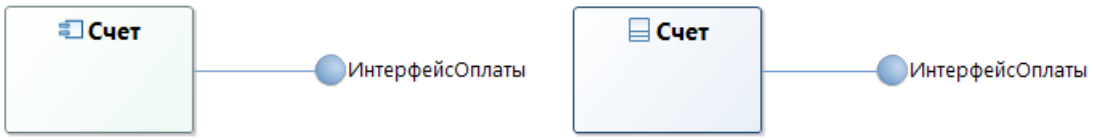


Рис. 11.20. интерфейса реализации

Для своей работы компонент или класс может использовать один или несколько интерфейсов. Такие интерфейсы называются требуемыми или импортируемыми.

Отношение использования (**Usage**) показывает, что класс или компонент использует (импортирует) интерфейс (рис. 11.21).

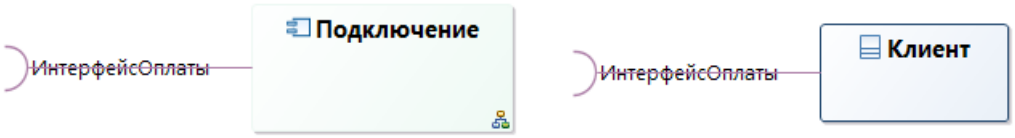


Рис. 11.21. Отношение использования интерфейса

Отношение соединения (**Connector**) связывает два изображения одного и того же интерфейса, выступающего в роли предоставляемого и требуемого интерфейса (рис. 11.22).

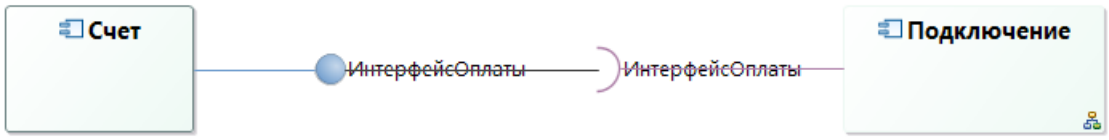


Рис. 11.22. Отношение соединения

Соединение изображений интерфейса возможно только после того, как интерфейс связан с компонентами или классами с помощью отношений реализации и использования.

Взаимодействие компонентов или классов может быть показано также с помощью элементов портов (**Port**) и отношения переопределения (**Redefined**). Вместо интерфейса на каждый компонент (или класс) помещается элемент порт. В отличие от интерфейса, т.к. порты разные, то, в общем случае, имена их тоже могут быть разными. В данном примере будем использовать одно имя для обоих портов. Порт источник (на рисунке справа) связывается с портом целью (стрелка направлена от порта источника к порту цели) с помощью отношения переопределения (рис. 11.23).

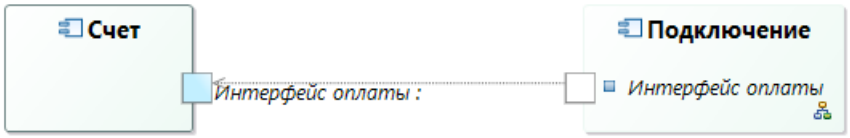


Рис. 11.23. Взаимодействие компонентов с помощью портов

Порты также используются для группировки предоставляемых и требуемых интерфейсов одним компонентом или классом. Например, компонент **Счет** предоставляет два интерфейса, которые можно подключить к порту, используя отношение реализации интерфейса (рис. 11.24).

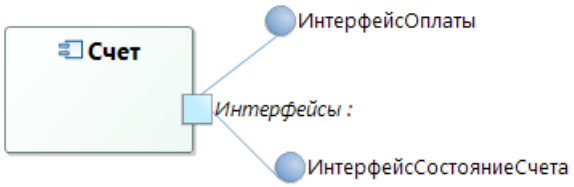


Рис. 11.24. Порт, используемый для группировки интерфейсов

Отношение обобщения (**Generalization**) используется для связи, либо двух классов, либо двух компонентов. Оно показывает связь между более общим (родительским) и частным (дочерним) элементом. Например, если родительский компонент предоставляет некоторый интерфейс, то этот же интерфейс будут предоставлять и дочерние компоненты. Но реализация интерфейса может быть разной (рис. 11.25).

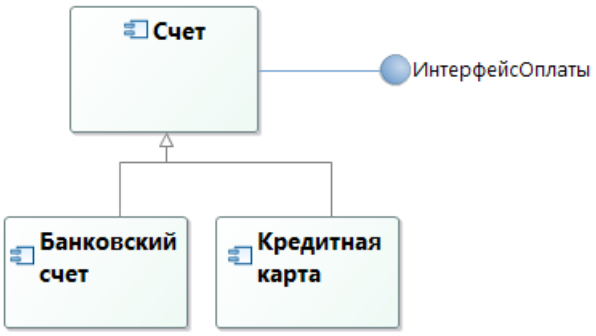


Рис. 11.25. Отношение обобщения

12. Диаграмма развертывания (Deployment Diagram)

Диаграмма развертывания показывает размещение артефактов (физических объектов, созданных человеком) по вычислительным узлам распределенной системы и взаимодействие узлов и артефактов.

12.1. Создание диаграммы развертывания

Добавим на диаграмму пакетов пакет **Физические объекты**. Нажмем правую кнопку мышки на этом элементе, выберем пункт меню **New** и далее в подменю пункт **Deployment diagram** (рис. 12.1).

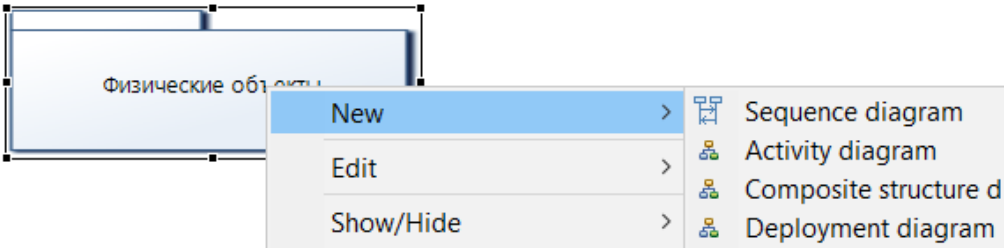


Рис. 12.1. Создание диаграммы развертывания внутри пакета

В открывшемся окне New Deployment Diagram переименуем диаграмму на **Развертывание** (рис. 12.2).

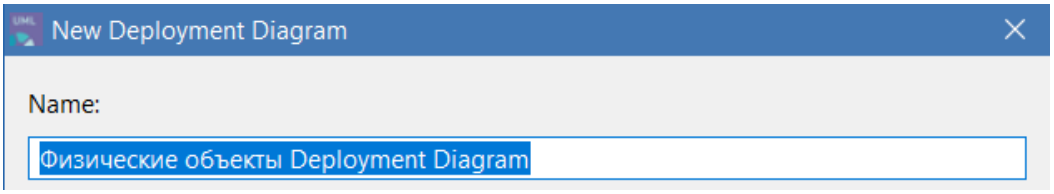


Рис. 12.2. Окно новой диаграммы

В Model Explorer увидим новый пакет и в нем созданную диаграмму (рис. 12.3).

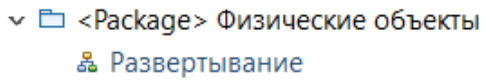


Рис. 12.3. Изображение пакета и диаграммы в Model Explorer

12.2. Элементы диаграммы развертывания

Диаграмма развертывания имеет палитру инструментов, состоящую из четырех разделов (рис. 12.4).

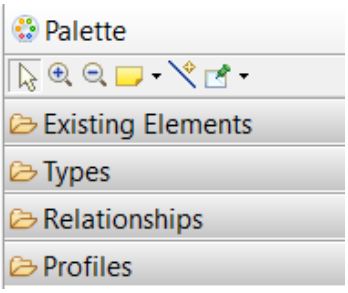


Рис. 12.4. Разделы палитры инструментов диаграммы развертывания

Первый раздел позволяет использовать на диаграмме существующие элементы (рис. 12.5).

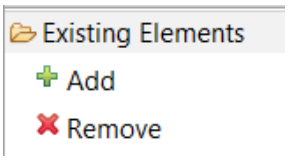


Рис. 12.5. Первый раздел палитры инструментов

Элемент добавить (**Add**) используется для добавления существующих элементов (компонентов и пакетов) также как при добавлении существующих элементов на диаграмму компонентов (подраздел 21.2 Элементы диаграммы компонентов).

Элемент удаления (**Remove**) используется для удаления элемента диаграммы. Кликните на этот элемент, а потом на элемент диаграммы. Элемент будет удален с диаграммы, но не из модели, поэтому его можно будет вернуть на диаграмму с помощью элемента Add.

Второй раздел содержит основные типы элементов диаграммы развертывания (рис. 12.6).

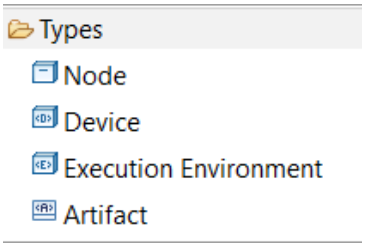


Рис. 12.6. Второй раздел палитры инструментов

Элемент узел (**Node**) используется для показа некоторого вычислительного узла. Существует два типа узлов: устройство и среда выполнения.

Элемент устройство (**Device**) используется для показа некоторого вычислительного устройства (сервер, локальный компьютер, процессор) или некоторого физического устройства, подключенного к компьютеру (принтер, монитор, датчик), или некоторого физического компонента компьютера (память, дисковод).

Элемент среда выполнения (**Execution Environment**) используется для показа некоторой программной среды (операционная система, веб-браузер, веб-сервер, система управления базами данных).

Элемент артефакт (**Artifact**) – это любой физический объект, размещенный в узле (обычно некоторый файл).

Четвертый раздел палитры инструментов содержит элемент стереотип (**Stereotype**) (рис. 12.7).

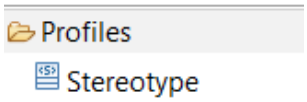


Рис. 12.7. Четвертый раздел палитры инструментов

С его помощью можно установить стереотип для любого элемента диаграммы. Кликните на этот элемент, а потом на элемент диаграммы. В открывшемся окне Select выберите нужный стереотип и кликните на кнопку Add, а потом на Finish (рис. 12.8).

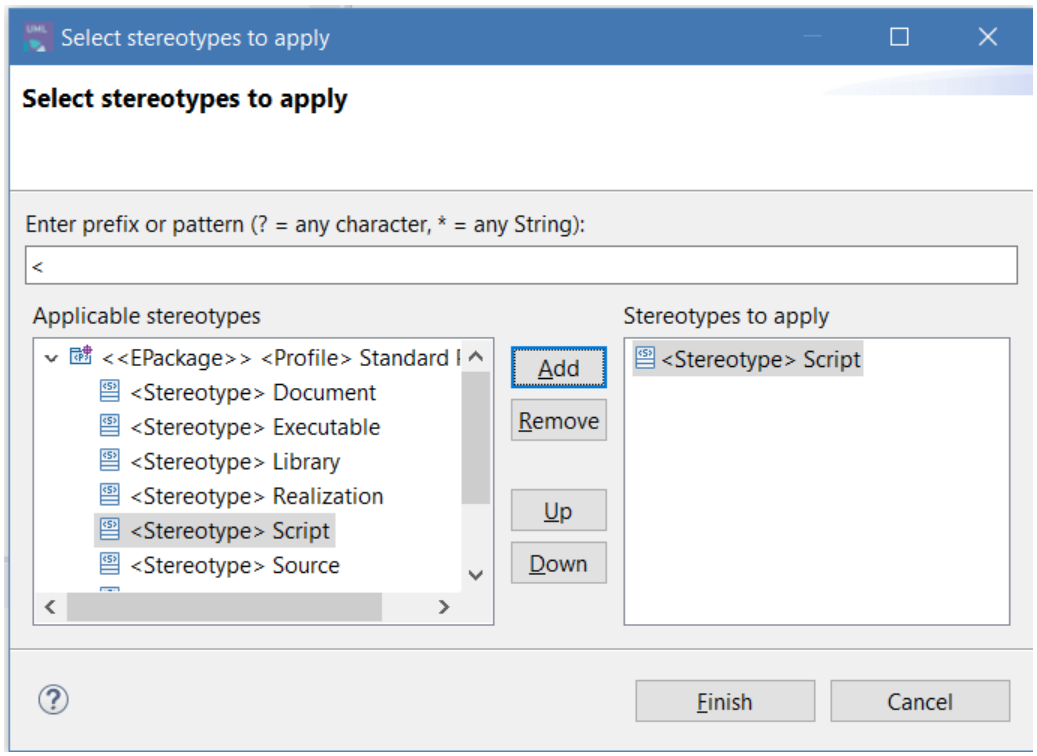


Рис. 12.8. Окно выбора стереотипа

Список возможных стереотипов будет зависеть от выбранного элемента диаграммы. На рисунке показаны стереотипы для артефакта. Стереотип элемента показывается на диаграмме (рис. 12.9).

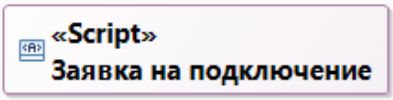


Рис. 12.9. Стереотип артефакта

Стереотип элемента может быть также установлен в окне редактирования свойств элемента, открываемому по двойному клику на элементе, или виде Properties (рис. 12.10).

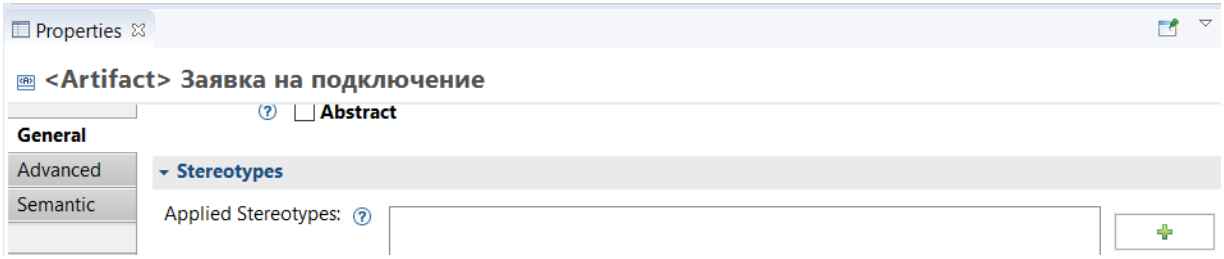


Рис. 12.10. Задание стереотипа в виде Properties

12.3. Отношения между элементами диаграммы развертывания

Отношение между элементами диаграммы развертывания определяются с помощью элементов, представленных в третьем разделе палитры инструментов (рис. 12.11).

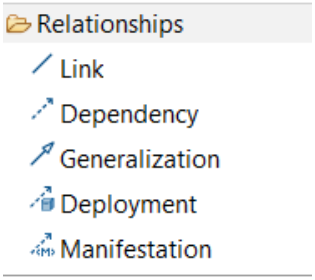


Рис. 12.11. Отношения на диаграмме развертывания

Элемент связь (**Link**) показывает взаимодействие между любыми элементами (узлами, артефактами, компонентами, пакетами) на диаграмме. С помощью имени связи можно показать среду, через которую осуществляется взаимодействие узлов. Например, компьютер клиента связан через Интернет с веб-сервером, на котором расположен сайт интернет-провайдера (рис. 12.12).

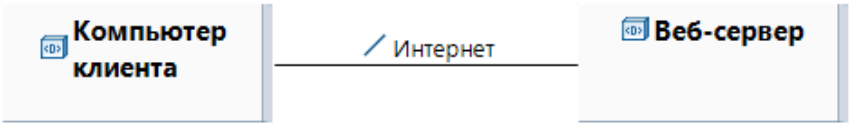


Рис. 12.12. Отношение связи

Имя связи может быть задано в виде Properties или окне редактирования свойств элемента Edit, открываемому по двойному клику на элементе связи. Обратите внимание, что в имени нет символа «/» (рис. 12.13).



Рис. 12.13. Имя связи

Элемент зависимость (**Dependency**) показывает зависимось между компонентами и узлами, а также между узлами. Компоненты должны располагаться на некотором узле (рис. 12.14).



Рис. 12.14. Отношение зависимости

Если один узел находится внутри другого узла, то его можно непосредственно разместить на диаграмме внутри узла, а можно воспользоваться отношением зависимости (рис. 12.15).



Рис. 12.15. Расположение одного узла в другом узле

Элемент обобщение (**Generalization**) используется для показа связи между общим и частным элементами. С помощью него могут быть связаны два узла, два компонента, два артефакта. Например, можно показать какие типы серверов используются в системе (рис. 12.16).

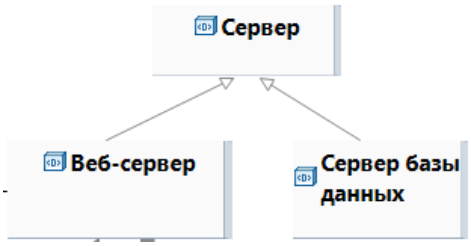


Рис. 12.16. Отношение обобщения

Элемент размещение (**Deployment**) используется для показа размещения артефакта на некотором узле. Артефакт может быть также непосредственно показан внутри узла, тогда связь размещение не нужна (рис. 12.17).



Рис. 12.17. Отношение размещения

Элемент манифестации (**Manifestation**) устанавливает отношение между компонентом и артефактом, которое означает, что артефакт является физической реализацией компонента (рис. 12.18).

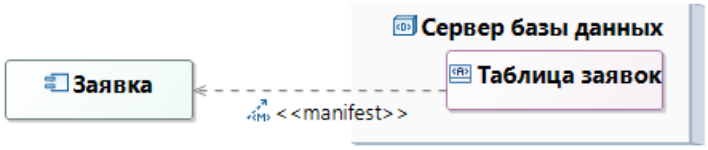


Рис. 12.18. Отношение манифестации

Обратите внимание, что в виде Model Explorer элементы второго раздела (различные узлы) появляются внутри пакета, в которм создана диаграмма, а все отношения на одном уровне с элементом пакета (рис. 12.19).

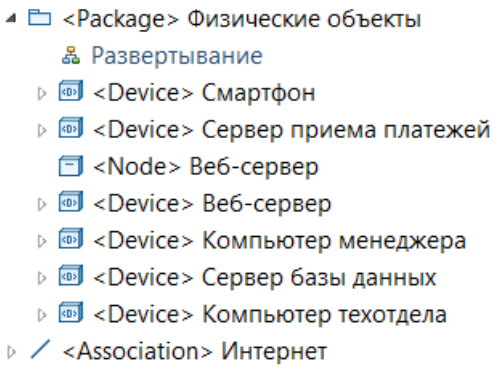


Рис. 12.19. Описание узлов внутри пакета

12.4. Пример диаграммы развертывания

В примере на рис. 12.20 показано взаимодействие смартфона клиента, трех серверов и двух комьютеров с помощью отношения связи. При этом смартфон клиента, веб-сервер и сервер приема платежей общаются друг с другом через интернет. Веб-сервер, сервер базы данных и компьютеры менеджера и техотдела находятся внутри одной локальной сети.

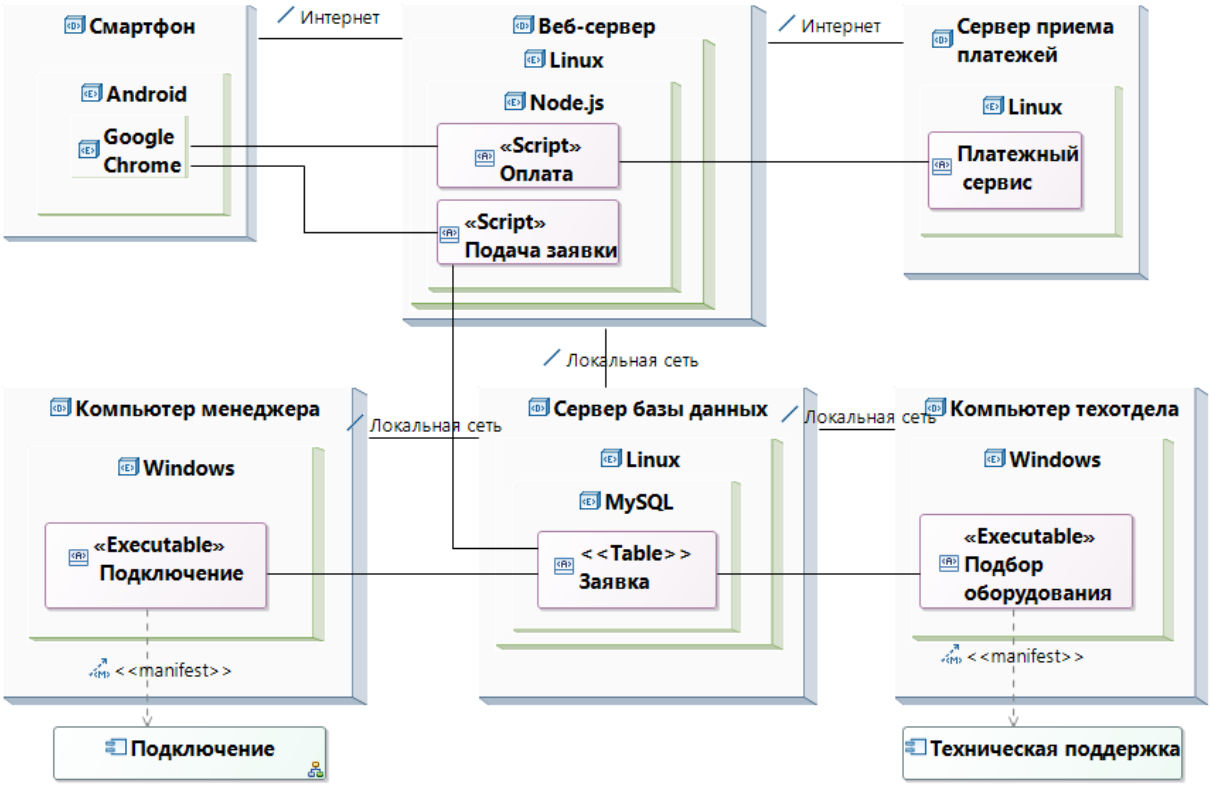


Рис. 12.20. Пример диаграммы развертывания

Клиент подает заявку со своего смартфона через браузер Google Chrome. Клиент также может выполнить оплату через браузер. При этом задействован некоторый платежный сервис. Выполнение оплаты и подача заявки клиентом реализуется с помощью скриптов, работающих в среде поддержки серверной части веб-приложения Node.js.

На смартфоне работает операционная система Android. Сервера используют ОС Linux, а на компьютерах менеджера и технического отдела стоит ОС Windows.

Менеджер и техотдел работают с исполняемыми файлами приложений. Эти артефакты имеют стереотип Executable. На диаграмме также показано с помощью отношения манифестации какие компоненты реализуют эти артефакты.

Заявки хранятся в таблице, поддерживаемой системой управления базой данных MySQL. Т.к. в UML Designer не нашлось стереотипа Table, то название стереотипа было записано в имени артефакта Заявка.

Взаимодействие с таблицей **Заявка** скрипта **Подача заявки**, приложения **Подключение** и приложения **Подбор оборудования** показано с помощью отношения **с**вязи.

Отношение связи также использовано, чтобы показать обращение браузера **Google Chome** к скриптам **Оплата** и **Подача заявки**, а также, обращение скрипта **Оплата** к артефакту **Платежный сервис**.

Литература

1. Буч Г., Рамбо Д., Якобсон И. Язык UML. Руководство пользователя. М.: ДМК Пресс, 2008. — 494 с. ISBN: 5-94074-334-X
2. Кватрани Т. Rational Rose 2000 и UML. Визуальное моделирование. М.: ДМК Пресс, 2009. — 176 с. ISBN: 5-94074-131-2
3. Розенберг Д., Скотт К. Применение объектного моделирования с использованием UML и анализ прецедентов. М.: ДМК Пресс, 2007. ISBN: 5-94074-050-2
4. Буч Г., Рамбо Д., Якобсон И. Введение в UML от создателей языка. 2-е изд. М.: ДМК Пресс, 2015. — 496 с. ISBN: 978-5-97060-157-0
5. Джим Арлоу, А. Нейштадт. UML 2 и Унифицированный процесс. Практический объектно-ориентированный анализ и проектирование, 2-е издание. - СПб: Символ-Плюс, 2007. – 624 с.
6. Леоненков А.В. Самоучитель UML2. BHV-СПб, 2007, - 576 с.
7. Гома Х. UML Проектирование систем реального времени 2 изд. М.: ДМК Пресс, 2016. — 159 с. ISBN: 978-5-97060-220-1
8. Фаулер М., Скотт К. UML. Основы, 3-е издание. - СПб: Символ-Плюс, 2005. — 192 с. ISBN: 5-93286-060-X
9. UML Designer. http://www.umldesigner.org/
10. Стандарты языка UML. http://www.omg.org/spec/UML

Св. план 2019 г., поз. 97

Давыдовский Михаил Альбинович
Никольская Марина Николаевна

ПРОЕКТИРОВАНИЕ ПРОГРАММНОЙ СИСТЕМЫ В UML DESIGNER

Учебное пособие